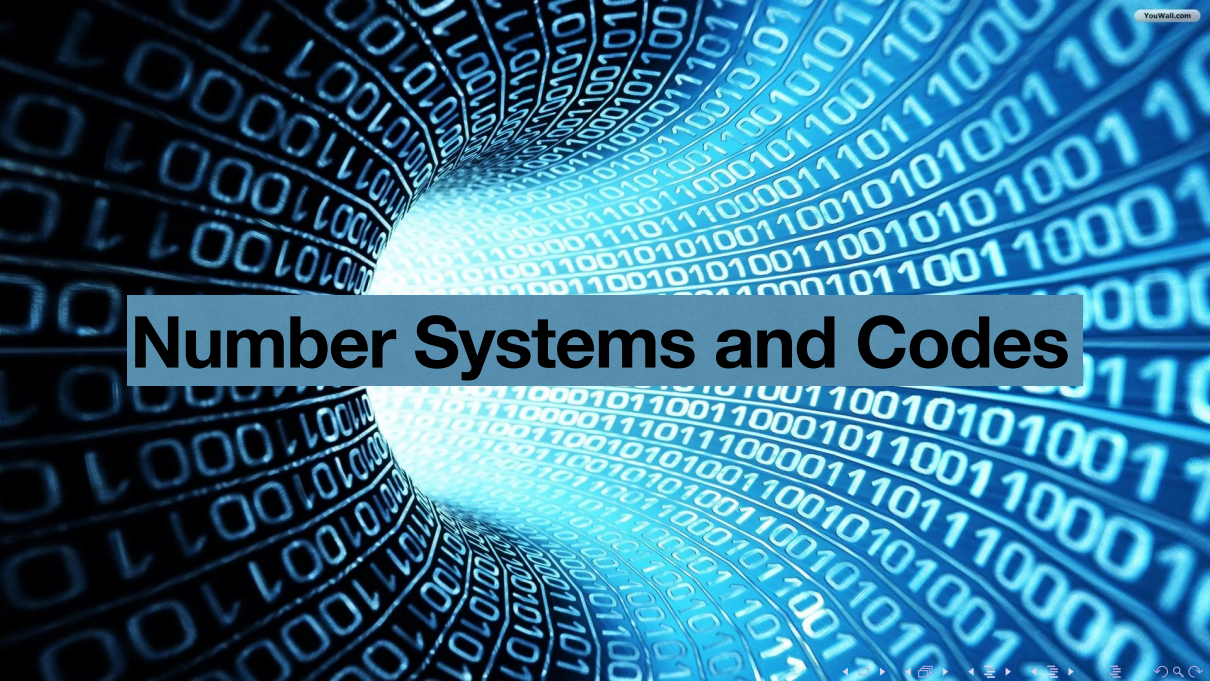
Index

1. Digital Logic
   - Number Systems
   - Switching Algebra
   - Logic Minimization
2. DL Circuit Design
   - Combinational Circuits
   - Sequential Circuits
3. Instruction Set Architecture
4. Datapath
   - Processor Datapath and Pipelining
   - Pipeline Hazards
   - Advanced Pipelining
5. Memory Handling
   - Cache
   - DRAM
   - VRAM

Digital Logic Design + Computer Architecture

Sayandeep Saha

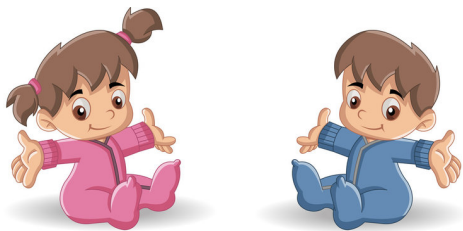
Assistant Professor
Department of Computer Science and Engineering
Indian Institute of Technology Bombay



Number Systems and Codes

Baby Step

- **Let's go back to the days when we were 2 years old…**
  - Learning numbers again….but in a new way…



Numbers in Computing

- We normally use the decimal number system.
- But computers understand only bits…
- How to compute on bits??

Generalization of Number Representation

- Numbers are represented in a “base”.
- Decimal (Base 10): $953.78_{10} = 9 \times 10^2 + 5 \times 10^1 + 3 \times 10^0 + 7 \times 10^{-1} + 8 \times 10^{-2}$
- Binary (Base 2):

$$1011.11_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2}$$
$$= 8 + 0 + 2 + 1 + \tfrac{1}{2} + \tfrac{1}{4} = 11\tfrac{3}{4} = 11.75_{10}$$

- The general case — base b number:
$(N)_b = a_{q-1}b^{(q-1)} + a_{q-2}b^{(q-2)} + \cdots + a_2b^2 + a_1b^1 + a_0b^0 + a_{-1}b^{-1} + \cdots a_{-p}b^{-p}, \quad 0 \le a_i < b, b > 1$
  - $a_{(q-1)}$ is called the Most Significant Digit (MSD)
  - $a_{(0)}$ is called the Least Significant Digit (MSD)
  - $a_{(-1)}$ — $a_{(-p)}$ are digits in the fractional part.

Generalization of Number Representation

Base				
2	4	8	10	12
0000	0	0	0	0
0001	1	1	1	1
0010	2	2	2	2
0011	3	3	3	3
0100	10	4	4	4
0101	11	5	5	5
0110	12	6	6	6
0111	13	7	7	7
1000	20	10	8	8
1001	21	11	9	9
1010	22	12	10	α
1011	23	13	11	β
1100	30	14	12	10
1101	31	15	13	11
1110	32	16	14	12
1111	33	17	15	13

Base Conversion

- Octal (b=8) to Decimal (b=10): $(432.2)_8 = 4 \times 8^2 + 3 \times 8^1 + 2 \times 8^0 + 2 \times 8^{-1} = (282.25)_{10}$
- Binary (base 2) to Decimal (b=10): $(1010.011)_2 = 2^3 + 2^1 + 2^{-2} + 2^{-3} = (10.375)_{10}$

Base Conversion: Decimal to…

To Binary:

$2 \,/\, 53$
$2 \,/\, 26$ rem. $= 1 = a_0$
$2 \,/\, 13$ rem. $= 0 = a_1$
$2 \,/\, 6$ rem. $= 1 = a_2$ $\quad 53_{10} = 110101_2$
$2 \,/\, 3$ rem. $= 0 = a_3$
$2 \,/\, 1$ rem. $= 1 = a_4$
0 rem. $= 1 = a_5$

Convenient way of writing

Integer	Remainder
41	
20	1
10	0
5	0
2	1
1	0
0	1

101001 = answer

To Octal:

153	
19	1
2	3
0	2

$= (231)_8$

Base Conversion

- General Rule: for $b_1 > b_2$

$$(N)_{b_1} = a_{q-1}b_2^{q-1} + a_{q-2}b_2^{q-2} + \cdots + a_1 b_2^1 + a_0 b_2^0$$

$$\frac{(N)_{b_1}}{b_2} = \underbrace{a_{q-1}b_2^{q-2} + a_{q-2}b_2^{q-3} + \cdots + a_1}_{Q_0} + \frac{a_0}{b_2}$$

$$\left(\frac{Q_0}{b_2}\right)_{b_1} = \underbrace{a_{q-1}b_2^{q-3} + a_{q-2}b_2^{q-4} + \cdots}_{Q_1} + \frac{a_1}{b_2}$$

Base Conversion: Fractions

To Binary:

	Integer		Fraction	Coefficient
$0.6875 \times 2 =$	1	+	0.3750	$a_{-1} = 1$
$0.3750 \times 2 =$	0	+	0.7500	$a_{-2} = 0$
$0.7500 \times 2 =$	1	+	0.5000	$a_{-3} = 1$
$0.5000 \times 2 =$	1	+	0.0000	$a_{-4} = 1$

Answer: $(0.6875)_{10} = (0.a_{-1}a_{-2}a_{-3}a_{-4})_2 = \underline{(0.1011)_2}$

- Important!!! If a number has both integer and fraction parts, then conversion of these two parts are done separately.
- Simplest way of converting from base b_1 to b_2, is to convert from b_1 to base 10 and then base 10 base b_2.

To Octal:

$0.513 \times 8 = 4.104$

$0.104 \times 8 = 0.832$

$0.832 \times 8 = 6.656$

$0.656 \times 8 = 5.248$

$0.248 \times 8 = 1.984$

$0.984 \times 8 = 7.872$

$(0.513)_{10} = (0.406517\ldots)_8$

$$(N)_{b_1} = a_{-1}b_2^{-1} + a_{-2}b_2^{-2} + \cdots + a_{-p}b_2^{-p}$$

$$b_2 \cdot (N)_{b_1} = a_{-1} + a_{-2}b_2^{-1} + \cdots + a_{-p}b_2^{-p+1}$$

Quiz Time

- Convert $(231.3)_4$ to base 7

Quiz Time

- Convert $(231.3)_4$ to base 7
- Ans:

$$(231.3)_4 = 2 \times 4^2 + 3 \times 4^1 + 1 \times 4^0 + 3 \times 4^{-1} = (45.75)_{10}$$

$$\begin{array}{l} 45 \\ 6 \quad rem = 3 \\ 0 \quad rem = 6 \end{array} \uparrow \quad \Rightarrow \quad (63)_7$$

$$\begin{array}{ll} .75 \times 7 = 5.25 & intpart = 5 \\ 0.25 \times 7 = 1.75 & intpart = 1 \\ 0.75 \times 7 = 5.25 & intpart = 5 \end{array} \quad \Rightarrow \quad (.5151\cdots)_7$$

$$\boxed{(63.5151\cdots)_7}$$

Quiz Time

- Convert $(0.625)_{10}$ to base 2

Quiz Time

- Convert $(0.625)_{10}$ to base 2

$$
\begin{aligned}
.625 \times 2 &= 1.25 \quad intpart = 1 \\
0.25 \times 2 &= 0.5 \quad intpart = 0 \\
0.5 \times 2 &= 1.00 \quad intpart = 1
\end{aligned}
\Rightarrow (.101)_2
$$

Octal and Hexadecimal Numbers

- Base 8 and base 16 are very useful in computing.
- Hex: 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F
- Binary to Hex/Octal and vice versa:
  -

$$(673.124)_8 = (\underbrace{110}_{6}\ \underbrace{111}_{7}\ \underbrace{011}_{3}\ .\ \underbrace{001}_{1}\ \underbrace{010}_{2}\ \underbrace{100}_{4})_2$$

$$(306.D)_{16} = (\underbrace{0011}_{3}\ \underbrace{0000}_{0}\ \underbrace{0110}_{6}\ .\ \underbrace{1101}_{D})_2$$

Binary Arithmetic

Bits		*Sum*	*Carry*	*Difference*	*Borrow*	*Product*
a	b	$a+b$		$a-b$		$a \cdot b$
0	0	0	0	0	0	0
0	1	1	0	1	1	0
1	0	1	0	1	0	0
1	1	0	1	0	0	1

Binary Arithmetic

Binary addition:

$$\begin{array}{rl} 1111 & = \text{carries of } 1 \\ 1111.01 & = (15.25)_{10} \\ 0111.10 & = (\ 7.50)_{10} \\ \hline 10110.11 & = (22.75)_{10} \end{array}$$

Binary subtraction:

$$\begin{array}{rl} 1 & = \text{borrows of } 1 \\ 10010.11 & = (18.75)_{10} \\ 01100.10 & = (12.50)_{10} \\ \hline 00110.01 & = (\ 6.25)_{10} \end{array}$$

Binary Arithmetic

Binary multiplication:

$$
\begin{array}{r}
11001.1 = (25.5)_{10} \\
\times \quad 110.1 = (\ 6.5)_{10} \\
\hline
110011 \\
000000 \\
110011 \\
110011 \\
\hline
10100101.11 = (165.75)_{10}
\end{array}
$$

Binary division:

$$
\begin{array}{rl}
 & 10110 = \text{quotient} \\
11001 \;\big|\; & 1000100110 \\
 & \;\;11001 \\
\hline
 & 00100101 \\
 & \;\;\;11001 \\
\hline
 & \;\;0011001 \\
 & \;\;\;\;11001 \\
\hline
 & \;\;\;\;00000 \quad = \text{remainder}
\end{array}
$$

Complements

- Simplest way to represent negative numbers.
- The most intuitive way to represent a negative number is to use an extra bit for sign — sign-magnitude representation
- But the computation with this representation is little complex, we'll see this later.
- Complements are more handy
- (b-1)'s complement of $(N)_b$: $(b^n - 1) - N$, where n is the number of digits.
- b's complement of $(N)_b$: $b^n - N$, where n is the number of digits

Complements

- Let's say we want to compute $(1234)_{10} - (110)_{10}$
- First we take the 10'complement of 110: $10^4 - 110 = 9890$ (Important!!! Number of digits has to be the same)
- Now add: $1234 + 9890 = 11124$
- $11124 > 10000$: so there will be an end carry in the addition — just discard the carry
- 11124 —> 1124, which is the answer.
- Now, what is the purpose????

Complements — in the binary world

- 2's complement of N: $2^n - N$
- 1's complement of N: $(2^n - 1) - N$
- We have a super easy way to compute these: can you guess why?

Complements in the binary world

- 2's complement of N: $2^n - N$
- 1's complement of N: $(2^n - 1) - N$
- We have a super easy way to compute these: can you guess why?
  - Observe that for any n
    - $(2^n - 1)$ is basically 1111… n times.
    - Now if you subtract N, all the bits of N are flipped.
    - Example: 1111 - 1011 = 0100
  - So, we do not need to do any subtraction really, —just flip the bits.
  - 2's complement = 1's complement + 1

Complements in the binary world

- Compute $M - N$
- Step 1: compute 2's complement of N: $2^n - N = comp(N) + 1$
- Step 2: $M + (2^n - N) = 2^n + (M - N)$
- Now if $M \geq N$: the sum will be $> 2^n$ — so there will be a carry. Just discard it and output the result.
- If $M < N$ the result is $2^n + (N - M) < 2^n$. This is basically the 2's complement of $(N - M)$. No carry will be produced. The output will be 2's complement of the result, with a negative sign in the front.

Complements in the binary world

- Let X = 1010100, Y = 1000011; compute X-Y and Y - X.

Complements in the binary world

- Let X = 1010100, Y = 1000011; compute X-Y and Y - X.

X =	1010100
2's complement of Y =	+ 0111101
Sum =	10010001
Discard end carry 2^7 =	−10000000
Answer: $X - Y$ =	0010001
Y =	1000011
2's complement of X =	+ 0101100
Sum =	1101111

There is no end carry.

Answer: $Y - X = -$(2's complement of 1101111) $= -0010001$

- Important!!! We are doing unsigned subtraction!!
- Food of thought: we can do the same with 1's complement too!!, then why 2's complement???

Complements in the binary world

- We can do the same with 1's complement too!!, then why 2's complement???
-

	Positive Integers (all systems)		Negative Integers		
$+N$		$-N$	Sign and Magnitude	2's Complement N^*	1's Complement $\bar{N}$
+0	0000	−0	1000	——	1111
+1	0001	−1	1001	1111	1110
+2	0010	−2	1010	1110	1101
+3	0011	−3	1011	1101	1100
+4	0100	−4	1100	1100	1011
+5	0101	−5	1101	1011	1010
+6	0110	−6	1110	1010	1001
+7	0111	−7	1111	1001	1000
		−8	——	1000	——

- For signed magnitude, we use 4 bits to represent [-7,7] with the 4th bit being the sign bit

Complements in the binary world

- We can do the same with 1's complement too!!, then why 2's complement???
-

	Positive Integers (all systems)		Negative Integers		
$+N$		$-N$	Sign and Magnitude	2's Complement N^*	1's Complement $\bar{N}$
+0	0000	−0	1000	——	1111
+1	0001	−1	1001	1111	1110
+2	0010	−2	1010	1110	1101
+3	0011	−3	1011	1101	1100
+4	0100	−4	1100	1100	1011
+5	0101	−5	1101	1011	1010
+6	0110	−6	1110	1010	1001
+7	0111	−7	1111	1001	1000
		−8	——	1000	——

- For 2's complement, we still use 4 bits to represent [-7,7]; but here the encoding is different.
- 1's complement has a negative 0

Complements in the binary world

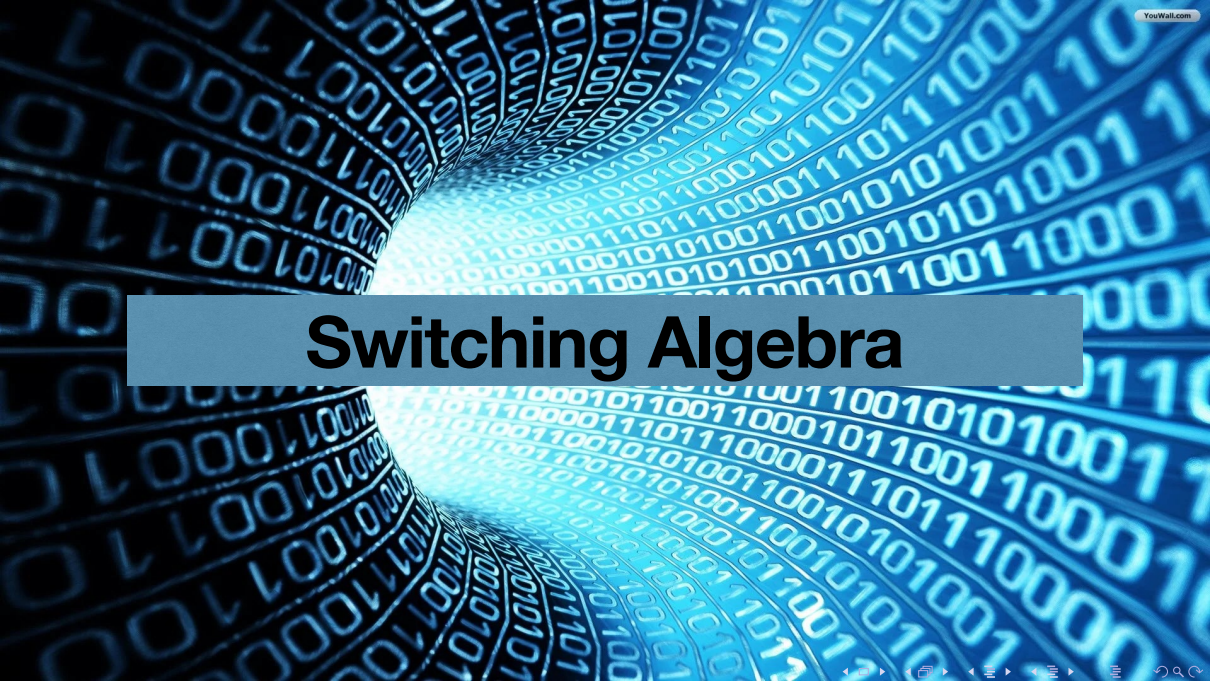
System	Range
Unsigned	$[0, 2^N - 1]$
Sign/Magnitude	$[-2^{N-1} + 1, 2^{N-1} - 1]$
Two's Complement	$[-2^{N-1}, 2^{N-1} - 1]$

Thank you

Digital Logic Design + Computer Architecture

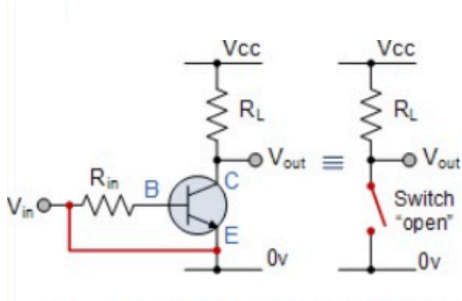
Sayandeep Saha

Assistant Professor
Department of Computer Science and Engineering
Indian Institute of Technology Bombay



Switching Algebra

Switches



Algebra of Switches

- **Warning!!! Too informal**: An algebraic system consists of a set and some well-defined operators on the elements of the set and possibly some relations defined over the elements.
- We will be studying one of the simplest system defined over the set {0, 1} — but ironically this set is going to complicate the rest of your life. :P (just joking)
- **Set**: {0,1}
- **Operators**:
  - AND, OR (Binary)
  - NOT (Unary)

$$0+0=0, \quad 0+1=1, \quad 1+0=1, \quad 1+1=1.$$

$$0\cdot 0=0, \quad 0\cdot 1=0, \quad 1\cdot 0=0, \quad 1\cdot 1=1.$$

$$0'=1, \quad 1'=0.$$

Algebra of Switches: Basic Properties

- **Idempotency**: $x + x = x$
  $x \,.\, x = x$

How do we prove this property?
Perfect induction*: proving a theorem by verifying every combination of values that the variables may assume*

Proof of $x + x = x$: $1 + 1 = 1$ and $0 + 0 = 0$

- If x is a switching variable, then: $x + 1 = 1$
  $x \,.\, 0 = 0$
  $x + 0 = x$
  $x \,.\, 1 = x$

- **Commutativity**: $x + y = y + x$
  $x \,.\, y = y \,.\, x$

Algebra of Switches: Basic Properties

- **Associativity**: $(x + y) + z = x + (y + z)$
  $(x \,.\, y) \,.\, z = x \,.\, (y \,.\, z)$
- **Complementation**: $x + x' = 1$
  $x \,.\, x' = 0$
- **Distributivity**: $x \,.\, (y + z) = x \,.\, y + x \,.\, z$
  $x + y \,.\, z = (x + y) \,.\, (x + z)$

Proofs are easy, you can write down the entire truth table

x	y	z	xy	xz	$y+z$	$x(y+z)$	$xy+xz$
0	0	0	0	0	0	0	0
0	0	1	0	0	1	0	0
0	1	0	0	0	1	0	0
0	1	1	0	0	1	0	0
1	0	0	0	0	0	0	0
1	0	1	0	1	1	1	1
1	1	0	1	0	1	1	1
1	1	1	1	1	1	1	1

Algebra of Switches: Principle of Duality

- **Observe**: Preceding properties grouped in pairs
- One statement can be obtained from the other by interchanging operations OR and AND and constants 0 and 1
- The two statements are said to be dual of each other
- **Implication**: necessary to prove only one of each pair of statements

Algebra of Switches: Some Basic Laws

- **Switching expression**: combination of finite number of switching variables and constants via switching operations (AND, OR, NOT)
  - Any constant or switching variable is a switching expression
  - If T_1 and T_2 are switching expressions, so are T_1', T_2', T_1+T_2 and T_1T_2
  - No other combination of constants and variables is a switching expression

- **Absorption law**: $x + xy = x$
  $x(x + y) = x$

Proof: $x + xy = x1 + xy$ [basic property]
$= x(1 + y)$ [distributivity]
$= x1$ [commutativity and basic property]
(Note we write in the law y + 1 = y, so we apply commutativity)
$= x$ [basic property]

Algebra of Switches: Some Basic Laws

- **Switching expression**: combination of finite number of switching variables and constants via switching operations (AND, OR, NOT)
  - Any constant or switching variable is a switching expression
  - If T_1 and T_2 are switching expressions, so are T_1', T_2', T_1+T_2 and T_1T_2
  - No other combination of constants and variables is a switching expression

- **Absorption law**: $x + xy = x$
  $x(x + y) = x$

Proof: $x + xy = x1 + xy$ [basic property]
$= x(1 + y)$ [distributivity]
$= x1$ [commutativity and basic property]
(Note we write in the law y + 1 = y, so we apply commutativity)
$= x$ [basic property]

Simplification of Switching Expressions

- **Law 2:** $x + x'y = x + y$, $x(x' + y) = xy$
- **Law 3**: $xy + x'z + yz = xy + x'z$, $(x + y)(x' + z)(y + z) = (x + y)(x' + z)$. **(Consensus Theorem)**
- **Law 4**: $(x')' = x$. **(Involution)**
- **Law 5**: $(x+y)' = x'y'$, $(xy)' = x' + y'$ **(De-Morgan's Theorem)**

Food of thought: Can we extend it for n variables???

x	y	x'	y'	$x + y$	$(x + y)'$	$x'y'$
0	0	1	1	0	1	1
0	1	1	0	1	0	0
1	0	0	1	1	0	0
1	1	0	0	1	0	0

Try the proofs!!!

Simplification of Switching Expressions

Example: Simplify $T(x,y,z) = (x + y)[x'(y' + z')]' + x'y' + x'z'$

Simplification of Switching Expressions

Example: Simplify $T(x,y,z) = (x + y)[x'(y' + z')]' + x'y' + x'z'$

$$
\begin{aligned}
(x + y)[x'(y' + z')]' + x'y' + x'z' &= (x + y)(x + yz) + x'y' + x'z' \\
&= (x + xyz + yx + yz) + x'y' + x'z' \\
&= x + yz + x'y' + x'z' \\
&= x + yz + y' + z' \\
&= x + z + y' + z' \\
&= x + y' + 1 \\
&= 1
\end{aligned}
$$

Thus, $T(x,y,z) = 1$, independently of the values of the variables

Simplification of Switching Expressions

Example: Prove $xy + x'y' + yz = xy + x'y' + x'z$

Simplification of Switching Expressions

Example: Prove $xy + x'y' + yz = xy + x'y' + x'z$

- From consensus theorem, $x'z$ can be added to LHS
- Consensus theorem can be applied again to first, third and fourth terms in $xy + x'y' + yz + x'z$ to eliminate yz and reduce it to RHS

Simplification of Switching Expressions

Example: Prove $xy + x'y' + yz = xy + x'y' + x'z$

- From consensus theorem, $x'z$ can be added to LHS
- Consensus theorem can be applied again to first, third and fourth terms in $xy + x'y' + yz + x'z$ to eliminate yz and reduce it to RHS

- LHS : $xy + x'y' + yz = xy + (x'y' + yz) = xy + (y'x' + (y')'z + x'z) = xy + y'x' + yz + x'z = (xy + x'z + yz) + x'y' = xy + x'z + x'y' = RHS$

Switching Functions

- **Switching function $f(x_1, x_2, \ldots, x_n)$**: values assumed by an expression for all combinations of variables $x_1, x_2, \ldots, x_n$
- **Complement function**: $f'(x_1, x_2, \ldots, x_n)$ assumes value 0 (1) whenever $f(x_1, x_2, \ldots, x_n)$ assumes value 1 (0)
- **Logical sum of two functions**: $f(x_1, x_2, \ldots, x_n) + g(x_1, x_2, \ldots, x_n) = 1$ for every combination in which either f or g or both equal 1
- **Logical product of two functions**: $f(x_1, x_2, \ldots, x_n) \cdot g(x_1, x_2, \ldots, x_n) = 1$ for every combination for which both f and g equal 1

x	y	z	f	g	f'	$f+g$	fg
0	0	0	1	0	0	1	0
0	0	1	0	1	1	1	0
0	1	0	1	0	0	1	0
0	1	1	1	1	0	1	1
1	0	0	0	1	1	1	0
1	0	1	0	0	1	0	0
1	1	0	1	1	0	1	1
1	1	1	1	0	0	1	0

Canonical Forms

- Truth tables are one of the most standard way of representing switching functions.
- But a functions with n variables require 2^n rows each of size $(n+1)$ bits. — quite some overhead.
- **Let's consider the problem of deriving a compact expression of a function from a given truth table**

Decimal code	x	y	z	f
0	0	0	0	1
1	0	0	1	0
2	0	1	0	1
3	0	1	1	1
4	1	0	0	0
5	1	0	1	0
6	1	1	0	1
7	1	1	1	1

$$f = x'y'z' + x'yz' + x'yz + xyz' + xyz$$

- Find the sum (OR) of all terms for which the function evaluates to 1.
  - In this case, for (0, 2, 3, 6, 7).
- Each term is a product of the variables on which the function depends
- Variable x_i appears in uncomplemented (complemented) form in the product if has value 1 (0) in the combination

Canonical Forms: Sum of Products

- **Minterm**: a product term that contains each of the *n* variables as factors in either complemented or uncomplemented form
  - It assumes value 1 for exactly one combination of variables
  - In the following table 000, 010, 011, 110, 111 are minterms, which are usually represented with their decimal equivalent (0,2,3,6,7). In terms of variables *x'y'z', x'yz', x'yz, xyz', xyz*

- **Canonical sum-of-products**: sum of all minterms derived from combinations for which function is 1
  - Also called **disjunctive normal expression**

- **Compact representation**: $\sum (0,2,3,6,7)$

Decimal code	x	y	z	f
0	0	0	0	1
1	0	0	1	0
2	0	1	0	1
3	0	1	1	1
4	1	0	0	0
5	1	0	1	0
6	1	1	0	1
7	1	1	1	1

Canonical Forms: Product of Sums

- **Maxterm**: a sum term that contains each of the *n* variables in either complemented or uncomplemented form
  - It assumes value 0 for exactly one combination of variables
  - Variable x_i appears in uncomplemented (complemented) form in the sum if it has value 0 (1) in the combination
- **Canonical product-of-sums**: product of all maxterms derived from combinations for which function is 0
  - Also called **conjunctive normal expression**

- **Compact representation**: $\prod(1,4,5)$

$$f = (x + y + z')(x' + y + z)(x' + y + z')$$

Decimal code	x	y	z	f
0	0	0	0	1
1	0	0	1	0
2	0	1	0	1
3	0	1	1	1
4	1	0	0	0
5	1	0	1	0
6	1	1	0	1
7	1	1	1	1

Derivation of SOP Forms

1. Examine each term: if it is a minterm, retain it; continue to next term

2. In each product which is not a minterm: check the variables that do not occur; for each x_i that does not occur, multiply the product by $(x_i + x_i')$

3. Multiply out all products and eliminate redundant terms

Example: $T(x,y,z) = x'y + z' + xyz$

$$= x'y(z + z') + (x + x')(y + y')z' + xyz$$
$$= x'yz + x'yz' + xyz' + xy'z' + x'yz' + x'y'z' + xyz$$
$$= x'yz + x'yz' + xyz' + xy'z' + x'y'z' + xyz$$

Canonical product-of-sums obtained in a dual manner

Example:

$T = x'(y' + z)$

$= (x' + yy' + zz')(y' + z + xx')$

$= (x' + y + z)(x' + y + z')(x' + y' + z)(x' + y' + z')(x + y' + z)(x' + y' + z)$

$= (x' + y + z)(x' + y + z')(x' + y' + z)(x' + y' + z')(x + y' + z)$

Transforming Canonical Forms

- **Example**: Find the canonical product-of-sums for

$$T(x,y,z) = x'y'z' + x'y'z + x'yz + xyz + xy'z + xy'z'$$

$$T = (T')' = [(x'y'z' + x'y'z + x'yz + xyz + xy'z + xy'z')']'$$

Complement T' consists of minterms not contained in T. Thus,

$$T = [x'yz' + xyz']' = (x + y' + z)(x' + y' + z)$$

- Canonical forms are unique
- **Two switching functions are equivalent if and only if their corresponding canonical forms are identical**

Introducing Exclusive-OR (XOR)

Exclusive-OR: modulo-2 addition, i.e., $A \oplus B = 1$ if either A or B is 1, but not both.

Commutativity: $A \oplus B = B \oplus A$

Associativity: $(A \oplus B) \oplus C = A \oplus (B \oplus C) = A \oplus B \oplus C$

Distributivity: $(AB) \oplus (AC) = A(B \oplus C)$

If $A \oplus B = C$, then

$$A \oplus C = B$$
$$B \oplus C = A$$
$$A \oplus B \oplus C = 0$$

Exclusive-OR of an even (odd) number of elements, whose value is 1, is 0 (1)

Functional Completeness

- A set of operations is **functionally complete** (or **universal**) if and only if every switching function can be expressed by operations from this set
- Every switching function can be expressed in canonical form consisting of a finite number of switching variables, constants and operations OR, AND, NOT

- **Example**: Set {+, ., ’}
- **Example**: Set {+, ’} since using De Morgan’s theorem, $x \cdot y = (x' + y')'$. Thus, + and ’ can replace the . in any switching function
- **Example**: Set {., ’} for similar reasons
- **Example**: NAND since $\text{NAND}(x,x) = x'$ and $\text{NAND}[\text{NAND}(x,y),\text{NAND}(x,y)] = xy$
- **Example**: NOR since $\text{NOR}(x,x) = x'$ and $\text{NOR}[\text{NOR}(x,y),\text{NOR}(x,y)] = x + y$

What Can be done with Switching Algebra?

Isomorphism: Two algebraic systems are isomorphic if

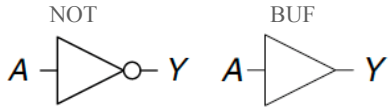
- For every operation in one system, there exists a corresponding operation in the second system
- To each element x_i in one system, there corresponds a unique element y_i in the other system, and vice versa
- If each operation and element in every postulate of one system is replaced by the corresponding operation and element in the other system, then the resulting postulate is valid in the second system

Thus, two algebraic systems are isomorphic if and only if they are identical except the labels and symbols used to represent the operations and elements.

- Switching algebra is isomorphic to proposition calculus — something which encodes declarative statements with values “True” or “False”, but never both.
  - “If it rains, then we play”

- **Most important**!!!Switching algebra is isomorphic to the **network of logic gates**, something which we build with transistors.

At The Gates



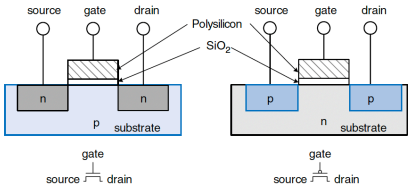
$Y = \overline{AB}$

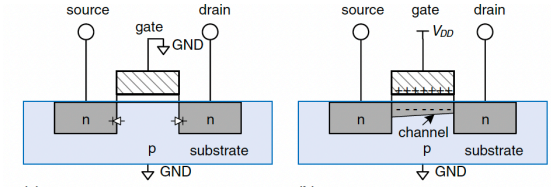
A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

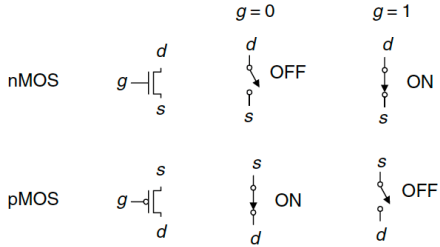
$Y = \overline{A+B}$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

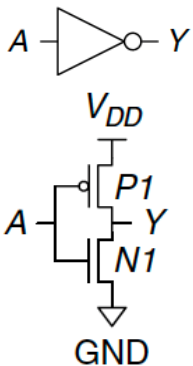
One Level Below

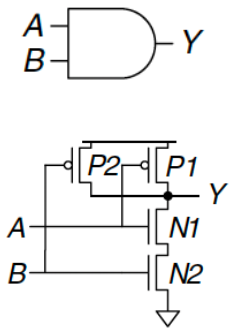


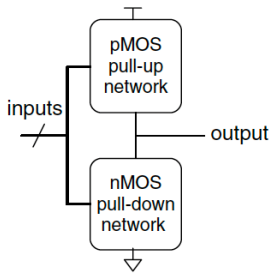




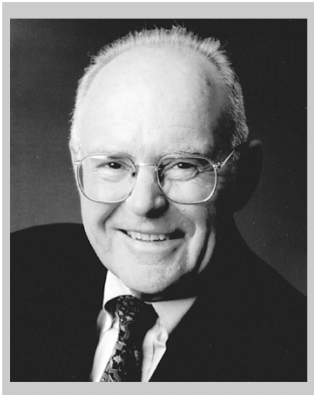
CMOS Level View







A Bit of GK

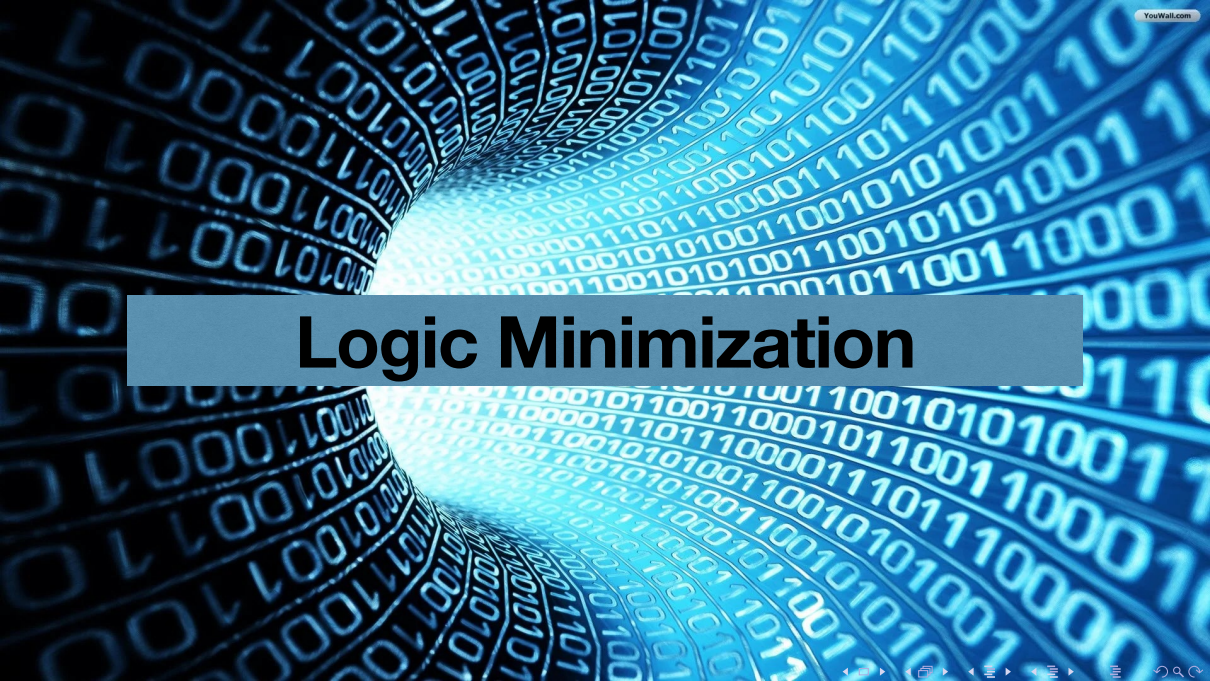


Thank you

Digital Logic Design + Computer Architecture

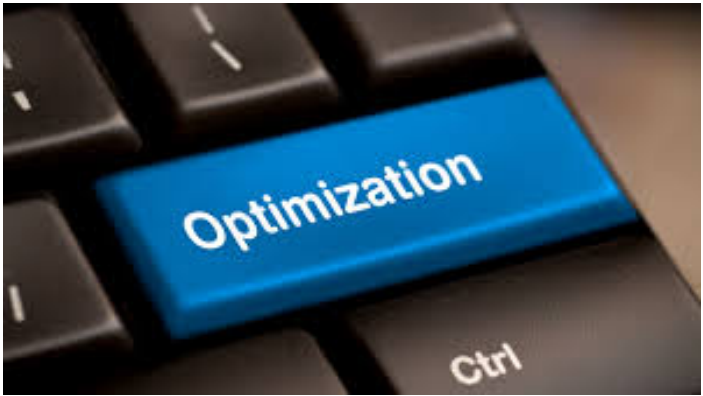
Sayandeep Saha

Assistant Professor
Department of Computer Science and Engineering
Indian Institute of Technology Bombay



Logic Minimization

Life of an Engineer



Logic Minimization: Why?

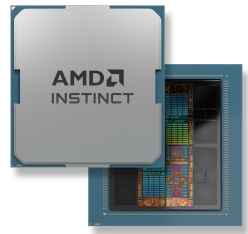
- Consider a switching expression. **How many gates do you need to implement this?** Consider each gate is 2-input, 1 output except the NOTs — 5 OR, 12 ANDs, 3 NOTs

$$f(x, y, z) = x'yz' + x'y'z' + xy'z' + x'yz + xyz + xy'z$$

- Now consider the following expression:' $x'z' + y'z' + yz + xz$
- Observe that both implements the same logic function!!! Now you need 4 ORs, 4 ANDs, and 3 NOTs.
- Can you do better?? — Yes $f(x, y, z) = x'z' + xy' + yz$
- Turns out that there can be more such expressions.
- Lower gate count => Lower transistor count => Lower area (and perhaps less power, and time)…
- So, now we have an engineering problem in hand — **how to minimize the switching expressions???**

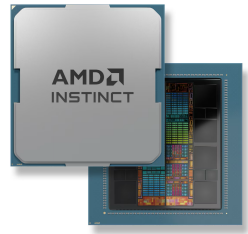
Bigger Picture

- Modern circuits contains billions of gates — e.g. AMD Instinct is a GPU processor containing 146,000,000,000 transistors; so a few billions of gates (if not trillions)…
- How do people minimized their gate network…Fortunately we have tools for that.
- Today we will be studying some of the fundamental techniques behind these tools.



Bigger Picture

- Modern circuits contains billions of gates — e.g. AMD Instinct is a GPU processor containing 146,000,000,000 transistors; so a few billions of gates (if not trillions)…
- How do people minimized their gate network…Fortunately we have tools for that.
- Today we will be studying some of the fundamental techniques behind these tools.
  - Of course, a very very rudimentary intro



The Map Method

- **Karnaugh map**: modified form of truth table
- Combine terms using the **Aa + Aa' = A (combining theorem)**

z \ xy	00	01	11	10
0	0	2	6	4
1	1	3	7	5

(*a*) Location of minterms in a three-variable map.

z \ xy	00	01	11	10
0			1	1
1			1	

(*b*) Map for function $f(x,y,z) = \sum(2,6,7) = yz' + xy$.

yz \ wx	00	01	11	10
00	0	4	12	8
01	1	5	13	9
11	3	7	15	11
10	2	6	14	10

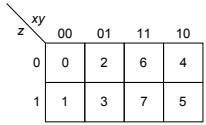
(*c*) Location of minterms in a four-variable map.

yz \ wx	00	01	11	10
00		1	1	1
01		1	1	
11			1	
10			1	

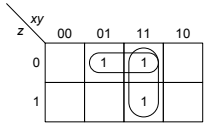
(*d*) Map for function $f(w,x,y,z) = \sum(4,5,8,12,13,14,15) = wx + xy' + wy'z'$.

The Map Method

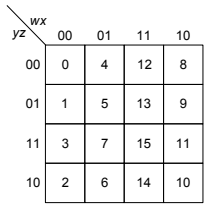
- **Karnaugh map**: modified form of truth table
- Combine terms using the **Aa + Aa' = A (combining theorem)**
- **Cube**:
  - Collection of 2^m cells, each adjacent to ***m*** cells of the collection
  - The cube is said to **cover** the cells it is involved with
  - Expressed by a product of **n-m literals** for a function containing **n variables**
  - **m literals** not in the product said to be eliminated



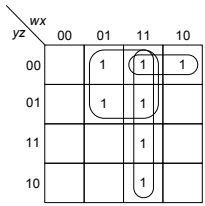
(*a*) Location of minterms in a three-variable map.



(*b*) Map for function $f(x,y,z) = \sum(2,6,7) = yz' + xy$.



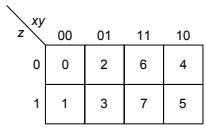
(*c*) Location of minterms in a four-variable map.



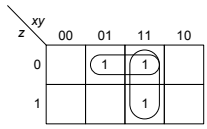
(*d*) Map for function $f(w,x,y,z) = \sum(4,5,8,12,13,14,15) = wx + xy' + wy'z'$.

The Map Method

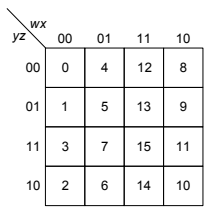
- **Karnaugh map**: modified form of truth table
- Combine terms using the **Aa + Aa' = A (combining theorem)**
- **Cube**:
  - Collection of **2^m** cells, each adjacent to ***m*** cells of the collection
  - The cube is said to **cover** the cells it is involved with
  - Expressed by a product of **n-m literals** for a function containing **n variables**
  - **m literals** not in the product said to be eliminated
  - **More Clarification**:
    - Consider the squares 2 and 6 in Fig (a)
    - The minterms are z'x'y and z'xy
    - Now apply the **combining theorem**.
    - Literal x and x' are eliminated.
    - The result is a 2-cube.



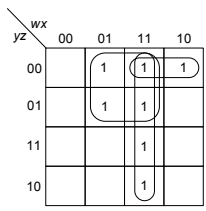
(*a*) Location of minterms in a three-variable map.



(*b*) Map for function $f(x,y,z) = \sum(2,6,7) = yz' + xy$.

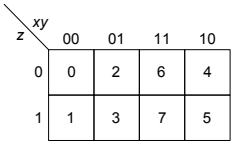


(*c*) Location of minterms in a four-variable map.

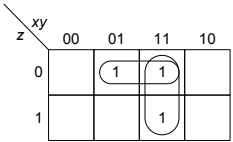


(*d*) Map for function $f(w,x,y,z) = \sum(4,5,8,12,13,14,15) = wx + xy' + wy'z'$.

The Map Method



(*a*) Location of minterms in a three-variable map.



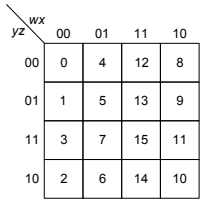
(*b*) Map for function $f(x,y,z) = \sum(2,6,7) = yz' + xy$.

- **Example**: $f = yz' + xy$
  - **Use of cell 6 in forming both cubes justified by idempotent law**
  - Corresponding algebraic manipulations:

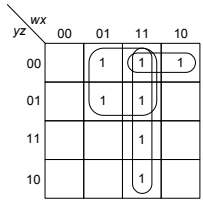
$$
\begin{aligned}
f &= x'yz' + xyz' + xyz \\
&= x'yz' + \underline{xyz' + xyz'} + xyz \text{ (idempotent law)} \\
&= yz'(x' + x) + xy(z' + z) \\
&= yz' + xy
\end{aligned}
$$

The Map Method

- **Example**: w'xy'z' + w'xy'z + wxy'z' + wxy'z = xy'(w'z' + w'z + wz' + wz) = xy'
- **Trick**:
  - In a cube, just keep the variables not changing their value.



(c) Location of minterms in a four-variable map.

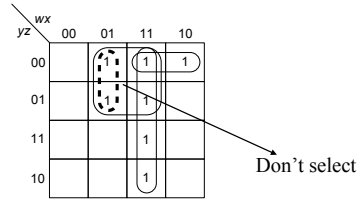


(*d*) Map for function $f(w,x,y,z) = \sum(4,5,8,12,13,14,15) = wx + xy' + wy'z'$.

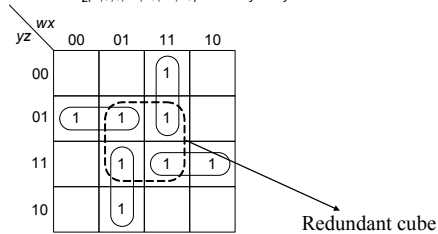
The Map Method

Rules for minimization:

- **Step 1**: cover those 1 cells by cubes that cannot be combined with other 1 cells; continue to 1 cells that have a single adjacent 1 cell (thus can form cubes of only two cells)
- **Step 2—:** Combine 1 cells that yield cubes of four cells, but are not part of any cube of eight cells, and so on..
  - A cube contained in a larger cube must never be selected
  - A cube contained in any combination of other cubes already selected in the cover is redundant **(consensus theorem)**
  - If there are more than one way of covering the map with cubes, select the cover with larger cubes
  - **Minimal expression**: collection of cubes that are as large and as few in number as possible, such that each 1 cell is covered by at least one cube
  - **Irredundant expressions**:
    - An SOP from where no term or literal can be deleted.
    - Not necessarily minimal
  - **Minimal and irredundant expressions may not be unique**
  - **But a minimal expression is always irredundant.**

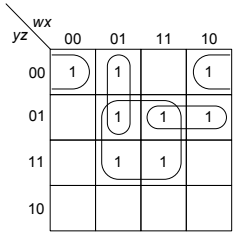


(*d*) Map for function $f(w,x,y,z)$
$=\sum(4,5,8,12,13,14,15) = wx + xy' + wy'z'$.

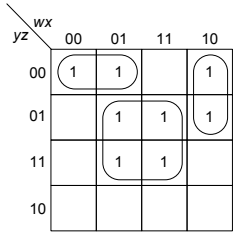


Let's try this..

The Map Method



(*a*) $f = x'y'z' + w'xy' + wy'z + xz$ is an irredundant expression.



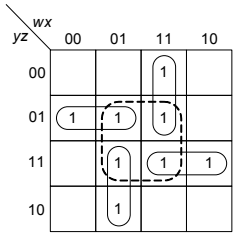
(*b*) $f = w'y'z' + wx'y' + xz$ is the unique minimal expression.

Example: Two irredundant expressions for $f(w,x,y,z) = \sum(0,4,5,7,8,9,13,15)$

The Map Method

Example: $f(w,x,y,z) = \sum (1,5,6,7,11,12,13,15)$

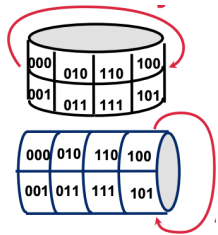
- Only one irredundant form: f = wxy' + wyz + w'xy + w'y'z



The Map Method: Earth is not Flat

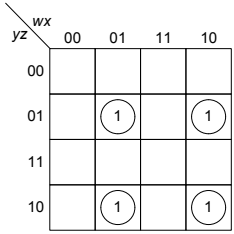
A \ BC	00	01	11	10
0	000	001	011	010
1	100	101	111	110

A \ BC	00	01	11	10
0	1			1
1				

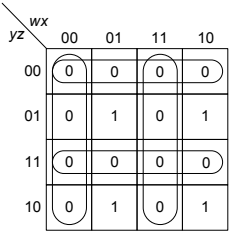


Minimal Product-of-Sums

- **Dual procedure**: product of a minimum number of sum factors, provided there is no other such product with the same number of factors and fewer literals
  - **Variable corresponding to a 1 (0) is complemented (uncomplemented)**
  - Cubes are formed of 0 cells
- **Example**: either one of minimal sum-of-products or minimal product-of-sums can be better than the other in literal count



(*a*) Map of $f(x,y,z) = \sum 5,6,9,10)$
$= w'xy'z + wx'y'z + w'xyz' + wx'yz'$.



(*b*) Map of $f(x,y,z)$
$=\prod(0,1,2,3,4,7,8,11,12,13,14,15)$
$= (y + z)(y' + z')(w + x)(w' + x')$.

Let's Try it..

- Implement $f(A, B, C, D) = \sum (0,2,8,12,13)$ with minimum number of gates.

Let's Try it..

- Implement **complement of** $f(A, B, C, D) = \prod(7,9,13)$.

Don’t-care Combinations

- **Don’t-care combination ϕ:** combination for which the value of the function is not specified.
  - Either input combinations may be invalid
  - Or precise output value is of no importance

- Since each don’t-care can be specified as either 0 or 1
  - a function with k don’t-cares corresponds to a class of 2^k distinct functions.
  - Our aim is to choose the function with the minimal representation
- Assign 1 to some don’t-cares and 0 to others in order to increase the size of the selected cubes whenever possible
- No cube containing only don’t-care cells may be formed

Code Converter

Example: code converter from BCD to excess-3 code
Combinations 10 through 15 are don't-cares

Truth table

Decimal	*BCD inputs*				*Excess-3 outputs*			
number	w	x	y	z	f_4	f_3	f_2	f_1
0	0	0	0	0	0	0	1	1
1	0	0	0	1	0	1	0	0
2	0	0	1	0	0	1	0	1
3	0	0	1	1	0	1	1	0
4	0	1	0	0	0	1	1	1
5	0	1	0	1	1	0	0	0
6	0	1	1	0	1	0	0	1
7	0	1	1	1	1	0	1	0
8	1	0	0	0	1	0	1	1
9	1	0	0	1	1	1	0	0

Code Converter

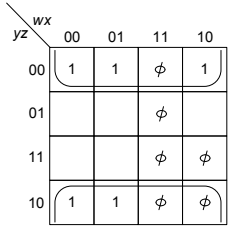
Example: code converter from BCD to excess-3 code
Combinations 10 through 15 are don't-cares

Truth table

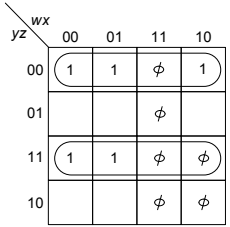
Decimal	*BCD inputs*				*Excess-3 outputs*			
number	w	x	y	z	f_4	f_3	f_2	f_1
0	0	0	0	0	0	0	1	1
1	0	0	0	1	0	1	0	0
2	0	0	1	0	0	1	0	1
3	0	0	1	1	0	1	1	0
4	0	1	0	0	0	1	1	1
5	0	1	0	1	1	0	0	0
6	0	1	1	0	1	0	0	1
7	0	1	1	1	1	0	1	0
8	1	0	0	0	1	0	1	1
9	1	0	0	1	1	1	0	0

$$f_1 = \sum(0,2,4,6,8) + \sum_{\phi}(10,11,12,13,14,15)$$
$$f_2 = \sum(0,3,4,7,8) + \sum_{\phi}(10,11,12,13,14,15)$$
$$f_3 = \sum(1,2,3,4,9) + \sum_{\phi}(10,11,12,13,14,15)$$
$$f_4 = \sum(5,6,7,8,9) + \sum_{\phi}(10,11,12,13,14,15)$$

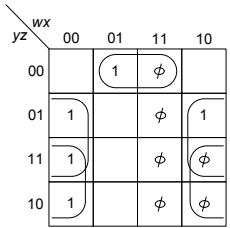
Code Converter



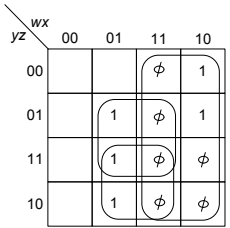
f_1 map



f_2 map



f_3 map

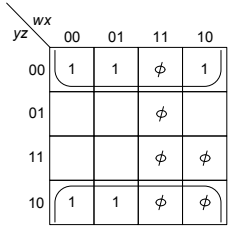


f_4 map

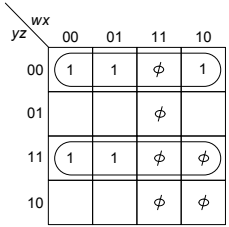
Increase the size of the cubes without making it necessary to increase the number of cubes, than would be required with fewer don't cares assigned one.

Lets do it!!!

Code Converter

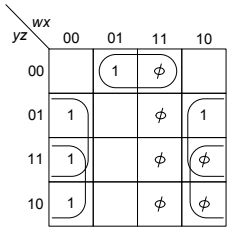


f_1 map

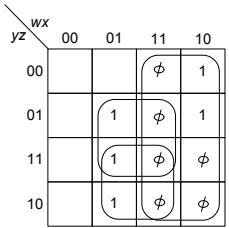


f_2 map

Increase the size of the cubes without making it necessary to increase the number of cubes, than would be required with fewer don't cares assigned one.



f_3 map

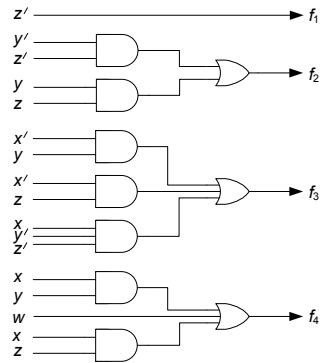


f_4 map

$$f_1 = z'.$$
$$f_2 = y'z' + yz$$
$$f_3 = x'y + x'z + xy'z'$$
$$f_4 = w + xy + xz$$

Logic Network for Code Converter

Two-level AND-OR realization:

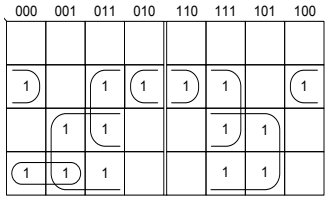


Five-variable Map

General five-variable map

yz \ vwx	000	001	011	010	110	111	101	100
00	0	4	12	8	24	28	20	16
01	1	5	13	9	25	29	21	17
11	3	7	15	11	27	31	23	19
10	2	6	14	10	26	30	22	18

Example: Minimize $f(v,w,x,y,z) = \sum(1,2,6,7,9,13,14,15,17,22,23,25,29,30,31)$



$f(v,w,x,y,z) = x'y'z + wxz + xy + v'w'yz'$

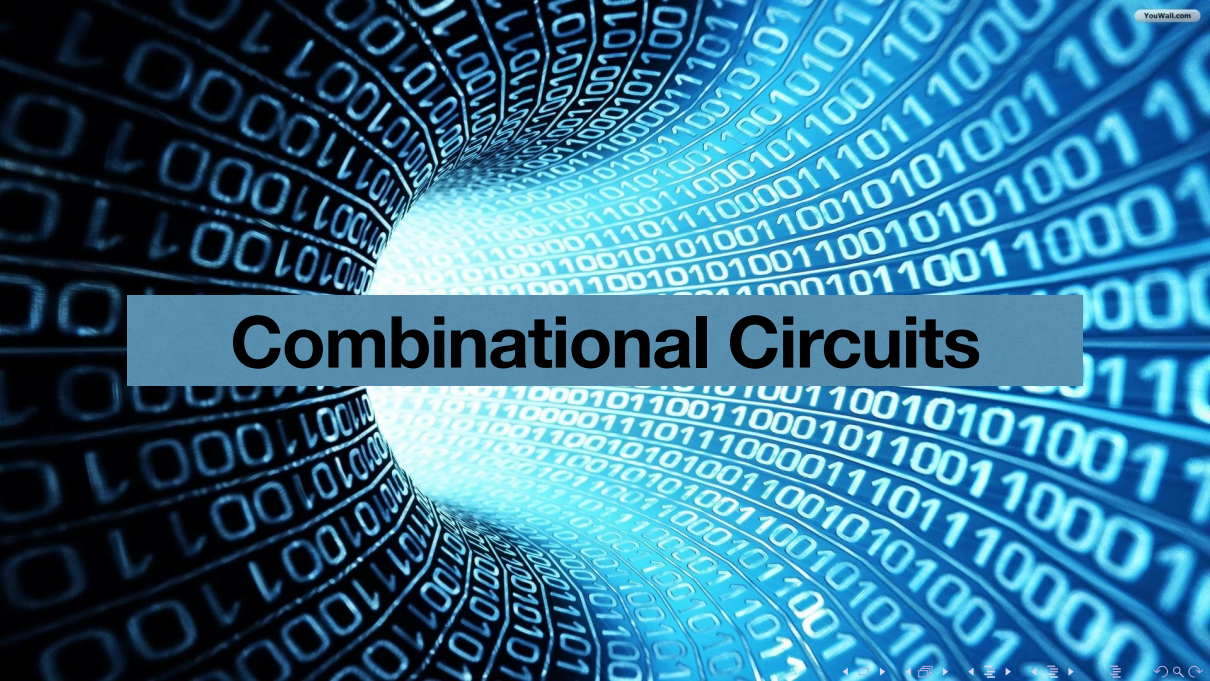
Limitation of Simple Maps

- Maps are useful up to 5-6 variables, after that the calculation becomes formidable..
- How many cells are there in a 6 variable map??
- We also need something which is more amenable to a computer program.

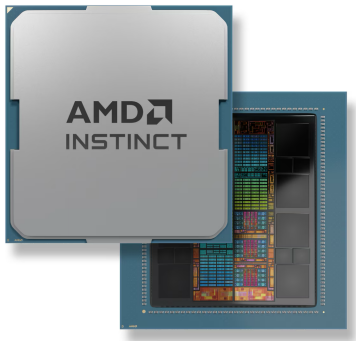
Digital Logic Design + Computer Architecture

Sayandeep Saha

Assistant Professor
Department of Computer Science and Engineering
Indian Institute of Technology Bombay


YouWall.com
Combinational Circuits

Do You Want to Design Some Day?



Design with Gates

- **Logic gates**: perform logical operations on input signals
- **Positive (negative) logic polarity**: constant 1 (0) denotes a high voltage and constant 0 a low (high) voltage
- **Combinational circuits:** No memorization
- **Synchronous sequential circuits**: have memory; driven by a clock that produces a train of equally spaced pulses
- **Propagation delay**: time to propagate a signal through a gate
- **Asynchronous circuits**: are almost free-running and do not depend on a clock; controlled by initiation and completion signals

Exclusive-OR (XOR) and XNOR

Exclusive-OR: modulo-2 addition, i.e., $A \oplus B = 1$ if either A or B is 1, but not both.

Commutativity: $A \oplus B = B \oplus A$

Associativity: $(A \oplus B) \oplus C = A \oplus (B \oplus C) = A \oplus B \oplus C$

Distributivity: $(AB) \oplus (AC) = A(B \oplus C)$

If $A \oplus B = C$, then

$$A \oplus C = B$$
$$B \oplus C = A$$
$$A \oplus B \oplus C = 0$$

Exclusive-OR of an even (odd) number of elements, whose value is 1, is 0 (1)

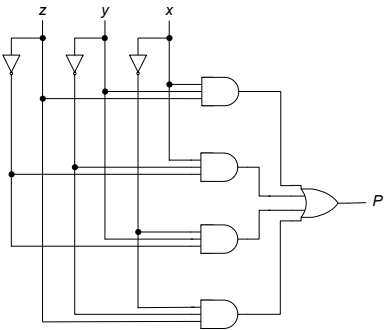
XNOR: Complement of XOR

Combinational Circuits: Parity-bit Generator

Parity-bit generator: produces output value 1 if and only if an odd number of its inputs have value 1

z \ xy	00	01	11	10
0	0	1	0	1
1	1	0	1	0

(*a*) Map.



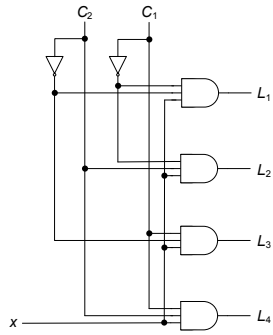
(*b*) Implementation.

$$P = x'y'z + x'yz' + xy'z' + xyz = x \oplus y \oplus z$$

Combinational Circuits: Serial to Parallel

Serial-to-parallel converter: distributes a sequence of binary digits on a serial input to a set of different outputs, as specified by external control signals

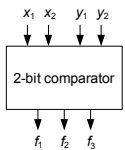
Control		*Output lines*				*Logic equations*
C_1	C_2	L_1	L_2	L_3	L_4	
0	0	x	0	0	0	$L_1 = xC_1'C_2'$
0	1	0	x	0	0	$L_2 = xC_1'C_2$
1	0	0	0	x	0	$L_3 = xC_1C_2'$
1	1	0	0	0	x	$L_4 = xC_1C_2$



Combinational Circuits: Comparators

***n*-bit comparator**: compares the magnitude of two numbers X and Y, and has three outputs f_1, f_2, and f_3

- $f_1 = 1$ iff $X > Y$
- $f_2 = 1$ iff $X = Y$
- $f_3 = 1$ iff $X < Y$



(*a*) Block diagram.

$f_1 = ?$

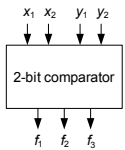
$f_2 = ?$

$f_3 = ?$

Combinational Circuits: Comparators

***n*-bit comparator**: compares the magnitude of two numbers X and Y, and has three outputs f_1, f_2, and f_3

- $f_1 = 1$ iff $X > Y$
- $f_2 = 1$ iff $X = Y$
- $f_3 = 1$ iff $X < Y$



(*a*) Block diagram.

y_1y_2 \ x_1x_2	00	01	11	10
00	2	1	1	1
01	3	2	1	1
11	3	3	2	3
10	3	3	1	2

(*b*) Map for f_1, f_2, and f_3.

$f_1 = ?$

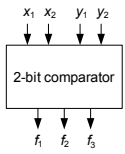
$f_2 = ?$

$f_3 = ?$

Combinational Circuits: Comparators

***n*-bit comparator**: compares the magnitude of two numbers X and Y, and has three outputs f_1, f_2, and f_3

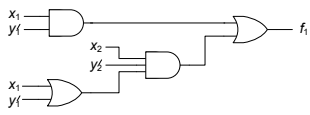
- $f_1 = 1$ iff $X > Y$
- $f_2 = 1$ iff $X = Y$
- $f_3 = 1$ iff $X < Y$



(*a*) Block diagram.

y_1y_2 \ x_1x_2	00	01	11	10
00	2	1	1	1
01	3	2	1	1
11	3	3	2	3
10	3	3	1	2

(*b*) Map for f_1, f_2, and f_3.



(*c*) Circuit for f_1.

$$f_1 = x_1x_2y_2' + x_2y_1'y_2' + x_1y_1'$$
$$= (x_1 + y_1')x_2y_2' + x_1y_1'$$

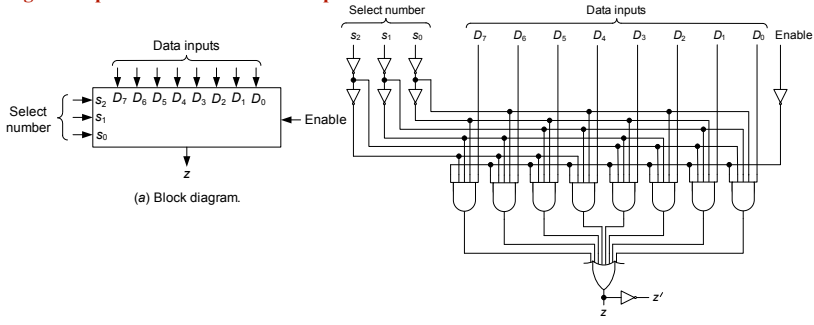
$$f_2 = x_1'x_2'y_1'y_2' + x_1'x_2y_1'y_2 + x_1x_2'y_1y_2' + x_1x_2y_1y_2$$
$$= x_1'y_1'(x_2'y_2' + x_2y_2) + x_1y_1(x_2'y_2' + x_2y_2)$$
$$= (x_1'y_1' + x_1y_1)(x_2'y_2' + x_2y_2)$$

$$f_3 = x_2'y_1y_2 + x_1'x_2'y_2 + x_1'y_1$$
$$= x_2'y_2(y_1 + x_1') + x_1'y_1$$

Combinational Circuits: Multiplexers

Multiplexer: electronic switch that connects one of *n* inputs to the output
Data selector: application of multiplexer

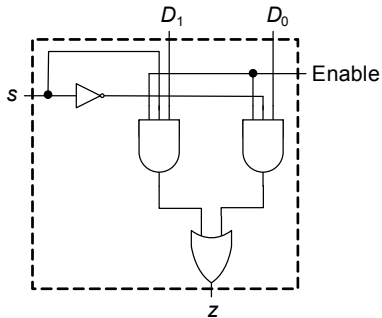
- *n* data input lines, $D_0, D_1, \ldots, D_{n-1}$
- *m* select digit inputs $s_0, s_1, \ldots, s_{m-1}$
- 1 output
- **Can you design a simple data selectors with 2 input data lines?**



Combinational Circuits: Multiplexers

Data selectors: can implement arbitrary switching functions

Example: implementing two-variable functions



$z = sD_1 + s'D_0$

If $s = A$, $B = D_0$, and $B' = D_1$, then $z = A \oplus B$.

Implementing Switching Function with Mux

To implement an *n*-variable function: a data selector with n-1 select inputs and 2^{n-1} data inputs

Implementing three-variable functions:

$$z = s_2's_1'D_0 + s_2's_1D_1 + s_2s_1'D_2 + s_2s_1D_3$$

Example: $s_1 = A$, $s_2 = B$, $D_0 = C$, $D_1 = 1$, $D_2 = 0$, $D_3 = C'$

$$z = A'B'C + AB' + ABC'$$
$$= AC' + B'C$$

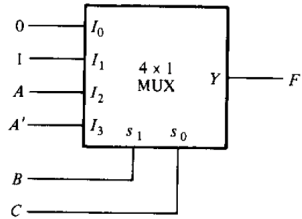
General case: Assign n-1 variables to the select inputs and last variable and constants 0 and 1 to the data inputs such that desired function results

Implementing Switching Function with Mux

$$F(A, B, C) = \sum (1,3,5,6)$$

Minterm	A	B	C	F
0	0	0	0	0
1	0	0	1	1
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	1
7	1	1	1	0

	I_0	I_1	I_2	I_3
A'	0	(1)	2	(3)
A	4	(5)	(6)	7
	0	1	A	A'



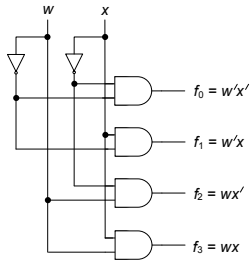
Decoders

Decoders with *n* inputs and 2^n outputs: for any input combination, only one output is 1

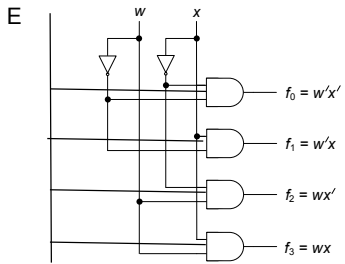
Useful for:

- Routing input data to a specified output line, e.g., in addressing memory
- Basic building blocks for implementing arbitrary switching functions
- Code conversion
- Data distribution

Example: 2-to-4- decoder

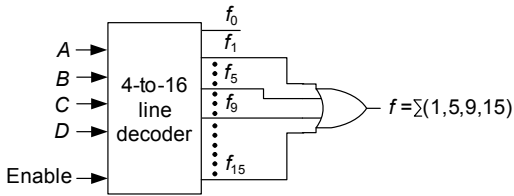


Decoders with Enable



Realizing Arbitrary Functions

Idea: Realize a distinct minterm at each output



Adders: Half Adder

Add two variables and generate the sum and carry

x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$$S = x \oplus y$$

$$C = xy$$

Adders: Full Adder

Add two variables and an input carry..

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Try it yourself…!!!

Adders: Full Adder

Add two variables and an input carry..

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$$S = x \oplus y \oplus z$$

$$C = xy + yz + zx$$

Adders: Full Adder with Half Adders

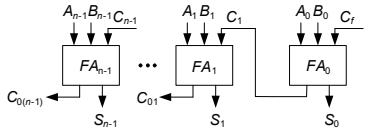
Use two half adder and something else to generate a full adder

Try it yourself…!!!

Ripple Carry Adder

Ripple-carry adder: Stages of full adders

- C_f: forced carry
- $C_{0(n-1)}$: overflow carry



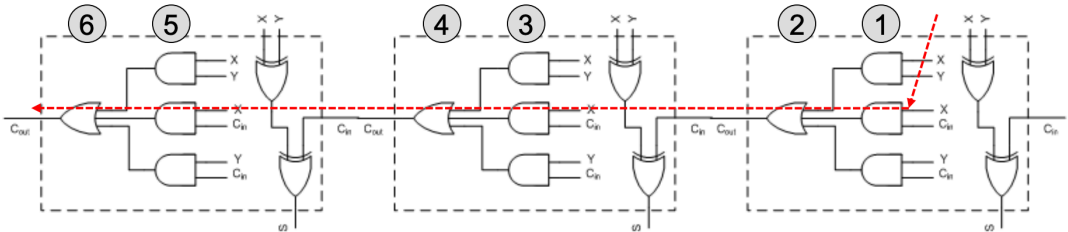
$$S_i = A_i \oplus B_i \oplus C_i$$

$$C_{0i} = A_i B_i + B_i C_{0i} + C_{0i} A_i$$

Time required:

- **Carry propagation takes longest time — in the worst case, the carry propagates through all the stages**
- Time per full adder: 2 units (assuming each gate takes one unit of time)
  - Time for carry generation
  - Assumption: two level circuit realisation with 2 input gates
- Time for ripple-carry adder: **2*n* units**

Ripple Carry Adder



https://ee.usc.edu/~redekopp/ee457/slides/EE457Unit2b_FastAdders.pdf

Carry Lookahead Adder

Carry-lookahead adder: several stages simultaneously examined and their carries generated in parallel

- Generate signal $D_i = A_iB_i$
- Propagate signal $T_i = A_i \oplus B_i$
- Thus, $C_{0i} = D_i + T_iC_i$

To generate carries in parallel: convert recursive form to nonrecursive

$$
\begin{aligned}
C_{0i} &= D_i + T_iC_i \\
C_i &= C_{0(i-1)} \\
C_{0i} &= D_i + T_i(D_{i-1} + T_{i-1}C_{i-1}) \\
&= D_i + T_iD_{i-1} + T_iT_{i-1}(D_{i-2} + T_{i-2}C_{i-2}) \\
&= D_i + T_iD_{i-1} + T_iT_{i-1}D_{i-2} + T_iT_{i-1}T_{i-2}C_{i-2} \\
\ldots \quad & \ldots\ldots \\
C_{0i} &= D_i + T_iD_{i-1} + T_iT_{i-1}D_{i-2} + \ldots + T_iT_{i-1}T_{i-2}\ldots T_0C_f
\end{aligned}
$$

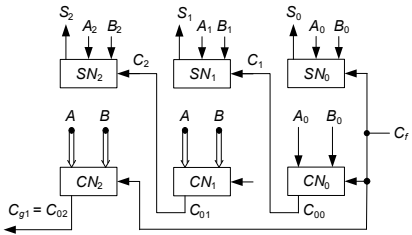
Thus, $C_{0i} = 1$ if it has been generated in the i^{th} stage or originated in a preceding stage and propagated to all subsequent stages

Carry Lookahead Adder

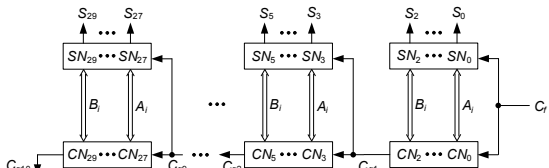
Implementation of lookahead for the complete adder impractical:

- Divide the *n* stages into groups
- Full carry lookahead within group
- Ripple carry between groups

Example: Three-digit adder group with full carry lookahead



(*a*) Block diagram of initial three-stage group

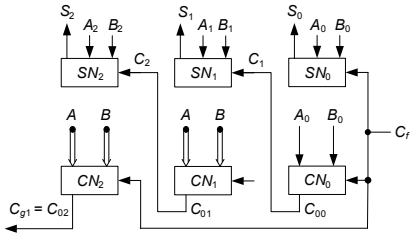


Carry Lookahead Adder

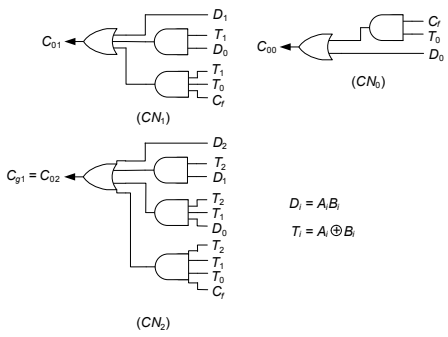
Implementation of lookahead for the complete adder impractical:

- Divide the *n* stages into groups of 3-bit adders
- Full carry lookahead within group
- Ripple carry between groups

Example: Three-digit adder group with full carry lookahead



(*a*) Block diagram of initial three-stage group



(*b*) The carry networks

Time taken:

- 4 time units for C_{g1} why 4?
- Only 2 time units for C_{g2} and other group carries why?

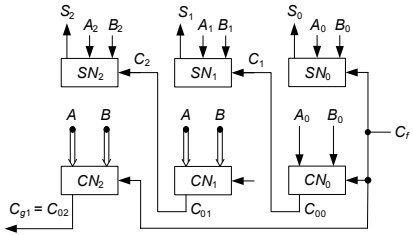
Carry Lookahead Adder

Implementation of lookahead for the complete adder impractical:

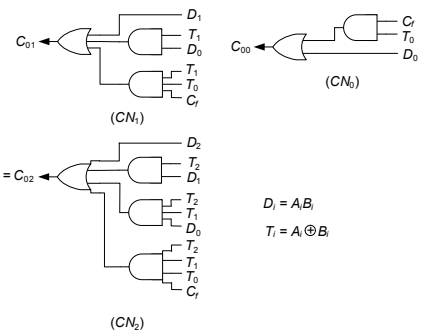
- Divide the *n* stages into groups of 3-bit adders
- Full carry lookahead within group
- Ripple carry between groups

Example: Three-digit adder group with full carry lookahead

- XOR - 2 unit delay
- AND - 1 unit delay
- OR - 1 unit delay
- No restriction on the number of inputs



(*a*) Block diagram of initial three-stage group



(*b*) The carry networks

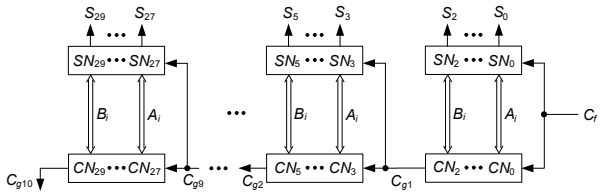
Time taken:

- 4 time units for C_{g1} why 4?
- Only 2 time units for C_{g2} and other group carries why?

Carry Lookahead Adder

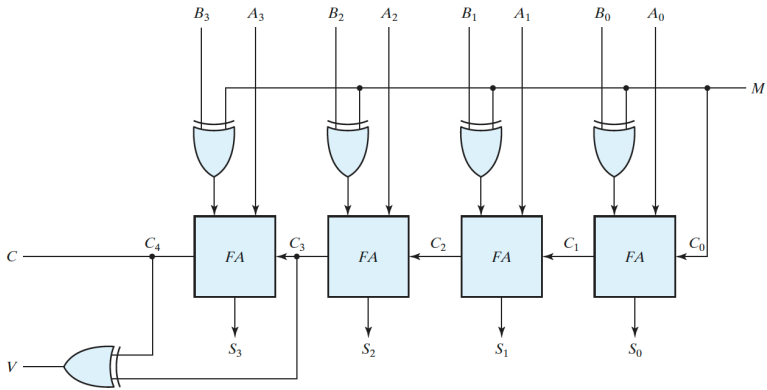
Example: divide n stages into groups of **three bits** for a **30 bit adder**

- **Very very important!!!**
  - First stage requires more time (2 units extra to generate T_i).
  - Last stage requires more time (2 extra units for the final sum, for the other units it's not coming in the critical path)
- **Time taken**: $4 + 2n/3$ time units
- 50% additional hardware for a threefold speedup



Adder Subtractor

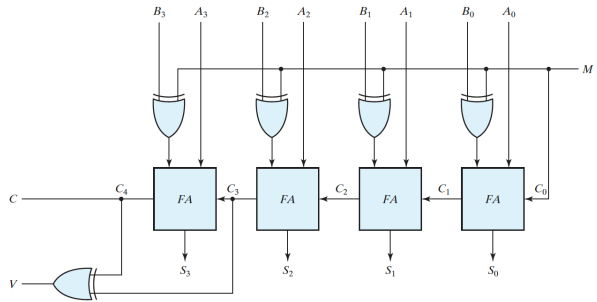
- Implements two's complement subtraction
- The idea is to have both an adder and subtractor in the same circuit
- **Overflow detection**



Adder Subtractor: The Overflow

- **Overflow**: When two numbers with n digits each are added and the sum is a number occupying n + 1 digits, we say that an overflow occurred.
- This is true for binary or decimal numbers, signed or unsigned
- An overflow condition can be detected by observing the carry into the sign bit position and the carry out of the sign bit position. If these two carries are not equal, an overflow has occurred.
- If the numbers are unsigned
  - C bit detects the overflow
- If the numbers are signed
  - V bit detections the overflow

carries:	0	1	carries:	1	0
+70	0	1000110	−70	1	0111010
+80	0	1010000	−80	1	0110000
+150	1	0010110	−150	0	1101010

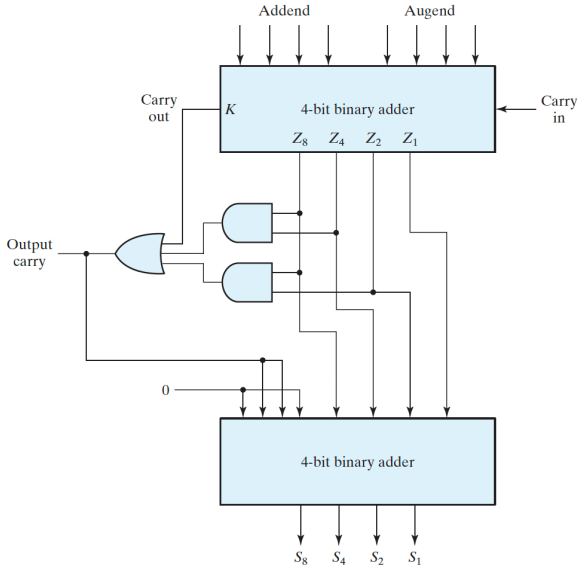


BCD Adder

- **Adds two BCD Numbers**
- First add in binary and then convert the sum to BCD
- Values beyond 9 requires “correction”
  - For values beyond 9, 0110 needs to added to generate the BCD encoded sum
- We use a bit C to detect when correction is needed
- C = 1, when
  - Carry K or the binary sum is 1
  - Z8 = 1 and Z4 = 1
  - Z8 = 1 and Z2 = 1
- **C = K + Z8Z4 + Z8Z2**

Binary Sum					BCD Sum					Decimal
K	Z_8	Z_4	Z_2	Z_1	*C*	S_8	S_4	S_2	S_1	
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	2
0	0	0	1	1	0	0	0	1	1	3
0	0	1	0	0	0	0	1	0	0	4
0	0	1	0	1	0	0	1	0	1	5
0	0	1	1	0	0	0	1	1	0	6
0	0	1	1	1	0	0	1	1	1	7
0	1	0	0	0	0	1	0	0	0	8
0	1	0	0	1	0	1	0	0	1	9
0	1	0	1	0	1	0	0	0	0	10
0	1	0	1	1	1	0	0	0	1	11
0	1	1	0	0	1	0	0	1	0	12
0	1	1	0	1	1	0	0	1	1	13
0	1	1	1	0	1	0	1	0	0	14
0	1	1	1	1	1	0	1	0	1	15
1	0	0	0	0	1	0	1	1	0	16
1	0	0	0	1	1	0	1	1	1	17
1	0	0	1	0	1	1	0	0	0	18
1	0	0	1	1	1	1	0	0	1	19

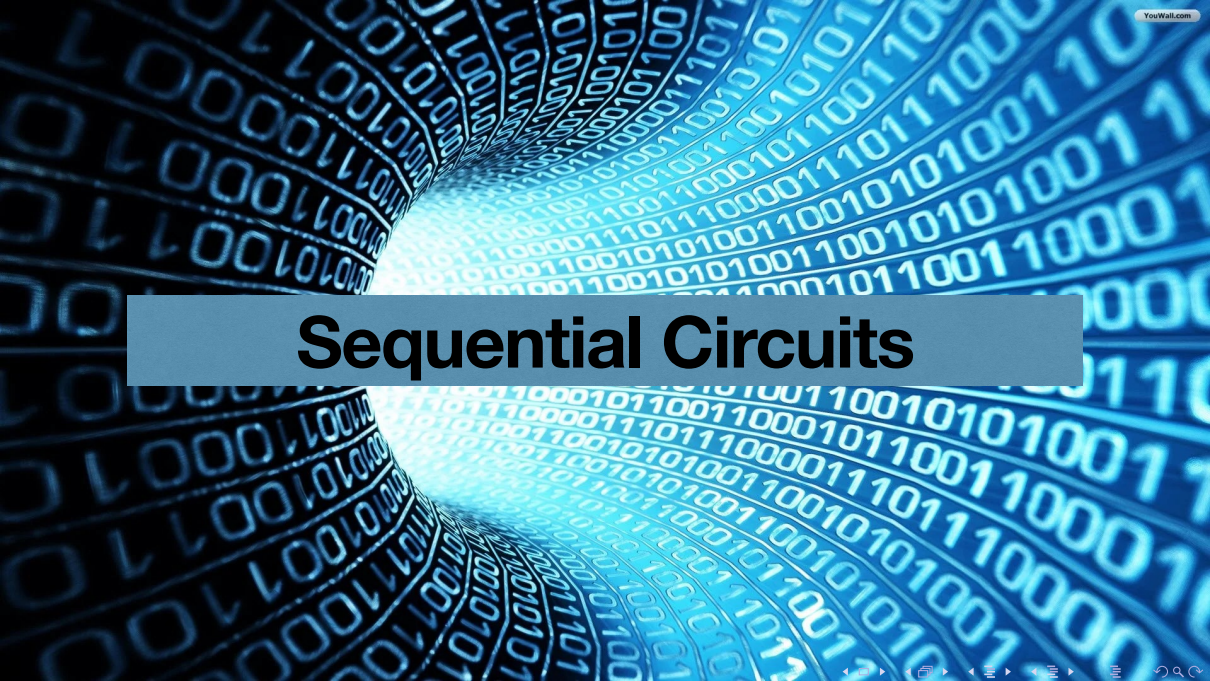
BCD Adder


Addend
Augend
Carry out
K
4-bit binary adder
Carry in
Z_8 Z_4 Z_2 Z_1
Output carry
0
4-bit binary adder
S_8 S_4 S_2 S_1

Digital Logic Design + Computer Architecture

Sayandeep Saha

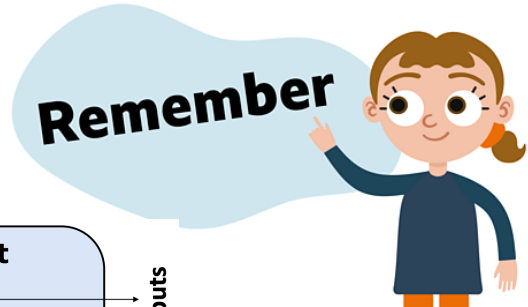
Assistant Professor
Department of Computer Science and Engineering
Indian Institute of Technology Bombay

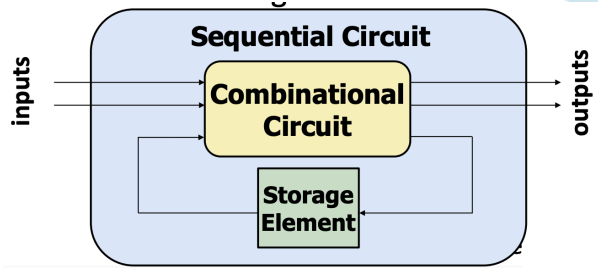


Sequential Circuits

A Circuit that Remembers

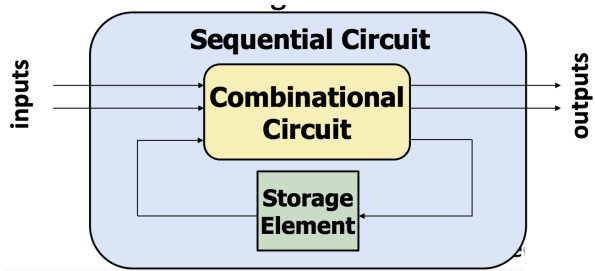
- **How do you remember things?**
  - Memory
- **Can we design a circuit which remembers?**
  - A formal way to model this capability is called a **state**
  - **So we will be modelling circuits to create a state.**



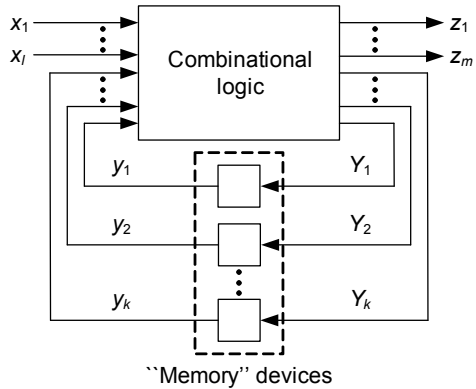


A Circuit that Remembers

- **Every digital logic you see in real life is sequential**
  - Your processors — that you going to see in the rest of the course
  - Your washing machine — it remembers your setting and washes accordingly
  - Your elevator — it remembers which floors to stop
  - Your ATM machine — it remembers your choice and updates your account after despatching money



Sequential Circuits



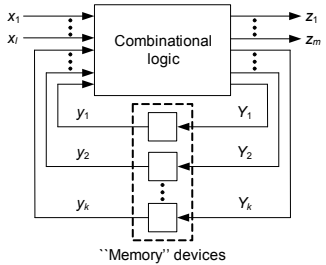
Sequential Circuits

To generate the Y's: memory devices must be supplied with appropriate input values

- **Characteristic table/functions**: switching functions that describe the impact of x_i's and y_j's on the memory-element input
- **Excitation table**: its entries are the values of the memory-element inputs

Most widely used memory elements: **flip-flops**, which are made of **latches**

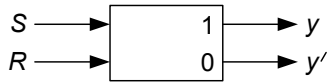
- **Latch**: remains in one state indefinitely until an input signals directs it to do otherwise



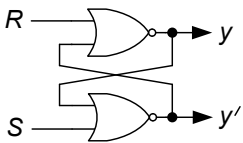
Memory Element: Latches

Latch: remains in one state indefinitely until an input signals directs it to do otherwise

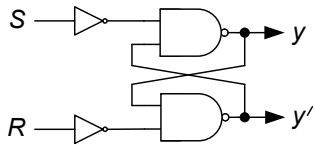
Set-reset of *SR* latch:



(*a*) Block diagram.



(*b*) NOR latch.



(*c*) NAND latch.

Memory Element: Latches

Characteristic table and excitation requirements:

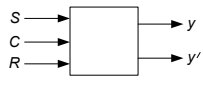
$y(t)$	$S(t)$	$R(t)$	$y(t+1)$
0	0	0	0
0	0	1	0
0	1	1	?
0	1	0	1
1	1	0	1
1	1	1	?
1	0	1	0
1	0	0	1

$$RS = 0$$
$$y(t+1) = R'y(t) + S$$

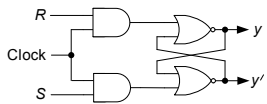
Circuit change		*Required value*	
From:	*To:*	S	R
0	0	0	–
0	1	1	0
1	0	0	1
1	1	–	0

Clocked *SR* latch: all state changes synchronized to clock pulses

- Restrictions placed on the length and frequency of clock pulses: so that the circuit changes state no more than once for each clock pulse



(*a*) Block diagram.



(*b*) Logic diagram.

Memory Element: Latches

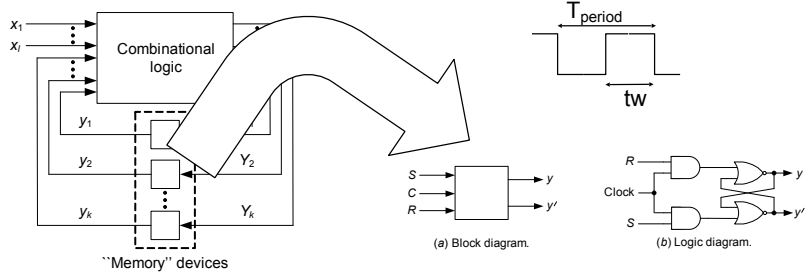
Why is the (1,1) input forbidden?

$y(t)$	$S(t)$	$R(t)$	$y(t+1)$
0	0	0	0
0	0	1	0
0	1	1	?
0	1	0	1
1	1	0	1
1	1	1	?
1	0	1	0
1	0	0	1

$$RS = 0$$
$$y(t+1) = R'y(t) + S$$

1. If **R=S=1, Q** and **Q'** will both settle to 1, which **breaks** our invariant that **Q** = !**Q'**
2. If **S** and **R** transition back to 0 at the same time, **Q** and **Q'** begin to oscillate between 1 and 0 because their final values depend on each other (**metastability**)
   - This eventually settles depending on **variation in the circuits**

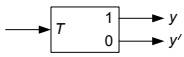
Memory Element: Latches



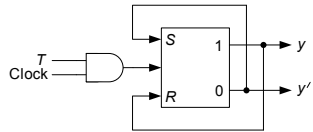
- A **clock** is a periodic signal that is used to keep time in sequential circuits.
- **Duty Cycle** is the ratio of t_w/T_{period}
- We want to keep t_w small so that in the same clock pulse only a single computation is performed.
- We want to keep T_{period} sufficient so that there is enough time for the next input to be computed.

Memory Element: T Latch

Value 1 applied to its input triggers the latch to change state



(*a*) Block diagram.



(*b*) Deriving the T latch from the clocked SR latch.

Excitations requirements:

Circuit change		*Required*
From:	*To:*	*value T*
0	0	0
0	1	1
1	0	1
1	1	0

"Q" is basically "y"

Characteristic Table

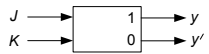
T Flip-Flop

T	**Q(t + 1)**	
0	$Q(t)$	No change
1	$Q'(t)$	Complement

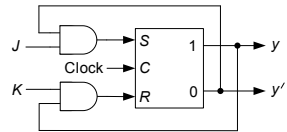
$$y(t+1) = Ty'(t) + T'y(t)$$
$$= T \oplus y(t)$$

Memory Element: JK Latch

Unlike the *SR* latch, $J = K = 1$ is permitted: when it occurs, the latch acts like a trigger and switches to the complement state



(*a*) Block diagram.



(*b*) Constructing the JK latch from the clocked SR latch.

Excitation requirements:

Circuit change		*Required value*	
From:	*To:*	*J*	*K*
0	0	0	–
0	1	1	–
1	0	–	1
1	1	–	0

"Q" is basically "y"

Characteristic Table

***JK* Flip-Flop**			
J	***K***	***Q*(*t* + 1)**	
0	0	$Q(t)$	No change
0	1	0	Reset
1	0	1	Set
1	1	$Q'(t)$	Complement

Can you write the characteristic equation?

$$y(t+1) = Jy(t)' + K'y(t)$$

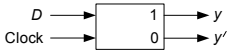
D Latch — The Latch of Your Life

The next state of the D latch is equal to its present excitation:

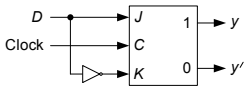
$$y(t+1) = D(t)$$

D Flip-Flop

D	Q(t + 1)	
0	0	Reset
1	1	Set



(*a*) Block diagram.



(*b*) Transforming the JK latch to the D latch.

Excitation Table

Q(t)	Q(t+1)	D
0	0	0
0	1	1
1	0	0
1	1	1

How is Your Clock?

Clocked latch: changes state only in synchronization with the clock pulse and no more than once during each occurrence of the clock pulse

Duration of clock pulse: determined by circuit delays and signal propagation time through the latches

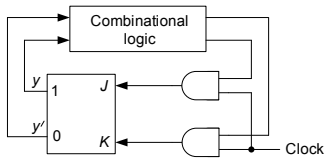
- Must be long enough to allow latch to change state, and
- Short enough so that the latch will not change state twice due to the same excitation

Excitation of a *JK* latch within a sequential circuit:

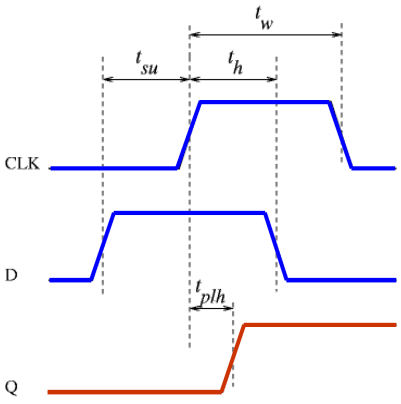
- Length of the clock pulse must allow the latch to generate the *y*'s
- But should not be present when the values of the *y*'s have propagated through the combinational circuit

How fast/slow should be the clock really?

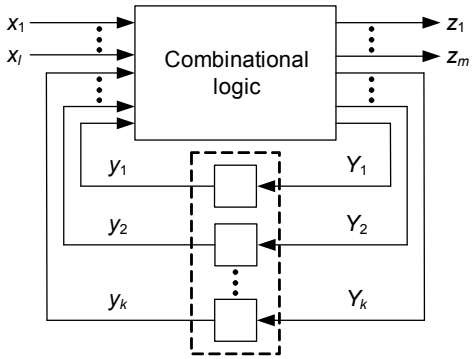
But when does the flip-flop changes its state???



All in One



D Flip–flop (edge–triggered)
(positive edge triggering)



``Memory'' devices

Delay to make sure all is well

- **Setup time**, t_{su}, is the time period prior to the clock becoming active (edge or level) during which the flip-flop inputs must remain stable.

- **Hold time**, t_h, is the time after the clock becomes inactive during which the flip-flop inputs must remain stable.

- Setup time and hold time define a *window of time during which the flip-flop inputs cannot change* – quiescent interval.

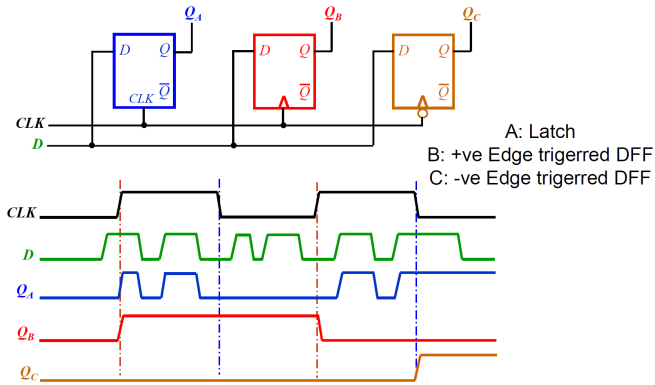
More Delay

- **Propagation delay, t_{pHL} and t_{pLH}** , has the same meaning as in combinational circuit – beware propagation delays usually will not be equal for all input to output pairs. There can be two propagation delays: $t_{C\text{-}Q}$ (*clock*→*Q* delay) and $t_{D\text{-}Q}$ (*data*→*Q* delay).

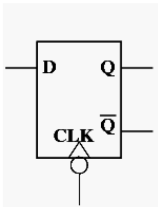
- For a level or pulse triggered latch:
  - Data input should remain stable till the clock becomes inactive.
  - Clock should remain active till the input change is propagated to Q output. That is, active period of the clock,

$$t_w > \max\{t_{pLH}, t_{pHL}\}$$

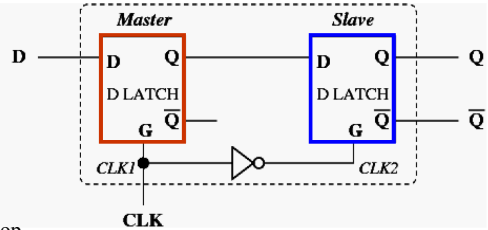
The Triggering Dilemma



Master Slave Flip-Flop



Negative edge triggered flipflop

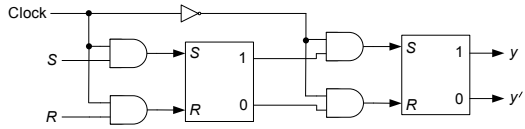


At a given time, only one latch is alive (either master or slave)

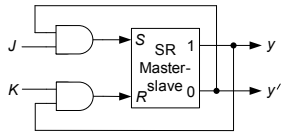
Master Slave Flip-Flop

Master-slave flip-flop: a type of synchronous memory element that eliminates the timing problems by isolating its inputs from its outputs

Master-slave *SR* flip-flop:



Master-slave *JK* flip-flop: since master-slave *SR* flip-flop suffers from the problem that both its inputs cannot be 1, it can be converted to a *JK* flip-flip



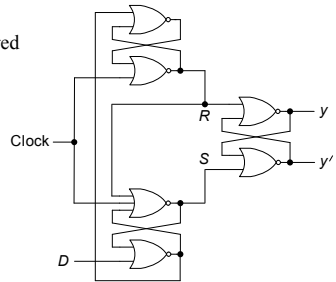
Edge Triggered Flip-Flop

Positive (negative) edge-triggered *D* flip-flip: stores the value at the *D* input when the clock makes a 0 -> 1 (1 -> 0) transition

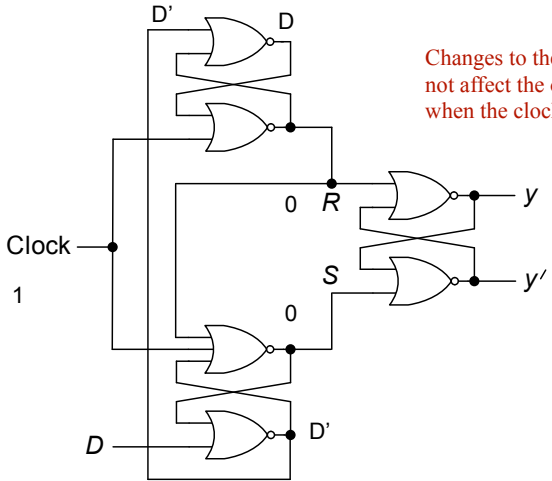
- Any change at the *D* input after the clock has made a transition does not have any effect on the value stored in the flip-flop

A negative edge-triggered *D* flip-flop:

- When the clock is high, the output of the bottommost (topmost) NOR gate is at *D'* (*D*), whereas the *S*-*R* inputs of the output latch are at 0, causing it to hold previous value
- When the clock goes low, the value from the bottommost (topmost) NOR gate gets transferred as *D* (*D'*) to the *S* (*R*) input of the output latch
  - Thus, output latch stores the value of *D*
- If there is a change in the value of the *D* input after the clock has made its transition, the bottommost NOR gate attains value 0
  - However, this cannot change the *SR* inputs of the output latch

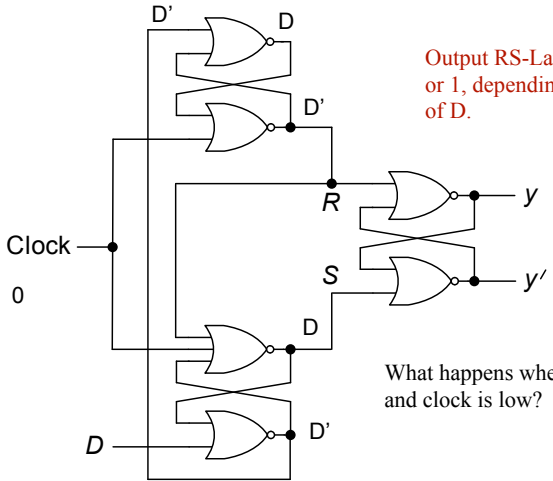


Edge Triggered Flip-Flop



Changes to the input D does not affect the output y and y' when the clock is high.

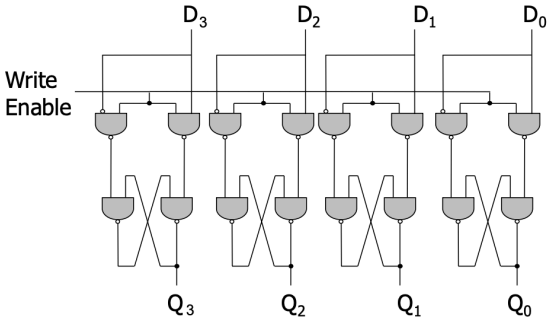
Edge Triggered Flip-Flop

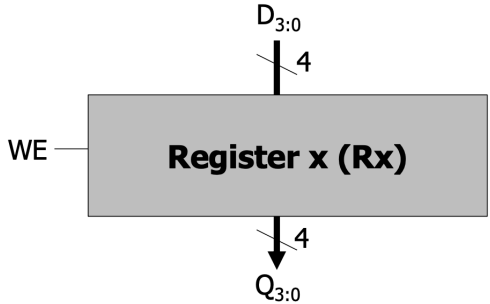


Output RS-Latch stores a 0 or 1, depending on the value of D.

What happens when D changes and clock is low?

Registers: Your Main Sequential Element





- Used to store data
- Basically an **array of D-flip-flops**
- You can **load data**, **reset it to zero**, and **shift it to left and right**

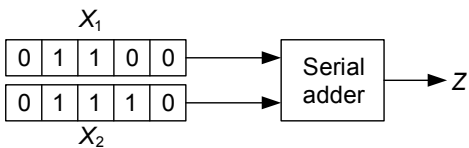
Sequential Circuits and Finite State Machines

Sequential circuit: its outputs a function of external inputs as well as stored information (aka. State)

Finite-state machine (FSM): abstract model to describe the synchronous sequential machines. It has finite memory.

Serial binary adder example: You are given a 1-bit adder. But you have to add n-bit numbers

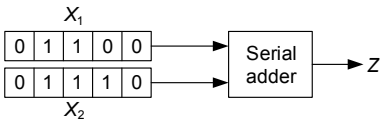
- First, decide what to remember??
- Then decide how many bits to remember??



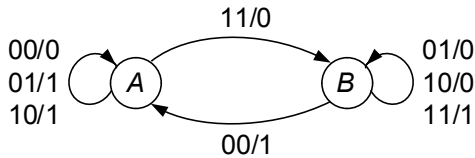
$$
\begin{array}{rccccccc}
 & t_5 & t_4 & t_3 & t_2 & t_1 & & \\
 & 0 & 1 & 1 & 0 & 0 & = & X_1 \\
+ & 0 & 1 & 1 & 1 & 0 & = & X_2 \\
\hline
 & 1 & 1 & 0 & 1 & 0 & = & Z
\end{array}
$$

Sequential Circuits and Finite State Machines

- Let **A** denote the state of the adder at t_i if the carry 0 is generated at t_{i-1}
- Let **B** denote the state of the adder at t_i if the carry 1 is generated at t_{i-1}

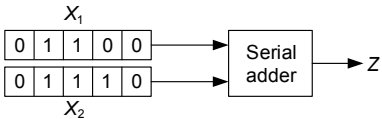


$$
\begin{array}{rccccccc}
 & t_5 & t_4 & t_3 & t_2 & t_1 & & \\
 & 0 & 1 & 1 & 0 & 0 & = & X_1 \\
+ & 0 & 1 & 1 & 1 & 0 & = & X_2 \\
\hline
 & 1 & 1 & 0 & 1 & 0 & = & Z
\end{array}
$$



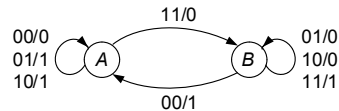
Sequential Circuits and Finite State Machines

- Let **A** denote the state of the adder at t_i if the carry 0 is generated at t_{i-1}
- Let **B** denote the state of the adder at t_i if the carry 1 is generated at t_{i-1}

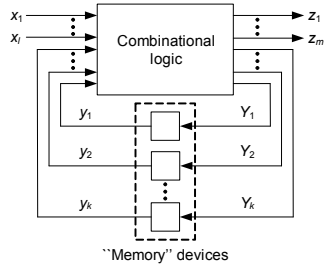


	NS, z			
PS	$x_1x_2 = 00$	01	11	10
A	$A, 0$	$A, 1$	$B, 0$	$A, 1$
B	$A, 1$	$B, 0$	$B, 1$	$B, 0$

$$
\begin{array}{rccccc}
 & t_5 & t_4 & t_3 & t_2 & t_1 & \\
 & 0 & 1 & 1 & 0 & 0 & = X_1 \\
+ & 0 & 1 & 1 & 1 & 0 & = X_2 \\
\hline
 & 1 & 1 & 0 & 1 & 0 & = Z
\end{array}
$$



Sequential Circuits and Finite State Machines



Input variables: $\{x_1, x_2, .., x_l\}$

Input configuration, symbol, pattern or vector: ordered l-tuple of 0's and 1's

Input alphabet: set of $p = 2^l$ distinct input patterns

- Thus, input alphabet $I = \{I_1, I_2, .., I_p\}$
- Example: for two variables x_1 and x_2
  - $I = \{00, 01, 10, 11\}$

Output variables: $\{z_1, z_2, .., z_m\}$

Output configuration, symbol, pattern or vector: ordered m-tuple of 0's and 1's

Output alphabet: set of $q = 2^m$ distinct output patterns

- Thus, output alphabet $O = \{O_1, O_2, .., O_q\}$

Synthesis of Synchronous Sequential Circuits

Main steps:

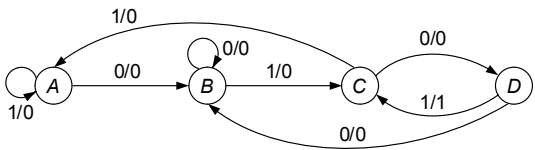
1. From a word description of the problem, form a **state diagram** or **table**
2. Select a **state assignment** and determine the type of memory elements
3. Derive **transition** and **output** tables
4. Derive an **excitation table** and obtain **excitation** and **output functions** from their respective tables
5. Draw a **circuit diagram**

Synthesis of Synchronous Sequential Circuits

One-input/one-output sequence detector: produces output value 1 every time sequence 0101 is detected, else 0

- Example: 010101 -> 000101

State diagram and state table:



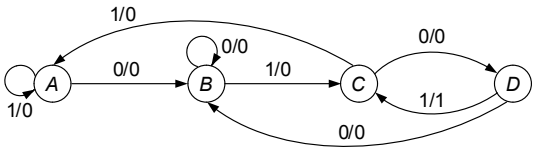
PS	NS, z	
	$x=0$	$x=1$
A	$B,0$	$A,0$
B	$B,0$	$C,0$
C	$D,0$	$A,0$
D	$B,0$	$C,1$

Synthesis of Synchronous Sequential Circuits

One-input/one-output sequence detector: produces output value 1 every time sequence 0101 is detected, else 0

- Example: 010101 -> 000101

State diagram and state table:



	NS, z	
PS	$x=0$	$x=1$
A	$B,0$	$A,0$
B	$B,0$	$C,0$
C	$D,0$	$A,0$
D	$B,0$	$C,1$

Transition and output tables:

	Y_1Y_2		z	
y_1y_2	$x=0$	$x=1$	$x=0$	$x=1$
$A \rightarrow 00$	01	00	0	0
$B \rightarrow 01$	01	11	0	0
$C \rightarrow 11$	10	00	0	0
$D \rightarrow 10$	01	11	0	1

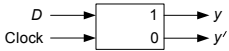
D Latch — The Latch of Your Life

The next state of the D latch is equal to its present excitation:

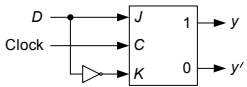
$$y(t+1) = D(t)$$

D Flip-Flop

D	Q(t + 1)	
0	0	Reset
1	1	Set



(*a*) Block diagram.



(*b*) Transforming the JK latch to the D latch.

Excitation Table

Q(t)	Q(t+1)	D
0	0	0
0	1	1
1	0	0
1	1	1

Synthesis of Synchronous Sequential Circuits

Excitation Table

Q(t)	Q(t+1)	D
0	0	0
0	1	1
1	0	0
1	1	1

- Let us use DFF as our state elements
- We need 2 DFFs as our state is 2 bit
- Now how to set the inputs of the DFFs??

	Y_1Y_2		z	
y_1y_2	$x=0$	$x=1$	$x=0$	$x=1$
$A \rightarrow 00$	01	00	0	0
$B \rightarrow 01$	01	11	0	0
$C \rightarrow 11$	10	00	0	0
$D \rightarrow 10$	01	11	0	1

Synthesis of Synchronous Sequential Circuits

Excitation Table

Q(t)	Q(t+1)	D
0	0	0
0	1	1
1	0	0
1	1	1

- Let us use DFF as our state elements
- We need 2 DFFs as our state is 2 bit
- Now how to set the inputs of the DFFs??

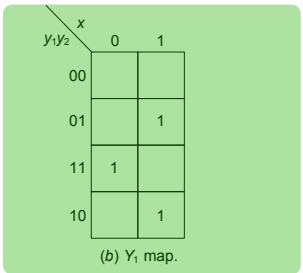
y_1y_2	Y_1Y_2 $x=0$	Y_1Y_2 $x=1$	z $x=0$	z $x=1$
$A \rightarrow 00$	01	00	0	0
$B \rightarrow 01$	01	11	0	0
$C \rightarrow 11$	10	00	0	0
$D \rightarrow 10$	01	11	0	1

D(Y1)		D(Y2)	
x = 0	x = 1	x=0	x=1
0	0	1	0
0	1	1	1
1	0	0	0
0	1	1	1

Synthesis of Synchronous Sequential Circuits

Excitation Table

Q(t)	Q(t+1)	D
0	0	0
0	1	1
1	0	0
1	1	1



(*b*) Y_1 map.

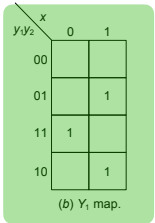
y_1y_2	Y_1Y_2 $x=0$	Y_1Y_2 $x=1$	z $x=0$	z $x=1$
$A \rightarrow 00$	01	00	0	0
$B \rightarrow 01$	01	11	0	0
$C \rightarrow 11$	10	00	0	0
$D \rightarrow 10$	01	11	0	1

D(Y1)		D(Y2)	
x = 0	x = 1	x=0	x=1
0	0	1	0
0	1	1	1
1	0	0	0
0	1	1	1

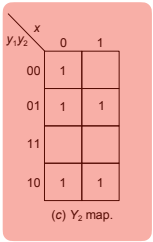
Synthesis of Synchronous Sequential Circuits

Excitation Table

Q(t)	Q(t+1)	D
0	0	0
0	1	1
1	0	0
1	1	1



(b) Y_1 map.



(c) Y_2 map.

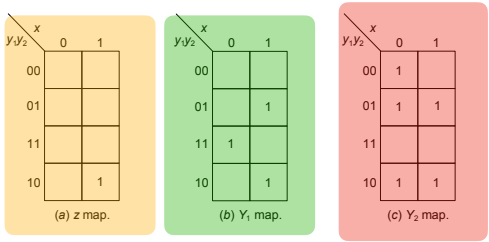
y_1y_2	Y_1Y_2 $x=0$	Y_1Y_2 $x=1$	z $x=0$	z $x=1$
$A \rightarrow 00$	01	00	0	0
$B \rightarrow 01$	01	11	0	0
$C \rightarrow 11$	10	00	0	0
$D \rightarrow 10$	01	11	0	1

D(Y1)		D(Y2)	
x = 0	x = 1	x=0	x=1
0	0	1	0
0	1	1	1
1	0	0	0
0	1	1	1

Synthesis of Synchronous Sequential Circuits

Excitation Table

Q(t)	Q(t+1)	D
0	0	0
0	1	1
1	0	0
1	1	1



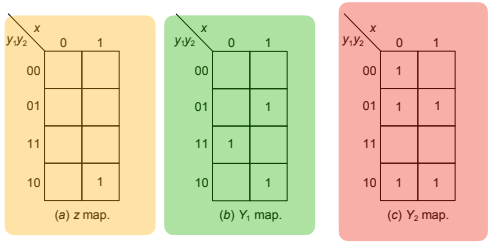
y_1y_2	Y_1Y_2 $x=0$	Y_1Y_2 $x=1$	z $x=0$	z $x=1$
$A \rightarrow 00$	01	00	0	0
$B \rightarrow 01$	01	11	0	0
$C \rightarrow 11$	10	00	0	0
$D \rightarrow 10$	01	11	0	1

D(Y1)		D(Y2)	
x = 0	x = 1	x=0	x=1
0	0	1	0
0	1	1	1
1	0	0	0
0	1	1	1

Synthesis of Synchronous Sequential Circuits

Excitation Table

Q(t)	Q(t+1)	D
0	0	0
0	1	1
1	0	0
1	1	1



$$z = xy_1y_2'$$
$$Y_1 = x'y_1y_2 + xy_1'y_2 + xy_1y_2'$$
$$Y_2 = y_1y_2' + x'y_1' + y_1'y_2$$

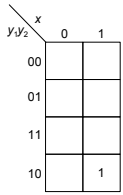
y_1y_2	Y_1Y_2 $x=0$	Y_1Y_2 $x=1$	z $x=0$	z $x=1$
$A \rightarrow 00$	01	00	0	0
$B \rightarrow 01$	01	11	0	0
$C \rightarrow 11$	10	00	0	0
$D \rightarrow 10$	01	11	0	1

D(Y1)		D(Y2)	
x = 0	x = 1	x=0	x=1
0	0	1	0
0	1	1	1
1	0	0	0
0	1	1	1

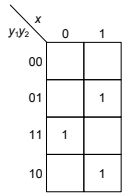
Synthesis of Synchronous Sequential Circuits

Excitation Table

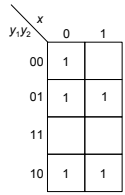
Q(t)	Q(t+1)	D
0	0	0
0	1	1
1	0	0
1	1	1



(a) z map.



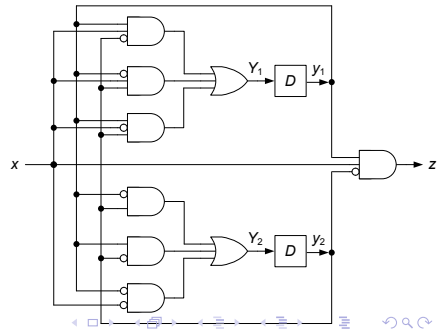
(b) Y_1 map.



(c) Y_2 map.

$$z = xy_1y_2'$$
$$Y_1 = x'y_1y_2 + xy_1'y_2 + xy_1y_2'$$
$$Y_2 = y_1y_2' + x'y_1' + y_1'y_2$$

Logic Diagram



	Y_1Y_2		z	
y_1y_2	$x=0$	$x=1$	$x=0$	$x=1$
$A \rightarrow 00$	01	00	0	0
$B \rightarrow 01$	01	11	0	0
$C \rightarrow 11$	10	00	0	0
$D \rightarrow 10$	01	11	0	1

D(Y1)		D(Y2)	
x = 0	x = 1	x=0	x=1
0	0	1	0
0	1	1	1
1	0	0	0
0	1	1	1

Synthesis of Synchronous Sequential Circuits

Another state assignment:

	Y_1Y_2		z	
y_1y_2	$x=0$	$x=1$	$x=0$	$x=1$
$A \rightarrow 00$	01	00	0	0
$B \rightarrow 01$	01	10	0	0
$C \rightarrow 10$	11	00	0	0
$D \rightarrow 11$	01	10	0	1

$z = xy_1y_2$

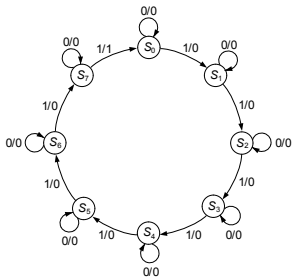
$Y_1 = x'y_1y_2' + xy_2$

$Y_2 = x'$

Binary Counter

One-input/one-output modulo-8 binary counter: produces output value 1 for every eighth input 1 value

State diagram and state table:



	NS		$Output$	
PS	$x=0$	$x=1$	$x=0$	$x=1$
S_0	S_0	S_1	0	0
S_1	S_1	S_2	0	0
S_2	S_2	S_3	0	0
S_3	S_3	S_4	0	0
S_4	S_4	S_5	0	0
S_5	S_5	S_6	0	0
S_6	S_6	S_7	0	0
S_7	S_7	S_0	0	1

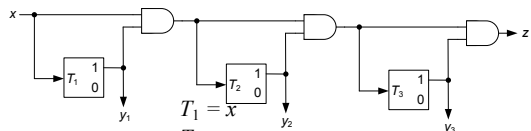
Binary Counter

Transition and output tables:

PS	NS		z		$T_3T_2T_1$	
$y_3y_2y_1$	$x=0$	$x=1$	$x=0$	$x=1$	$x=0$	$x=1$
000	000	001	0	0	000	001
001	001	010	0	0	000	011
010	010	011	0	0	000	001
011	011	100	0	0	000	111
100	100	101	0	0	000	001
101	101	110	0	0	000	011
110	110	111	0	0	000	001
111	111	000	0	1	000	111

Excitation table for *T*

Circuit change		*Required*
From:	*To:*	*value T*
0	0	0
0	1	1
1	0	1
1	1	0



$T_1 = x$

$T_2 = xy_1$

$T_3 = xy_1y_2$

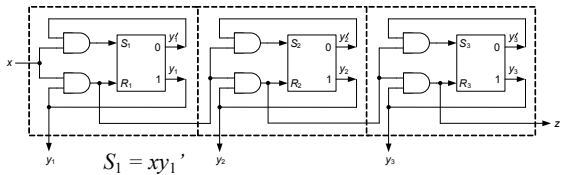
$z = xy_1y_2y_3$

Binary Counter with SR Flip Flops

PS	NS		z		$x=0$			$x=1$		
$y_3y_2y_1$	$x=0$	$x=1$	$x=0$	$x=1$	S_3R_3	S_2R_2	S_1R_1	S_3R_3	S_2R_2	S_1R_1
000	000	001	0	0	0–	0–	0–	0–	0–	10
001	001	010	0	0	0–	0–	–0	0–	10	01
010	010	011	0	0	0–	–0	0–	0–	–0	10
011	011	100	0	0	0–	–0	–0	10	01	01
100	100	101	0	0	–0	0–	0–	–0	0–	10
101	101	110	0	0	–0	0–	–0	–0	10	01
110	110	111	0	0	–0	–0	0–	–0	–0	10
111	111	000	0	1	–0	–0	–0	01	01	01

Excitation table for *SR* flip-flops and logic diagram:

Circuit change		*Required value*	
From:	*To:*	S	R
0	0	0	–
0	1	1	0
1	0	0	1
1	1	–	0



$S_1 = xy_1'$

$R_1 = xy_1$

$S_2 = xy_1y_2'$

$R_2 = xy_1y_2$

$S_3 = xy_1y_2y_3'$

$R_3 = z = xy_1y_2y_3$

But…Life is Beautiful End of the day…

```
always @(posedge clk) begin
    if (rst)
        count <= 3'b000;        // Reset to 0
    else if (count == 3'b111)   // If 7, wrap back to 0
        count <= 3'b000;
    else
        count <= count + 1'b1;  // Increment
end
```

Generic Template for Writing State Machines

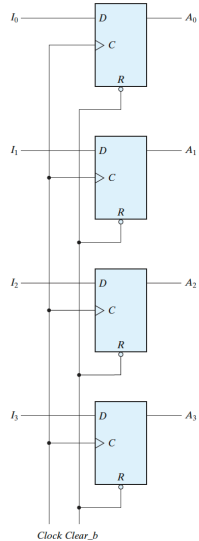
```
module simple_counter(clk,rst, enable, out);
 input clk;
 input rst;
 input enable;


 output reg out;

 reg [<num_state_bits>:0] state, nxt_state;
 parameter S0 = 2'b00;
 parameter S1 = 2'b01;
 parameter S2 = 2'b10;
 parameter S3 = 2'b11;
 parameter S4 = …
              :
              :
```

```
always @(posedge clk) begin
   if (rst) begin
      state <= S0;
   end
   else if (enable) begin
      state <= nxt_state;
   end
end
```

```
always@(*) begin
    case(state)
       S0:
          begin
             If (<inp_sig == 1>) begin
                nxt_state = S1;
                out = 0;
             end
          end
       S1:
          begin
             nxt_state = S2;
             out = 1;
          end
                :
                :
       default:
          begin
             nxt_state = S0;
             out = 0;
          end
    endcase

 end

endmodule
```

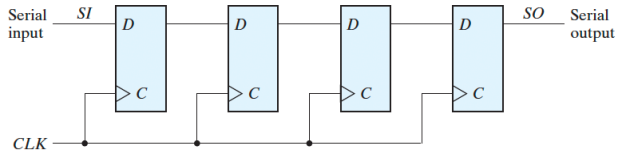
Registers: Your Main Sequential Element

- 4-bit register
- Asynchronous Reset
- On a clock tick, the data in I0, I1, I2, I3 gets available in A0, A1, A2, A3.



Shift Registers

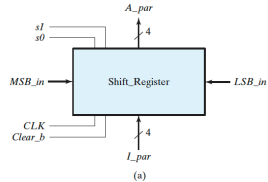
- Shift the bits left and right
- Again, very easy to write in Verilog

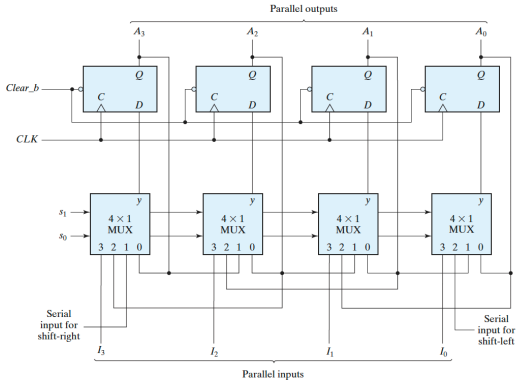


Shift Registers

- Universal Shift Register

Mode Control		
s_1	s_0	Register Operation
0	0	No change
0	1	Shift right
1	0	Shift left
1	1	Parallel load





Sequential Circuit as a Controller

Control element: streamlines computation by providing appropriate control signals

Example: digital system that computes the value of $(4a + b)$ modulo 16

- a, b: four-bit binary number
- 4-bit output

First let's decide what are the components needed

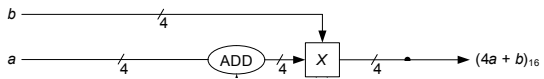
Sequential Circuit as a Controller

Control element: streamlines computation by providing appropriate control signals

Example: digital system that computes the value of $(4a + b)$ modulo 16

- a, b: four-bit binary number
- X: register containing four flip-flops
- A 4-bit parallel adder

- x: number stored in X
- Register can be loaded with: either b or $a + x$
- **But we have to stop after 4 additions — who tells that??**



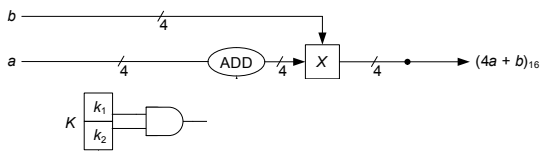
Sequential Circuit as a Controller

Control element: streamlines computation by providing appropriate control signals

Example: digital system that computes the value of $(4a + b)$ modulo 16

- a, b: four-bit binary number
- X: register containing four flip-flops
- A 4-bit parallel adder

- x: number stored in X
- Register can be loaded with: either b or $a + x$
- **But we have to stop after 4 additions — who tells that??**
- K: modulo-4 binary counter, whose output L equals 1 whenever the count is 3 modulo 4 — **but where does it connect to??**



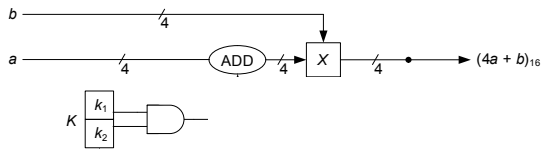
Sequential Circuit as a Controller

Control element: streamlines computation by providing appropriate control signals

Example: digital system that computes the value of $(4a + b)$ modulo 16

- a, b: four-bit binary number
- X: register containing four flip-flops
- A 4-bit parallel adder

- x: number stored in X
- Register can be loaded with: either b or $a + x$
- **But we have to stop after 4 additions — who tells that??**
- K: modulo-4 binary counter, whose output L equals 1 whenever the count is 3 modulo 4 — **but where does it connect to??**



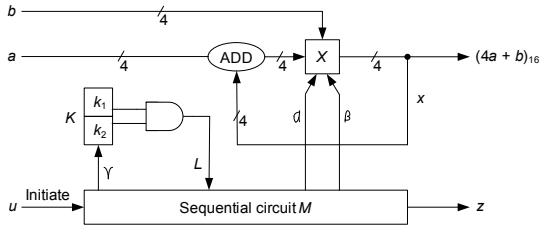
Sequential Circuit as a Controller

Control element: streamlines computation by providing appropriate control signals

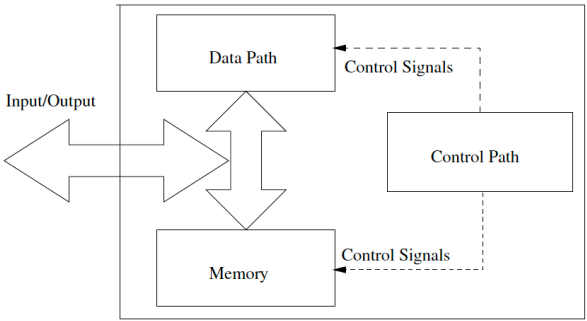
Example: digital system that computes the value of $(4a + b)$ modulo 16

- a, b: four-bit binary number
- X: register containing four flip-flops
- A 4-bit parallel adder

- x: number stored in X
- Register can be loaded with: either b or $a + x$
- **But we have to stop after 4 additions — who tells that??**
- K: modulo-4 binary counter, whose output L equals 1 whenever the count is 3 modulo 4 — **but where does it connect to??**
- **We need another sequential circuit as a controller to the circuit**



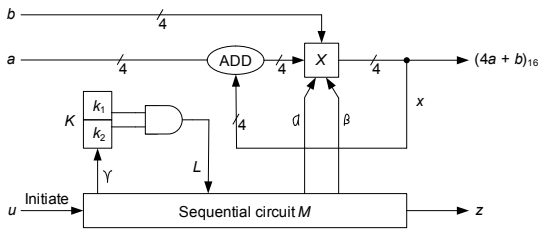
General View of a Hardware



Sequential Circuit as a Controller

Sequential circuit *M*:

- Input u: initiates computation
- Input L: gives the count of K
- Outputs: α, β, γ, z
- When $\alpha = 1$: contents of b transferred to X
- When $\beta = 1$: values of x and a added and transferred back to X
- When $\gamma = 1$: count of K increased by 1
- $z = 1$: whenever final result available in X

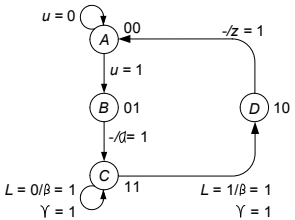


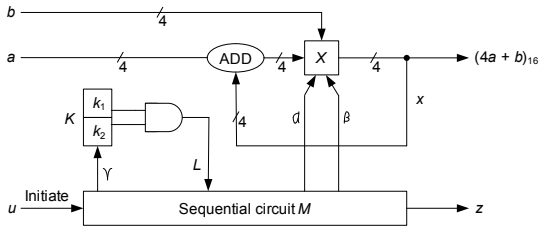
Sequential Circuit as a Controller

Sequential circuit *M*:

- K, u, z: initially at 0
- When $u = 1$: computation starts by setting $\alpha = 1$
  - Causes b to be loaded into X
- To add a to x: set $\beta = 1$ and $\gamma = 1$ to keep track of the number of times a has been added to x
- After four such additions: $z = 1$ and the computation is complete
- At this point: $K = 0$ to be ready for the next computation

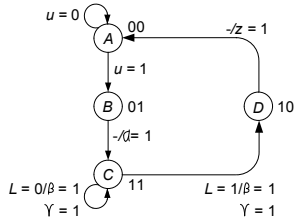
State diagram:





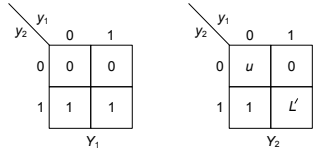
Sequential Circuit as a Controller

State assignment, transition table, maps and logic diagram:



PS y_1y_2	NS Y_1Y_2
00	0*u*
01	11
11	1*L*′
10	00

(*a*) Transition table.



(*b*) Maps for Y_1 and Y_2.

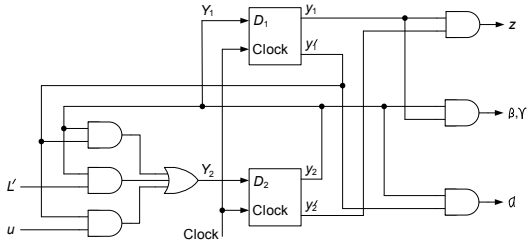
$\alpha = y_1'y_2$

$\beta = \gamma = y_1y_2$

$z = y_1y_2'$

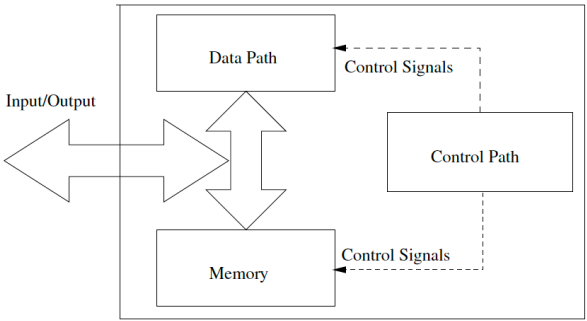
$Y_1 = y_2$

$Y_2 = y_1'y_2 + uy_1' + L'y_2$



(*c*) Logic diagram.

General View of a Hardware



Binary GCD Algorithm

Input: Integers u and v
Output: Greatest Common Divisor of u and v: $z = gcd(u, v)$

```
1  while (u!=v) do
2      if u and v are even then
3          z = 2gcd(u/2, v/2)
4      end
5      else if (u is odd and v is even) then
6          z = gcd(u, v/2)
7      end
8      else if (u is even and v is odd) then
9          z = gcd(u/2, v)
10     end
11     else
12         if (u ≥ v) then
13             z = gcd((u − v)/2, v)
14         end
15         else
16             z = gcd(u, (v − u)/2)
17         end
18     end
19 end
```

Binary GCD Algorithm: Closer to Hardware

Input: Integers u and v
Output: Greatest Common Divisor of u and v: $z = gcd(u, v)$

```
1  while (u!=v) do
2      if u and v are even then
3          z = 2gcd(u/2, v/2)
4      end
5      else if (u is odd and v is even) then
6          z = gcd(u, v/2)
7      end
8      else if (u is even and v is odd) then
9          z = gcd(u/2, v)
10     end
11     else
12         if (u ≥ v) then
13             z = gcd((u − v)/2, v)
14         end
15         else
16             z = gcd(u, (v − u)/2)
17         end
18     end
19 end
```

Input: Integers u and v
Output: Greatest Common Divisor of u and v: $z = gcd(u, v)$

```
1  register X_R, Y_R;
2  X_R = u; Y_R = v; count = 0;
3  while (X_R! = Y_R) do
4      if (X_R[0] = 0 and Y_R[0] = 0) then
5          X_R = right shift(X_R)
6          Y_R = right shift(Y_R)
7          count = count + 1
8      end
9      else if (X_R[0] = 1 and Y_R[0] = 1) then
10         Y_R = right shift(Y_R)
11     end
12     else if (X_R[0] = 0 and Y_R[0] = 1) then
13         X_R = right shift(X_R)
14     end
15     else
16         if (X_R ≥ Y_R) then
17             X_R = right shift(X_R − Y_R)
18         end
19         else
20             Y_R = right shift(Y_R − X_R)
21         end
22 end
23 while (count > 0) do
24     X_R = left shift(X_R)
25     count = count − 1
26 end
```

Binary GCD Algorithm: Closer to Hardware

Input: Integers u and v
Output: Greatest Common Divisor of u and v: $z = gcd(u, v)$

```
1  while (u!=v) do
2      if u and v are even then
3          z = 2gcd(u/2, v/2)
4      end
5      else if (u is odd and v is even) then
6          z = gcd(u, v/2)
7      end
8      else if (u is even and v is odd) then
9          z = gcd(u/2, v)
10     end
11     else
12         if (u ≥ v) then
13             z = gcd((u − v)/2, v)
14         end
15         else
16             z = gcd(u, (v − u)/2)
17         end
18     end
19 end
```

How to interpret as hardware?

Keep the count of the 2's getting multiplied

Input: Integers u and v
Output: Greatest Common Divisor of u and v: $z = gcd(u, v)$

```
1  register X_R, Y_R;
2  X_R = u; Y_R = v; count = 0;
3  while (X_R! = Y_R) do
4      if (X_R[0] = 0 and Y_R[0] = 0) then
5          X_R = right shift(X_R)
6          Y_R = right shift(Y_R)
7          count = count + 1
8      end
9      else if (X_R[0] = 1 and Y_R[0] = 1) then
10         Y_R = right shift(Y_R)
11     end
12     else if (X_R[0] = 0 and Y_R[0] = 1) then
13         X_R = right shift(X_R)
14     end
15     else
16         if (X_R ≥ Y_R) then
17             X_R = right shift(X_R − Y_R)
18         end
19         else
20             Y_R = right shift(Y_R − X_R)
21         end
22 end
23 while (count > 0) do
24     X_R = left shift(X_R)
25     count = count − 1
26 end
```

Constructing the Datapath

Input: Integers u and v
Output: Greatest Common Divisor of u and v: $z = gcd(u, v)$

```
1  register X_R, Y_R;
2  X_R = u; Y_R = v; count = 0;
3  while (X_R! = Y_R) do
4      if (X_R[0] = 0 and Y_R[0] = 0) then
5          X_R = right shift(X_R)
6          Y_R = right shift(Y_R)
7          count = count + 1
8      end
9      else if (X_R[0] = 1 and Y_R[0] = 1) then
10         Y_R = right shift(Y_R)
11     end
12     else if (X_R[0] = 0 and Y_R[0] = 1) then
13         X_R = right shift(X_R)
14     end
15     else
16         if (X_R ≥ Y_R) then
17             X_R = right shift(X_R − Y_R)
18         end
19         else
20             Y_R = right shift(Y_R − X_R)
21         end
22 end
23 while (count > 0) do
24     X_R = left shift(X_R)
25     count = count − 1
26 end
```

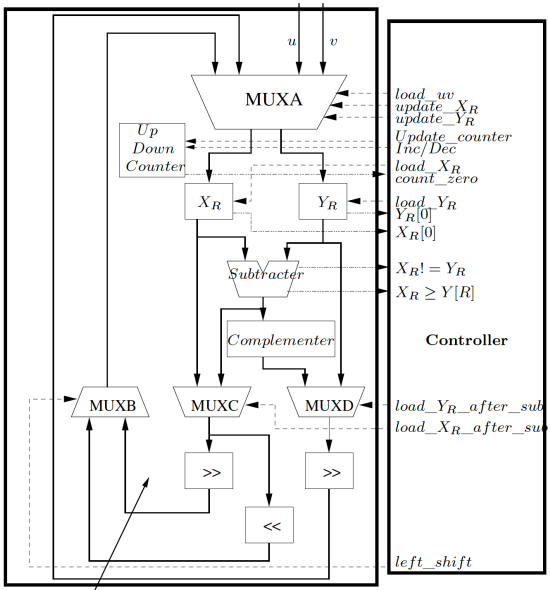
- **Required hardware components**
  - Two Registers for holding u and v (sequential)
- **Datapath elements**
  - Subtractor
  - Complementer
  - Right shifter
  - Left shifter
  - Counter (sequential)
  - Multiplexors — to route the control signals

Constructing the Controller

```
1  register X_R, Y_R;
2  X_R = u; Y_R = v; count = 0;                          /* State 0 */
3  while (X_R != Y_R) do
4      if (X_R[0] = 0 and Y_R[0] = 0) then               /* State 1 */
5          X_R = right shift(X_R)
6          Y_R = right shift(Y_R)
7          count = count + 1
8      end
9      else if (X_R[0] = 1 and Y_R[0] = 1) then          /* State 2 */
10         Y_R = right shift(Y_R)
11     end
12     else if (X_R[0] = 0 and Y_R[0] = 1) then          /* State 3 */
13         X_R = right shift(X_R)
14     end
15     else                                              /* State 4 */
16         if (X_R ≥ Y_R) then
17             X_R = right shift(X_R − Y_R)
18         end
19         else
20             Y_R = right shift(Y_R − X_R)
21         end
22 end
23 while (count > 0) do                                  /* State 5 */
24     X_R = left shift(X_R)
25     count = count − 1
26 end
```

- **Observation**
  - Whenever there is an if-else block, we allocate a state
  - Why? Because they dictate the control flow
  - The while loop is handled by the counter
    - Give the counter value to the controller and it tells you when to stop
  - **Note**: there might be cases when you have no if-else statements, but you have to sequentialize the computation
    - Remember the 4a+b example.
    - Whenever you need to sequentialise, allocate states.

Constructing the Hardware Diagram



Input: Integers u and v
Output: Greatest Common Divisor of u and v: $z = gcd(u, v)$

```
1  register X_R, Y_R;
2  X_R = u; Y_R = v; count = 0;
3  while (X_R! = Y_R) do
4      if (X_R[0] = 0 and Y_R[0] = 0) then
5          X_R = right shift(X_R)
6          Y_R = right shift(Y_R)
7          count = count + 1
8      end
9      else if (X_R[0] = 1 and Y_R[0] = 1) then
10         Y_R = right shift(Y_R)
11     end
12     else if (X_R[0] = 0 and Y_R[0] = 1) then
13         X_R = right shift(X_R)
14     end
15     else
16         if (X_R ≥ Y_R) then
17             X_R = right shift(X_R − Y_R)
18         end
19         else
20             Y_R = right shift(Y_R − X_R)
21         end
22 end
23 while (count > 0) do
24     X_R = left shift(X_R)
25     count = count − 1
26 end
```

Constructing the State Transition Table

Present State	Next State					Output Signals										
	0____	100_	110_	101_	111_	*load uv*	*update X_R*	*update Y_R*	*load X_R*	*load Y_R*	*load_X_R after_sub*	*load_Y_R after_sub*	*Update counter*	*Inc /Dec*	*left shift*	*count zero*
S_0	S_5	S_1	S_2	S_3	S_4	1	0	0	1	1	0	0	0	_	_	_
S_1	S_5	S_1	S_2	S_3	S_4	0	1	1	1	1	0	0	1	1	_	_
S_2	S_5	S_1	S_2	S_3	S_4	0	0	1	0	1	0	0	0	_	_	_
S_3	S_5	S_1	S_2	S_3	S_4	0	1	0	1	0	0	0	0	_	_	_
S_4 ($X_R \geq Y_R$)	S_5	S_1	S_2	S_3	S_4	0	1	0	1	0	1	0	0	_	_	_
S_4 ($X_R < Y_R$)	S_5	S_1	S_2	S_3	S_4	0	0	1	0	1	0	1	0	_	_	_
S_5	S_5	S_5	S_5	S_5	S_5	0	0	0	0	0	0	0	1	0	1	0

- **Controller receives 4 input signals**
  - XR != YR
  - XR[0]
  - YR[0]
  - XR >= YR

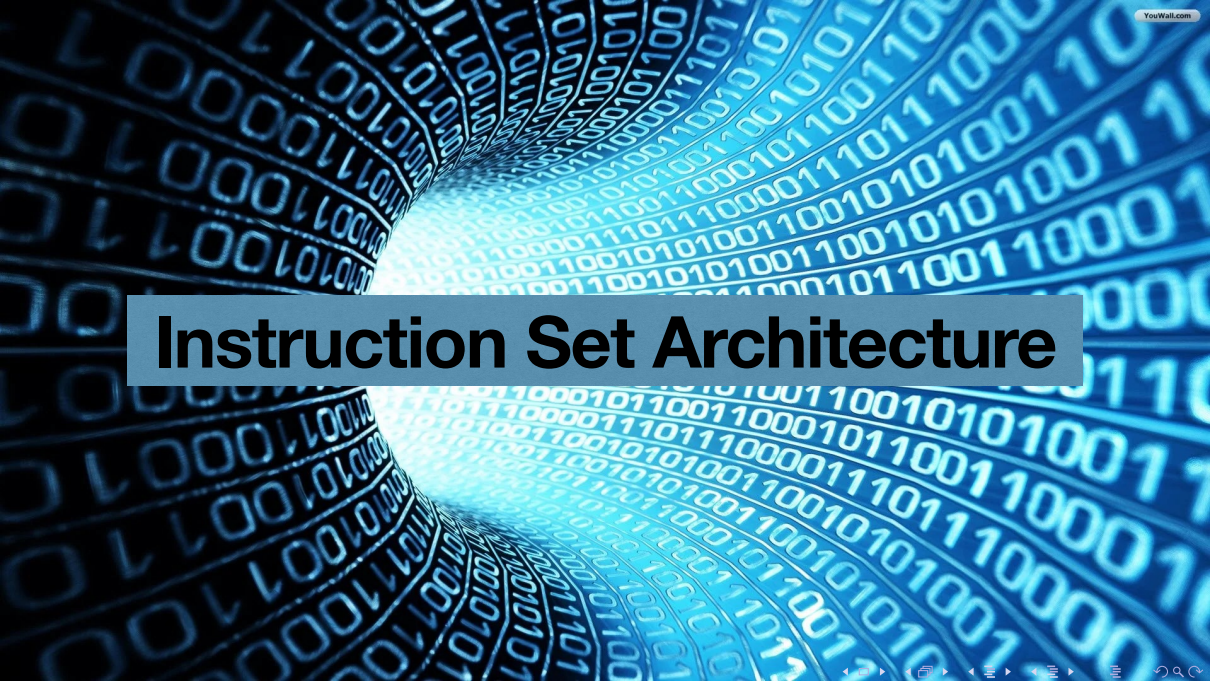
- Count zero is also a control signal, but it goes to the controller can comes back to the MUX as it is. So it was shown as output

Now go and code it down

Digital Logic Design + Computer Architecture

Sayandeep Saha

Assistant Professor
Department of Computer Science and Engineering
Indian Institute of Technology Bombay


YouWall.com
Instruction Set Architecture

How to talk to a Computer?

- **Computers can be given “instructions”**
- We have a set of instructions for every computer — called **instruction set**
- **When you write a program, you write instructions..**
  - More details later…
- Every instruction some hardware circuit implemented inside the processor to get its job done.
- **Instruction Set Architecture**: specifies the set of instructions a processor understands, their encoding, how they access memory etc…

Image generated by ChatGPT

What happens when you write a program

- **Say we write:**
  - a = b + c;

- There is a software program called **compiler**
  - Takes our code and encodes in terms of the instructions available for the computer
  - `add reg1, reg2, reg3`

- There is another program called **assembler** which converts the instruction (sequence) to bits
- 0101110000110101

Image generated by ChatGPT

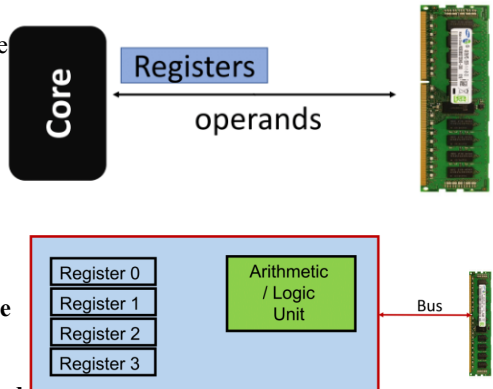
How to talk to a Computer?

- **Instruction Set Architecture**: specifies the set of instructions a processor understands, their encoding, how they access memory etc…
  - **End of the day even your ChatGPT is a sequence of instructions** (many billions or trillions).
- Instruction set is basically an **abstraction layer**
  - **Hides the complexity of hardware from the software designers,**
  - **Interfaces the software and hardware.**

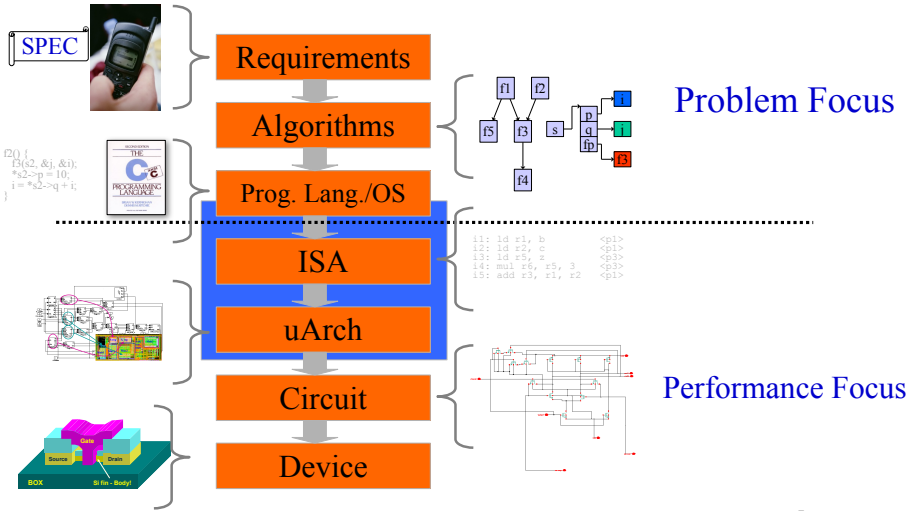
Image generated by ChatGPT

Let's get into the processor a bit

- It is a sequential circuit with a **limited** number of registers.
- It interacts with an external "memory".
- Every instruction operates on some **operands** and generate results.
- Results and operands are stored in **registers**.
- **But they can also be in memory as the number of registers are limited**
- Note that typically such memory **(**called **DRAM** or **Dynamic Random Access Memory)** is **off chip —outside the processor**
- **To operate, you have to bring the data from memory and store the results back**

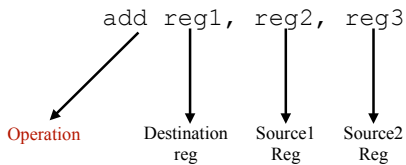


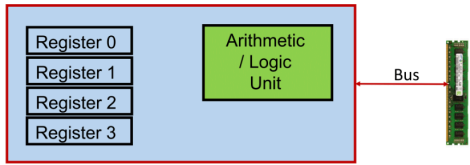
The Big Picture



Dissection of an Instruction

- Let's focus on the simplistic view of the processor





- Most of the arithmetic/logical instructions can take this form — not all though

Instruction Set Architectures (ISA)

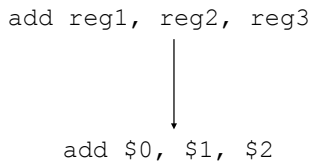
- There are many…
  - Intel uses **X86**
  - Apple uses a version of **AArch64** (ARM)
  - The entire world of embedded processors like ST-Microelectronics uses ARM
  - Now **RISC-V** is becoming a mainstream trend.
  - We shall study MIPS — a simple to understand ISA

Instruction Set Architectures (ISA)

- We shall study MIPS — a simple to understand ISA
  - Great for beginning…
  - Similar to ARM
  - Still in use in the embedded devices
    - Your smart card
    - Modems
    - Bitcoin-wallets

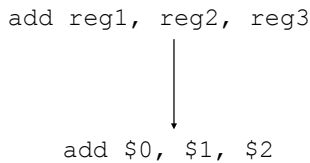
Now let's write some MIPS

- We shall name the registers as `$0`, `$1`, or `$a0`, `$g1` etc…
- Now we shall try something a bit more complex…



Now let's write some MIPS

- Let's compute: `a = b+c-d`
- No idea? — get idea :P



Now let's write some MIPS

- Let's compute: `a = b+c-d`
- Assume we have add and sub instructions taking two sources and one destination register

```
add $0, $1, $2
sub $0, $1, $2
```

Now let's write some MIPS

- Let's compute: `a = b+c-d`
- Assume we have add and sub instructions taking two sources and one destination register

```
add $0, $1, $2
sub $0, $1, $2
```

- First' let's simplify :
  - `t = b+c`
  - `a = t-d`
- **Observe**: I use a temporary register…
- Now, I can map to instructions..
  - `add $r0, $r1, $r2 //t = b+c`
  - `sub $d0, $r0, $r3 //a = t-d`

Now let's write some MIPS

- Let's try: `f = (g+h)-(i+j)`

Now let's write some MIPS

- Let's try: `f = (g+h)-(i+j)`

  - `add $r0, $r1, $r2 //x = g+h`
  - `add $r3, $r4, $r5 //y = i+j`
  - `Sub $r0, $r0, $r3 //f = x-y`

- **Food of thought**: Well, do I really need to reuse registers???

Ok…A Few MIPS Details…

- We have 32 registers in the processor
  - So we have to reuse registers, no other option…
  - Typically, registers are 32-bits…
- But why don't we have infinite number of registers
  - Well, every piece of register is a real hardware…

- **But**: Why 32??

Ok…A Few MIPS Details…

- We have 32 registers in the processor
  - So we have to reuse registers, no other option…
  - Typically, registers are 32-bits…
  - Each instruction also encoded in 32 bits

- **But**: Why 32??

- But why don't we have infinite number of registers
  - Well, every piece of register is a real hardware…

The choice depends on several factors, like the speed of the execution, the usage and size of memory, the size of code, the encoding and decoding of instructions….**It's not a random choice**…

Immediate Instructions…

- `b = a + 7`

```
addi $r0, $r1, 7
```

- We don't need a register for the constant…
  - Can you tell me why?? Just guess…

Immediate Instructions…

- `b = a + 7`

```
addi $r0, $r1, 7
```

- We don't need a register for the constant…
  - Can you tell me why?? Just guess…

- `i` stands for 'immediate'
- The constant is in **2's complement form and of 16 bits.**
- **Question: Do I need a `subi` instruction??**

Zero Is Very Special in Our Life…

- MIPS has a register which is called `$zero`
  - It stores 0
  - What is the purpose?
    - Well, a lot…you will see
  - A simple use of `$zero`

```
add $r1, $r0, $zero  // a = b
```

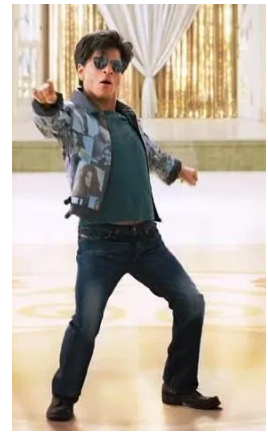
- But again, why???

Zero Is Very Special in Our Life…

- MIPS has a register which is called `$zero`
  - It stores 0
  - What is the purpose?
    - Well, a lot…you will see
  - A simple use of `$zero`

```
add $r1, $r0, $zero  // a = b
```

- But again, why??? — just not needed



a=b….The Pseudo-Instructions

- You can still write…

```
move $r1, $r0   // a = b
```

- But it is a pseudo-instruction
- Internally it converts to `add`
- Once again an engineering choice
- There are many such pseudo-instructions. See:

https://en.wikibooks.org/wiki/MIPS_Assembly/Pseudoinstructions

Logical Instructions

- Your good old Boolean algebra

sll, srl, and, or, nor, andi, ori etc

No not instruction ☺, well not is nor with one operand=0

- Remember: **These are bitwise operations…**
  - **Treats the operands as bit strings instead of numbers**

Logical Instructions

- Your good old Boolean algebra

sll, srl, and, or, nor, andi, ori etc

No not instruction ☺, well not is nor with one operand=0

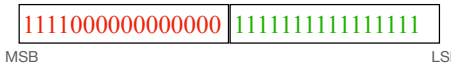
- Remember: **These are bitwise operations…**
  - **Treats the operands as bit strings instead of numbers**

Critical Thinking…

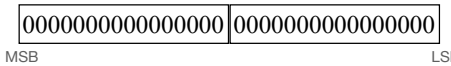
- We have seen that constants are 16 bits…
- But registers are 32-bits…
- How to store a 32-bit constant in a register???
  - Let's say the constant is:
    - 11110000000000001111111111111111
    - In Hex: 0xF000FFFF
- **Info**: You have the following new instruction:
  - `lui $r0, const // loads cosnt in the upper 16 bits of the register $r0`

Critical Thinking…

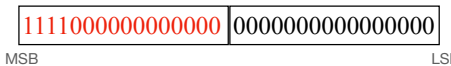
- Think, how the data will be represented inside your register…



  - Initially The register `$r0` is at (simplifying assumption…does not matter)



  - Now do: `lui $r0, 0xF000`

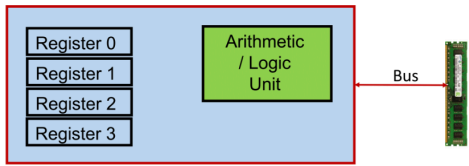


  - **Now do,** `addi $r0, 0xFFFF`

  - You can also do `ori`

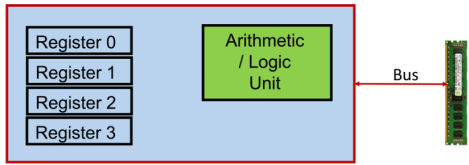
How to Use Your Memory??

- Recall, that MIPS only have 32 registers.
- Have you ever cared about counts while declaring variables in your program? — No way…
- Then how things work?
  - How can every program fits itself in 32 registers?



How to Use Your Memory??

- **Solution**:
  - Just store things in an external memory
  - Fetch the data to registers whenever it is required
  - Store the results after processing.
  - But still something is missing here…What is that??



How to Use Your Memory??

- Name this person?

How to Use Your Memory??

- Name this person?
  - John Luis von Neumann

How to Use Your Memory??

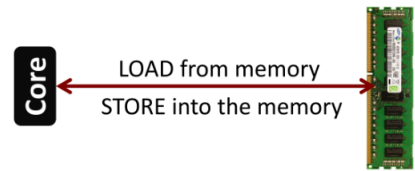
- In the old days, “programming” involved actually changing a machine’s physical configuration:
  - by flipping switches or connecting wires.
  - Memory only stored data that was being operated on.
- Then around 1944, **John von Neumann and others got the idea to encode instructions in a format that could be stored in memory just like data. — Stored program paradigm**
  - The processor interprets and executes instructions from memory

How to Use Your Memory??

- In the old days, “programming” involved actually changing a machine’s physical configuration:
  - by flipping switches or connecting wires.
  - Memory only stored data that was being operated on.
- Then around 1944, **John von Neumann and others got the idea to encode instructions in a format that could be stored in memory just like data. — Stored program paradigm**
  - The processor interprets and executes instructions from memory

Memory Instructions

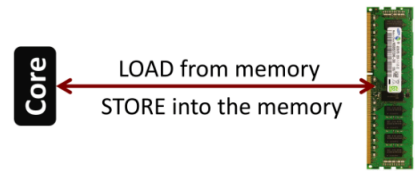
- **Load-Store Architecture**:
  - Load your data to process
  - Store it back…
- Instructions are handled in a slightly different manner….will come to that…




```
lw $t0, 1($a0)     # $t0 = Memory[$a0 + 1]
sw $t0, 1($a0)     # Memory[$a0 + 1] = $t0
```

Memory Instructions

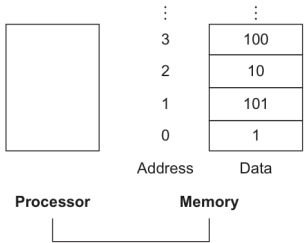
- **Load-Store Architecture**:
  - Load your data to process
  - Store it back…
- Instructions are handled in a slightly different manner….will come to that…
- But, a critical question:
  - How do you know where to find the data inside memory?




```
lw $t0, 1($a0)     # $t0 = Memory[$a0 + 1]
sw $t0, 1($a0)     # Memory[$a0 + 1] = $t0
```

Memory Instructions

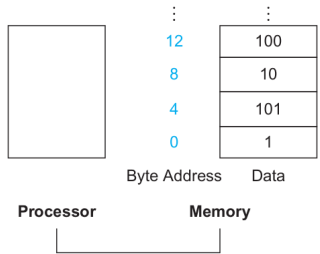
- But, a critical question:
  - How do you know where to find the data inside memory?
    - Memory has addresses
    - Think it like a large contiguous array…
    - **Every byte in memory has an unique address**
      - **Byte-addressable**
  - **BTW, each address is 32-bit in MIPS**



Memory Instructions

```
lw $t0, 1($a0)     # $t0 = Memory[$a0 + 1]
sw $t0, 1($a0)     # Memory[$a0 + 1] = $t0
```

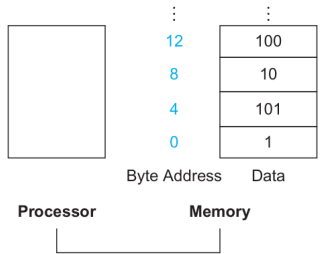
- The `lw` is interpreted as "load word"
  - MIPS also have other variants like "load byte" (`lb`)
- Data comes in `$t0`.
- But what is the 1(`$a0`) part signify?
  - `$a0` is the *base address* of the location you want to read from memory
  - 1 is called the *offset.*
- But why don't you read directly?



Memory Instructions

```
lw $t0, 1($a0)     # $t0 = Memory[$a0 + 1]
sw $t0, 1($a0)     # Memory[$a0 + 1] = $t0
```

- The `lw` is interpreted as "load word"
  - MIPS also have other variants like "load byte" (`lb`)
- Data comes in `$t0`.
- But what is the 1(`$a0`) part signify?
  - `$a0` is the *base address* of the location you want to read from memory
  - 1 is called the *offset.*
- But why don't you read directly?
  - Again a design choice, to ease compilation, programming, and hardware design…

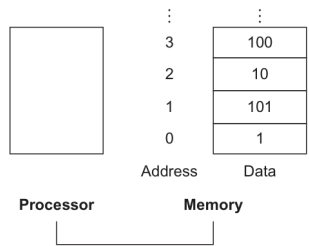


Memory Instructions: Word vs. Byte

```
lw $t0, 1($s0)        lb $t0, 1($s0)
```

- `lw` is interpreted as "load word"
- `lb` is "load byte"
- For the lw, we need the base+offset ($s0 + 1) to be **always divisible by 4 — word alignment**
- Why?
- Nothing such for lb

- **What Lies Beneath?**
  - `lb` just read the byte in the calculated address
  - `lw` reads four consecutive bytes starting from the calculated address.
- Why word alignment — again, it simplifies hardware OS, compiler….

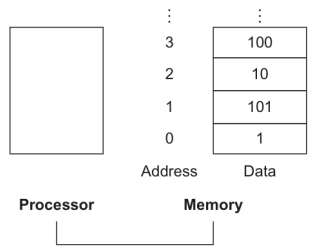


Memory Instructions: Word vs. Byte

```
lw $t0, 1($s0)        lb $t0, 1($s0)
```

- `lw` is interpreted as "load word"
- `lb` is "load byte"
- For the lw, we need the base+offset ($s0 + 1) to be **always divisible by 4 — word alignment**
- Why?
- Nothing such for lb

- **What Lies Beneath?**
  - `lb` just read the byte in the calculated address
  - `lw` reads four consecutive bytes starting from the calculated address.
- Why word alignment — again, it simplifies hardware OS, compiler….



Endianness (Byte ordering within a word)

- Big Endian: address of most significant byte = word address (xx00 = Big end of word), MIPS

- Little Endian: address of least significant byte = word address (xx00 = Little end of word), x86

Just for an example, do not take it for granted …

```
unsigned int i = 1;
char *c = (char*)&i;  // reading the LSB
Printf (“%d”, *c);
```

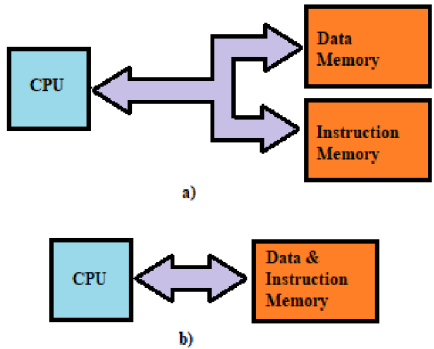
```
unsigned int i = 12345678;
char *c = (char*)&i;
Printf (“%d”, *c);
```

```
unsigned int i = 1;
char *c = (char*)&i;  // reading the LSB
Printf (“%d”, *c);
Little endian: 1
Big endian: 0
```

```
unsigned int i = 12345678;
char *c = (char*)&i;
Printf (“%d”, *c);
Little endian: 78
Big endian: 12
```

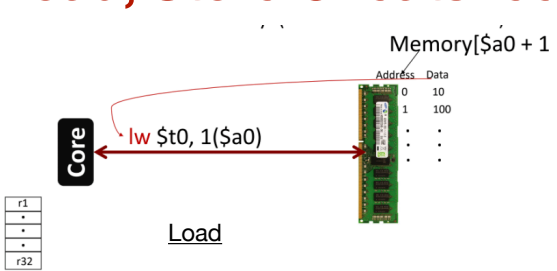
Another Important Point…

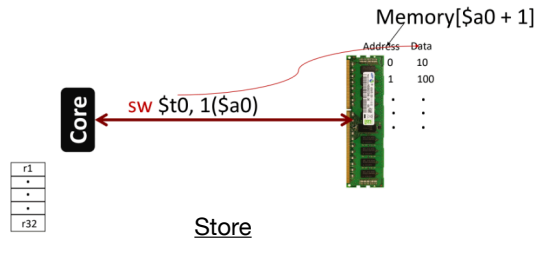
- Ok, **Von Neumann** said, data and code both are stored in the same memory.
  - In practice, this may lead to an issue — at a specific interval of time, **you can either fetch a data or an instruction**.
    - Affects parallelisation
- **What if you separate the data and instruction memory and buses?**
  - That is called **Harvard Architecture.**
  - Modern commercial systems use a combination of both
    - RAM stores both instruction and data
    - But there are other intermediate memory (**caches**) which are separated for instruction and data



Source: Internet

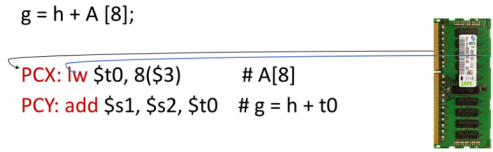
Load, Store Cheatsheet





Program Counter

Points to the next instruction in the memory to be fetched



Load+Store+Instruction-fetch

Summary…

- Data and instructions at the same place
- Registers are limited — 32 bit wide
- Instructions are 32 bit wide
- Registers are accessed by names
- Memory is accessed by addresses

Decision Making…

- If, else statements in your program…
  - How they are interpreted as instructions??

beq (branch equals to) and
bne (branch not equals to)

```
beq $t0, $t1, L1
bne $t0, $t1, L1
```

Decision Making…

beq $t0, $t1, L1

goto L1 (statements labeled as L1) if $t0 equals $t1

bne $t0, $t1, L1

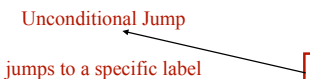
goto L1 (statements labeled as L1) if $t0 does not equal to $t1

Simple Example…

- Let's compile:
  - `if (i == j) f = g + h; else f = g - h;`

Assume:

`$s0` has `i`, `$s1` has `j`, `$s2` has `g`, `$s3` has `h`, `$s4` has `f`




```
        beq $s0, $s1, if_equal     # if i == j, jump to if_equal
        sub $s4, $s2, $s3          # else: f = g - h
        j end_if                   # jump to end
if_equal:
        add $s4, $s2, $s3          # f = g + h
end_if:
```

Decision Making…

- So you can check conditions:
  - `If (x = 0)..`
  - `If (x != 0)..`
  - `If (x = y)..`
  - `If (x != y)…`
- But how about the following code??

```
if (a < b)

  c=1
else
  c=0
```

Decision Making…

```
if (a < b)

  c=1
else
  c=0
```

- Set on less than (`slt`)
  - `slt  $t0, $s3, $s4  # $t0 = 1 if $s3 < $s4`

  `slti  $t0,$s2,10  # $t0 = 1 if $s2 < 10`

- After using `slt`, we can use the `beq` or `bne`

Simple Example…

- Let's compile:
  - `if (i < j) f = g + h; else f = g - h;`

Assume:

`$s0` has `i`, `$s1` has `j`, `$s2` has `g`, `$s3` has `h`, `$s4` has `f`

```
        slt $t0, $s0, $s1          # $t0 = 1 if i < j
        beq $t0, $zero, ELSE       # if $t0 == 0, i >= j, jump to ELSE
        add $s4, $s2, $s3          # f = g + h
        j END_IF                   # jump to END_IF
ELSE:
        sub $s4, $s2, $s3          # f = g - h
END_IF:
```

Dealing With Loops

- Let's first see how we deal with **arrays**…
  - `f = h + A[8]`

Assume:

- `$t0` has `A[8]`, `$s5` has base address of the array A, `$s4` has `f`, `$s3` has `h`
- Also assume "`A[8]`" as `uint8_t` (a byte)

```
lbu $t0, 8($s5)              # Load word A[8] with byte offset
add $s4, $s3, $t0            # f = h + A[8]
```

- But what is "`A[8]`" is `int (4 bytes)`?????

```
lw $t0, 32($s5)              # Load A[8], 8 * 4 = 32 (word)offset
add $s4, $s3, $t0            # f = h + A[8]
```

Dealing With Loops

- Let's consider:
  - `while (A[i] > k) i = i+1;`

Performs left logical shift by two bits..why??

Assume:

`$s0` has `i`,

`$t1` has address of `A[i]`

`$t2` has `A[i]`

`$s6` has `k`

```
LOOP:
    sll $t1, $s0, 2          # $s0 = i, i*4 for word offset
    add $t1, $s5, $t1        # Compute address A[i]
    lw $t2, 0($t1)           # Load A[i] (integer)
    slt $t3, $t2, $s6        # $t3 = 1 if A[i] < k
    bne $t3, $zero, END_LOOP # if A[i] < k, exit loop
    addi $s0, $s0, 1         # i = i + 1
    j LOOP
END_LOOP:
```

Dealing With Loops

- What happens if:
  - `while (A[i] == k) i = i+1;`

Assume:

`$s0` has `i`,

`$t1` has address of `A[i]`

`$t2` has `A[i]`

`$s6` has `k`

Dealing With Loops

- What happens if:
  - `while (A[i] == k) i = i+1;`

Assume:

`$s0` has `i`,

`$t1` has address of `A[i]`

`$t2` has `A[i]`

`$s6` has `k`

```
    LOOP:
     sll $t1, $s0, 2          # $s0 = i, i*4 for word offset
     add $t1, $s5, $t1        # Compute address A[i]
     lw $t2, 0($t1)           # Load A[i] (integer)
     bne $t2, $s6, END_LOOP   # if A[i] != k, exit loop
     addi $s0, $s0, 1         # i = i + 1
     j LOOP
END_LOOP:
```

More on Jumping…

- What happens if:
  - `while (A[i] == k) i = i+1;`

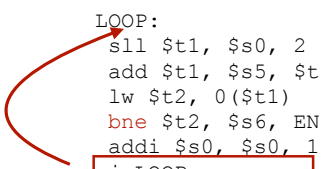
Assume:

`$s0` has `I`,

`$t1` has address of `A[i]`

`$t2` has `A[i]`

`$s6` has `k`



```
LOOP:
    sll $t1, $s0, 2          # $s0 = i, i*4 for word offset
    add $t1, $s5, $t1        # Compute address A[i]
    lw $t2, 0($t1)           # Load A[i] (integer)
    bne $t2, $s6, END_LOOP   # if A[i] != k, exit loop
    addi $s0, $s0, 1         # i = i + 1
    j LOOP
END_LOOP:
```

- Normally:
  - PC, PC+4, PC+8,….
  - But jump instruction loads a new value to the PC
    - It's the offset in the program where the exception should divert (the label is basically that)

More on Jumping…

- What happens if:
  - `while (A[i] == k) i = i+1;`

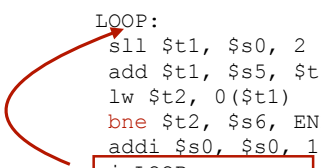
Assume:

`$s0` has `I`,

`$t1` has address of `A[i]`

`$t2` has `A[i]`

`$s6` has `k`



```
LOOP:
    sll $t1, $s0, 2          # $s0 = i, i*4 for word offset
    add $t1, $s5, $t1        # Compute address A[i]
    lw $t2, 0($t1)           # Load A[i] (integer)
    bne $t2, $s6, END_LOOP   # if A[i] != k, exit loop
    addi $s0, $s0, 1         # i = i + 1
    j LOOP
END_LOOP:
```

- Normally:
  - PC, PC+4, PC+8,….
  - But jump instruction loads a new value to the PC
    - It's the offset in the program where the exception should divert (the label is basically that)
  - But jumping is even more exotic…Let's see why

More on Jumping…

More on Jumping…Working with Functions

```
int sum(int a, int b)
{
    int c=a+b;
    return c;
}
void main (void)
{
    int i=1;
    int j=2;
    int k = sum(i,j);
   // …..
}
```

Function call jumps to a location in your code

- **Caller**: One who calls the function
- **Callee**: The function which is being called

- **Anatomy of a Function Call**:
  - Put parameters in a place where the function can access them.
  - Transfer control to the function.
  - Acquire the storage resources needed for the function.
  - Perform the desired task.
  - Put the result value in a place where the caller program can access it.
  - Return control to the point of origin, since a function can be called from several points in a program.

Working with Functions — The MIPS Case

- **MIPS Support for Function Call**:
  - `$a0–$a3`: four argument registers in which to pass parameters
  - `$v0–$v1`: two value registers in which to return values
  - `$ra`: one **return address** register to return to the point of origin
- **Ways of Jumping..**:
  - `jal Label`: Jump and link
  - `jr $ra`: Jump back to the return address stored in `$ra`

Working with Functions — The MIPS Case

- **Ways of Jumping..**:
  - `jal Label`:
    - First, save `PC+4` in `$ra`
    - The instruction to be executed next is at `Label`
  - `jr $ra`: Jump back to the return address stored in `$ra` `(PC + 4)`

Working with Functions — The MIPS Case

Complete Picture

```
int sum(int a)
{
    int c=a+4;
    return c;
}
void main (void)
{
    int i=2;
    int k = sum(i);
}
```

```
sum:
 PC+100: addi $v0, $a0, 4      # c = a + 4, return in $v0
 PC+104: jr $ra                # return to PC+12

main:
 PC+4:  li $a0, 2              # i = 2
 PC+8:  jal sum                # call sum(i); $ra = PC+12
 PC+12: addi $s1, $v0, 0       # k = return value (k = 6)
```

Working with Functions — The MIPS Case

- **MIPS Support for Function Call**:
  - `$a0–$a3`: four argument registers in which to pass parameters
  - `$v0–$v1`: two value registers in which to return values
  - `$ra`: one **return address** register to return to the point of origin
- **Ways of Jumping..**:
  - `jal Label`: Jump and link
  - `jr $ra`: Jump back to the return address stored in `$ra`

Functions — More Parameters

- What if there are more than 4 parameters?
- What if the function needs more registers (beyond two return registers) to operate?
- **Remember**: caller must have its state restored after callee finishes.

```
int leaf_example (int g, int h, int i, int j, int k)
{
    int f;
    f = (g + h) - (i + j) + k;
    return f;
    }

    void main (void)
{
    int g=2;
    int h=3;
    int i=1;
    int j=1;
    int k=3;
    int l = leaf_example(g,h,i,j,k);
}
```

Functions — More Parameters

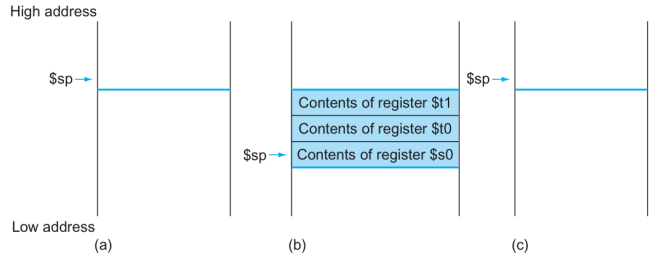
```
int leaf_example (int g, int h, int i, int j,
int k)
{
    int f;
    f = (g + h) - (i + j) + k;
    return f;
    }

    void main (void)
{
    int g=2;
    int h=3;
    int i=1;
    int j=1;
    int k=3;
    int l = leaf_example(g,h,i,j,k);
}
```

What is this trick?

```
leaf_example:
    addi $sp, $sp, -8          # make space for s0, t0
    sw $s0, 4($sp)             # save s0
    sw $t0, 0($sp)             # save t0

    add $s0, $a0, $a1          # s0 = g + h
    add $t0, $a2, $a3          # t0 = i + j
    sub $v0, $s0, $t0          # v0 = (g + h) - (i + j)
    add $v0, $v0, $t2          # v0 += k (f = ... + k)

    lw $t0, 0($sp)             # restore t0
    lw $s0, 4($sp)             # restore s0
    addi $sp, $sp, 8           # deallocate stack
    jr $ra                     # return
```

Stack — What is it?

- A region in DRAM
- Grows in the lower direction
- Push and Pop are the main operations
- **Stack pointer**: A special register
  - A value denoting the most recently allocated address in a stack that shows where registers should be spilled or where old register values can be found.



Functions — More Parameters

- **MIPS Support for Function Call**:
  - We need registers beyond `$a3`
    - Use other registers `$s0`, `$t0`
    - But the caller function might be already have some values in them.
    - **Solution**: store them in memory (stack) before using in the function.
      - Restore back before exiting
  - This idea of saving registers is called **register spilling**

What is this trick?

```
leaf_example:
    addi $sp, $sp, -8          # make space for s0, t0
    sw $s0, 4($sp)             # save s0
    sw $t0, 0($sp)             # save t0

    add $s0, $a0, $a1          # s0 = g + h
    add $t0, $a2, $a3          # t0 = i + j
    sub $v0, $s0, $t0          # v0 = (g + h) - (i +
j)
    add $v0, $v0, $t2          # v0 += k (f = ... + k)

    lw $t0, 0($sp)             # restore t0
    lw $s0, 4($sp)             # restore s0
    addi $sp, $sp, 8           # deallocate stack
    jr $ra                     # return
```

Nested Procedures

```
main(){
a = a + f1(a);
}
f1(a) {
        a = a -  f2(a);  return a;}
f2(a) {
        a = a + f3(a); return a;}
f3(a) {
        a = a + 1;       return a;}
```

```
f1:
    f2's argument in $a0 to $a3
            jal f2
```

Nested Procedures

f1:

f2's argument in `$a0` to `$a3`

jal f2

…

f2:

f3's argument in `$a0` to `$a3`

jal f3

…

Nested Procedures

f1:

PC: f2's argument in `$a0` to `$a3`

PC+4: jal f2 // $ra = PC+8

…

f2:

PC+100: f3's argument in `$a0` to `$a3`

PC+104: jal f3 // `$ra` = PC+108

...

f3: …

```
jr $ra
```

Nested Procedures

f1:

PC: f2's argument in $a0 to $a3

PC+4: jal f2 // $ra = PC+8

…

f2:

PC+100: f3's argument in $a0 to $a3

PC+104: jal f3 // $ra = PC+108

jr $ra ☹ Oh no!!

...

f3: …

jr $ra

Nested Procedures

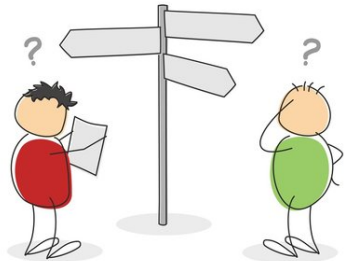
caller registers

callee registers

Why?

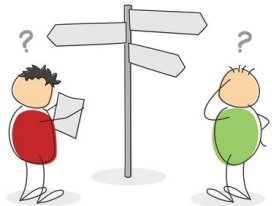
Callee does not know, registers used by callers, can be many callers too

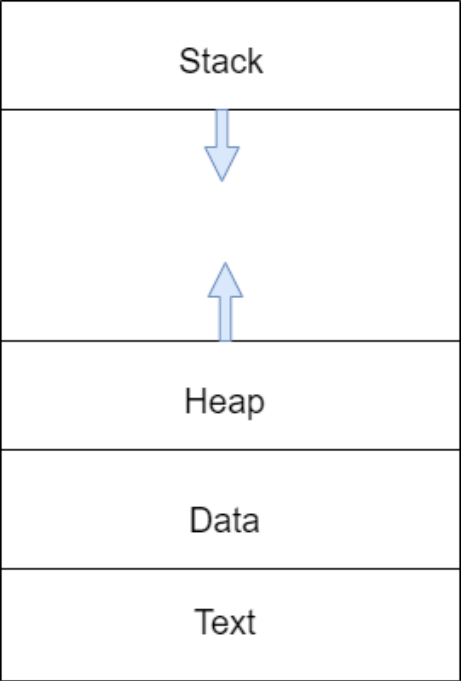
Caller does not know the callee's plan ☺



What to do?

- Push all the other registers that must be preserved onto the stack
  - Caller pushes any argument registers (`$a0-$a3`) or temporary registers (`$t0-$t9`) that are needed after the call.
  - Callee pushes the return address register `$ra` and any saved registers (`$s0-$s7`) used by the callee.
  - The $sp is adjusted to account for the number of registers put on the stack.
  - Everything is restored after the call

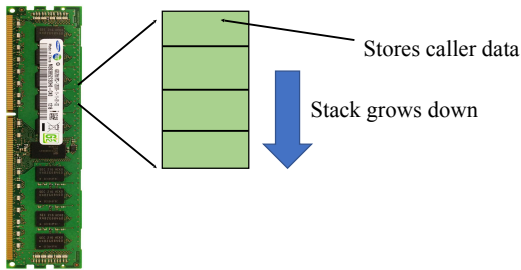




The loaded program

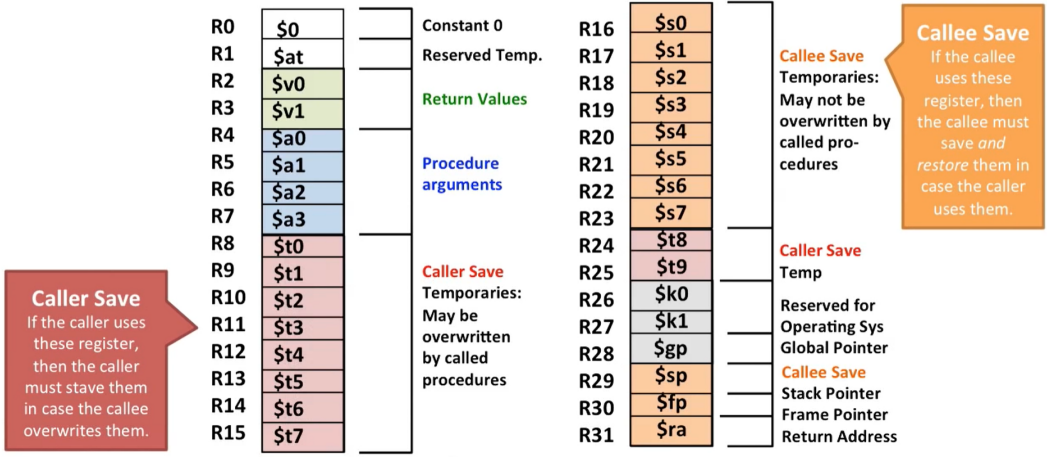
- System program that loads the executable into the memory.
- Every executable has a text, heap/stack data segments

MIPS way of handling it:
The Stack (part of DRAM, for each function call)

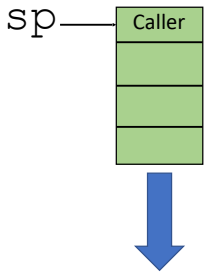


`$sp` (stack pointer) points to the address where stack ends

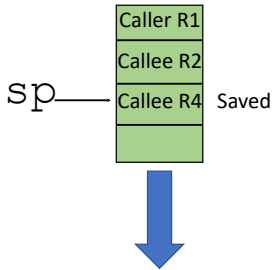
One per function, private memory area, else the same problem ☹


R0 $0
R1 $at
R2 $v0
R3 $v1
R4 $a0
R5 $a1
R6 $a2
R7 $a3
R8 $t0
R9 $t1
R10 $t2
R11 $t3
R12 $t4
R13 $t5
R14 $t6
R15 $t7
Constant 0
Reserved Temp.
Return Values
Procedure arguments
Caller Save Temporaries: May be overwritten by called procedures
Caller Save
If the caller uses these register, then the caller must stave them in case the callee overwrites them.
R16 $s0
R17 $s1
R18 $s2
R19 $s3
R20 $s4
R21 $s5
R22 $s6
R23 $s7
R24 $t8
R25 $t9
R26 $k0
R27 $k1
R28 $gp
R29 $sp
R30 $fp
R31 $ra
Callee Save Temporaries: May not be overwritten by called pro-cedures
Caller Save Temp
Reserved for Operating Sys
Global Pointer
Callee Save
Stack Pointer
Frame Pointer
Return Address
Callee Save
If the callee uses these register, then the callee must save and restore them in case the caller uses them.

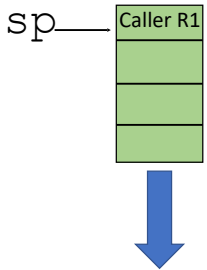
MIPS way of handling it: Before function call



MIPS way of handling it: Function call is ON

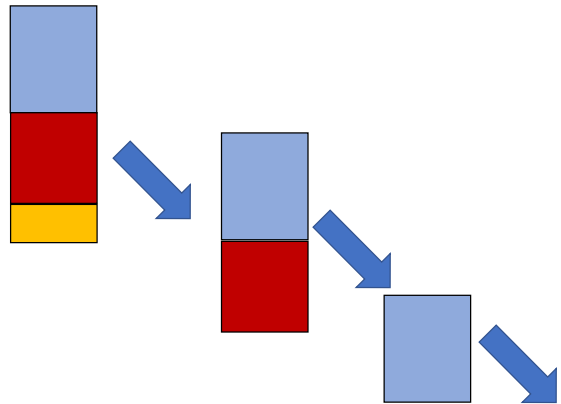


MIPS way of handling it: After the function call



Nested Functions (Remember main() is a function too ☺)

```
CS230 // jal cs230
{
    CS330 // jal cs330
      {
         CS430 // jal cs430
          {
          } //jr
       } //jr
} // jr
```



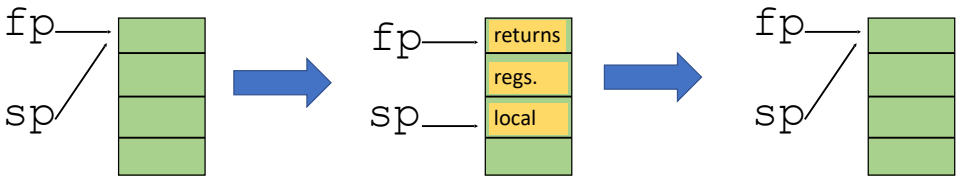
The final one: Frame pointer

- Stack also stores local variables and data structures (local arrays and structures) for a function along with the return address(es).

- Frame pointer ($fp) will get incremented and decremented based on the local arguments used.

The final one: Frame pointer

Frame pointer: Points to local variables and saved registers. Points to the highest address in the function frame. Stays there throughout the procedure. Stack pointer, moves around.



A Complete Example: Recursion

```
int fact (int n) {
  if (n < 1) return (1);
  else return (n * fact(n - 1)); }
```

```
fact:
    addi $sp, $sp, -8          # make space for ra and n
    sw $ra, 4($sp)             # save return address
    sw $a0, 0($sp)             # save n

    slti $t0, $a0, 1           # t0 = 1 if n < 1
    beq $t0, $zero, RECURSE    # if n >= 1, do recursion

    li $v0, 1                  # base case: return 1
    addi $sp, $sp, 8           # deallocate stack
    jr $ra                     # return

RECURSE:
    addi $a0, $a0, -1          # n - 1
    jal fact                   # recursive call

    lw $a0, 0($sp)             # restore n
    mul $v0, $a0, $v0          # n * fact(n - 1)
    lw $ra, 4($sp)             # restore return address
    addi $sp, $sp, 8           # deallocate stack
    jr $ra                     # return
```

1

A Complete Example: Recursion

```
int fact (int n) {
  if (n < 1) return (1);
  else return (n * fact(n - 1)); }
```

```
fact:
1   addi $sp, $sp, -8          # make space for ra and n
    sw $ra, 4($sp)             # save return address
    sw $a0, 0($sp)             # save n

    slti $t0, $a0, 1           # t0 = 1 if n < 1
    beq $t0, $zero, RECURSE    # if n >= 1, do recursion

    li $v0, 1                  # base case: return 1
    addi $sp, $sp, 8           # deallocate stack
    jr $ra                     # return

RECURSE:
2   addi $a0, $a0, -1          # n - 1
    jal fact                   # recursive call

    lw $a0, 0($sp)             # restore n
    mul $v0, $a0, $v0          # n * fact(n - 1)
    lw $ra, 4($sp)             # restore return address
    addi $sp, $sp, 8           # deallocate stack
    jr $ra                     # return
```

A Complete Example: Recursion

```
int fact (int n) {
  if (n < 1) return (1);
  else return (n * fact(n - 1)); }
```

```
fact:
1   addi $sp, $sp, -8          # make space for ra and n
    sw $ra, 4($sp)             # save return address
    sw $a0, 0($sp)             # save n

    slti $t0, $a0, 1           # t0 = 1 if n < 1
    beq $t0, $zero, RECURSE    # if n >= 1, do recursion

3   li $v0, 1                  # base case: return 1
    addi $sp, $sp, 8           # deallocate stack
    jr $ra                     # return

RECURSE:
2   addi $a0, $a0, -1          # n - 1
    jal fact                   # recursive call

    lw $a0, 0($sp)             # restore n
    mul $v0, $a0, $v0          # n * fact(n - 1)
    lw $ra, 4($sp)             # restore return address
    addi $sp, $sp, 8           # deallocate stack
    jr $ra                     # return
```

A Complete Example: Recursion

```
int fact (int n) {
  if (n < 1) return (1);
  else return (n * fact(n - 1)); }
```

- Do you notice something unusual??

```
fact:
1   addi $sp, $sp, -8          # make space for ra and n
    sw $ra, 4($sp)             # save return address
    sw $a0, 0($sp)             # save n

    slti $t0, $a0, 1           # t0 = 1 if n < 1
    beq $t0, $zero, RECURSE    # if n >= 1, do recursion

3   li $v0, 1                  # base case: return 1
    addi $sp, $sp, 8           # deallocate stack
    jr $ra                     # return

RECURSE:
2   addi $a0, $a0, -1          # n - 1
    jal fact                   # recursive call

    lw $a0, 0($sp)             # restore n
4   mul $v0, $a0, $v0          # n * fact(n - 1)
    lw $ra, 4($sp)             # restore return address
    addi $sp, $sp, 8           # deallocate stack
    jr $ra                     # return
```

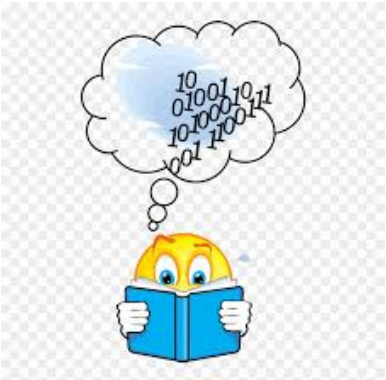
A Complete Example: Recursion

```
int fact (int n) {
  if (n < 1) return (1);
  else return (n * fact(n - 1)); }
```

- Can you tell me why there I do not `lw` the `$ra` in the base case???
  - `$ra` does not change in the base case

```
fact:
1   addi $sp, $sp, -8          # make space for ra and n
    sw $ra, 4($sp)             # save return address
    sw $a0, 0($sp)             # save n

    slti $t0, $a0, 1           # t0 = 1 if n < 1
    beq $t0, $zero, RECURSE    # if n >= 1, do recursion

3   li $v0, 1                  # base case: return 1
    addi $sp, $sp, 8           # deallocate stack
    jr $ra                     # return

RECURSE:
2   addi $a0, $a0, -1          # n - 1
    jal fact                   # recursive call

    lw $a0, 0($sp)             # restore n
4   mul $v0, $a0, $v0          # n * fact(n - 1)
    lw $ra, 4($sp)             # restore return address
    addi $sp, $sp, 8           # deallocate stack
    jr $ra                     # return
```

Interpreting Instructions

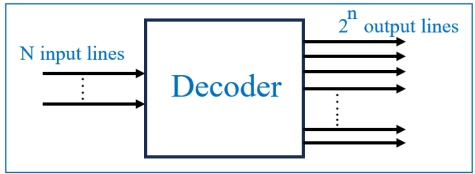


Why to Interpret?

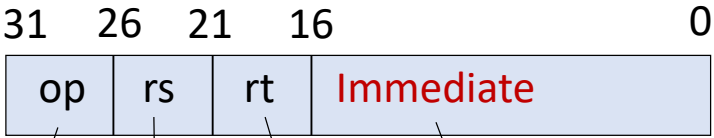
- Everything is a string of bits…
- Instructions are 32 bit strings
- How do the hardware know what is `$lw` and what is `$addi`
  - Unique bit string for each of them
- But how does the machine interpret them?

Decode it …

- Everything is a string of bits…
- Instructions are 32 bit strings
- How do the hardware know what is `$lw` and what is `$addi`
  - Unique bit string for each of them
- But how does the machine interpret them?



Instruction Decoding



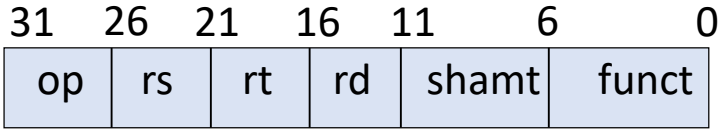
6 bits, maximum
64 operations
add
sub
mul
lw
bne …….

5 bits, 32 registers
(R0 to R31)

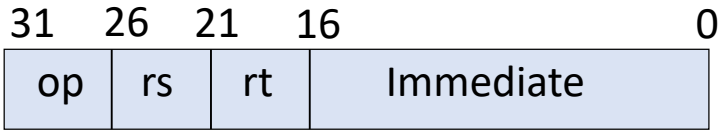
5 bits, 32 registers (R0 to R31)

16 bit constant

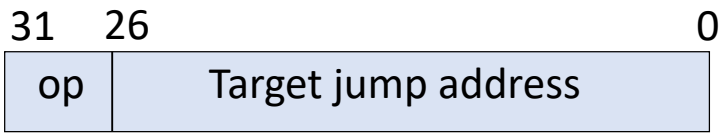
Let's have a look



R-type



I-type



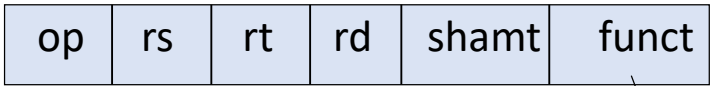
J-type

Good design demands good compromises

Instruction	Format	op	rs	rt	rd	shamt	funct	address
add	R	0	reg	reg	reg	0	32	n.a.
sub	R	0	reg	reg	reg	0	34	n.a.
addi	I	8	reg	reg	n.a.	n.a.	n.a.	constant
lw	I	35	reg	reg	n.a.	n.a.	n.a.	address
sw	I	43	reg	reg	n.a.	n.a.	n.a.	address

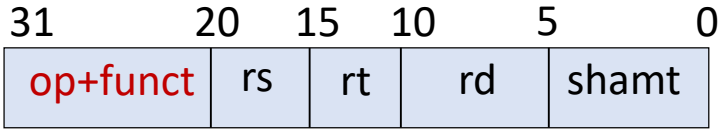
tells how to treat the last set of fields:
three fields or one field, still why funct ☹

MIPS encoding Anatomy

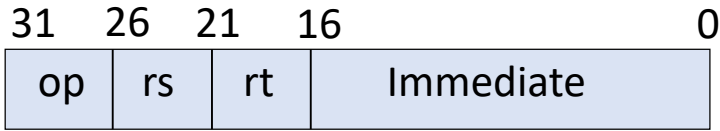


Why this field?
Wastage of space ☹

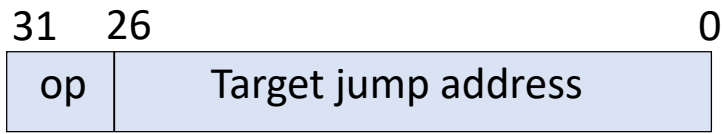
Why not?



R-type



I-type



J-type

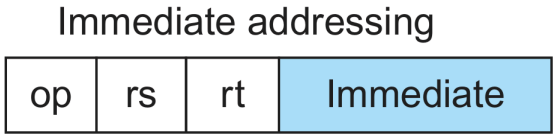
Potential Causes: Simple is The Best

- Every instruction looks the same
- Same number of bits for the **op** for all the instruction
- All the fields starts from the same location
- Uniform treatment for all instructions make the decoder hardware simple:
  - Believe me, you can save some MUXs

Addressing Modes

- Where is your data??

Addressing Modes

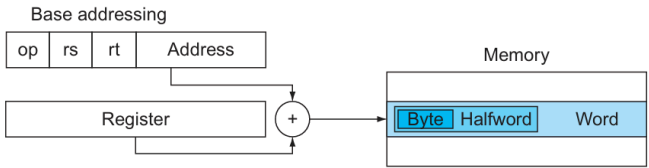


- Data is packed within instruction
- `addi $t0, $t0, 3`

Addressing Modes

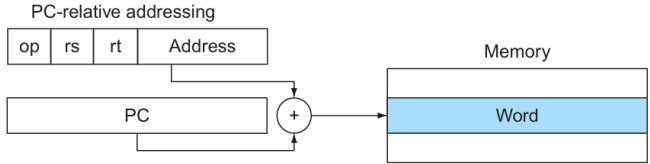
- Data is inside a register
- `add $t0, $t1, $s0`

Addressing Modes



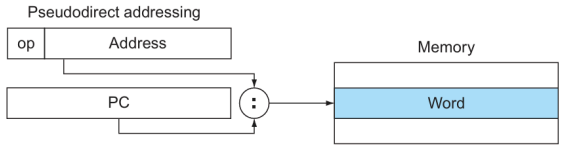
- Data is in memory
- Address is the sum of a register value + offset specified in the instruction.
- `lw $t0, 4($s0), lw $t0,($s0)`
- `lw/sw` have many versions for accessing byte halfword and word.
- Great for accessing arrays, structures, and with pointers

Addressing Modes



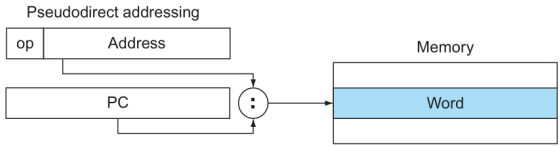
- Conditional branches
- The address field basically specifies an offset
- Adds a 16-bit address (sign extended to 26 bit and shifted left by 2 bits) to the PC
- `beq $t0, $zero, RECURSE`

Addressing Modes



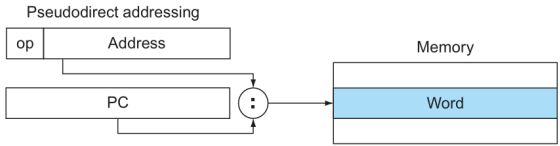
- Again for branching
- `jal label`
- Concatenates a 26-bit address shifted left 2 bits with the 4 upper bits of the PC
  - Why?

Addressing Modes



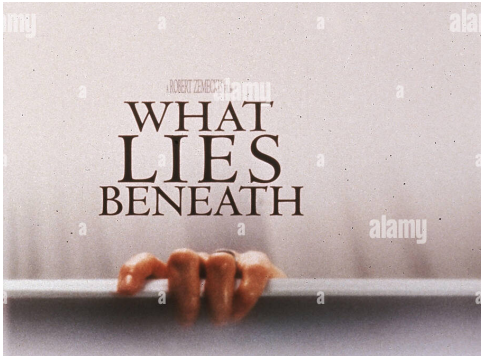
- Again for branching
- `jal label`
- Concatenates a 26-bit address shifted left 2 bits with the 4 upper bits of the PC
  - Why?
  - Remember word alignment of memory??????
    - 26 shifted by 2 bit is 28 bits with two zeros at the end — always divisible by 4
    - You can address 2^{28} bits = 256 MB
    - Add the upper 4 bits of PC — can access several 256 MB chunks..

Addressing Modes



- Again for branching
- `jal label`
- Concatenates a 26-bit address shifted left 2 bits with the 4 upper bits of the PC
  - Why?
  - Remember word alignment of memory??????
    - 26 shifted by 2 bit is 28 bits with two zeros at the end — always divisible by 4
    - You can address 2^{28} bits = 256 MB
    - Add the upper 4 bits of PC — can access several 256 MB chunks..

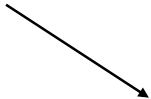
ISA Vs. Microarchitecture



ISA Vs. Microarchitecture

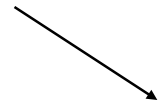
ISA

- What a programmer gets to see
- `add`, `lw`



Microarchitecture

- How the instructions are implemented
- How the `add` is implemented — ripple carry/carry-save



Hardware Design

- How to implement the ripple carry with gates
- Deals with gates, flip-flops

ISA Vs. Microarchitecture

ISA: Programmer's View

- Opcodes
- Registers
- Addressing modes
- Alignment
- Instruction types/formats
- Access control user/OS
- Address space

ISA is satisfies the need of assembly level programmers, compilers, assemblers

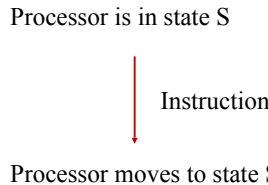
Microarchitecture

- Pipelining
- Caches
- Branch Predictors
- Prefetches
- Memory controllers

Microarchitecture is: how to bring speed..

Microarchitecture State

The information held in the processor at the end of an instruction to provide the processing context for the next instruction.



ISA Vs. Microarchitecture

ISA: Programmer's View

- Opcodes
- Registers
- Addressing modes
- Alignment
- Instruction types/formats
- Access control user/OS
- Address space

ISA is satisfies the need of assembly level programmers, compilers, assemblers

Microarchitecture

- Pipelining
- Caches
- Branch Predictors
- Prefetches
- Memory controllers

Microarchitecture is: how to bring speed..

ISA Vs. Microarchitecture

ISA: Responsibility of the Designer

- Register — ISA
- Memory access instruction — ISA

Microarchitecture: Responsibility of the Designer

- Cycles to access the register — Microarch
- Cycles to access memory — Microarch

Computer Architecture

ISA+ Microarchitecture

Design Challenges

- Closer to software
  - complex instructions — e.g., instruction for GCD
  - Easy for programmers (perhaps..)

Design Challenges

- Closer to software
  - complex instructions — e.g., instruction for GCD
  - Easy for programmers (perhaps..)
  - Complex microarchitecture
  - Complex hardware design

- Closer to hardware
  - Simple instructions
  - More instructions in a program
  - Simpler and clean hardware.

Design Challenges

- Closer to software
  - complex instructions — e.g., instruction for GCD
  - Easy for programmers (perhaps..)
  - Complex microarchitecture
  - Complex hardware design

- Closer to hardware
  - Simple instructions
  - More instructions in a program
  - Simpler and clean microarchitecture and hardware.

The RISC CISC Debate

The RISC CISC Debate

- **RISC:** Reduced Instruction Set Computer — MIPS, RISC-V
- **CISC:** Complex Instruction Set Computer — X86 [actually not fully]

- Why CISC:
  - Implementing commonly used instructions in hardware can lead to significant performance benefits.
  - For example, use of string instructions had significant performance benefits.

- Why RISC:
  - The rarely used instructions can be eliminated to save chip space --- on chip cache and large number of registers can be provided.

The RISC CISC Debate

- Rich instruction set:
  - Some simple, some very complex
- Complex addressing modes:
  - Orthogonal addressing (Every possible addressing mode for every instruction).
- Many instructions take multiple cycles:
- Instructions are of variable sizes
- Microcode control

The RISC CISC Debate

- Direct (absolute)
- Register indirect
- Indexed
- Scaled
- Autoincrement
- Autodecrement
- Memory indirect

```
ADD R1, (1001)
SUB R2, (R1)
ADD R1, (R2 + R3)
SUB R2, 100(R2)[R3]
ADD R1, (R2)+
SUB R2, -(R1)
ADD R1, @(R3)
```

And more …

The RISC CISC Debate— X86

12 addressing modes:

- Register.
- Immediate.
- Direct.
- Base.
- Base + Displacement.
- Index + Displacement.
- Scaled Index + Displacement.
- Based Index.
- Based Scaled Index.
- Based Index + Displacement.
- Based Scaled Index + Displacement.
- Relative.

Operand sizes:

- Can be 8, 16, 32, 48, 64, or 80 bits long.
- Also supports string operations.

Instruction Encoding:

- The smallest instruction is one byte.
- The longest instruction is 12 bytes long.
- The first bytes generally contain the opcode, mode specifiers, and register fields.
- The remainder bytes are for address displacement and immediate data.

The RISC CISC Debate— X86

- One instruction could do the work of several instructions.
  - For example, a single instruction could load two numbers to be added, add them, and then store the result back to memory directly.

- Many versions of the same instructions were supported:
  - For example, one version would read two numbers from memory, and store the result in a register. Another version would read one number from memory and the other from a register and store the result to memory.

The RISC CISC Debate

- Compilers were hard to build especially for machines with registers
  - Make machine do more work than software
  - Have instructions load and store directly to memory (memory-to-memory operations)
- Magnetic core memory was used as main memory which was slow and expensive
  - Minimize assembly code size
  - **Complex Instruction Set Computers (CISC)**

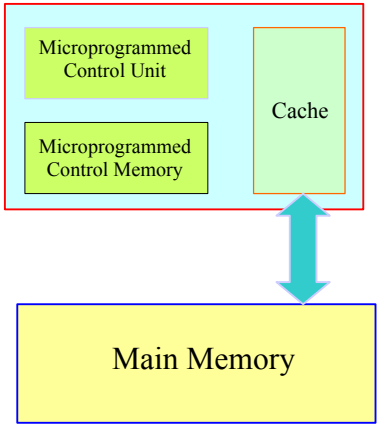
The RISC CISC Debate

- Compilers were improving:
  - Optimizing compilers could effectively use registers
- Caches:
  - Allowed main memory to be accessed at similar speeds to control memory
- Semiconductor memory was replacing magnetic core memory:
  - Reduced performance gap between control and main memory

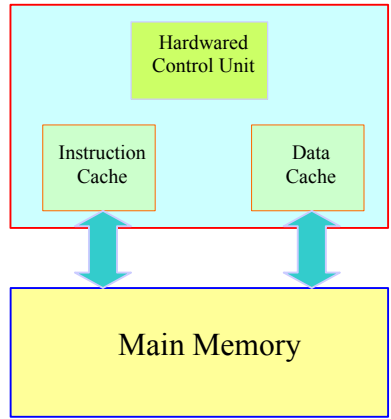
The RISC CISC Debate — RISC

- Small number of instructions
- Small number of addressing modes
- **Large number of registers** (>32)
- Instructions execute in one or two clock cycles
- Uniformed length instructions and fixed instruction format.
- **Register-Register Architecture:**
  - Separate memory instructions (load/store)
- Separate instruction/data cache
- **Hardwired control**
- **Pipelining**

The RISC CISC Debate



(a) CISC Organization

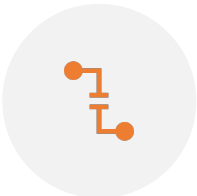


(b) RISC Organization

Implementation Issues

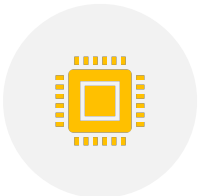
- A CISC processor typically devotes about half of its area to the control unit.
  - A RISC processor typically uses only about 10% of the area for the control unit, devoting to registers instead.
- Design and implementation time.
  - The simple control unit and circuitry of RISC result in faster design cycles.

Mystery



INTEL *CONVERTS* CISC ONES INTO RISC ONES, AND GENERATE MICROOPERATIONS.

INTELLIGENT CISC-RISC DECODER



CONSUMES AROUND 2% OF THE CHIP AREA. GOOD OR BAD?

How Easy?

A billion-dollar idea ☺

i) requires changes to microarchitecture.
ii) requires changes to ISA.
iii) both (i) and (ii)

Think about the trade-offs ☺
Does it affect the system stack?

The Other ISAs

World of ISAs

x86: Intel, AMD: Laptops, Desktops, Servers

ARM: Arm, Qualcomm, Apple, Samsung: Mobiles

btw ARM: Advanced RISC Machines ☺

RISC-V: Open-source ISA, what does it mean?

RISC-V, opensource

What is the license model?

The RISC-V ISA is free and open with a permissive license for use by anyone in all types of implementations. Designers are free to develop proprietary or open source implementations for commercial or other exploitations as they see fit. RISC-V International encourages all implementations that are compliant to the specifications.

Note that the use of the RISC-V trademark requires a license which is granted to members of RISC-V International for use with compliant implementations. The RISC-V specification is based around a structure which allows flexibility with modular extensions and additional custom instructions/extensions. If an implementation was based on the RISC-V specification but includes modifications beyond this framework, then it cannot be referenced as RISC-V.

Does that mean free for industry to use and play with, but then we pay if we produce a product using this ISA?

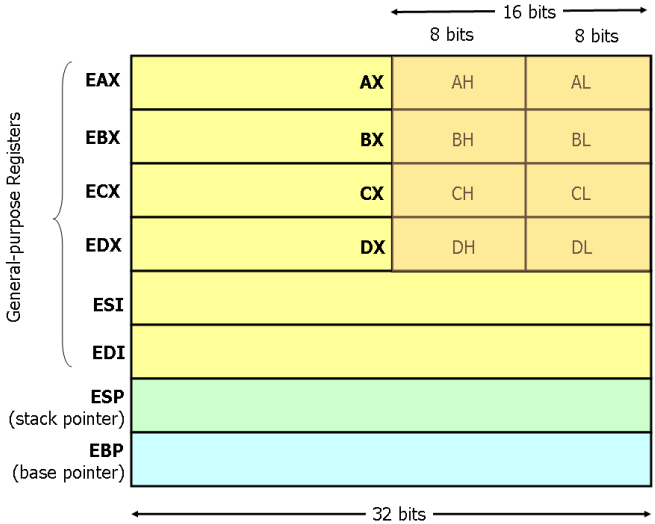
The RISC-V ISA is free for product use too. Those who want to use the RISC-V logo should join RISC-V International (see question No. 1).

If our company builds a RISC-V implementation, is it required to release its source code for the RISC-V core?

No, the source code can be completely closed.

x86 Registers: 80386

For 64-bits, rax, rbx



Subtle Differences

- x86 arithmetic/logic instructions: one operand should act as both source and destination

add $s0, $s1 // Add $s0 and $s1 and put it in $s0 ☺

- One of the operands can be in memory:

wow (programmer) or oh no (instruction size ☹) !!

add $s0, Mem[$s1] ☺

- No more fixed-length instructions ☹ can be 4/8/X bytes

Why?

Fixed Width or Variable Width

Variable: No fixed size

6 bytes for add, 2 bytes for load

Smaller code footprint (compact)

Fixed:

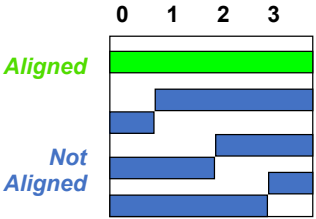
4 bytes for all, Larger code footprint, simple decoding

Fallacies

CISC instructions provide higher performance

Assembly language codes provide higher performance

Instruction Alignment: Why we need it?



Aligned:

x-byte access starting from an address y: y % x must be zero.

MIPS vs X86

MIPS does not allow unaligned accesses

x86 does not enforce alignment ☺

Whose job is to generate aligned/unaligned accesses?

MIPS vs X86

MIPS does not allow unaligned accesses

x86 does not enforce alignment ☺

Whose job is to generate aligned/unaligned accesses?

Compiler

Let’s go a bit deeper

Object of size s bytes at byte add. A is aligned if A mod s = 0

Alignment for faster transfer of data ?

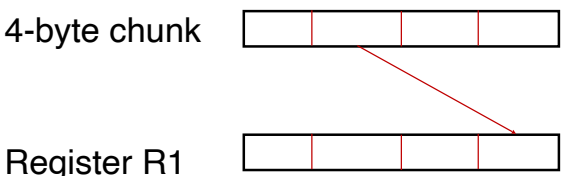
Why fast ??

Think about memory (caches if you know).

Memory operations and alignment network

LOADs and STOREs need an alignment network that makes sure data loaded/written are aligned.

lb R1, 1($s3)



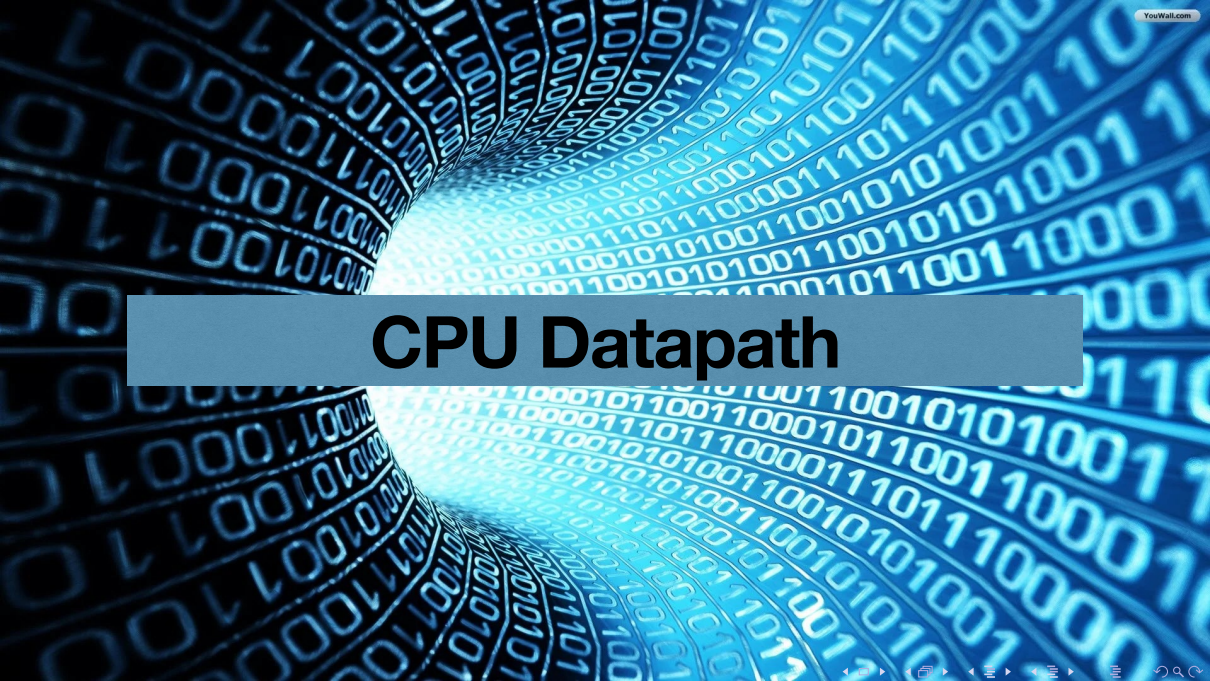
For the Curious ones

https://lemire.me/blog/2012/05/31/data-alignment-for-speed-myth-or-reality/

Digital Logic Design + Computer Architecture

Sayandeep Saha

Assistant Professor
Department of Computer Science and Engineering
Indian Institute of Technology Bombay



CPU Datapath

What is “Datapath” for a Processor?

- Same as the datapath control split we studied for GCD processor.
  - **Datapath processes the data**
  - **Controller tells how to process the data**
- But here it’s for processing the instructions in an ISA
- A datapath that can process all the instructions in an ISA

Image generated by ChatGPT

Datapath Elements

- We shall begin with the simplest case
  - Every instruction finishes in **one clock cycle**
  - The clock frequency is determined by the instruction finishes last
- Remember, again how we constructed the GCD datapath
- Let's first identity what are the components needed

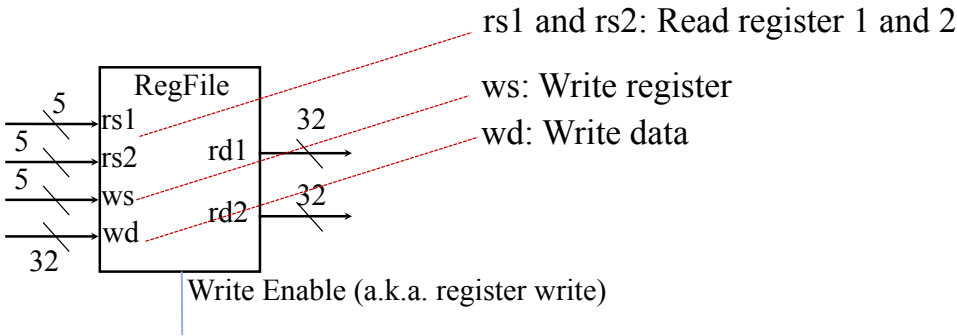
Image generated by ChatGPT

Datapath Elements

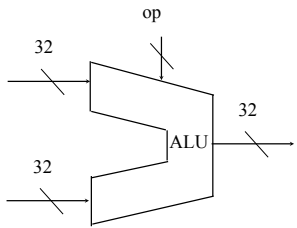
- Remember, again how we constructed the GCD datapath
- Let's first identity what are the components needed

Image generated by ChatGPT

Datapath Elements: Register File

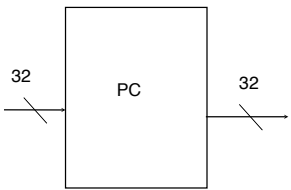


Datapath Elements: The ALU



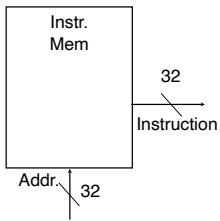
- All arithmetic and logical operation happens here
- Operation selection
- E.g.,
  - Op = 0: Y = A+B
  - OP = 1: Y = A- B
  - OP = 2; Y = A*B

Datapath Elements: Program Counter



- Remember PC register??
- It always points to the next instruction to be executed…
- But where is the next instruction???

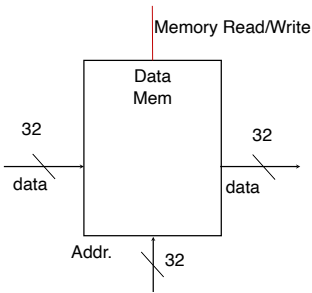
Datapath Elements: Instruction Memory



Remember: No writes to instruction memory

Not concerned about how programs are loaded into this memory.

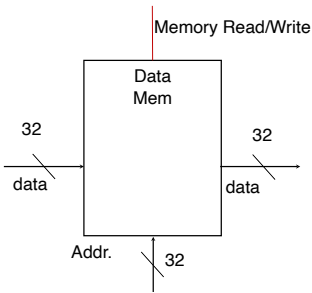
Datapath Elements: Data Memory



Why data and instruction memory and not one memory?

- Recall Harvard vs. Von Neuman
- Not so simple matter, will discuss later.

Datapath Elements: Data Memory



Why data and instruction memory and not one memory?

- Recall Harvard vs. Von Neuman
- Not so simple matter, will discuss later.

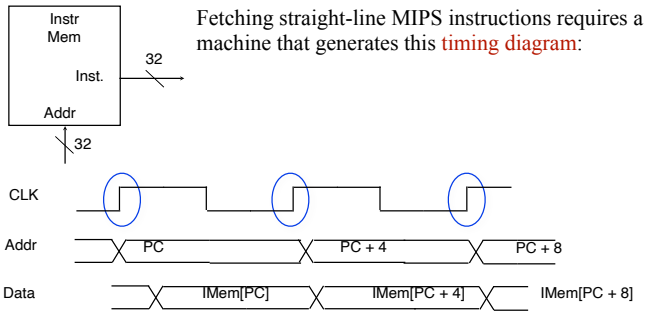
Datapath Elements: Buses

- Same as your public transport
- Transfer data and instructions
- Separate buses for address and data

Datapath Elements: Buses

- Same as your public transport
- Transfer data and instructions
- Separate buses for address and data
  - Why???

Datapath Elements: Timing Diagram

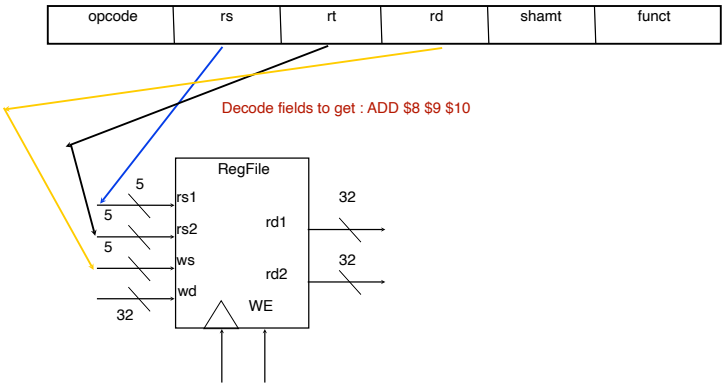


Fetching straight-line MIPS instructions requires a machine that generates this timing diagram:

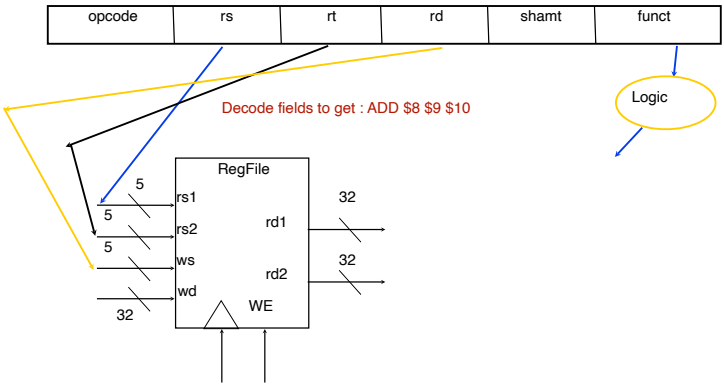
PC == Program Counter, points to next instruction.

- Every clock cycle we process one instruction (super simple)
- Sending the PC and getting the instruction from memory is called **Instruction Fetch**

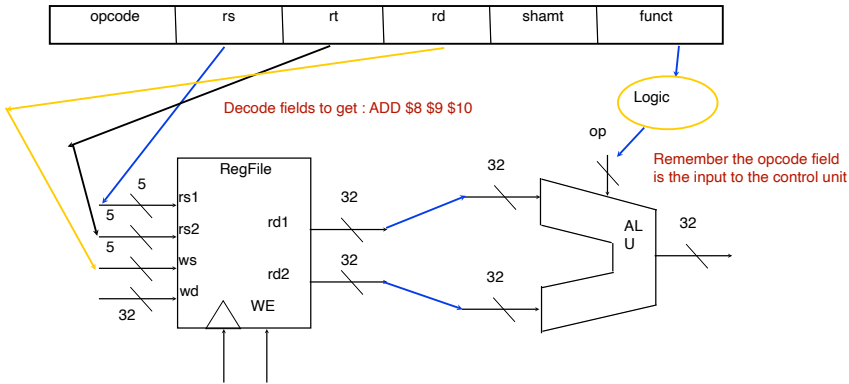
Datapath Elements: Decoding Instructions



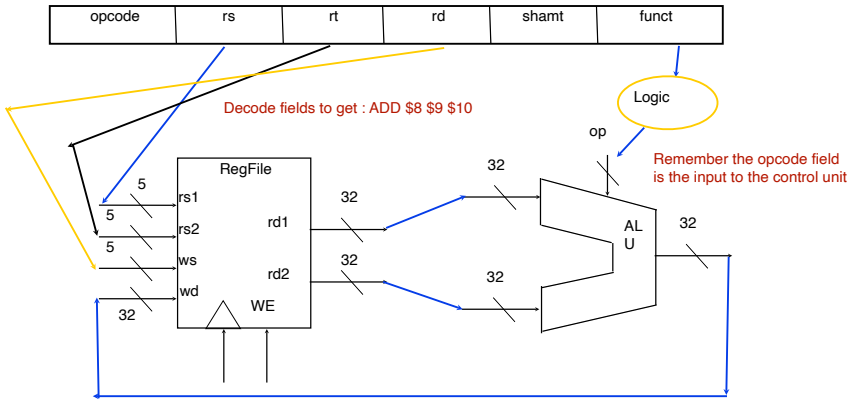
Datapath Elements: Decoding Instructions



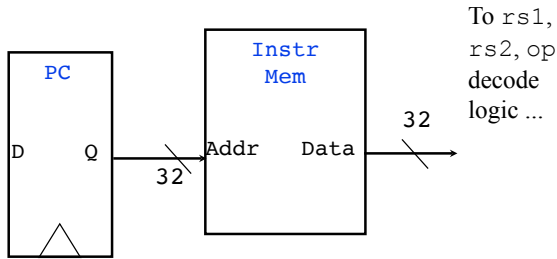
Datapath Elements: Executing Instructions



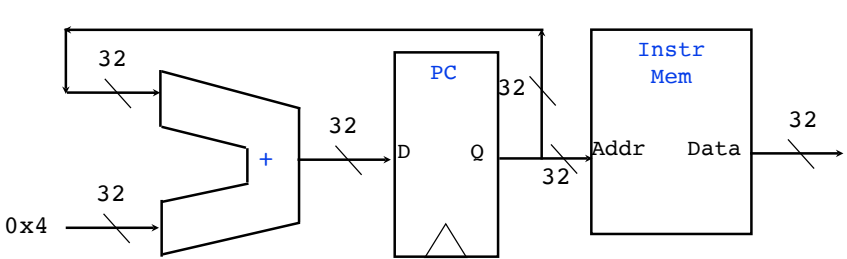
Datapath Elements: Executing Instructions



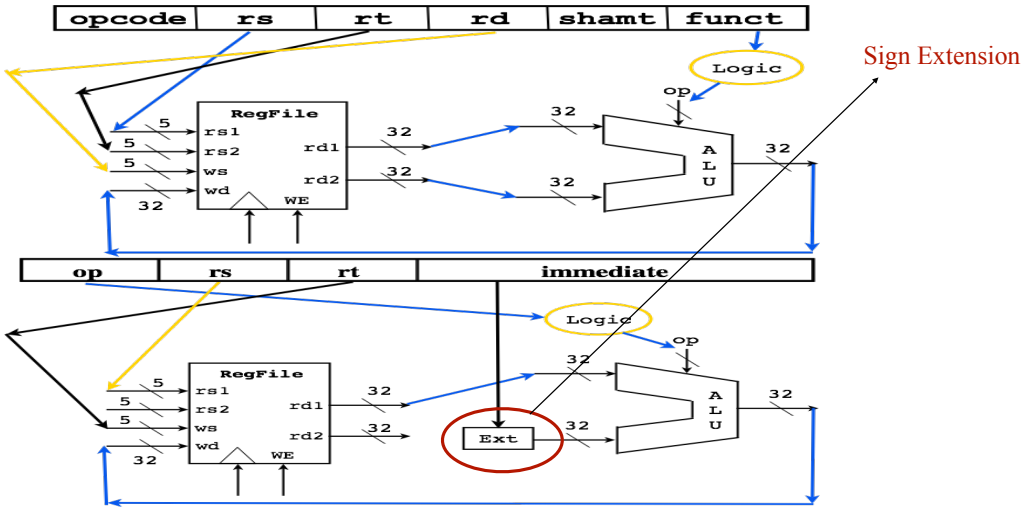
Datapath For Instruction Fetch



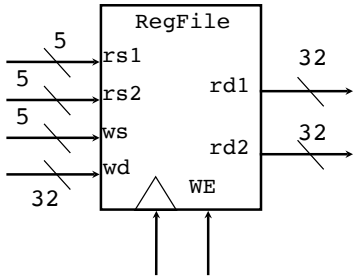
Datapath For Instruction Fetch

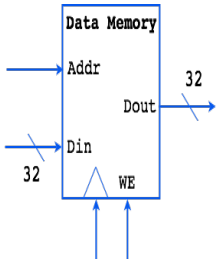


What about I format?



Loads from Memory

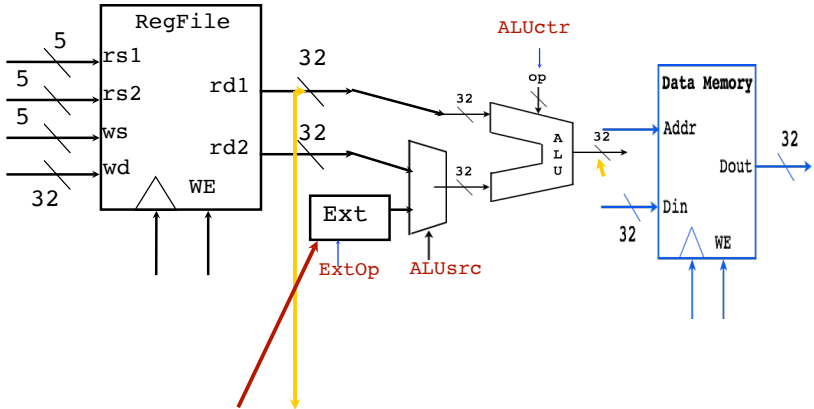




Syntax: LW $1, 32($2)

Action: $1 = M[$2 + 32]

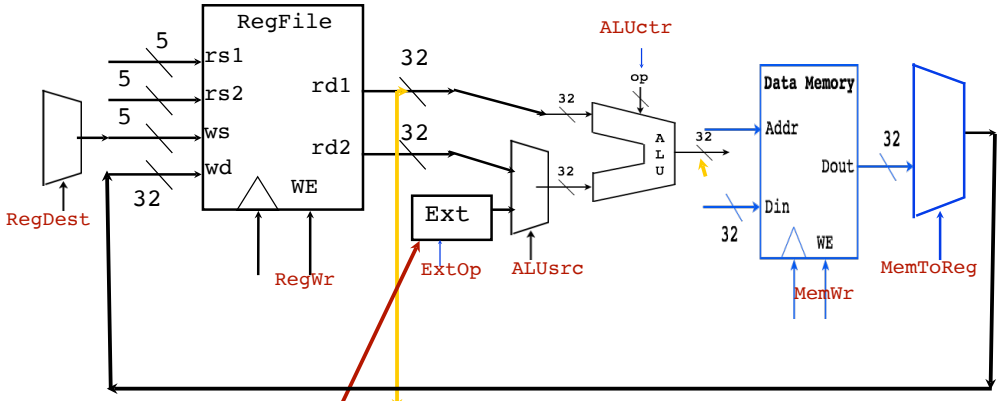
Loads from Memory



Syntax: LW $1, 32($2)

Action: $1 = M[$2 + 32]

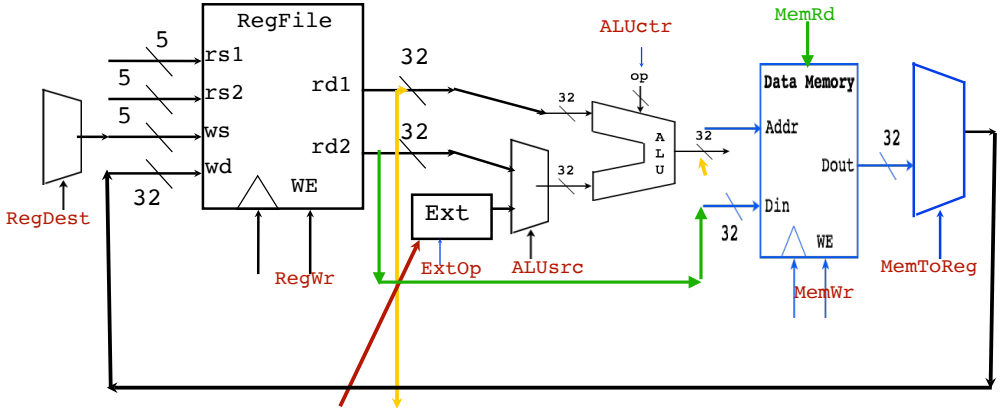
Loads from Memory



Syntax: LW $1, 32($2)

Action: $1 = M[$2 + 32]

Stores to Memory (with Load Datapath)

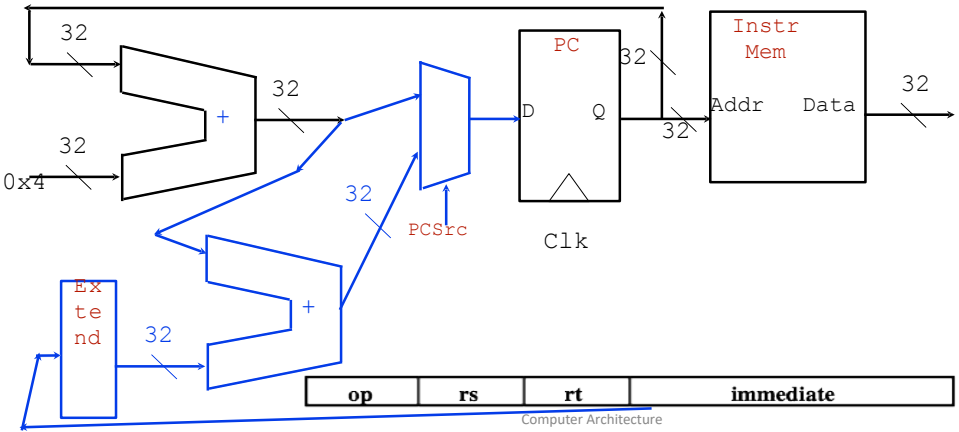


Syntax: SW $1, 32($2)

Action: M[$2 + 32] = $1

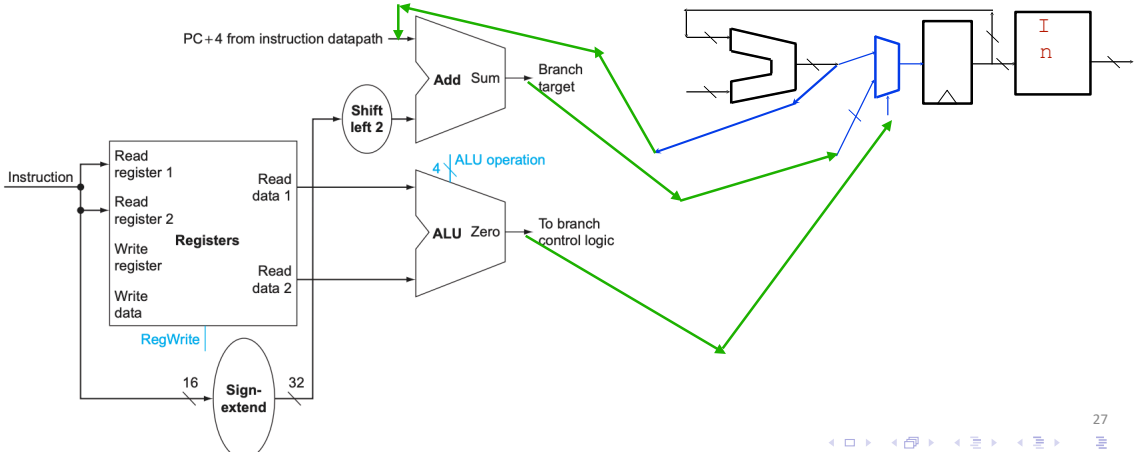
Branch Instructions

```
Syntax: BEQ $1, $2, 12
Action: If ($1 != $2), PC = PC + 4
Action: If ($1 == $2), PC = PC + 4 + 48
```

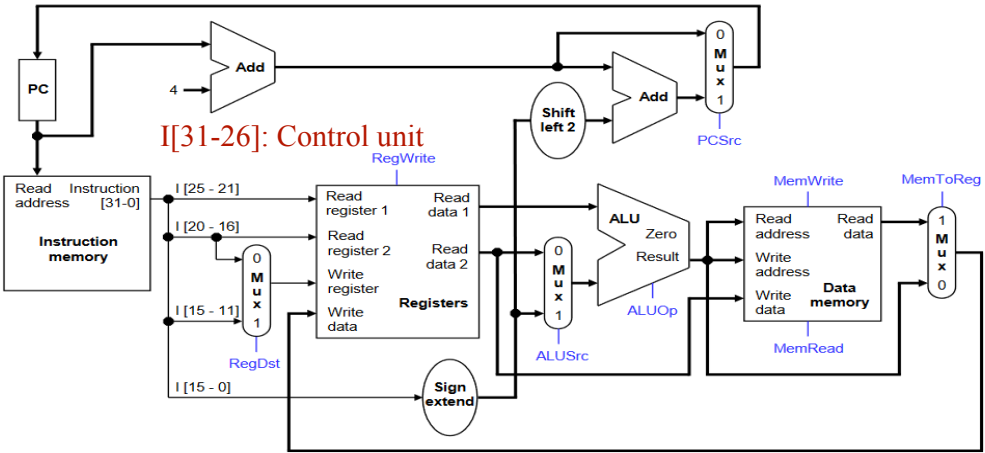


Branch Instructions

```
Syntax: BEQ $1, $2, 12
Action: If ($1 != $2), PC = PC + 4
Action: If ($1 == $2), PC = PC + 4 + 48
```

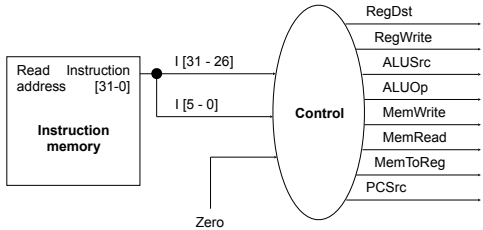


The Complete Picture



Control Signals So far

- MemRead
- MemWrite
- RegWrite
- MemtoReg
- RegDst
- ALUop, ALUSrc
- PCSrc



In Detail

- MemRead: Read from memory when assert
- MemWrite: Write into the memory when assert
- RegWrite: Reg. on Write register updated with the input, on assert
- MemtoReg: On assert, memory to register, on deassert, ALU to register
- RegDst: On assert, use rd field, on deassert use rt field
- ALUSrc: On assert, lower 16 bits of an inst., on deassert from the second register
- PCSrc: On assert, branch target, deassert, PC+4

Control Signal Table

Operation	RegDst	RegWrite	ALUSrc	ALUOp	MemWrite	MemRead	MemToReg
add	1	1	0	010	0	0	0
sub	1	1	0	110	0	0	0
and	1	1	0	000	0	0	0
or	1	1	0	001	0	0	0
slt	1	1	0	111	0	0	0
lw	0	1	1	010	0	1	1
sw	X	0	1	010	1	0	X
beq	X	0	0	110	0	0	X

Why not single cycle?

- The longest possible datapath is the clock cycle time.

What does it mean?

Why not single cycle?

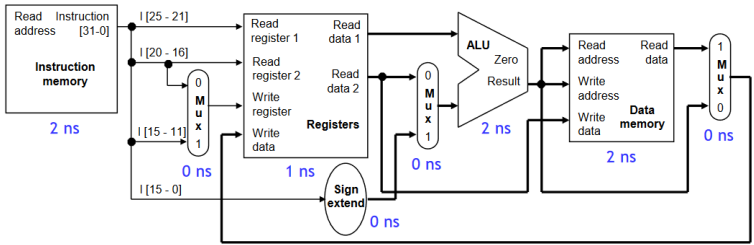
- For example, lw $t0, -4($sp) needs 8ns, assuming the delays shown here.

reading the instruction memory	2ns	
reading the base register $sp	1ns	
computing memory address $sp-4	2ns	8ns
reading the data memory	2ns	
storing data back to $t0	1ns	

one clock cycle: 8ns

Processor frequency: 125MHz

Cycle per Instruction (CPI): 1



An add instruction: no need of 8ns

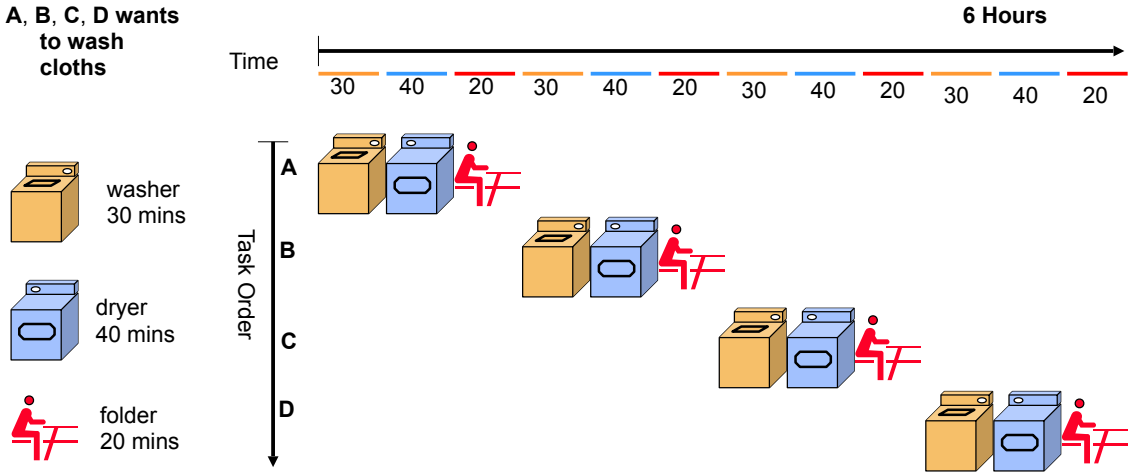
Why not single cycle?

- The longest possible datapath is the clock cycle time.

Violating *common case fast* — Basic principle of computer architecture.

Now what should we do??

Why not single cycle?



Single Vs. Multi-cycle: Where is the Gain?

Single cycle (Worst case)

Everyone takes 90 mins Full package = 90 minutes

Multi cycle (average case kinda)

One person: 20 to 90 mins avg = 53.33 minutes

Single to Multi Cycle

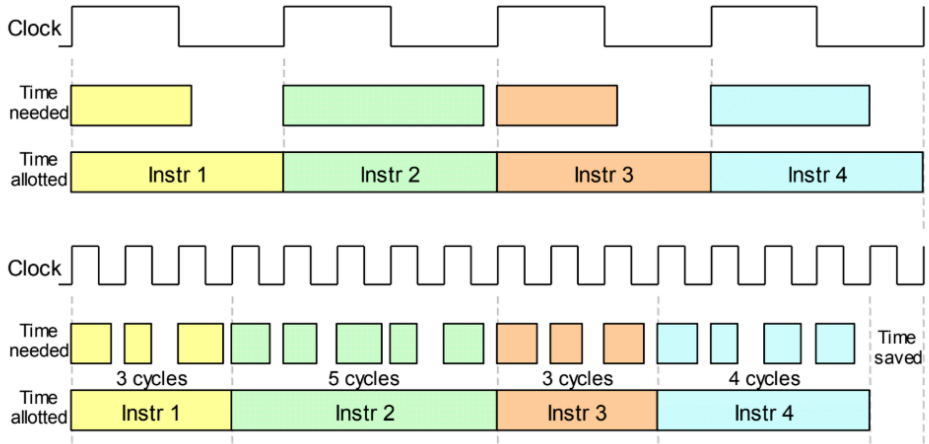
- Measuring Performance:

$$Execution\ Time = \#Instructios \times Cycles\ Per\ Instruction \times Clock\ Cycle\ Time$$

$$Speedup = \frac{Execution\ Time_{old}}{Execution\ Time_{new}}$$

  - Cycles Per Instruction (CPI) = 1 for single cycle processor
    - Very good, can't be better in simple scenarios
    - But still it's problematic — Why?
      - The clock cycle time is bounded by the length of the critical path of the datapath — here it is the load instruction
      - In simple words, the overall computation time will be poor!!!
  - How can we improve?
    - Multi-cycle datapath: Improve clock frequency — but poor CPI — not a great option

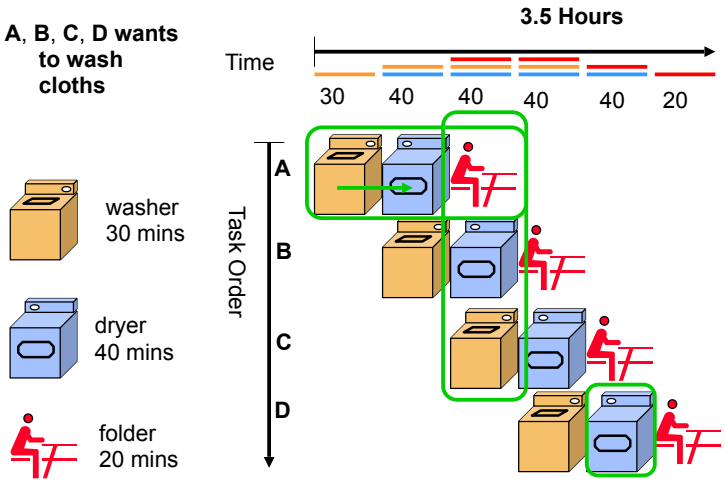
Single to Multi Cycle



Can We Have Both?

Faster clock rate and also CPI=1?

Basic Intuition



Main Observations

- Different computation stages which can have overlapped tasks
  - This is the point of speedup!!!
- The timing of each computation stage is determined by the most time consuming task
- **This is called pipelining**
- **Tricky**:
  - Each person still needs 90 mins.
  - But considering all A, B, C, D, the average execution time improves a lot.

Latency and Bandwidth (throughput)

- Latency
  - time it takes to complete one instance

- Throughput
  - number of computations done per unit time

Following slides are adapted (with modifications) from Biswa's slides

Let's Summarize

Single cycle: CPI: 1 , Cycle time: long

Multi cycle: CPI: >1, Cycle time: short

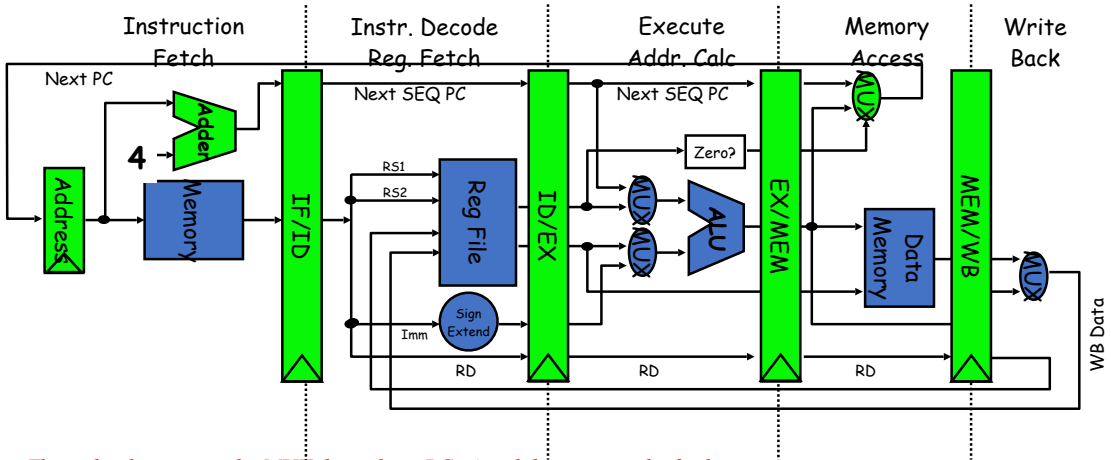
Pipelined: CPI: 1, Cycle time: short (improves throughput but not latency)

Pipelining and Richard Feynman

https://www.youtube.com/watch?v=9miKIWIYi4w

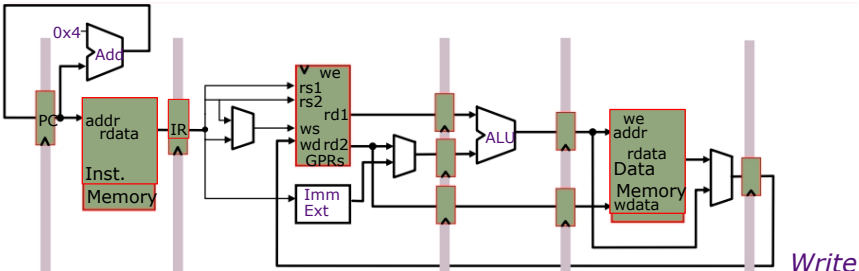
Jump to 1:25

Vanilla 5-stage pipeline



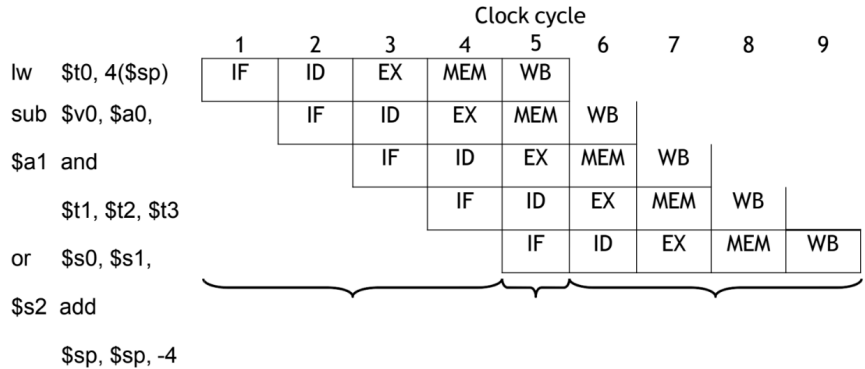
The right place to put the MUX that selects PC+4 and the target is the fetch stage.
The slide shows a vanilla 5-stage pipeline if we just take a single cycle datapath and divide it into five stages.

Resource Utilization

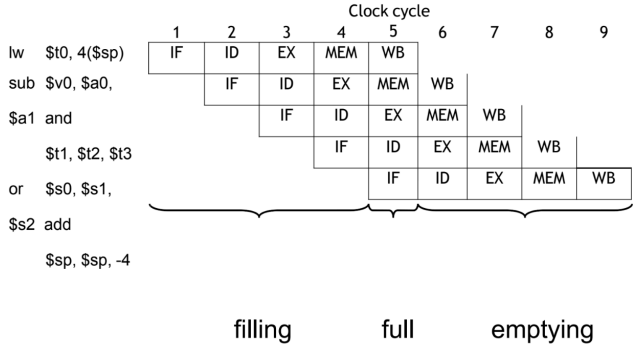


Resources	time	t0	t1	t2	t3	t4	t5	t6	t7	. . . .
	IF	I_1	I_2	I_3	I_4	I_5				
	ID		I_1	I_2	I_3	I_4	I_5			
	EX			I_1	I_2	I_3	I_4	I_5		
	MA				I_1	I_2	I_3	I_4	I_5	
	WB					I_1	I_2	I_3	I_4	I_5

Visualizing Pipeline



Visualizing Pipeline: Execution Time



For a k-stage pipeline executing N instructions

first instruction: K cycles

Next N-1 instructions: N-1 cycles, total = K + (N-1) cycles

Pipelined versus Single cycle CPU design

Instruction	Ifetch	Decode	Execute	Memory	Writeback	Total time
LOAD	200ns	100	200	200	100	800ns
STORE	200	100	200	200		700ns
ADD	200	100	200		100	600ns
BRANCH	200	100	200			500ns

Total latency in single cycle CPU: 3200 ns

Total latency in pipelined CPU (200ns clock cycle):
1000ns (1st instruction) + 3 X 200 ns (for next three) = 1600 ns

What's the big deal

Speedup = 3200ns/1600ns = 2X

What if we have a billion instructions?
Single cycle = 1 billion * 800ns = 800 seconds
Pipelined = 1000ns + (1 billion -1) * 200ns ~ 200 seconds

Speedup = 4X ☺

Let's include latch latency too

Inter-stage latch = 10ns

New clock cycle time in the pipelined design = 210ns

First instruction will get completed by 1040ns (five stages X 200 ns + four inter-stage latches * 10ns)

New Speedup = 800s/210s ~ 3.8X

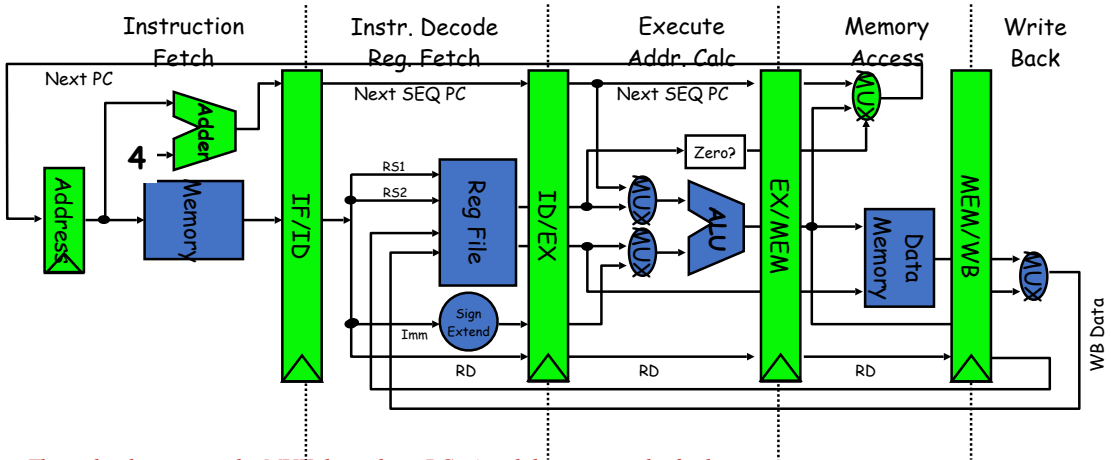
How to Divide the Datapath?

Suppose memory is significantly slower than other stages. For example, suppose

t_{IM}	= 10 units
t_{DM}	= 10 units
t_{ALU}	= 5 units
t_{RF}	= 1 unit
t_{RW}	= 1 unit

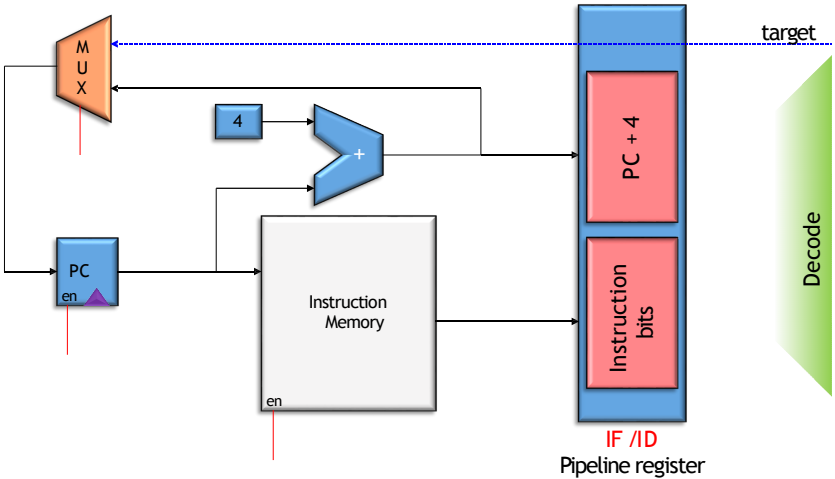
Since the slowest stage determines the clock, it may be possible to combine some stages without any loss of performance

Vanilla 5-stage pipeline

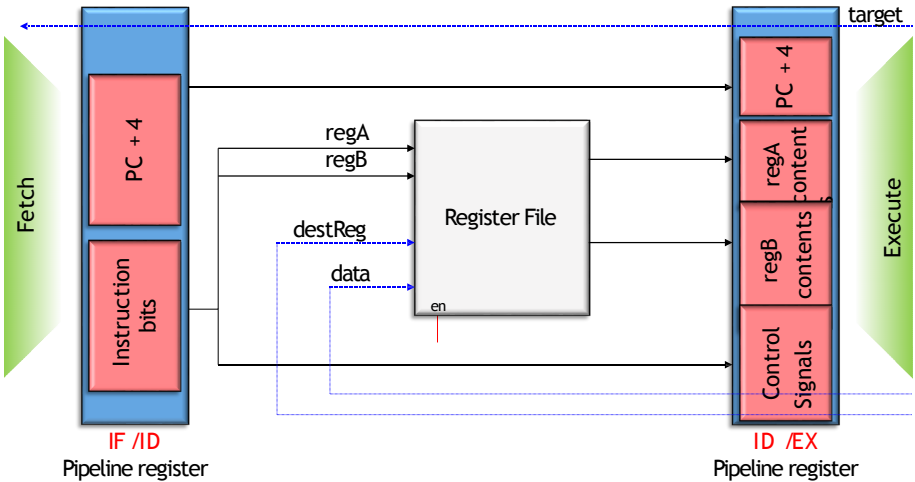


The right place to put the MUX that selects PC+4 and the target is the fetch stage.
The slide shows a vanilla 5-stage pipeline if we just take a single cycle datapath and divide it into five stages.

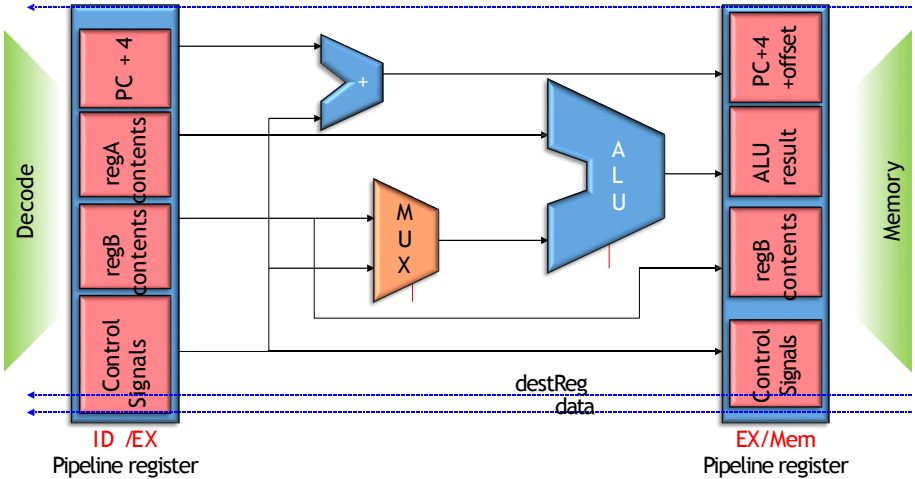
Stage-1: Fetch



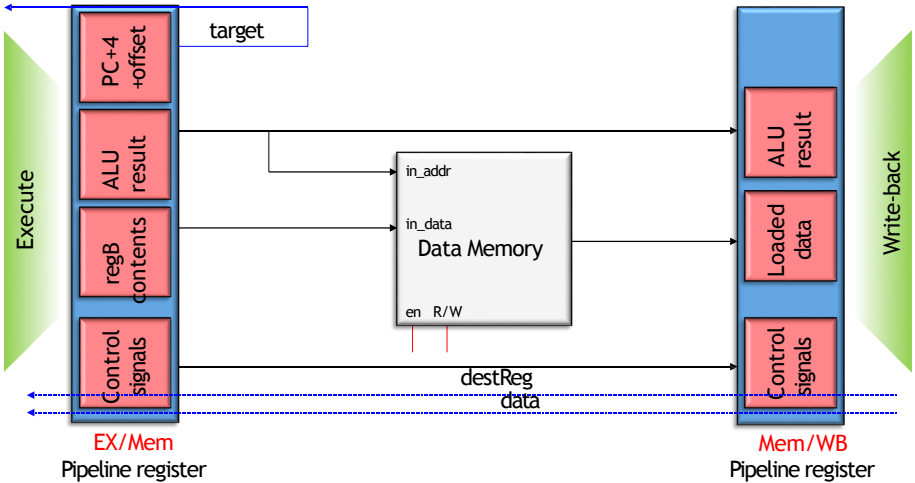
Stage 2: Decode



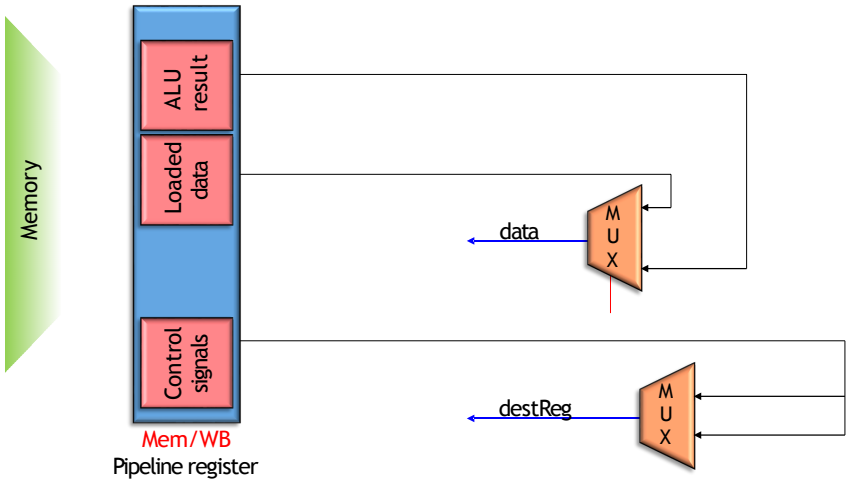
Stage 3: Execute



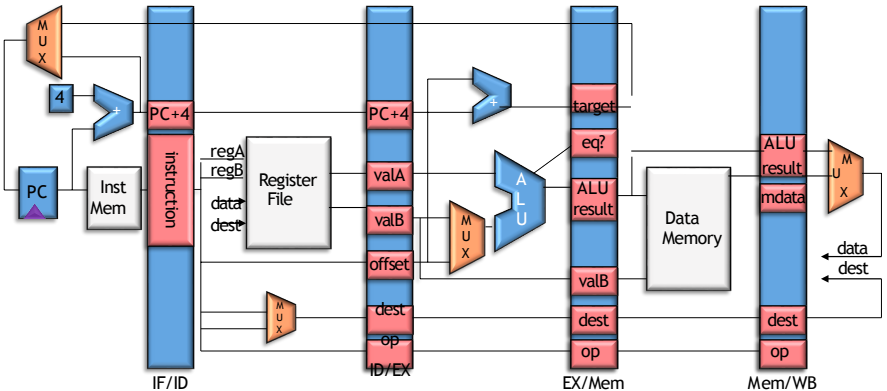
Stage 4: Memory Stage



Stage 5: Write-back



The Complete Picture



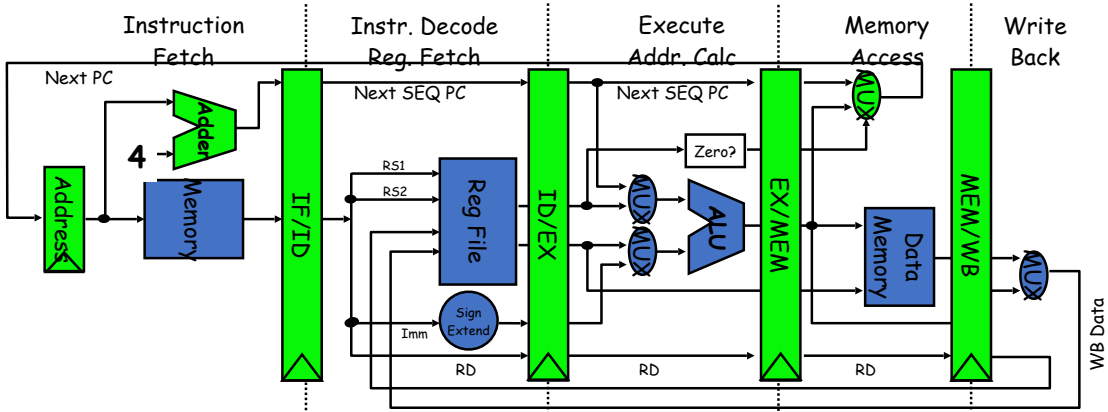
Digital Logic Design + Computer Architecture

Sayandeep Saha

Assistant Professor
Department of Computer Science and Engineering
Indian Institute of Technology Bombay

Pipeline Hazards

Pipeline Recap…



For a k-stage pipeline executing N instructions

first instruction: k cycles

Next N-1 instructions: N-1 cycles, total = K + (N-1) cycles

Pipeline In Real World

Limited resources

Stages does not take uniform time

Inter-instruction depenceny

Branches

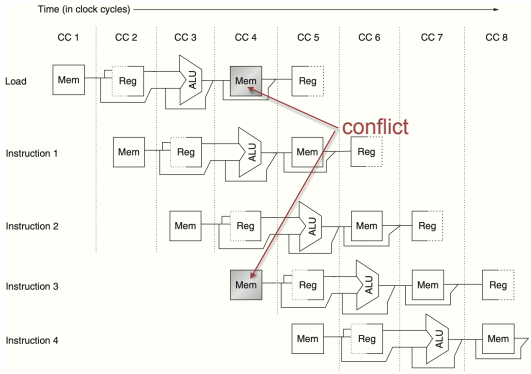
Limited resources

Pipeline Hazards

- Hazards are events in which prevents an instruction going down the pipeline. The pipeline is stalled
  - Structural hazards
  - Data hazards
  - Control hazards

Structural Hazards

- When two instructions wants to access the same resource at the same clock cycle.

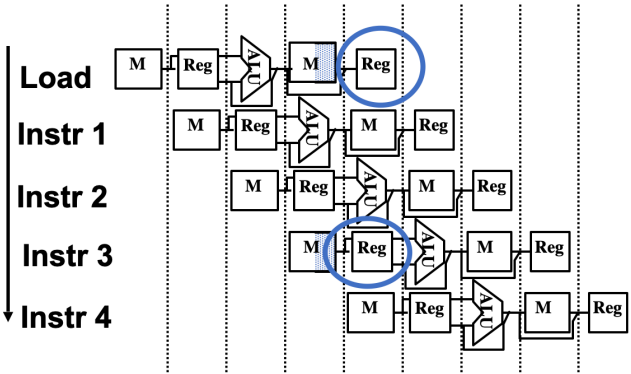


Instruction	Clock cycle number									
	1	2	3	4	5	6	7	8	9	10
Load instruction	IF	ID	EX	MEM	WB					
Instruction $i+1$		IF	ID	EX	MEM	WB				
Instruction $i+2$			IF	ID	EX	MEM	WB			
Instruction $i+3$				Stall	IF	ID	EX	MEM	WB	
Instruction $i+4$						IF	ID	EX	MEM	WB
Instruction $i+5$							IF	ID	EX	MEM
Instruction $i+6$								IF	ID	EX

- Issue: two instruction wants to access the memory simultaneously
  - One reading data
  - Other reading instruction.
- Solution: Separate instruction and data memory

Structural Hazards

- Can also happen for register files.



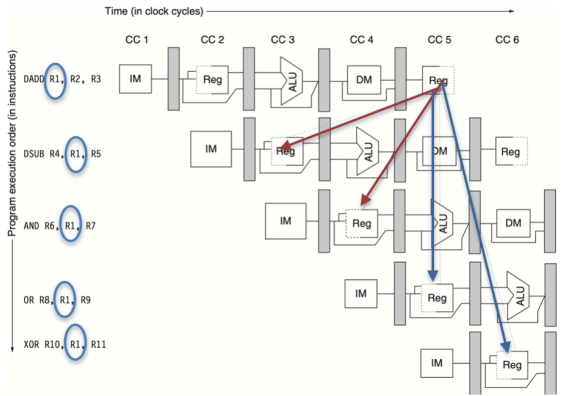
- Issue: conflict in ID and WB stage
  - Insufficient number of read/write ports
  - Read write in the same cycle, but no "write before read" convention.

  Solution:
  - Separate and multiple read write ports
  - Write in the first half of the clock cycle and read in the second half.

- Structural hazards are relatively rare in modern processors — compilers are smart.
- Only happens for less frequently used functional units

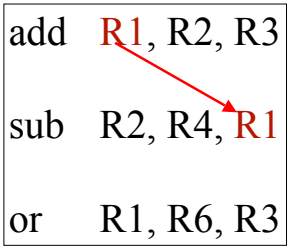
Data Hazards

- Hazards arising due to data dependency

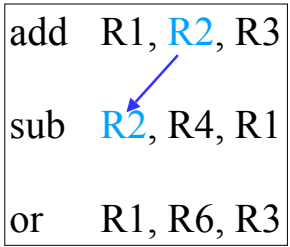


- The first DADD writes R1 at WB stage.
- All succeeding instructions reads the R1 result
- So, all instruction except the last OR and XOR has to wait for the WB of the first instruction.
- So we need two stall cycles or something else!!

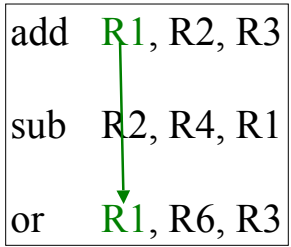
Data dependences



read-after-write
(RAW)
True dependence



write-after-read
(WAR)
Anti dependence



write-after-write
(WAW)
Output dependence

Data Hazards

Read-After-Write (RAW)
- Read must wait until earlier write finishes

Anti-Dependence (WAR)
- Write must wait until earlier read finishes. Not possible with vanilla 5-stage pipeline

- Output Dependence (WAW)
  - Earlier write can't overwrite later write
    Not possible with vanilla 5-stage pipeline)

Control Hazards

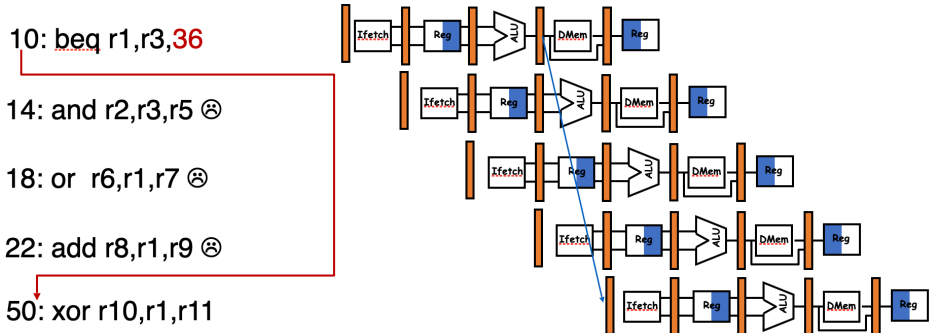
- Hazards arising due to branching…

- **Remember!!!** Branch target is not known during fetch.
- if a branch changes the PC to its target address, it is a *taken* branch.

- Else it is *untaken.*

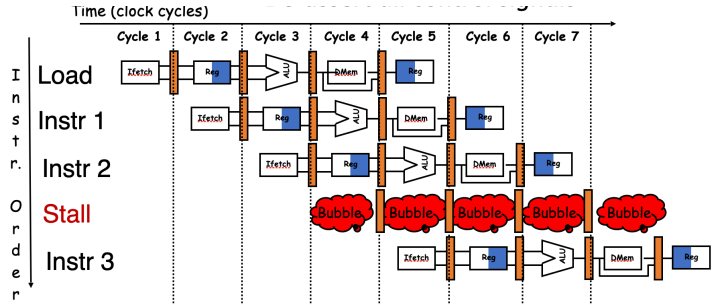
Control Hazards

- Hazards arising due to branching…
- What happens to the instructions at 14, 18, 22?



What is a Stall

- Putting bubbles in pipeline.
- Actually wasting clock cycles — in a stall cycle no instruction can enter the pipeline
- De-assart all control signals
- Compiler way — put a `nop` instruction. — eg, `sll $0 $0 (in MIPS)`



Control Hazard and Stalls

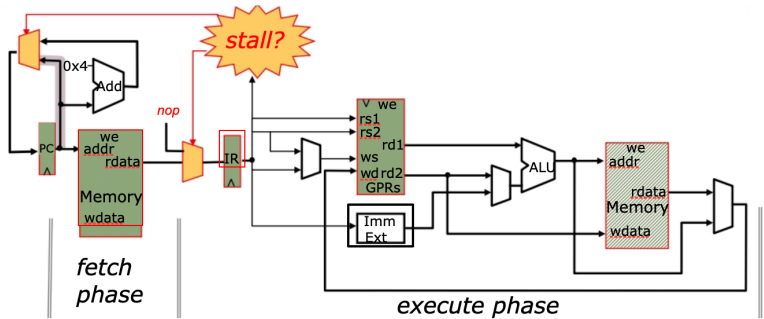
- The earliest we can get to know the branch outcome is at the end of ID stage (needs some simple hardware modification)

Branch instruction	IF	ID	EX	MEM	WB		
Branch successor		IF	IF	ID	EX	MEM	WB
Branch successor + 1				IF	ID	EX	MEM
Branch successor + 2					IF	ID	EX

This IF has to be undone — results in a stall cycle

How Stall is implemented in Pipelines

- Can be detected from the content of the pipeline registers.
- Upon detecting stall, just do not update the PC and dessert all control signals



Data Hazard Detector and stalls

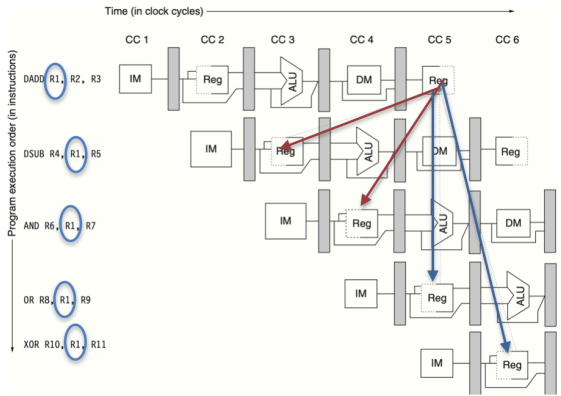
EX to DEC:

EX/MEM.RegisterRd = ID/EX.RegisterRs
EX/MEM.RegisterRd = ID/EX.RegisterRt

MEM to DEC:
MEM/WB.RegisterRd = ID/EX.RegisterRs
MEM/WB.RegisterRd = ID/EX.RegisterRt

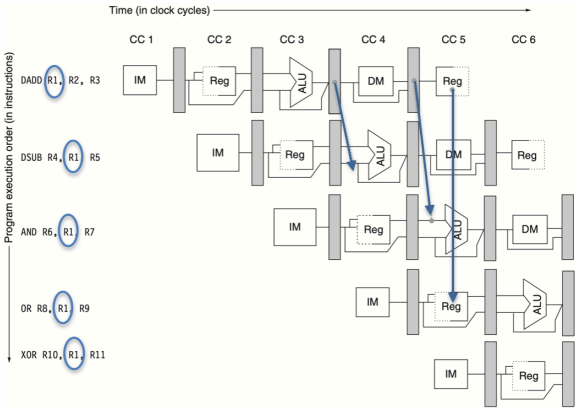
Data Hazards

- Hazards arising due to data dependency



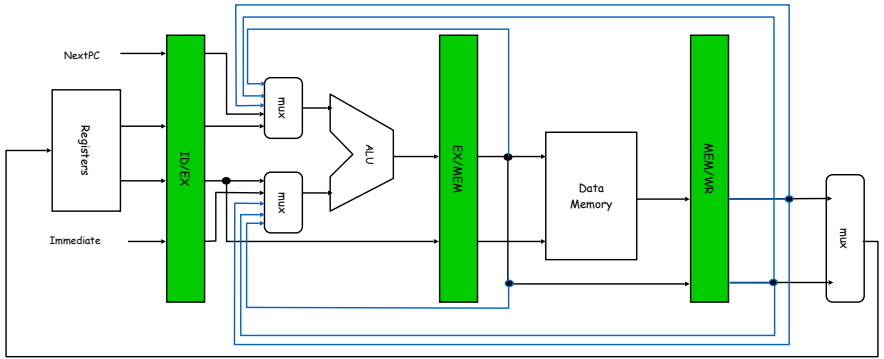
- The first DADD writes R1 at WB stage.
- All succeeding instructions reads the R1 result
- So, all instruction except the last OR and XOR has to wait for the WB of the first instruction.
- So we need two stall cycles or something else!!

Handling Data Hazards: Forwarding

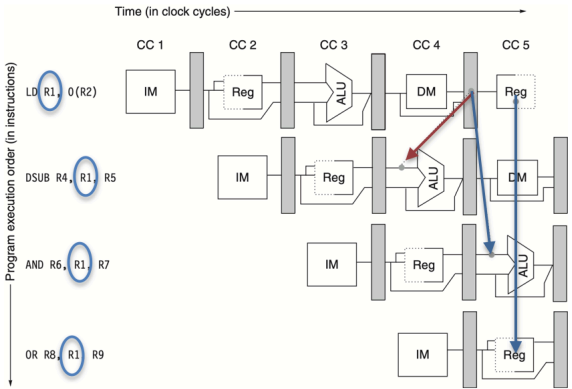


- No stalls!!
- Can be generalised — any functional unit generating data can forward to any other input whenever needed.

Handling Data Hazards: Forwarding



Can Forwarding Solve All the Problems?



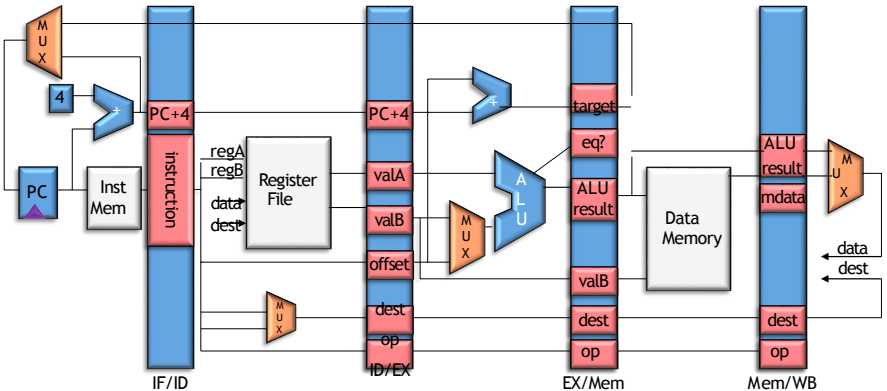
- No problems for the AND and OR — forwarding works fine
- But we cannot forward for the DSUB as it is backward in time.
- So one cycle stall is needed.

What Happens to Speedup with Stalls

$$Speedup = \frac{CPI\ Unpipelined}{CPI\ Ideal + stall\ cycles}$$

- CPI Ideal = 1
- Also assume stages are perfectly balanced so that if the unpipelined cycle time is T, the pipelined cycle time becomes T/k for a k stage pipeline. — so easy to cancel out T

The Complete Picture



Control Hazards

What do we need to calculate—next PC?

- For Jumps
  - Opcode, offset, and PC
- For Jump Register
  - Opcode and register value
- For Conditional Branches
  - Opcode, offset, PC, and register (for condition)
- For all others

Control Hazards

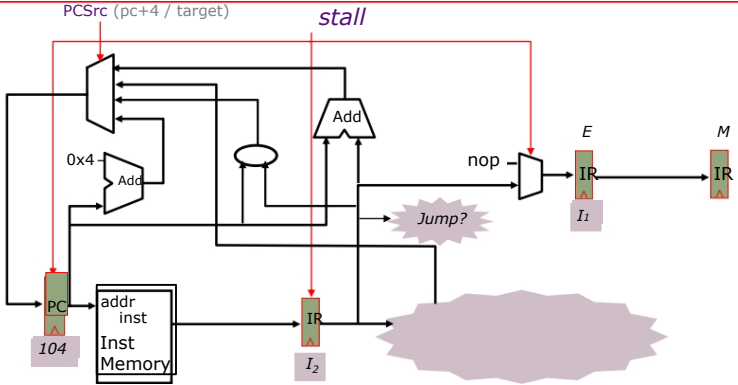
What do we need to calculate next PC?

- For Jumps
  - Opcode, offset, and PC
- For Jump Register
  - Opcode and register value
- For Conditional Branches
  - Opcode, offset, PC, and register (for condition)

In what stage do we know these?

- PC - Fetch
- Opcode, offset - Decode (or Fetch?)
- Register value - Decode
- Branch condition ((rs)==0) - Execute (or Decode?)

Speculate, PC=PC+4



I1	096	ADD
I2	100	J 304
I3	~~104~~	~~ADD~~
I4	304	ADD

What happens on mis-speculation, i.e., when next instruction is not PC+4?

kill *How? Insert NOPs*

Conditional branches

I_1	096	ADD
I_2	100	BEQZ r1 200
I_3	104	ADD
I_4	304	ADD

Branch condition is not known until the execute stage

Instructions between a branch instruction and the target are in the wrong-path if the branch is not taken

Again (stalls/NOPs)

	time t0	t1	t2	t3	t4	t5	t6	t7	. . . .
(I_1) 096: ADD	IF_1	ID_1	EX_1	MA_1	WB_1				
(I_2) 100: BEQZ 200		IF_2	ID_2	EX_2	MA_2	WB_2			
(I_3) 104: ADD			IF_3	ID_3	nop	nop	nop		
(I_4) 108:				IF_4	nop	nop	nop	nop	
(I_5) 304: ADD					IF_5	ID_5	EX_5	MA_5	WB_5

Resource Usage

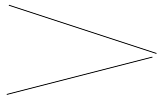
	time t0	t1	t2	t3	t4	t5	t6	t7	. . . .
IF	I_1	I_2	I_3	I_4	I_5				
ID		I_1	I_2	I_3	nop	I_5			
EX			I_1	I_2	nop	nop	I_5		
MA				I_1	I_2	nop	nop	I_5	
WB					I_1	I_2	nop	nop	I_5

What else can be done? Compiler?

Delayed branch: Define branch to take place AFTER a following instruction (used to be in early RISC processors)

branch instruction
sequential successor$_1$
sequential successor$_2$

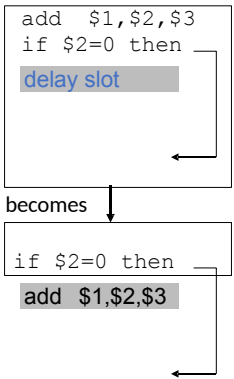
.....
sequential successor$_n$
branch target if taken



Branch delay of length *n*

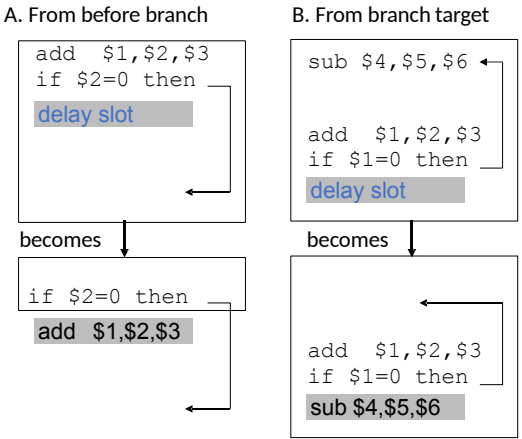
Scheduling Branch Delay Slots

A. From before branch



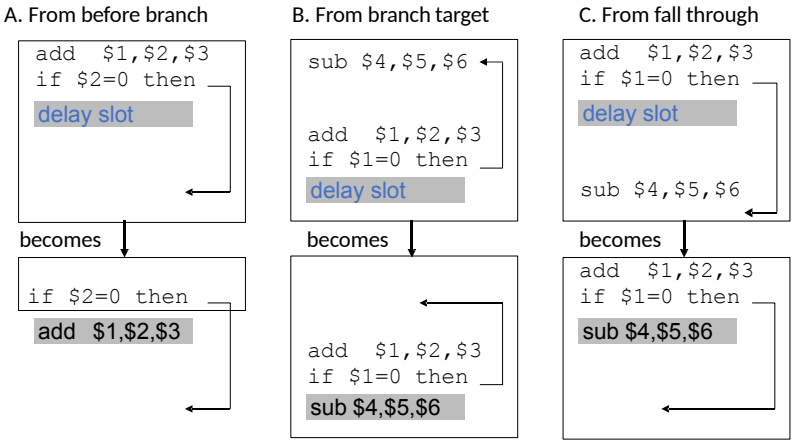
A is the best choice, fills delay slot & reduces instruction count (IC)

Scheduling Branch Delay Slots



A is the best choice, fills delay slot & reduces instruction count (IC)

Scheduling Branch Delay Slots



A is the best choice

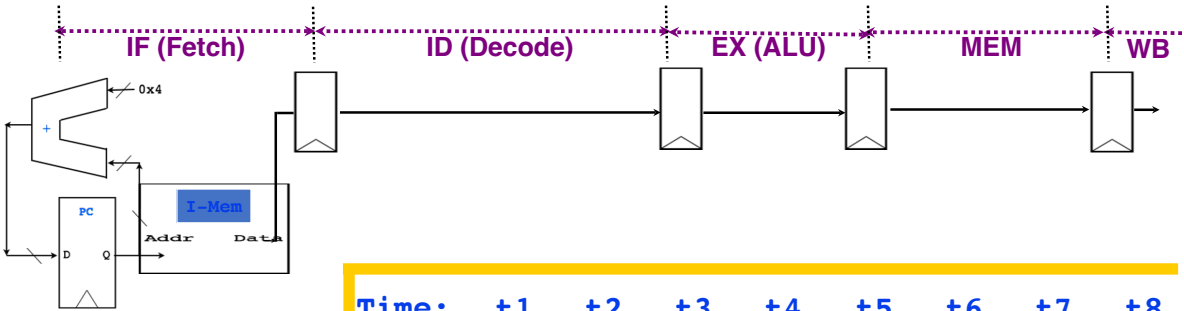
Do not put a branch in the delay slot :P

New Pipeline Speedup

$$\text{Pipeline Speedup} = \frac{\text{Pipeline Depth}}{1+\text{pipeline stalls because of branches}}$$

Pipeline stalls (branches) = Branch frequency X penalty

Branch instructions

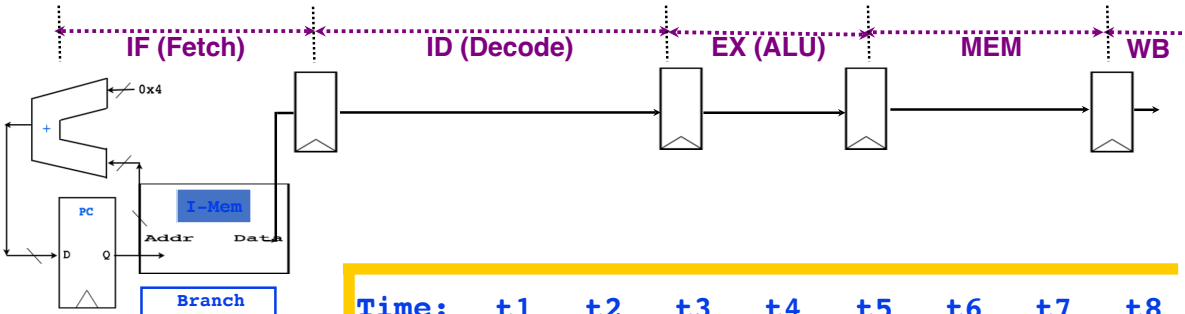


Time:	t1	t2	t3	t4	t5	t6	t7	t8
Inst								
I1:	IF	ID	EX	MEM	WB			
I2:		IF	ID					
I3:			IF					
I4:								
I5:								
I6:								

EX stage computes if branch is taken

If branch is taken, these instructions MUST NOT complete!

Branch Predictors



Time:	t1	t2	t3	t4	t5	t6	t7	t8
Inst								
I1:	IF	ID	EX	MEM	WB			
I2:		IF	ID					
I3:			IF					
I4:								
I5:								
I6:								

EX stage computes if branch is taken

If branch is taken, these instructions MUST NOT complete!

Branch Predictors

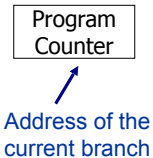
- Predict whether the next PC is a branch PC in the fetch stage
- But:
  - If it is branch, will it be taken?
  - What is the target address?
    - Not known at fetch…..

Branch Predictor: A bit deeper

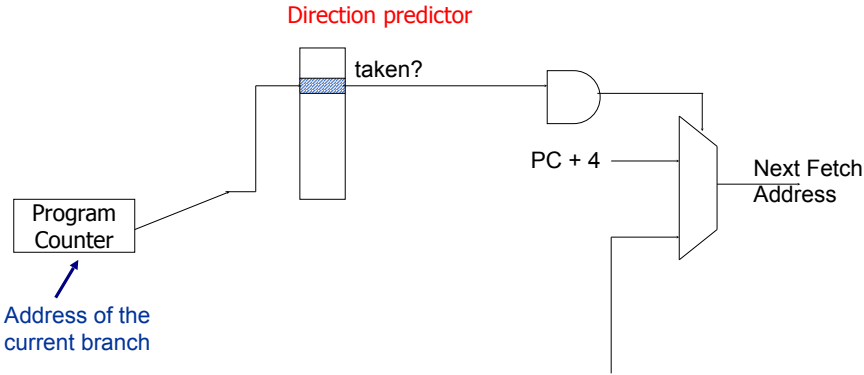
Three tasks

1. Is the PC a branch/jump? YES/NO
2. If Yes, can we predict the direction? Taken or not-taken
3. If taken, can we predict the target address?

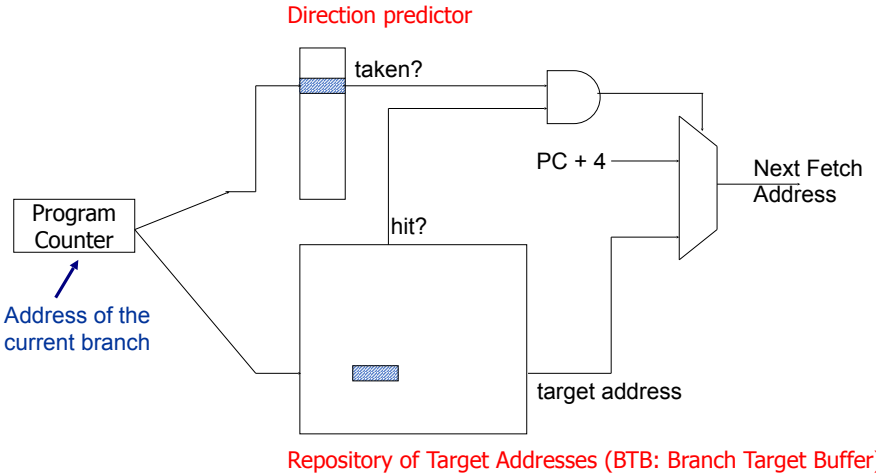
Branch Predictor: A bit deeper



Branch Predictor: A bit deeper



Branch Predictor: A bit deeper



Static (compiler) Direction Prediction

Always not-taken: Simple to implement: no need for BTB, no direction prediction

Low accuracy: ~30-40%

Always taken: No direction prediction, we need BTB though

Better accuracy: ~60-70%

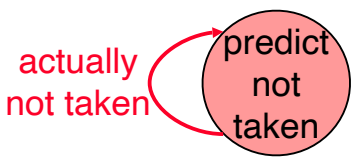
Backward branches (i.e., loop branches) are usually taken

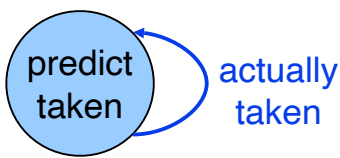
Dynamic Predictors

Microarchitectural way of predicting it.

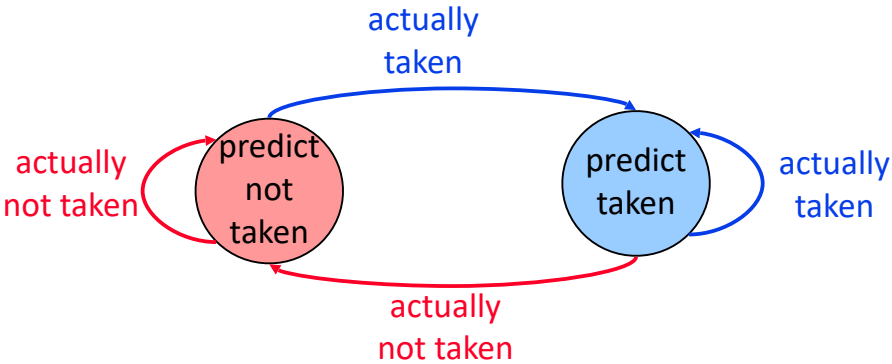
Simple one: Last time predictor

Last-time predictor





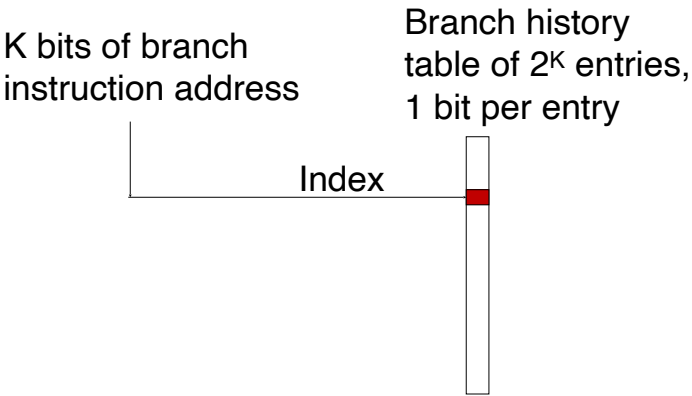
Last-time predictor



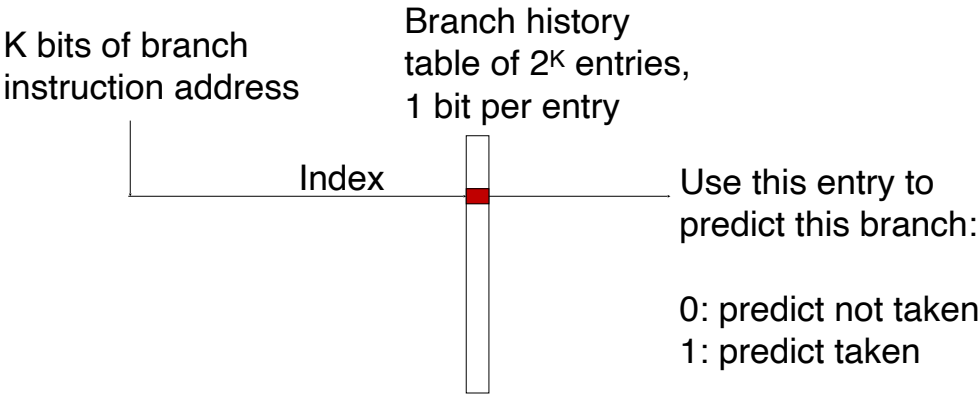
Implementation

K bits of branch instruction address

Implementation



Implementation



Performance of Last-time predictor

TTTTTTTTTTNNNNNNNNNN - 90% accuracy

Always mispredicts the last iteration and the first iteration of a loop branch

Accuracy for a loop with N iterations = (N-2)/N

+ Loop branches for loops with large number of iterations

-- Loop branches for loops will small number of iterations

TNTNTNTNTNTNTNTNTNTN — 0% accuracy

Performance: Calculating the CPI

20% of all instructions are branches, 85% accuracy

Last-time predictor CPI =

[1 + (0.20*0.15) * 2] =

1.06 (minimum two stalls to resolve a branch)

Types of Branches

	Conditional	Unconditional
Direct	if - then- else for loops (bez, bnez, etc)	procedure calls (jal) goto (j)
Indirect		return (jr)

Why do we need branch prediction?

- Allows useful work to be completed while waiting for a branch to resolve.

- Processors with deep pipelines
  - Intel Core 2 Duo: 14 stages
  - AMD Athlon 64: 12 stages
  - Intel Pentium 4: 31 stages
- Many cycles before branch is resolved
  - Wasting time if wait…
  - Would be good if can do some useful work…

Branch Prediction

- Key Idea: Predict branch outcome heuristically.
  - If successful, then we've gained a performance improvement.
  - Otherwise, discard instructions that have been executed speculatively.
    - Program execution state is still correct, all we've done is "waste" a little power.

Branch Prediction Strategies

- **Static:**
  - Decided before runtime
  - Examples:
    - Always-Not Taken
    - Always-Taken
    - Backwards Taken, Forward Not Taken (BTFNT)
- **Dynamic** (aka profile-driven prediction):
  - Prediction decisions may change during the execution of the program

What happens when a branch is mispredicted?

- On a mispredict:
  - No speculative state may commit
    - Squash instructions in the pipeline
    - Cannot allow stores to registers for instructions which would not get to commit
  - Need to handle exceptions appropriately

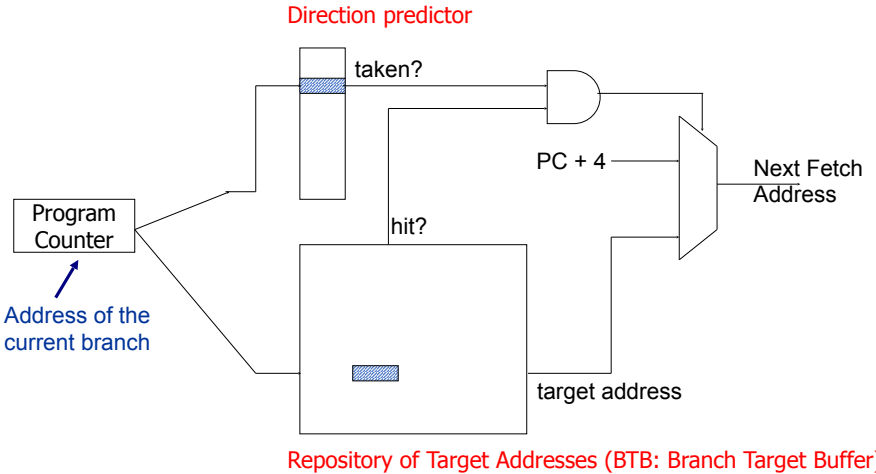
Direction Based Prediction

- **Pro:** Simple to implement
  - But, branch behaviour is often variable (dynamic) and depends on how the program is behaving recently.
  - Can't capture such behaviour at compile time with simple direction based prediction!
- **Need history (aka profile)-based prediction.**

Direction-Based Branch Prediction

- Which things exactly to predict?
  - **Direction:**
    - Taken / Not Taken
    - Can only be Direction
  - **Target Address**
    - PC+offset (Taken)/ PC+4 (Not Taken)
    - How implemented?
      - Using Branch Target Address Cache (BTAC) or Branch Target Buffer (BTB)

Branch Predictor: A bit deeper



Example: Branch Penalty Calculation

- Assume a MIPS pipeline using predict taken:
  - 16% of all instructions are branches:
    - 4% unconditional branches: 3 cycle penalty
    - 12% conditional:
      - 50% taken: 3 cycle penalty
      - 50% not taken: 0 cycle penalty

Solution

- For a sequence of N instructions:
  - 3 * 0.04 * N delays due to unconditional branches
  - 0.5 * 3 * 0.12 * N delays due to conditional taken
- Overall CPI=
  - 1.3*N
  - (or 1.3 cycles/instruction)
  - 30% Performance Hit!!!

History-based Branch Prediction

- An important example is State-based branch prediction:
- Consists of 2 parts:
  - "Predictor" to guess where/if instruction will branch (and to where)
  - "Recovery Mechanism": A way to fix mistakes

History-based Branch Prediction

- **One bit predictor:**
  - Use the outcome from the last time the branch instruction was executed.
- **Problem:**
  - Even if branch is almost always taken, we will be wrong at least twice
  - if branch alternates between taken, not taken
    - **We get 0% accuracy**

Example

- Let initial value = **T**
- Suppose actual outcome of branches

  NT, NT,NT,T,T,T

  - Predictions are: **T, NT,NT,NT,T,T**
    - 2 wrong (in red), 4 correct = 66% accuracy
- 2-bit predictors can do better
- In general, can have k-bit predictors.

1-bit Predictor: Exercise

- Program assumptions:
  - 23% loads and in ½ of cases, next instruction uses load value
  - 13% stores
  - 19% conditional branches
  - 2% unconditional branches
  - 43% other

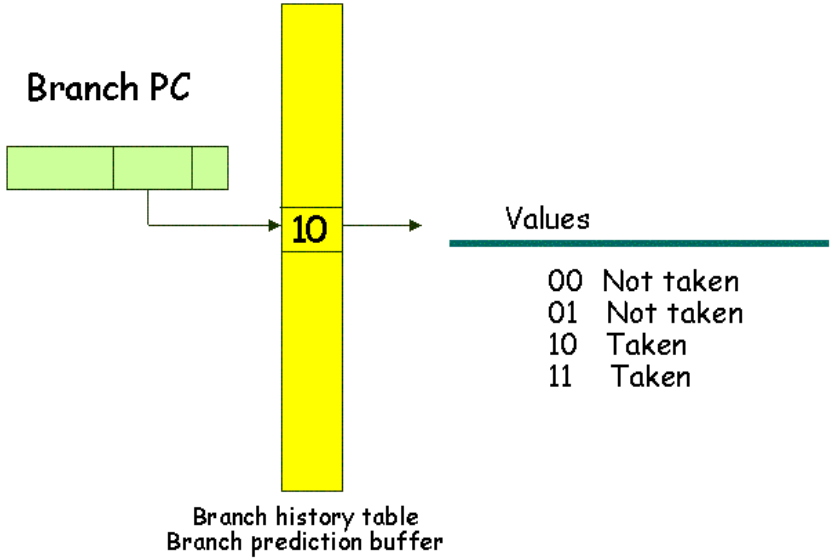
Exercise

- Machine Assumptions:
  - 5 stage pipe
    - Penalty of 1 cycle on use of load value immediately after a load.
    - Jumps are resolved in ID stage and incur a 1 cycle branch penalty.
    - 75% branch prediction accuracy and 1 cycle delay (penalty) on misprediction.

Solution

- CPI penalty calculation:
  - Loads:
    - 50% of the 23% of loads have 1 cycle penalty: 0.5*.23=0.115
  - Jumps:
    - All 2% of jumps have 1 cycle penalty: 0.02*1 = 0.02
  - Conditional Branches:
    - 25% of the 19% are mispredicted, have a 1 cycle penalty: 0.25*0.19*1 = 0.0475
- Total Penalty: 0.115 + 0.02 + 0.0475 = 0.1825
- Average CPI: 1 + 0.1825 = 1.1825

2-bit branch prediction

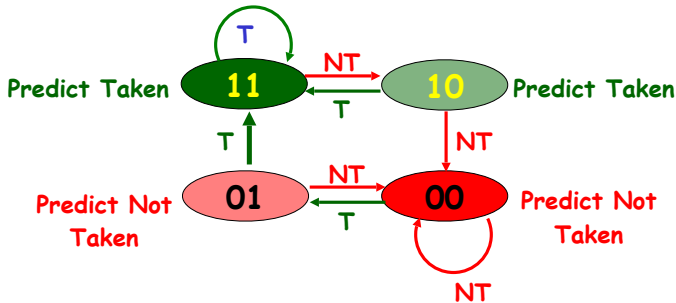


2-Bit Branch Prediction

- Approach: Prediction is changed only if mispredicted twice
- Adds hysteresis to decision making process

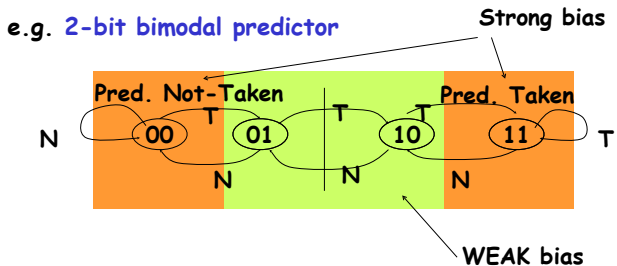
Red: stop, not taken

Green: go, taken



AKA Saturation Counter Predictor

- Observation: branches highly **bimodal**
- n-bit saturation counter
  - Hysteresis
  - n-bit entries in branch prediction table



n-bit Saturating Counter

- Values: $0 \sim 2^n-1$
- When the counter is greater than or equal to one-half of its maximum value,
  - the branch is predicted as taken. Otherwise, not taken.
- Studies have shown that the 2-bit predictors do almost as well

2-bit Predictor

- What is the prediction accuracy using a 4096 entry 2-bit branch predictor for a typical application?
  - 99% to 80% depending upon the application.
- Can an n-bit (n>2) predictor do better?
  - Not really! 2-bit predictors do almost as well as any n-bit predictors.
- How can then the accuracy of branch prediction be improved?
  - **Correlating branch predictor.**

Predictors in Simple Pipelines

- Initial pipelined processors, e.g. MIPS, SOLARIS, etc.:
  - Did only trivial branch predictions.
- Possible reasons could be:
  - The penalty of mispredictions not as severe as in deeper pipelined processors.
  - Sophisticated branch predictors did not exist.
- Advanced branch prediction techniques have now become very important:
  - With the use of deeper pipelines.
  - Introduction of superscalar processors.

Improving Accuracy of Branch Predictors

- It may be possible to improve the accuracy of branch prediction:
  - By observing the recent behavior of other branches.
- **Example:**

```
if (a==2){
        b=2;}
if(b==2}{
        b=0;}
```

The Main Idea

- **Record m most recently executed branches as taken or not taken,**
  - **Use that pattern to select the proper n-bit branch history table (BHT).**

Main Idea

- An (m,k) predictor:
  - Makes use of the outcomes observed for the last m branches:
    - Uses m number of k-bit predictors.
    - Behavior of a branch can be predicted by choosing from 2**m branch predictors.

Local and global history

- Local Behavior

  What is the predicted direction of Branch A given the outcomes of previous instances of Branch A ?

- Global Behavior

  What is the predicted direction of Branch Z given the outcomes of *all** previous branches A, B, …, X and Y?

  Number of previous branches tracked limited by the history length

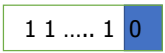
Two Level Branch Predictors

First level: Global branch history register (N bits)

The direction of last N branches

Second level: Table of saturating counters for each history entry

The direction the branch took the last time the same history was seen



GHR
(global history register)

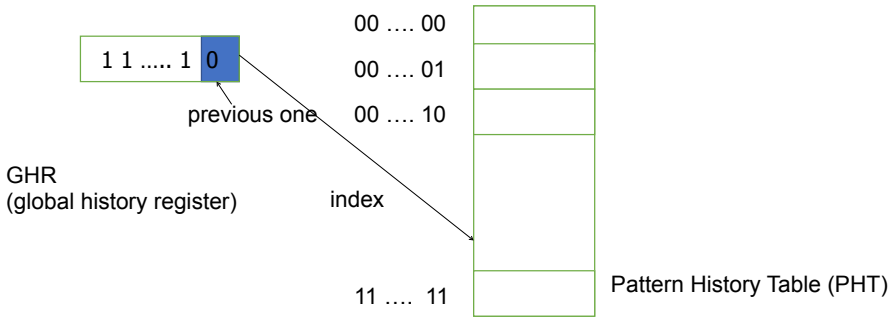
Two Level Branch Predictors

First level: Global branch history register (N bits)

The direction of last N branches

Second level: Table of saturating counters for each history entry

The direction the branch took the last time the same history was seen



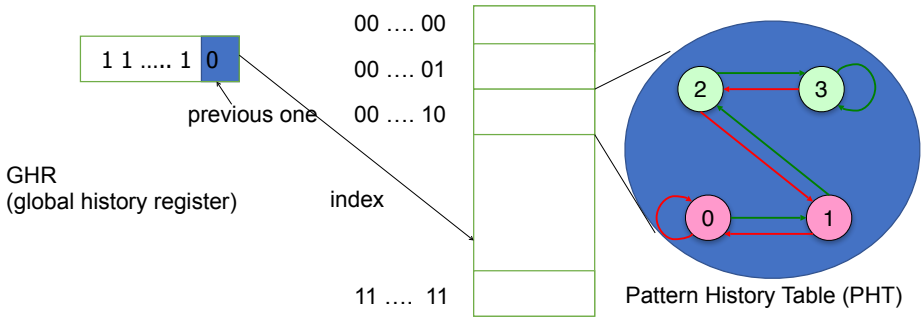
Two Level Branch Predictors

First level: Global branch history register (N bits)

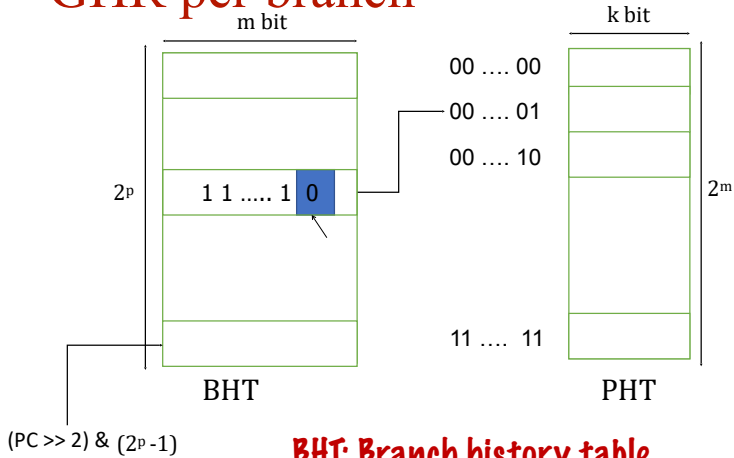
The direction of last N branches

Second level: Table of saturating counters for each history entry

The direction the branch took the last time the same history was seen



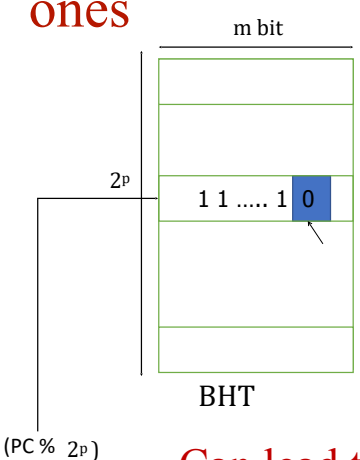
GHR per branch

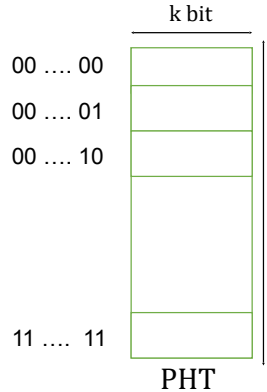


BHT: Branch history table

Mostly K=2, m =12 for example

Set of branches: One register for correlated ones

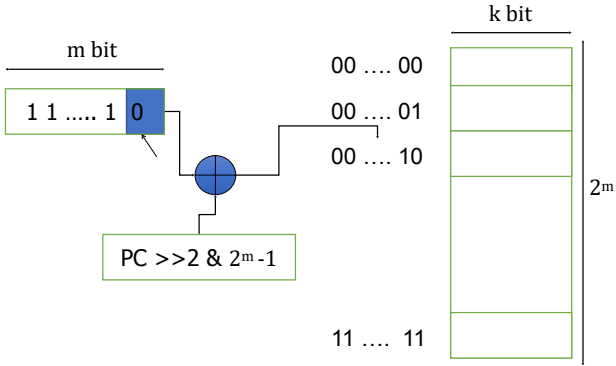





```
B1: if (aa == 2) aa = 0;
B2: if (bb == 2); bb=0;
B3: if (aa != bb) { … }
```

Can lead to positive/negative/neutral interference

Gshare is the answer



For a given history and for a given branch (PC) counters are trained

Few Important Points

Branch prediction happens at the IF stage.

We know the target outcome at the end of EX stage.

So BHT and PHT will be updated after EX stage for the corresponding PC. Any issues here?

Issue

I1 F D E
I2 F D E
I3 F D E

Lets assume I1 and I3 are branch instructions. I1 will update BHT and PHT in E stage, and I3 will probe BHT and PHT in F stage. To make sure PHT is updated correctly with the correct BHT entry, BHT entry is communicated till the E stage.

State-of-the-art

State of the art: Neural vs. TAGE

1970: Flynn
1972: Riseman/Foster
1979: Smith Predictor
1991: Two-level prediction
1993: gshare, tournament
1996: Confidence estimation
1996: Vary history length
1998: Cache exceptions
2001: Neural predictor
2004: PPM
2006: TAGE
2016: Still TAGE vs Neural

- Neural: AMD, Samsung
- TAGE: Intel?, ARM?
- Similarity
  - Many sources or “features”
- Key difference: how to combine them
  - TAGE: Override via partial match
  - Neural: integrate + threshold
- Every CBP is a cage match
  - Andre Seznec vs. Daniel Jimenez

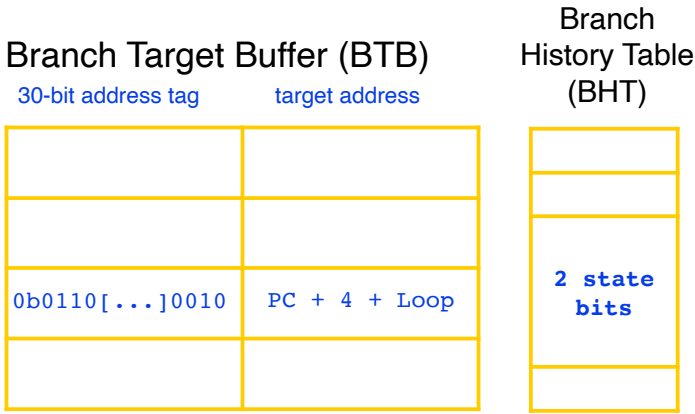
BTB (Target Address Predictor)

Address of branch instruction

`0b0110[...]01001000`

Branch instruction

`BNEZ R1 Loop`



BTB is probed in the fetch stage along with the direction predictor. A hit in the BTB means the PC is a branch PC.

Branch Target Buffer

- **For BTB to make a correct prediction, we need:**
  - **BTB hit:** the branch instruction should be in the BTB
  - **Prediction hit:** the prediction should be correct
  - **Target match:** the target address must not be changed from the last time
- Example: BTB hit ratio of 96%, 97% prediction hit, 1.2% of target change,

  The overall prediction accuracy = **0.96 * 0.97 *0.988 = 92%**

Branch Instruction Address	Branch Prediction Statistics	Branch Target Address
. . .	. . .	. . .

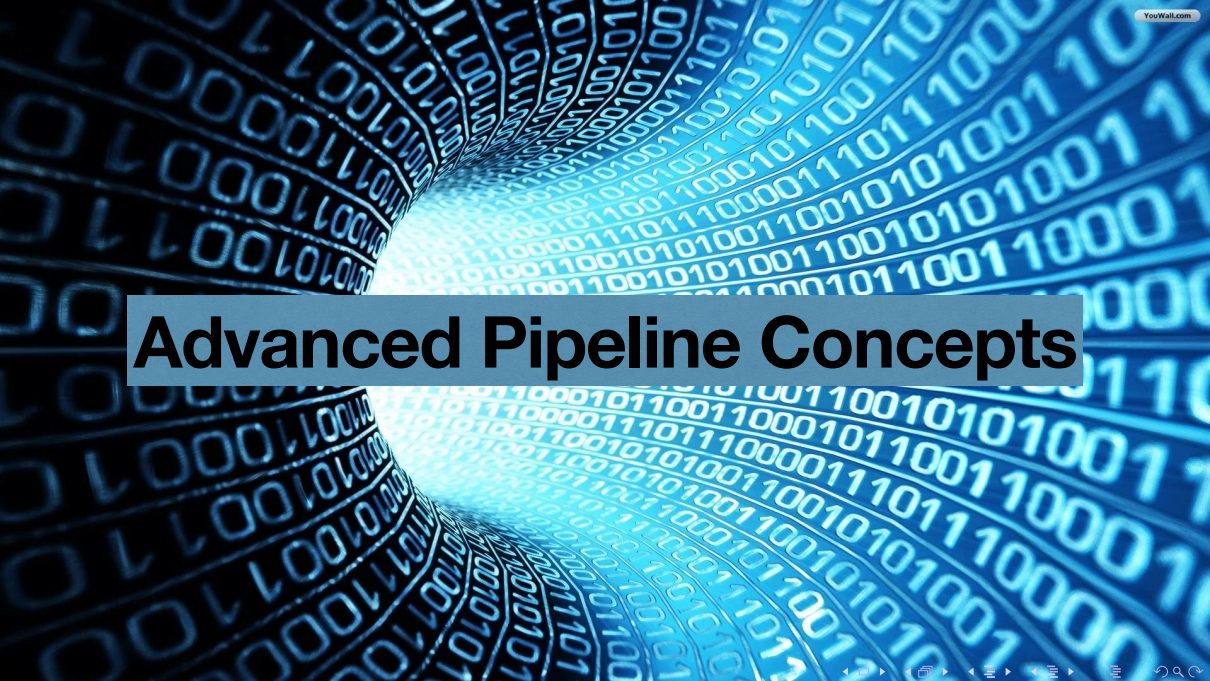
Book

Textbook reading: P & H, Chapter 4

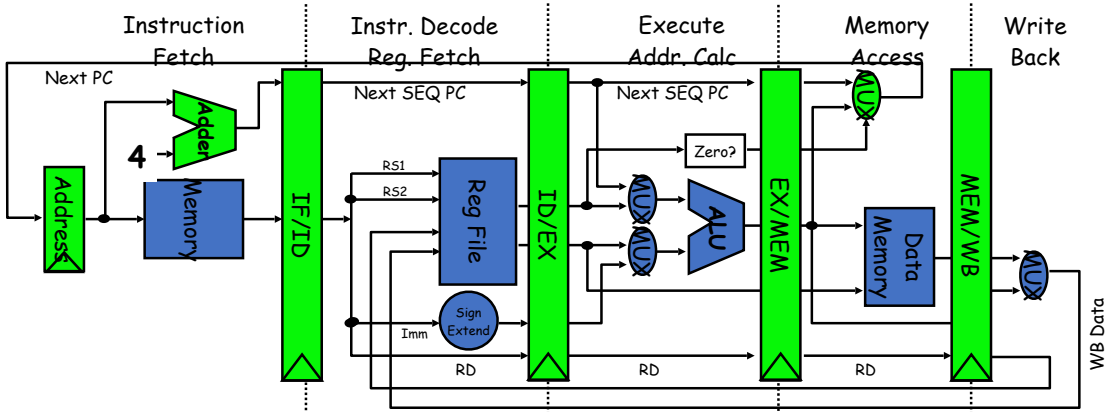
Digital Logic Design + Computer Architecture

Sayandeep Saha

Assistant Professor
Department of Computer Science and Engineering
Indian Institute of Technology Bombay


YouWall.com
Advanced Pipeline Concepts

Pipeline Recap…



For a k-stage pipeline executing N instructions

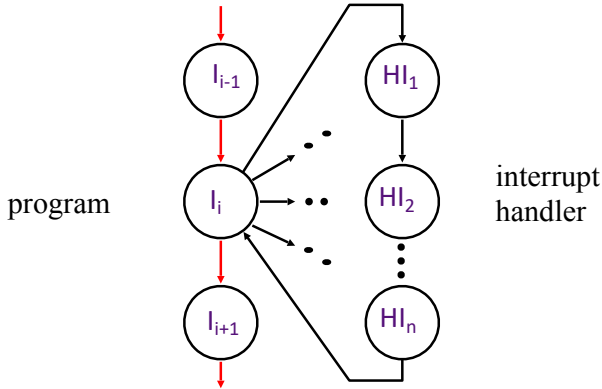
first instruction: k cycles

Next N-1 instructions: N-1 cycles, total = K + (N-1) cycles

Exception/Interrupt

An unscheduled event that disrupts program (instructions) in action.

Interrupt Handling

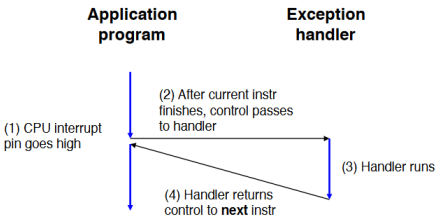


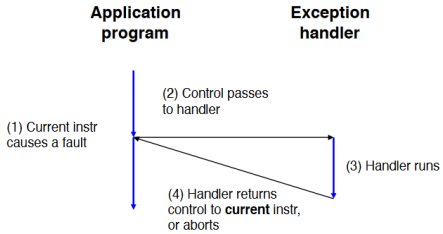
An *external or internal event* that needs to be processed. The event is usually unexpected or rare from program's point of view.

Types of Interrupts

- Asynchronous: an *external event*
  - input/output device service-request
  - timer expiration
  - power disruptions, hardware failure
- Synchronous: an *internal event (a.k.a. traps or exceptions)*
  - undefined opcode, privileged instruction
  - arithmetic overflow, FPU exception, misaligned memory access
  - *virtual memory exceptions:* page faults, TLB misses, protection violations
  - system calls, e.g., jumps into kernel

Interrupt and Exception





Interrupt Handler

Exception program counter (EPC): address of the offending instruction,

Saves EPC before enabling interrupts to allow nested interrupts

Need to mask further interrupts at least until EPC can be saved

Need to read a *status register* that indicates the cause of the interrupt

Handshake between processor and the OS

Processor:

stops the offending instruction,

makes sure all prior instructions complete,

flushes all the future instructions (in the pipeline)

Sets a register to show the cause

Saves EPC

Disables further interrupts

Jumps to pre-decided address (cause register or vectored)

Handshake between processor and the OS

OS:

Looks at the cause of the exception

Interrupt handler saves the GPRs

Handles the interrupt/exception

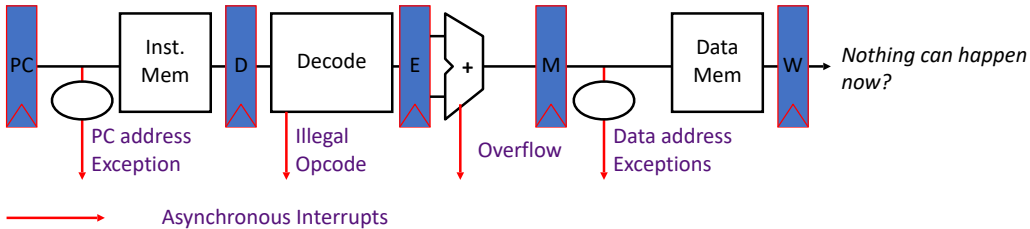
Calls RFE

Contd.

Uses a special indirect jump instruction RFE (*return-from-exception*) which

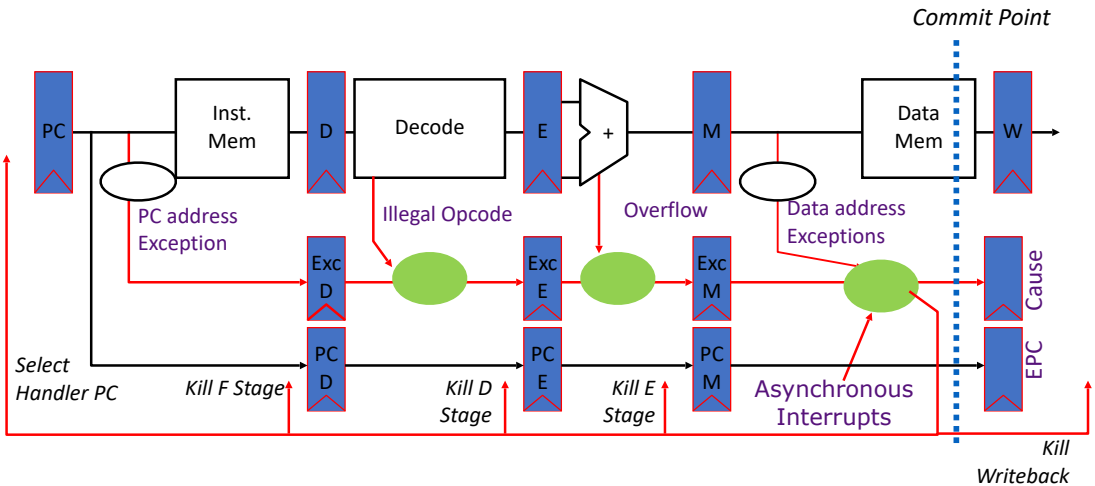
- enables interrupts
- restores the processor to the user mode

Exception handling and Pipelining



- When do we stop the pipeline for *precise* interrupts or exceptions?
- How to handle multiple simultaneous exceptions in different pipeline stages?
- How and where to handle external asynchronous interrupts?

Exception handling and Pipelining



Contd.

- Hold exception flags in pipeline until commit point for instructions that will be killed
- Exceptions in earlier pipe stages override later exceptions *for a given instruction*
- If exception at commit: update cause and EPC registers, kill all stages, inject handler PC into fetch stage

SuperScalar: More Instructions Per Cycle (IPC)

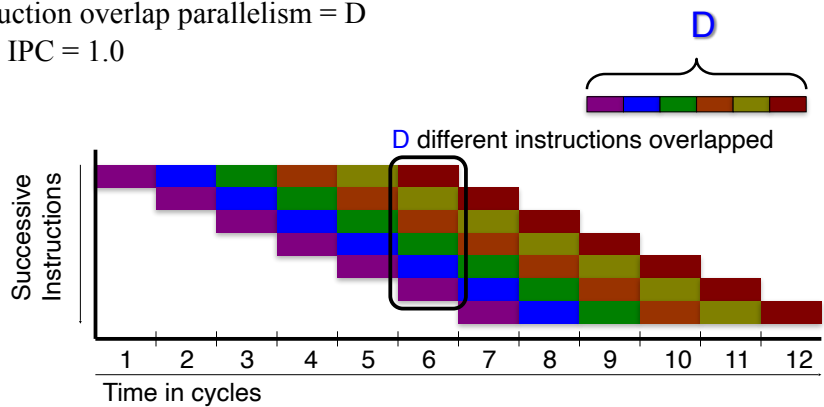
Beyond Scalar

- Scalar pipeline limited to CPI $\geq$ 1.0
  - Can never run more than 1 insn per cycle

- "Superscalar" can achieve CPI $\leq$ 1.0 (i.e., IPC $\geq$ 1.0)
  - *Superscalar* means executing multiple insns in parallel

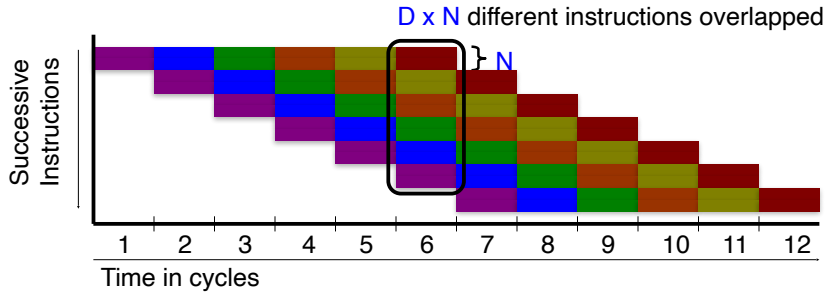
Instruction Level Parallelism (ILP)

- Scalar pipeline (baseline)
  - Instruction overlap parallelism = D
  - Peak IPC = 1.0



Superscalar Processor

- Superscalar (pipelined) Execution
  - Instruction parallelism = D x N
  - Peak IPC = N per cycle



What is the deal?

We get an IPC boost if the number of instructions fetched in one cycle are independent ☺

Complicates datapaths, multi-ported structures, complicates exception handling

Out of order (O3) processor: Pursuit of even higher IPC

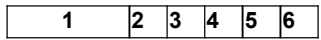
Out-of-order follows data-flow order

Example:

```
(1) r1 ← r4 / r7
(2) r8 ← r1 + r2
(3) r5 ← r5 + 1
(4) r6 ← r6 - r3
(5) r4 ← r5 + r6
(6) r7 ← r8 * r4
```

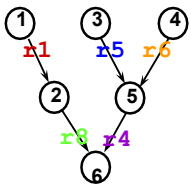
/* assume division takes 20 cycles */

In-order execution

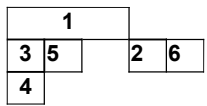


In-order (2-way superscalar)

Data Flow Graph



Out-of-order execution

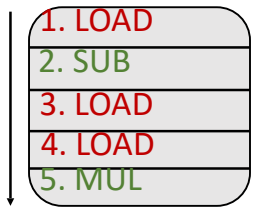


O3

Two or more instructions can execute in any order if they have no dependences (RAW, WAW, WAR)

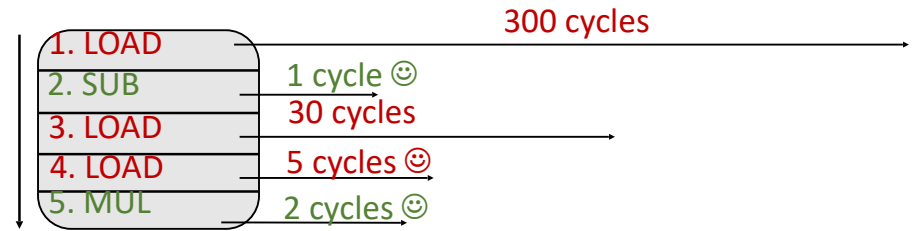
Completely orthogonal to superscalar/pipelining

O3 + Superscalar



In-order Instruction Fetch
(Multiple fetch in one cycle)

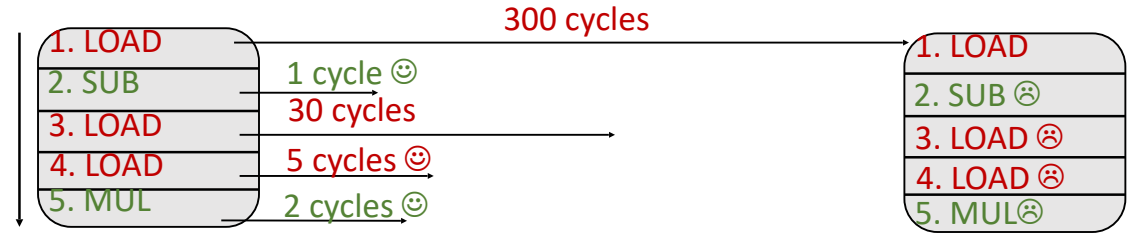
O3 + Superscalar



In-order Instruction Fetch
(Multiple fetch in one cycle)

Out-of-order execution

O3 + Superscalar



O3

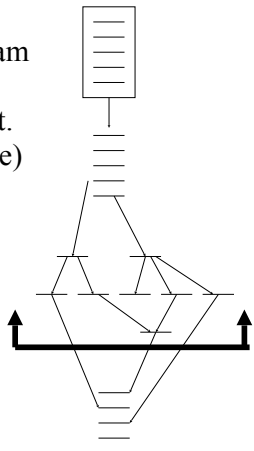
Program Form

Static program

dynamic inst.
Stream (trace)

execution
window

completed
instructions



Processing Phase

Dispatch/ dependences

inst. Issue

inst execution

inst. Reorder &
commit

The notion of Commit

After commit, the results of a committed instruction is visible to the programmer

and

the order at which instructions are fetched is also visible.

Why we need in-order commit?

Think about exceptions and precise exceptions

We should know till when we are done as per the programmer's view.

Performance: Time (Iron Law)

Time/Program =

Instructions/program X cycles/instruction X Time/cycle

Source code	ISA	microarch.
Compiler	microarch.	technology
ISA		

Example

Program p = one billion instructions

Processor takes one cycle per instruction

Processor clock is 4 GHz

CPU time = 10^{9} instructions X 1 cycle/instruction X 1/4 ns

= 0.25 second (4X faster)

Example

Program p = one billion instructions

Processor processes 10 instructions in one cycle

Processor clock is 4 GHz

CPU time = 10^{9} instructions X 0.10 cycle/instruction X 1/4 ns

= 0.025 second (40X faster)

Performance

- *Latency* (execution/response time): time to finish one task. It is additive (Performance = 1/latency)
- *Throughput* (bandwidth): number of tasks/unit time. It is not additive

Latency vs bandwidth

Latency vs Bandwidth, How they affect each other?

Latency helps bandwidth but not vice versa.

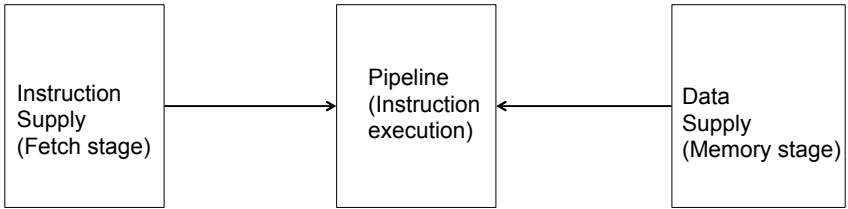
DRAM latency ↓ More # Accesses ~DRAM Bandwidth ↑

Bandwidth usually hurts latency

Queues - Bandwidth ↑ Increases latency ↓

Down the Memory Lane

The Ideal World

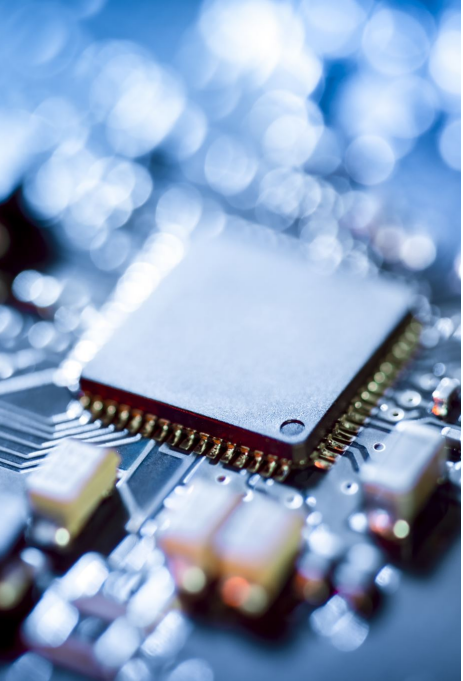


- Zero-cycle latency
- Infinite capacity
- Perfect control flow

- Zero-cycle latency
- Infinite capacity
- Infinite bandwidth

World of Memory Hierarchy: Why?

Before that What is memory?

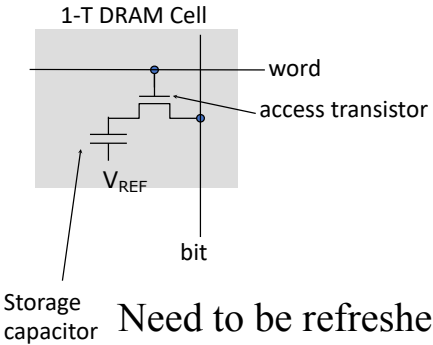


Semiconductor Memory

- Semiconductor memory began to be competitive in early 1970s
  - Intel formed to exploit market for semiconductor memory
  - Early semiconductor memory was Static RAM (SRAM). SRAM cell internals similar to a latch (cross-coupled inverters).

- First commercial Dynamic RAM (DRAM) was Intel 1103
  - 1Kbit of storage on single chip
  - charge on a capacitor used to hold value

One transistor DRAM



Denser

Value kept in one cell is as per the charge in the capacitor

Need to be refreshed periodically to maintain the charge

So dynamic RAM (DRAM)

DRAM

- Dynamic random-access memory
- Capacitor charge state indicates stored value
  - Whether the capacitor is charged or discharged indicates storage of 1 or 0
  - 1 capacitor
  - 1 access transistor
- Capacitor leaks
  - DRAM cell loses charge over time
  - DRAM cell needs to be refreshed

SRAMs (6T to 8T)

Static RAMs. No need of refresh as no capacitor.

Faster access (no capacitor)

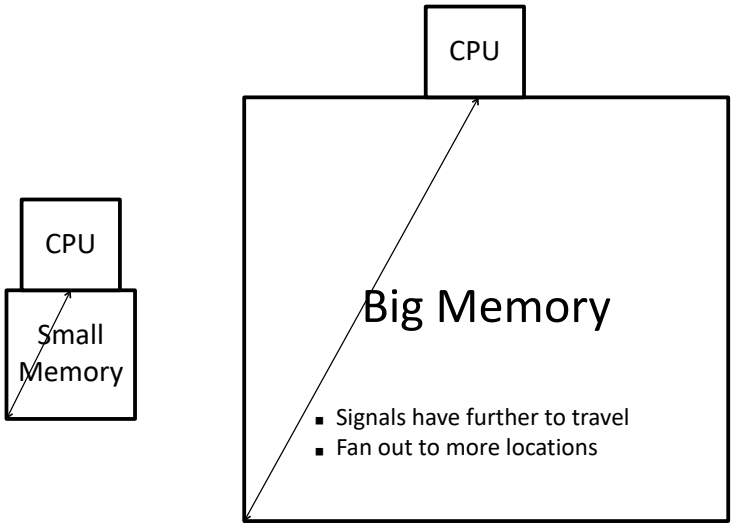
Density low as 6T: 1 bit compared to 1T: 1bit in DRAM

Costly than DRAM

The Problem

- Bigger is slower
  - SRAM, 512 Bytes, sub-nanosec
  - SRAM, KByte~MByte, ~nanosec
  - DRAM, Gigabyte, ~50 nanosec
  - Hard Disk, Terabyte, ~10 millisec
- Faster is more expensive (dollars and chip area)
  - SRAM, < 1000$ per GB
  - DRAM, < 20$ per GB
  - Hard Disk < 0.01$ per GB
  - These sample values scale with time
- Other technologies have their place as well
  - Flash memory, NVRAM, MRAM etc

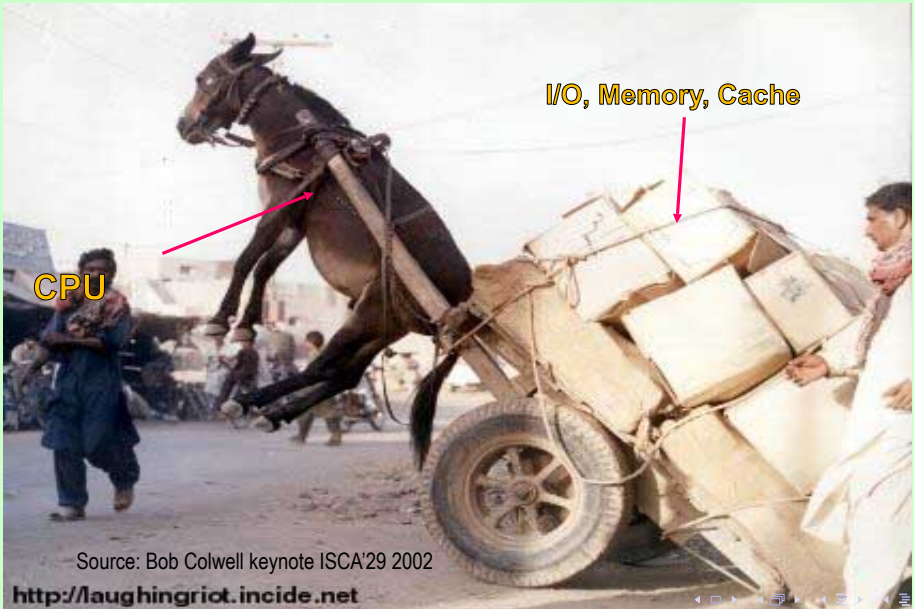
Size affects latency



An Unbalanced System

- Even a sophisticated processor may perform well below an ordinary processor:
  - Unless supported by matching performance by the memory system.
  - Unfortunately the gap is widening.
- Over the next few classes:
  - Study how memory system performance has been enhanced through various innovations and optimizations.

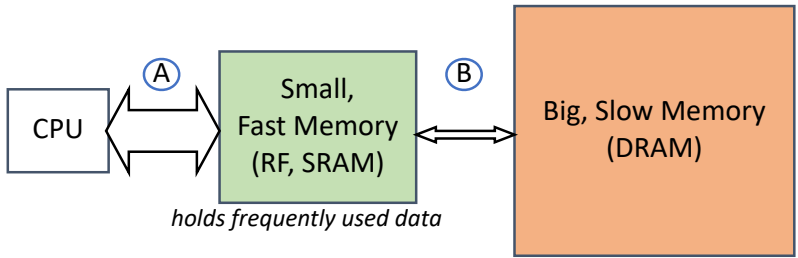
An Unbalanced System



Why Memory Hierarchy

- We want both fast and large
- But we cannot achieve both with a single level of memory
- Idea: Have multiple levels of storage (progressively bigger and slower as the levels are farther from the processor) and ensure most of the data the processor needs is kept in the fast(er) level(s)

Welcome to Memory Hierarchy



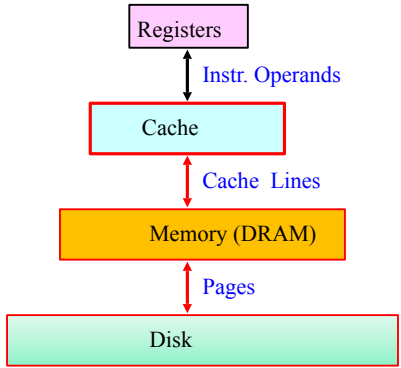
- *capacity*: Register << SRAM (Cache) << DRAM
- *latency*: Register << SRAM (Cache) << DRAM
- *bandwidth:* on-chip >> off-chip

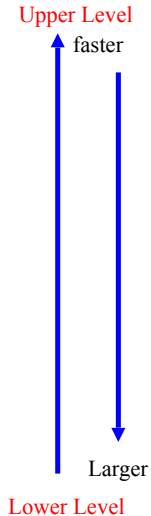
On a data access:

if data $\in$ fast memory $\Rightarrow$ low latency access *Cache*

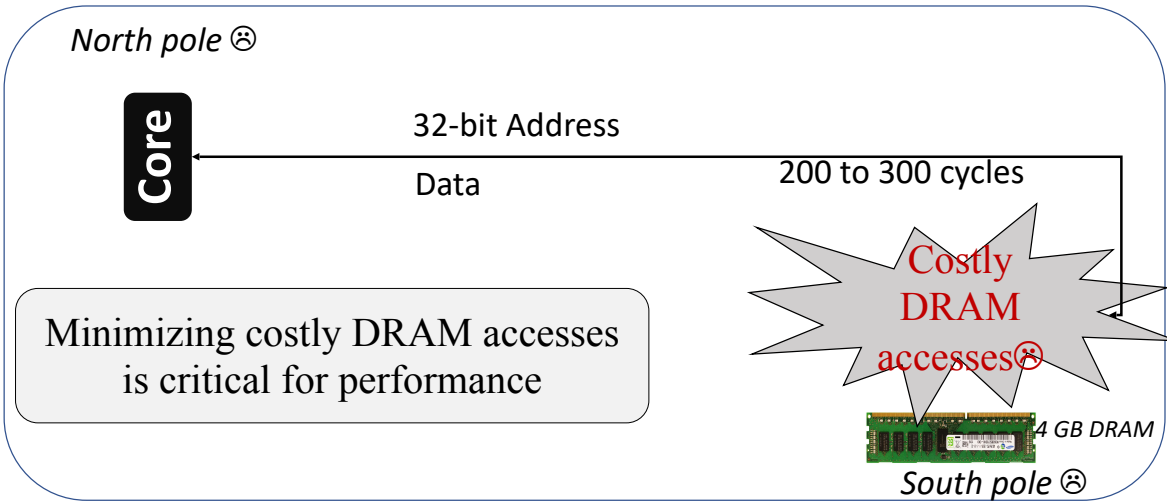
if data $\notin$ fast memory $\Rightarrow$ high latency access *DRAM*

Levels of the Memory Hierarchy



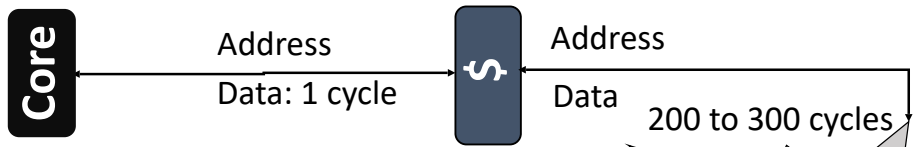


World with no caches



Cache with latency

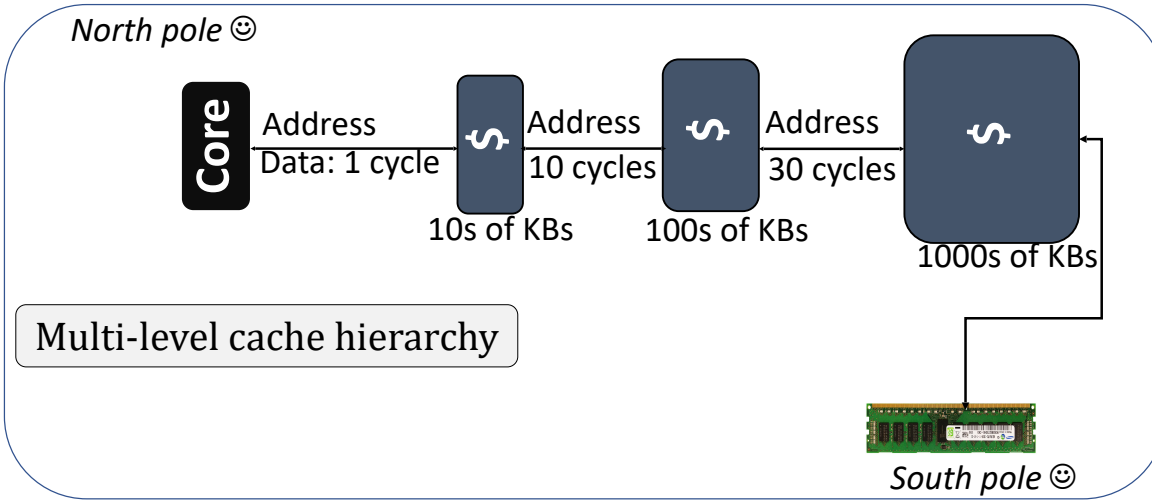
North pole ☺



32 to 64KB $ will be available in one to four cycles ☹

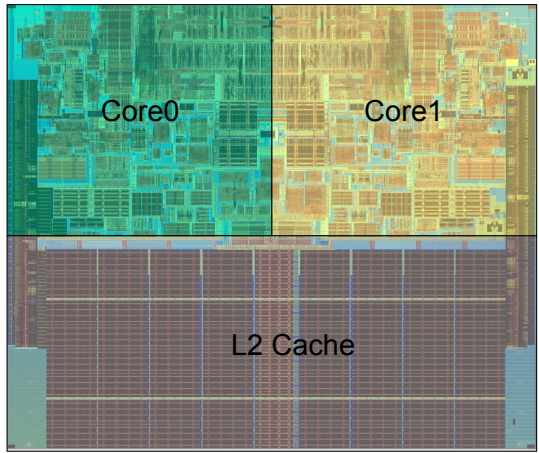
South pole ☺

Cache hierarchy with latency



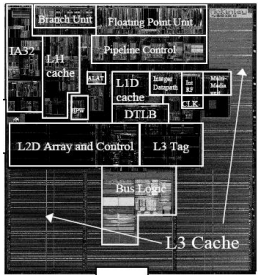
Case Study:Intel Core2 Duo

L1	32 KB, 8-Way, 64 Byte/ Line, LRU, WB 3 Cycle Latency
L2	4.0 MB, 16-Way, 64 Byte/Line, LRU, WB 14 Cycle Latency



Source: http://www.sandpile.org

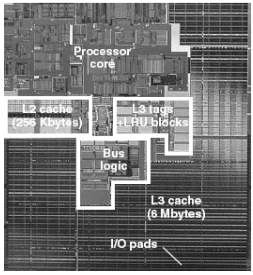
Case study: Intel Itanium 2



3MB
Version
180nm
421 mm^2

2002

6MB
Version
130nm
374 mm^2



Attribute	L1 instruction cache	L1 data cache	L2	L3
Size	16 Kbytes	16 Kbytes	256 Kbytes	6 Mbytes
Line size (bytes)	64	64	128	128
No. of ways	4	4	8	24
Replacement policy	Least recently used	Not recently used	Not recently used	Not recently used
Latency (no. of cycles)	1	Integer: 1 Floating-point: NA	Integer: 5 Floating-point: 6	14
Write policy	NA	Write through	Write back	Write back
Bandwidth (Gbytes/s)				
Read	48	24	48	48
Write	NA	24	48	48

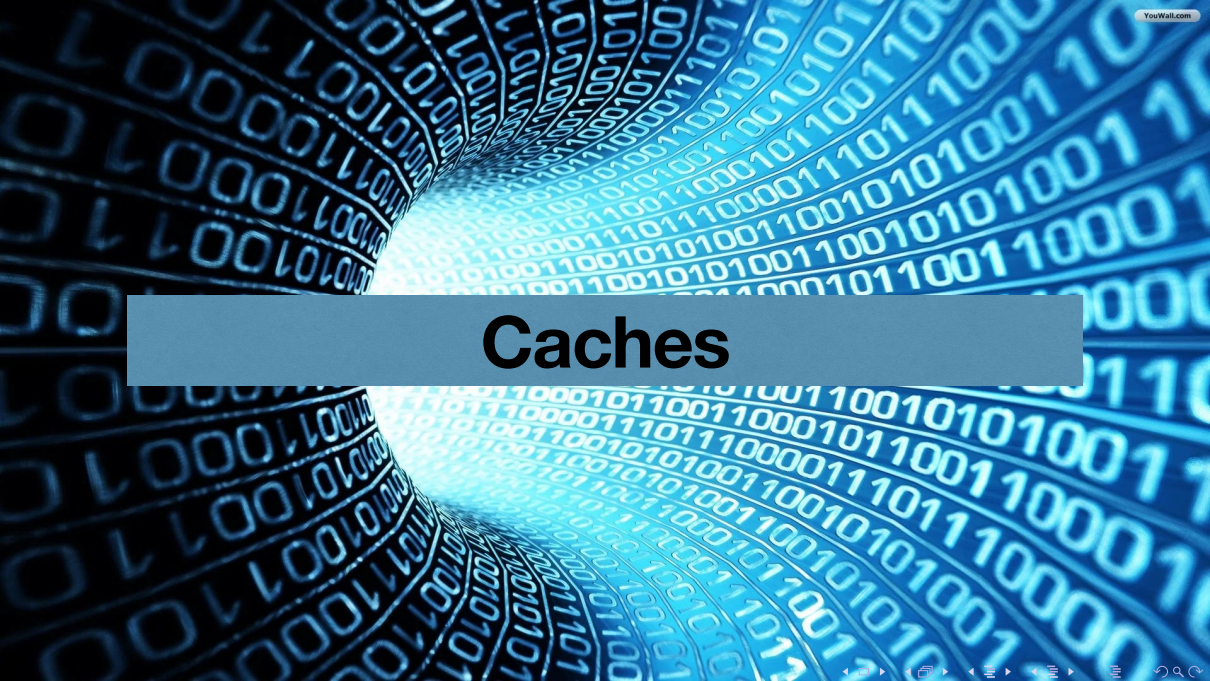
Book

P & H, Chapter 4

Digital Logic Design + Computer Architecture

Sayandeep Saha

Assistant Professor
Department of Computer Science and Engineering
Indian Institute of Technology Bombay



Caches

A Few Words About Performance

Performance — In our words

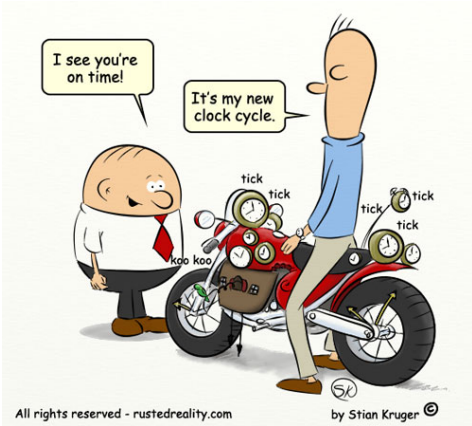
Which computer is faster?

The measure is *execution time*

“X is n times faster than Y”

$$n = \frac{Exetime_Y}{Exetime_X} = \frac{Perfromance_X}{Perfromance_Y}$$

Empirical Evaluation



- Benchmarks
- Metrics
- Simulators
- Latency and bandwidth

Evaluation

- To compare Processor A with Processor B by running programs
- How many programs?
- The programs that you care.
- What if I want to build a new one (processor, caches, DRAM) ?

World of Benchmarks

- SPEC CPU 2017 (https://www.spec.org/cpu2017/)

The **SPEC CPU® 2017** benchmark package contains SPEC's next-generation, industry-standardized, CPU intensive suites for measuring and comparing compute intensive performance, stressing a system's processor, memory subsystem and compiler.

SPECspeed: used for comparing time for a computer to complete single tasks

SPECrate: measure the throughput or work per unit of time.

What are there? From a GCC compiler, Gaming, Video compression, Chess(AI), Differential Equation solver, Numerical programs, searching genome sequence, quantum computer simulation and many more (SPEC 2006)

World of Benchmarks

CloudSuite (https://www.cloudsuite.ch/)

CloudSuite is a benchmark suite for cloud services. The benchmarks are based on real-world software stacks and represent real-world setups.

PARSEC (https://parsec.cs.princeton.edu/)

Benchmark suite composed of multithreaded programs. The suite focuses on emerging workloads and was designed to be representative of next-generation shared-memory programs for chip-multiprocessors.

World of Benchmarks

MobileBench (https://mobilebench.engineering.asu.edu/) comprising a selection of representative smart phone applications.

Many more application domain specific: Graph processing, ML perf,

Rules for Measuring Performance with Benchmarks

- No Source code modifications are allowed, or it is impossible
- Use one compiler, one language for all the benchmark programs and don't play with the flags

- Else, you can cheat!!!
- Also, use *benchmark suites*, not a single benchmark

Pitfalls of Benchmarks

Benchmark not representative of all

Your workload is I/O bound → SPECCPU is useless

Benchmark is too old

Need to be periodically refreshed

Non-benchmarks

- Application kernels: A small code fragment or part of the program
- Synthetic benchmark : Not part of any real program!!
- Micro-benchmark

World of Simulators

- Functional Simulator: Used to verify the correct execution of the program. Can not be used for performance evaluation.

- Performance simulators:

(i) Trace-driven: ChampSim (https://github.com/ChampSim/ChampSim)

(ii) Execution-driven: gem5, Multi2sim

Functional simulator is part of the performance simulators.

Evaluation Continued

Pick a *relevant* benchmark suite

Measure IPC of each program

Summarize the performance using:

- Arithmetic Mean (AM)
- Geometric Mean (GM)
- Harmonic Mean (HM)

Which one to choose?

Example

	ABM	AND
App. one	2	3
App. two	1.5	2
App. three	1.3	0.3
A.M.	1.60	1.76
G.M.	1.57	1.21
H.M.	1.54	0.72

AM on ratios

	X	Y
App. 1	1	100
App. 2	1000	10

Normalized to X	X	Y
App. 1	1	100
App. 2	1	0.01
AM	1	50.005

Y is 50 times faster than X

Normalized to Y	X	Y
App. 1	0.01	1
App. 2	100	1
AM	50.005	1

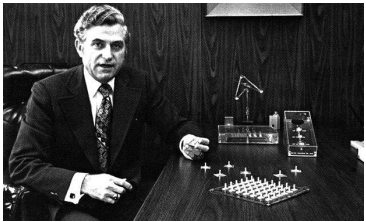
X is 50 times faster than Y

AM vs. GM

- GM of ratios is same as the ratio of the GMs
- Due to the aforementioned fact, the choice of reference does not matter if you go with GM.

Principles of Computer Design

Amdahl's Law (common case fast)

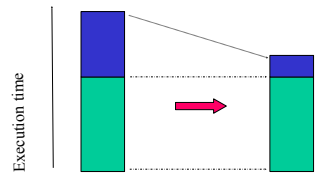


$$\text{Speedup}_{\text{overall}} = \frac{\text{Execution Time}_{\text{old}}}{\text{Execution Time}_{\text{new}}}$$

$$= \frac{1}{(1 - \text{Fraction}_{\text{enhanced}}) + \frac{\text{Fraction}_{\text{enhanced}}}{\text{Speedup}_{\text{enhanced}}}}$$

Amdahl's Law and Speedup

- Speedup: How much faster a machine will run due to an enhancement?
- Need to consider two things while using Amdahl's law:
  - 1st… **Fraction of the computation time that can use the enhancement**
    - If a program executes in 30 seconds and 15 seconds of exec. uses enhancement, fraction = ½
  - 2nd… **Improvement gained by enhancement**
    - If enhanced task takes 3.5 seconds and original task took 7secs, we say the speedup is 2.



Amdahl's Law: Example

- Floating point instructions improved to run 2 times faster.
- But, only 10% of actual instructions are FP

- $ExTime_{new} = ?$
- $Speedup_{new} = ?$

Example: Answer

- Floating point instructions improved to run 2X faster.
  - But only 10% of actual instructions are FP.

$$ExTime_{new} = ExTime_{old} \times (0.9 + 0.1/2) = 0.95 \times ExTime_{old}$$

$$Speedup_{overall} = \frac{1}{0.95} = 1.053$$

Example 2

A common transformation required in graphics processors is square root. Implementations of floating-point (FP) square root vary significantly in performance, especially among processors designed for graphics. Suppose FP square root (FSQRT) is responsible for 20% of the execution time of a critical graphics benchmark. One proposal is to enhance the FSQRT hardware and speed up this operation by a factor of 10. The other alternative is just to try to make all FP instructions in the graphics processor run faster by a factor of 1.6; FP instructions are responsible for half of the execution time for the application. The design team believes that they can make all FP instructions run 1.6 times faster with the same effort as required for the fast square root. Compare these two design alternatives.

Example 2

We can compare these two alternatives by comparing the speedups:

$$\text{Speedup}_{\text{FSQRT}} = \frac{1}{(1-0.2)+\frac{0.2}{10}} = \frac{1}{0.82} = 1.22$$

$$\text{Speedup}_{\text{FP}} = \frac{1}{(1-0.5)+\frac{0.5}{1.6}} = \frac{1}{0.8125} = 1.23$$

Improving the performance of the FP operations overall is slightly better because of the higher frequency.

Amdahl's Law

Which one will provide better overall speedup?

A. Small speedup on the large fraction of execution time.
B. Large speedup on the small fraction of execution time.
C. Does not matter.

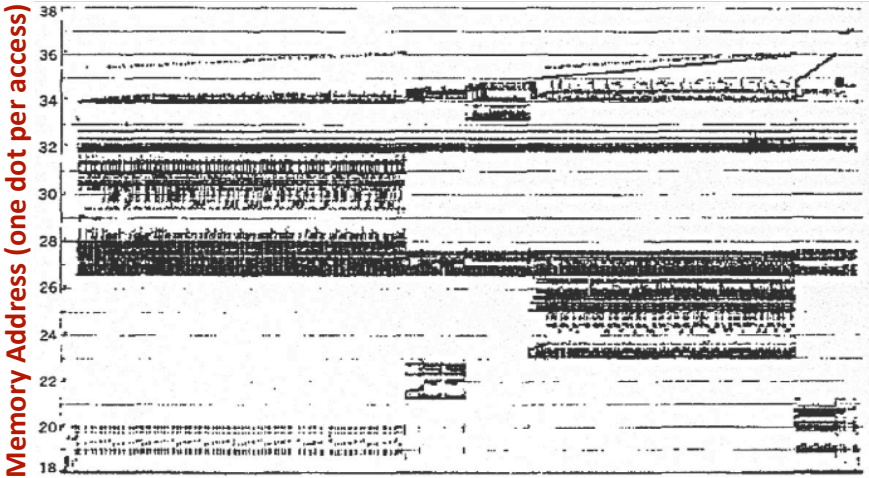
Depends on the difference between small and large. Mostly it is A.

Principle of Locality

- Programs tend to use the data and instructions they have used recently.

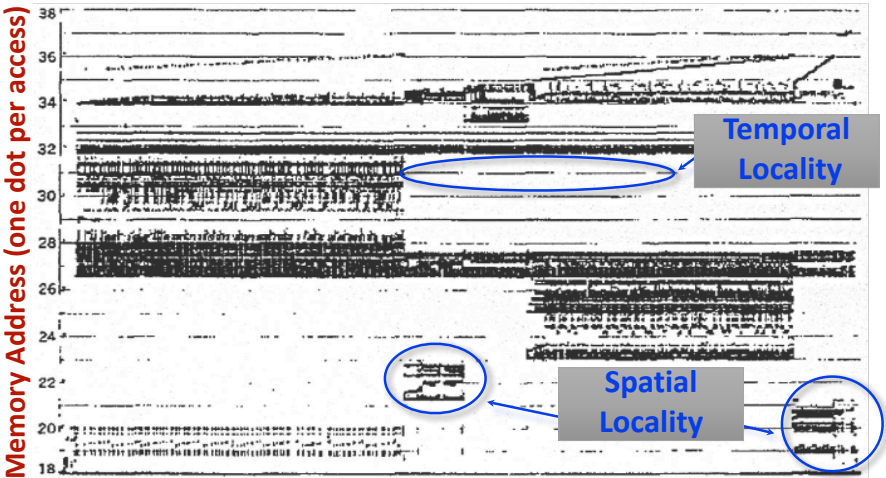
- So from the recent past, we can have a good idea of future
- Temporal Locality: Recently accessed items are to be used in near future.
- Spatial Locality: Items whose addresses are near to each other tend to referenced close together in time.

Let's look at the Applications (benchmarks)



Donald J. Hatfield, Jeanette Gerald: Program Restructuring for Virtual Memory. IBM Systems Journal 10(3): 168-192 (1971)

Locality



Donald J. Hatfield, Jeanette Gerald: Program Restructuring for Virtual Memory. IBM Systems Journal 10(3): 168-192 (1971)

Back to Memory

World with no caches

North pole ☹

Core

32-bit Address

Data

200 to 300 cycles

Costly DRAM accesses☹

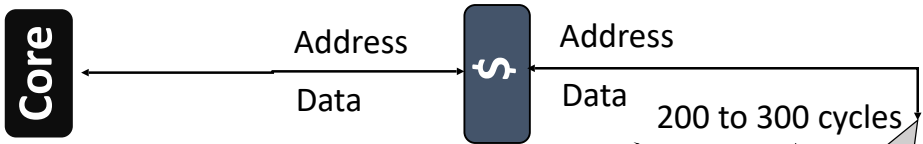
4 GB DRAM

South pole ☹

Minimizing costly DRAM accesses is critical for performance

Caching: Why does it works…?

North pole ☺



Caching is a speculation technique ☺
Works – if locality

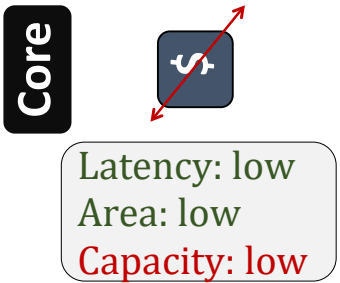
Do not ignore the common case

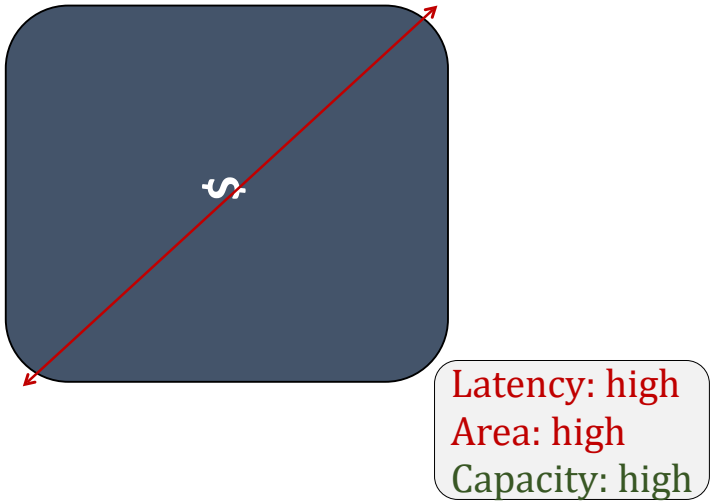
Reduction in DRAM accesses ~ Improvement in execution time

WRONG!

What if your program is not memory intensive

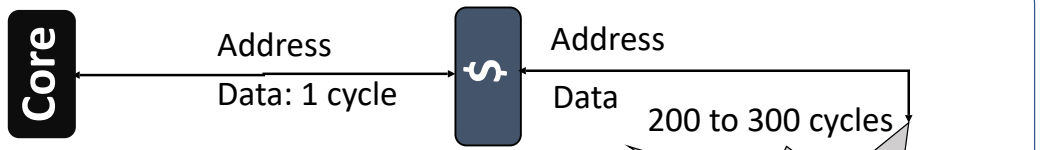
How big/small?





Cache with latency

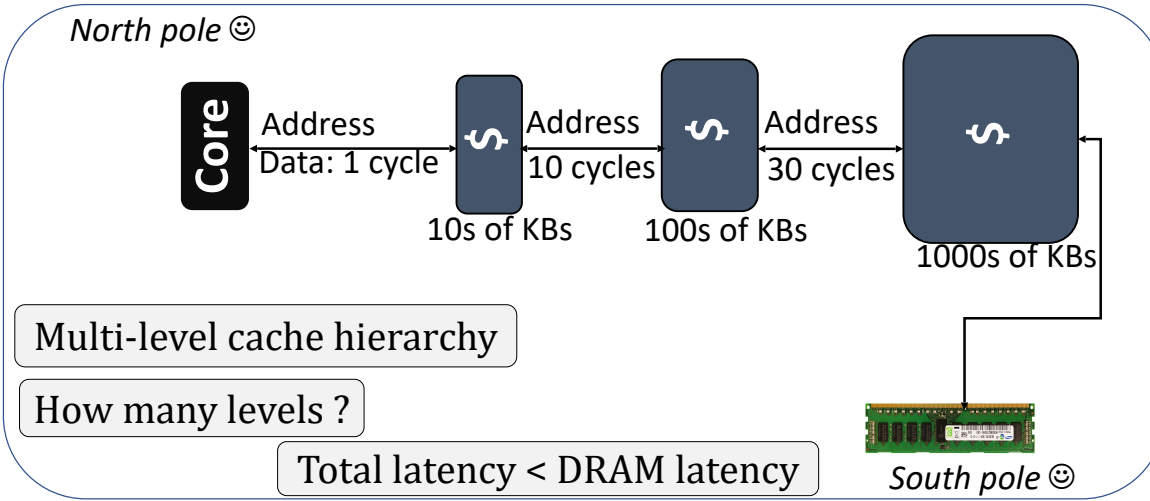
North pole ☺



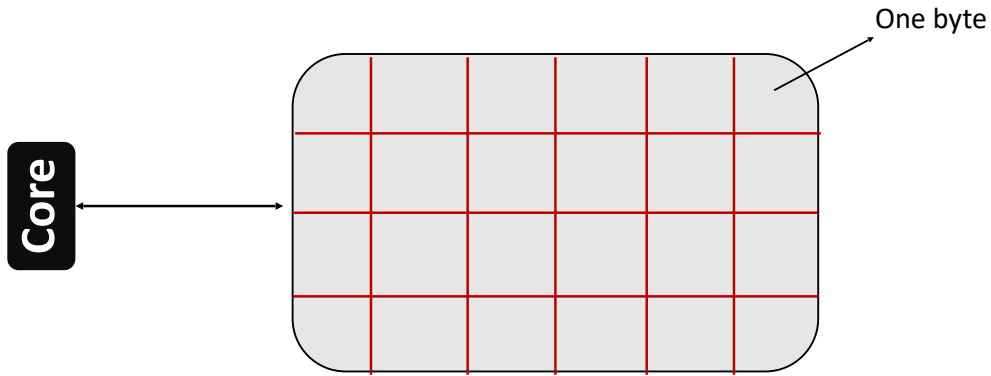
32 to 64KB $ will be available in one to four cycles ☹

South pole ☺

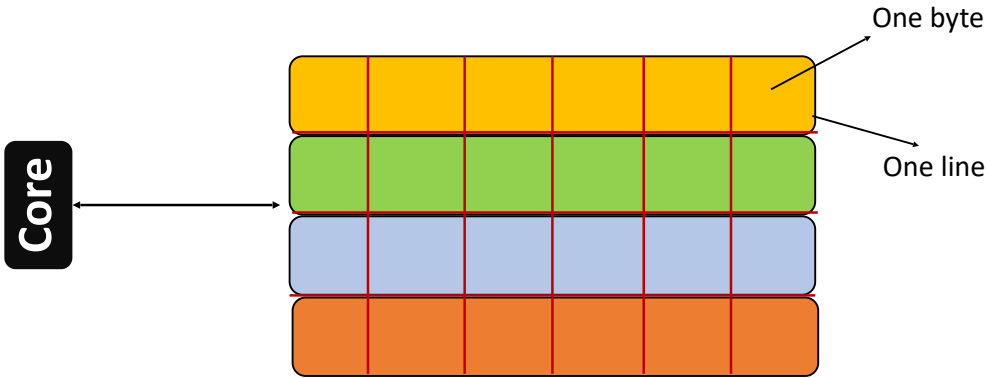
Cache hierarchy with latency



Accessing a cache

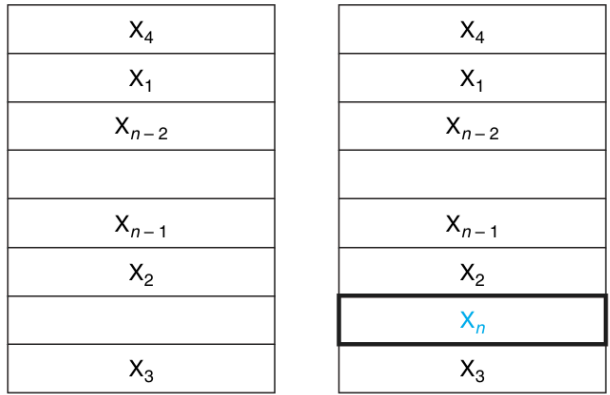


Bytes to blocks (lines)



Typical line size: 64 to 128 Bytes

Before and After You Access...

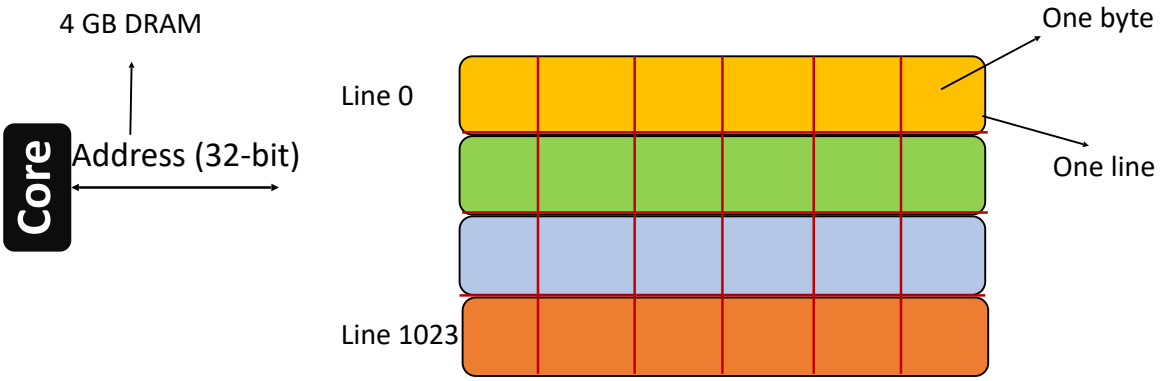


a. Before the reference to X_n b. After the reference to X_n

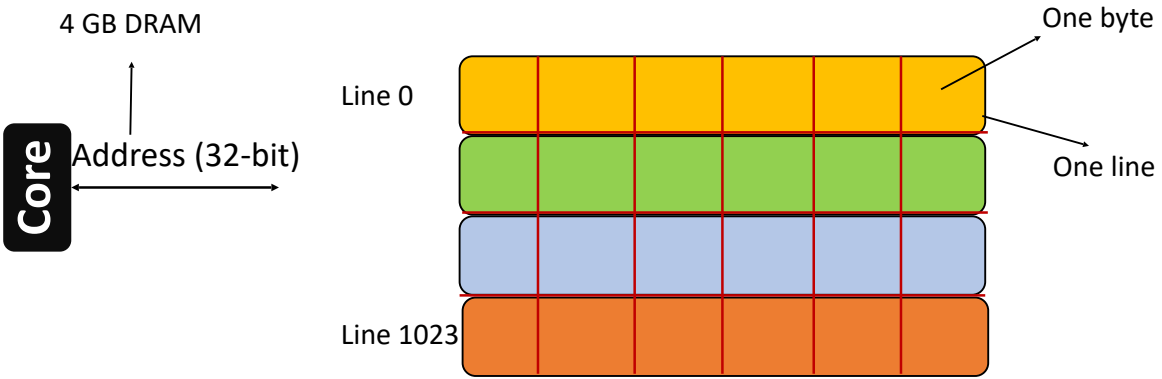
Accessing a cache

- Although cache blocks are of 16/128 bytes processor can still access 1 byte or one word — how?
  - Offset…
- How to efficiently utilize the limited memory? — every program wants to use it no?
  - Cache associativity and replacement and tags

A bit deeper: 1024 lines each of 32B



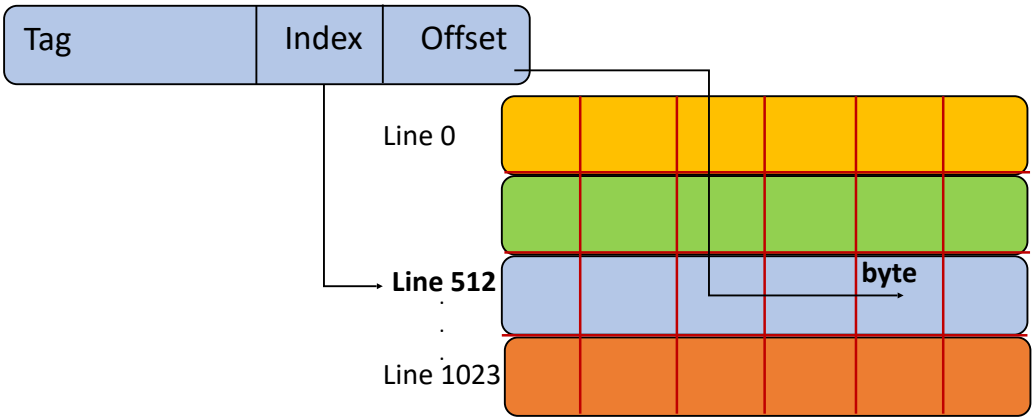
A bit deeper: 1024 lines each of 32B



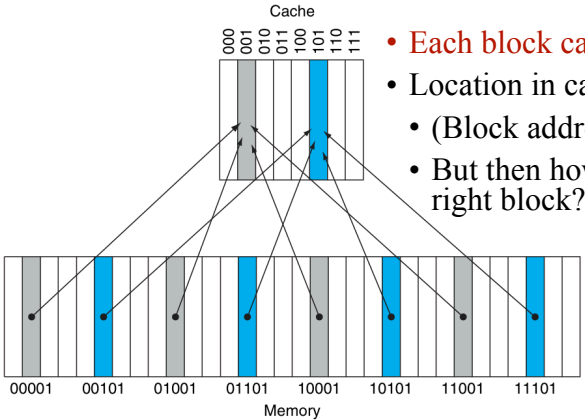
Line number (index): 10 bits

Byte offset (offset): 5 bits

Direct Mapped Cache

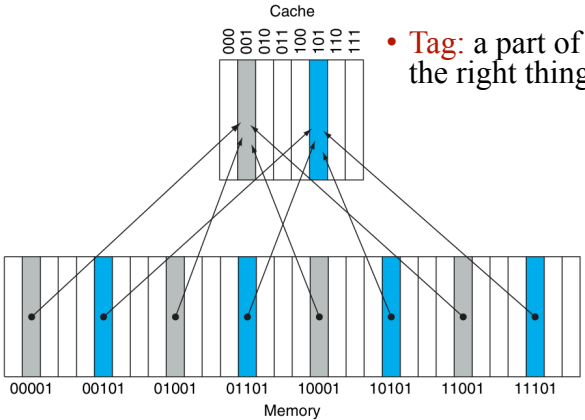


Direct Mapped Cache



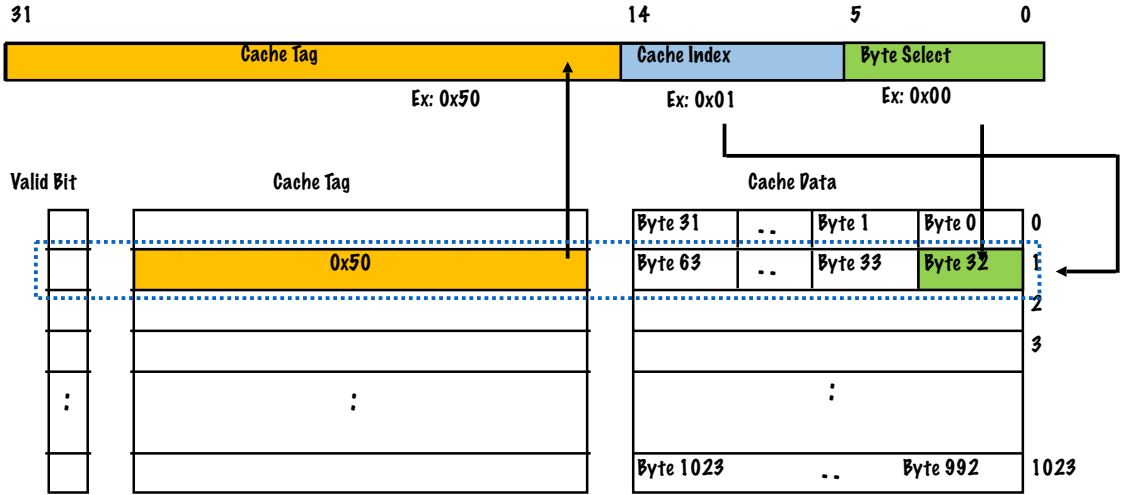
- Each block can go to exactly one place in the cache
- Location in cache:
  - (Block address)mod (number of blocks in the cache)
  - But then how do you know you are accessing the right block?

Direct Mapped Cache



- Tag: a part of your address which helps you identify the right thing…

Direct Mapped in Action



Accessing a Cache

Binary address of reference	Assigned cache block (where found or placed)
10110_{two}	$(10110_{two} \bmod 8) = 110_{two}$
11010_{two}	$(11010_{two} \bmod 8) = 010_{two}$
10110_{two}	$(10110_{two} \bmod 8) = 110_{two}$
11010_{two}	$(11010_{two} \bmod 8) = 010_{two}$
10000_{two}	$(10000_{two} \bmod 8) = 000_{two}$
00011_{two}	$(00011_{two} \bmod 8) = 011_{two}$
10000_{two}	$(10000_{two} \bmod 8) = 000_{two}$
10010_{two}	$(10010_{two} \bmod 8) = 010_{two}$
10000_{two}	$(10000_{two} \bmod 8) = 000_{two}$

Accessing a Cache

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	N		
111	N		

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

Index	V	Tag	Data
000	N		
001	N		
010	Y	11_{two}	Memory (11010_{two})
011	N		
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

Index	V	Tag	Data
000	Y	10_{two}	Memory (10000_{two})
001	N		
010	Y	11_{two}	Memory (11010_{two})
011	N		
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

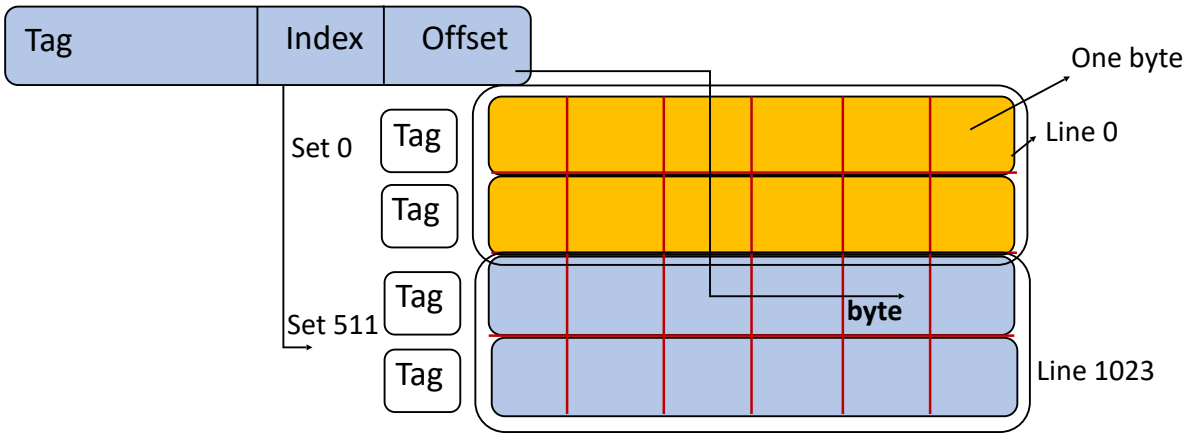
Accessing a cache

- Hit and Miss: you may (hit) or may not (miss) find the data inside the cache…
- When you access for the first time, it is always a miss (compulsory miss)
- When the cache is full, it will be another miss (capacity miss)
- When there is conflict, it can be miss again (conflict miss)

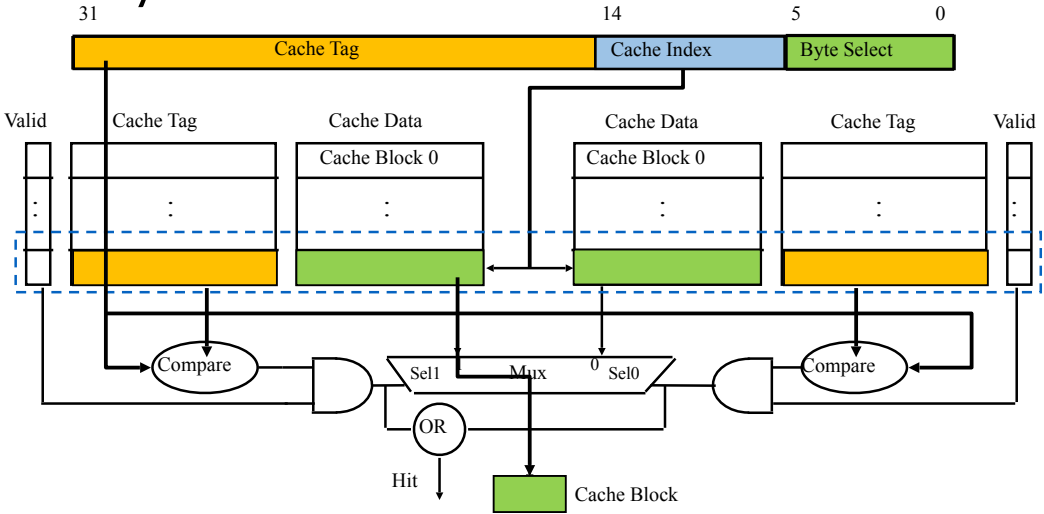
Concept of Valid Bit

- Does the cache word really contains something meaningful?
- Suppose you are just starting to run, at that time even if your tag matches, it might be useless data
- Valid bit: Indicates if a data is stale or useful

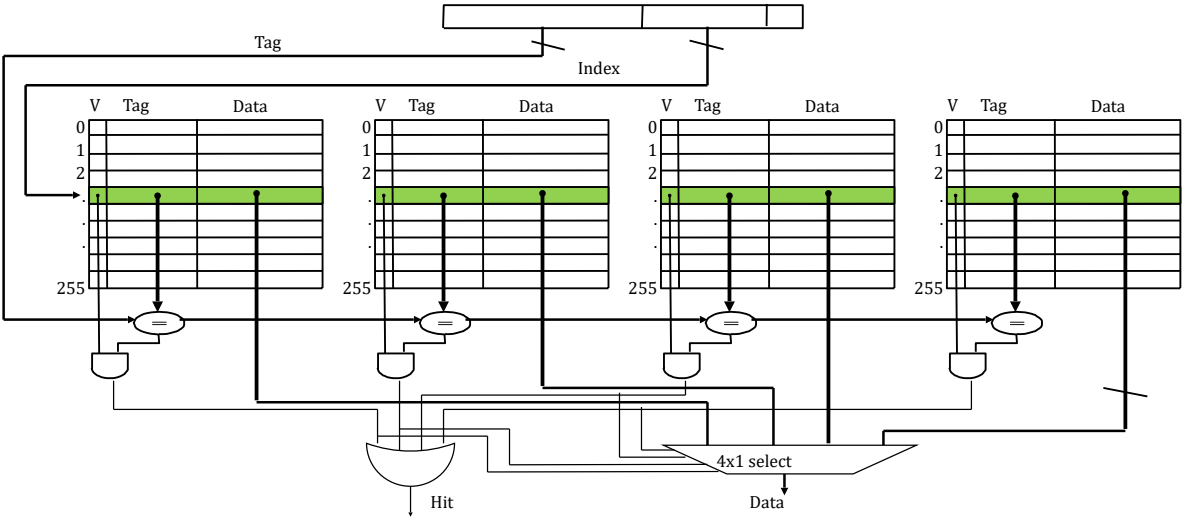
What if we have multiple ways?



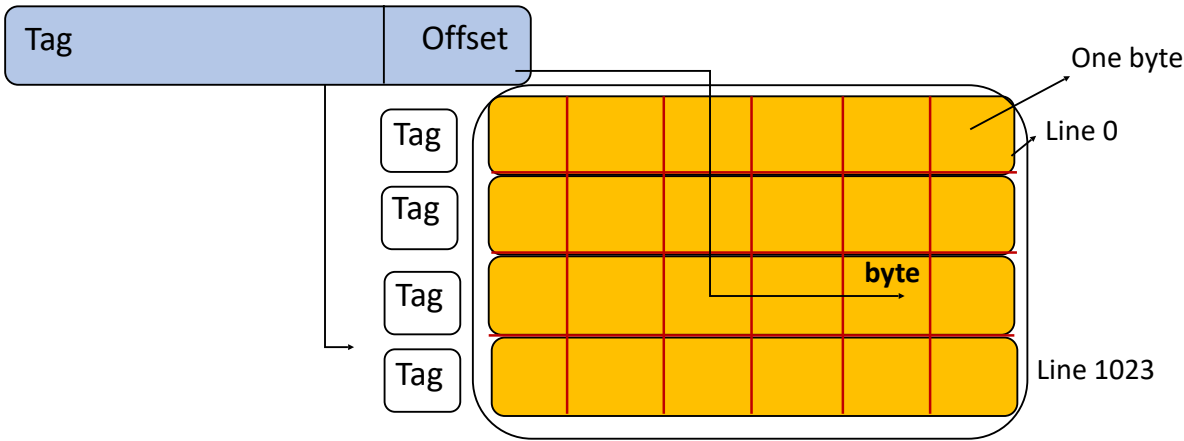
2-way associative in action



4-way associative: Just a better picture



Extreme: One cache, one set, fully associative



Knobs of interest

Line size, associativity, cache size

Tradeoff: latency, complexity, energy/power

Line size = one byte or cache size
Associativity = one or #lines
Cache size = Goal oriented: latency/bandwidth or capacity

Metrics

- Hit time: time taken to handle a hit
- Miss rate: What percentage of cache access results in a miss
- Miss penalty: How much time it takes to serve a miss

On a Miss, Replace a block, which block?

Think of each block in a set having a “priority”
Indicating how important it is to keep the block in the cache

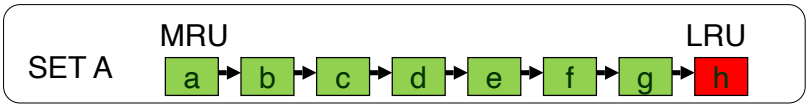
Key issue: How do you determine/adjust block priorities?

Ideally: Belady’s OPT policy, replace the block that will be used furthest in the future. No one knows the future though ☺

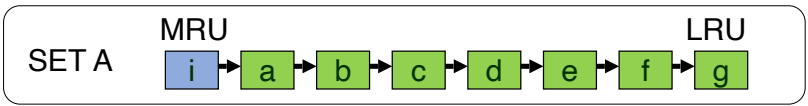
There are three key decisions in a set:
Insertion, promotion, eviction (replacement)

A simple LRU (Least-Recently-Used) Policy

Cache Eviction Policy: On a miss (block *i*), which block to evict (replace) ?



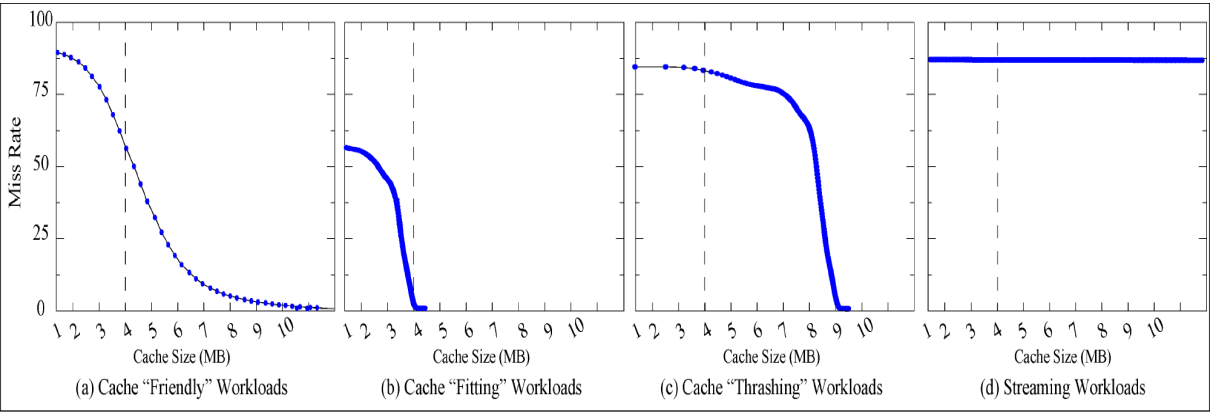
Cache Insertion Policy: New block *i* inserted into MRU.



Cache Promotion Policy: On a future hit (block *i*), promote to MRU

We need priority bits per block. For example, a 16-way cache will need four bit/ block LRU causes thrashing when working set > cache size

Types of Applications



Cache misses once more

Compulsory: first reference to a line (a.k.a. cold start misses)

- *misses that would occur even with infinite cache*

Capacity: cache is too small to hold all data

- *misses that would occur even under perfect (Belady's) replacement policy*

Conflict: misses that occur because of collisions due to line-placement strategy

- *misses that would not occur with ideal full associativity*

Cache knobs and Misses

- Larger cache size
  +reduces capacity and conflict misses?
  - hit time will increase
- Higher associativity
  +reduces conflict misses
  - increase hit time
- Larger line size
  +reduces compulsory misses
  - increases conflict misses and miss penalty

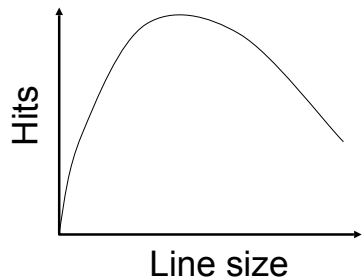
Line size

Too small blocks:

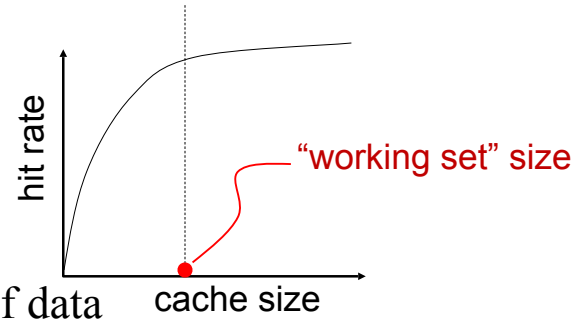
- don't exploit spatial locality well
- have larger tag overhead

Too large blocks:

- too few total # of blocks
  - likely-useless data transferred
  - Extra bandwidth/energy consumed

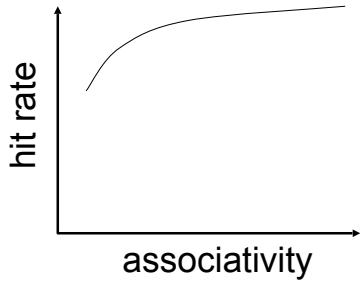


Cache size



Working set: the whole set of data the executing application references within a time interval

Associativity

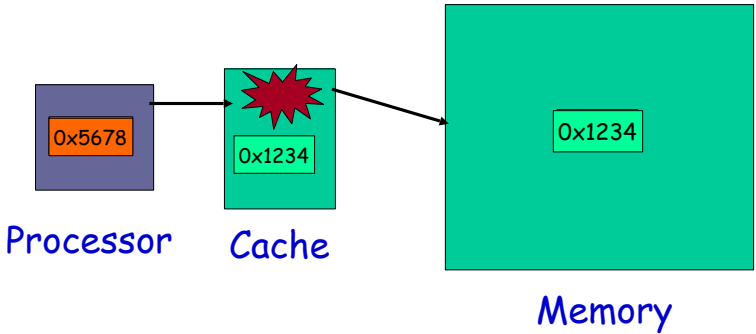


L1 cache: lower associativity, hit time
L3 cache: higher associativity

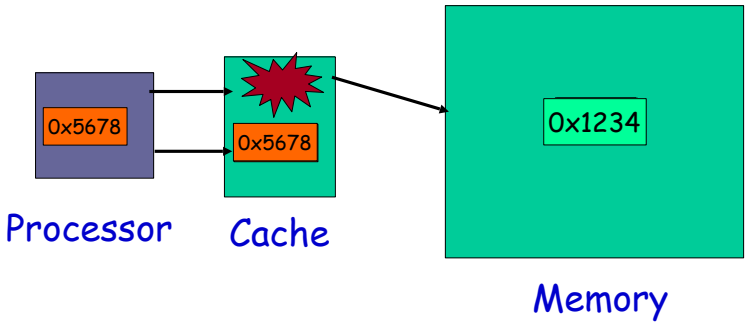
Cache Write Policies

- **Write-through:** Information is written to both the block in the cache and that in memory
- **Write-back:** Information is written back to memory only when a block frame is replaced:
  - **Uses a "dirty" bit to indicate whether a block was actually written to,**
  - Saves unnecessary writes to memory when a block is "clean"

Write-Through Policy



Write back Policy



Trade-offs

- **Write back:**
  - Faster because writes occur at the speed of the cache, not the memory.
  - Faster because multiple writes to the same block is written back to memory only once, uses less memory bandwidth.
- **Write through:**
  - Easier to implement

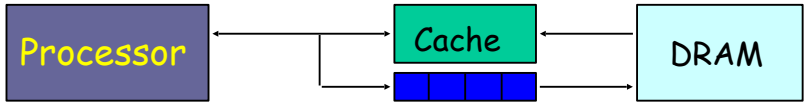
Write Allocate, No-write Allocate

- What happens on a write miss?
  - On a read miss, a block has to be brought in from a lower level memory
- **Two options:**
  - **Write allocate**: A block allocated in cache.
  - **No-write allocate**: No block allocation, but just written to main memory.

Write Allocate, No-write Allocate

- In no-write allocate,
  - Only blocks that are read from can be in cache.
  - Write-only blocks are never in cache.
- **Can either be used with write through and write back?**
  - Write-allocate used with write-back
  - No-write allocate used with write-through
- **Why does this make sense?**

Write Buffer



- **Processor:**
  - writes data into cache and write buffer
- **Memory controller:**
  - writes contents of the buffer to memory
- **Write buffer is a FIFO structure:**
  - Typically 4 to 8 entries
  - Desirable: Occurrence of Writes << DRAM write cycles
- **Memory system designer's nightmare:**
  - Write buffer saturation (i.e., Writes ~ DRAM write cycles)

Cache Performance Parameters

- AMAT is largely determined by:
  - **Cache miss rate:** number of cache misses divided by number of accesses.
  - **Cache hit time:** the time between sending address and data returning from cache.
  - **Cache miss penalty:** the extra processor stall cycles caused by access to the next-level cache.

Average Memory Access Time

- **AMAT:** The average time it takes for the processor to get a data item it requests.
  - Can vary considerably for different memory configurations due to various attributes of the memory hierarchy.
- AMAT can be expressed as:

$$\text{AMAT} = \text{Cache hit time} + \text{Miss rate} \times \text{Miss penalty}$$

Impact of Memory System on Processor Performance

CPU Performance $_{\text{with Memory Stall}}$ = CPI $_{\text{without stall}}$ + Memory Stall CPI

Memory Stall CPI

= Miss per inst × miss penalty

= % Memory Access/Instr × Miss rate × Miss Penalty

Example: Assume 20% memory acc/instruction, 2% miss rate, 400-cycle miss penalty. How much is memory stall CPI?

Memory Stall CPI= 0.2*0.02*400=1.6 cycles

CPU Performance with Memory Stall

CPU Performance $_{\text{with Memory Stall}}$ = CPI $_{\text{without stall}}$ + Memory Stall CPI

$$\text{CPU time} = \text{IC} \times \left(\text{CPI}_{\text{execution}} + \text{CPI}_{\text{mem_stall}}\right) \times \text{Cycle Time}$$

$$\text{CPI}_{\text{mem_stall}} = \text{Miss per inst} \times \text{miss penalty}$$

$$\text{CPI}_{\text{mem_stall}} = \text{Memory Inst Frequency} \times \text{Miss Rate} \times \text{Miss Penalty}$$

Performance Exercise 1

- Suppose:
  - Clock Rate = 200 MHz (5 ns per cycle), Ideal (no misses) CPI = 1.1 (including control stalls)
  - 50% arith/logic, 30% load/store, 20% control
  - 10% of data memory operations incur 50 cycles miss penalty
  - 1% of instruction memory operations also incur 50 cycles miss penalty
- Compute CPI with cache miss and AMAT.

Solution

- CPI = ideal CPI + average stalls per instruction =
  1.1(cycles/ns) + [0.30 (DataMops/ins)
  x 0.10 (miss/DataMop) x 50 (cycle/miss)] +[1
  (InstMop/ins)
  x 0.01 (miss/InstMop) x 50 (cycle/miss)]
  = (1.1 + 1.5 + .5) cycle/ins = 3.1
- Combined AMAT
  (normalised)=(1/1.3)x[1+0.01x50]+(0.3/1.3)x[1+0.1x50]=2.
  54

Exercise 2

- Assume 20% Load/Store instructions
- Assume CPI without memory stalls is 1
- Cache hit time = 1 cycle
- Cache miss penalty = 100 cycles
- Miss rate = 1%
- What is:
  - Stall cycles per instruction?

Exercise 2:Solution

- Average memory accesses per instruction = 1.2
- Stall cycles = 1.2 cycles
  - Instruction misses per instruction: $0.01 \times 100 = 1.0$ cycles / instruction.
  - Data misses per instruction: $0.20 \times 0.01 \times 100 = 0.2$ cycles/instr

Miss Rate Vs. MPKI

Miss rate

Misses per kilo instructions (MPKI)

But Why?

Miss Rate Vs. MPKI

Miss rate

$$\text{Miss Rate} = \frac{\text{\# of cache misses}}{\text{\# of cache accesses}}$$

MPKI

$$\text{MPKI} = \frac{\text{\# of cache misses}}{\text{\# of instructions executed}} \times 1000$$

What if you have only one memory access?

MPKI is often used for reporting performance evaluation results on benchmarks, as it takes care of the frequency of memory instructions real benchmarks

Memory Hierarchy Optimizations

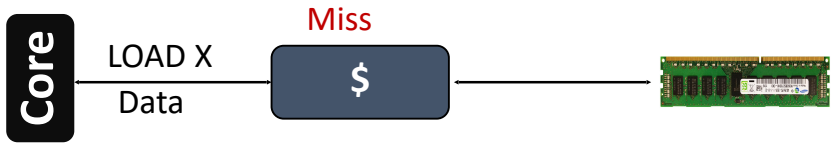
Reducing Miss Rates

- Techniques:
  - Larger block size
  - Larger cache size
  - Higher associativity

Reducing Miss Penalty

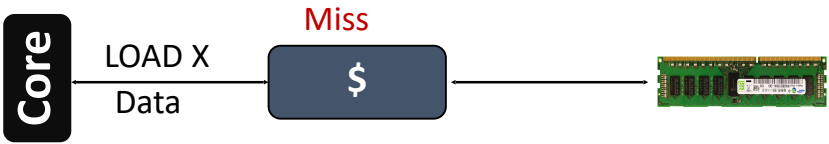
- Techniques:
  - Critical word first
  - Multilevel caches

On a miss: Critical Word first



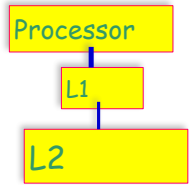
On a miss, respond with the word/byte requested to the core so that core can continue while fetching the rest of the block

On a miss: Early Restart



On a miss, fetch the words/bytes in normal order, but as soon as the requested word/byte of the block arrives, send it to the core.

Multi-Level Cache



- Add a second-level cache.
- L2 Equations:

AMAT = Hit Time_{L1} + Miss Rate_{L1} x Miss Penalty_{L1}

Miss Penalty_{L1} = Hit Time_{L2} + Miss Rate_{L2} x Miss Penalty_{L2}

AMAT = Hit Time_{L1} + Miss Rate_{L1} x (Hit Time_{L2} + Miss Rate_{L2} × Miss Penalty_{L2})

Multi-Level Cache: Some Definitions

- **Local miss rate:**
  - **M**isses in this cache divided by the total number of memory accesses to this cache (e.g. Miss rate$_{L2}$)
- **Global miss rate:**
  - Misses in this cache divided by the total number of memory accesses generated by the CPU
- L1 Global miss rate = L1 Local miss rate

Global vs. Local Miss Rates

- At lower level caches (L2 or L3), global miss rates provide more useful information:
  - **Indicate how effective is the cache in reducing AMAT.**
  - **Who cares if the miss rate of L3 is 50% as long as only 1% of processor memory accesses ever benefit from it?**

Performance Improvement Due to L2 Cache: Exercise

Assume:

- For 1000 memory instructions:
  - 40 misses in L1,
  - 20 misses in L2
- L1 hit time: 1 cycle,
- L2 hit time: 10 cycles,
- L2 miss penalty=100 cycles
- 1.5 memory references per instruction
- Assume ideal CPI=1.0

Find: Local miss rate of L2, AMAT, stall cycles per instruction, and those for case without L2 cache.

Solution

- **With L2 cache:**
  - Local miss rate of L2 = 50%
  - AMAT=1+(40/1500)X(10+50%X100)=1.92
  - Average Memory Stalls per Instruction = (1.92-1.0)x1.5=2.84
- **Without L2 cache:**
  - AMAT=1+40/1500X100=3.7
  - Average Memory Stalls per Inst=(3.7-1.0)x1.5=4.1
- **Perf. Improv. with L2** =(4.1+1)/(2.84+1)=32%

Note: We have not distinguished reads and writes. Access L2 only on L1 miss, i.e. write back cache…

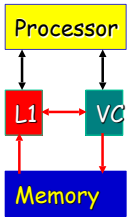
Multilevel Cache: Some Issues

- The speed (hit time) of L1 cache affects the clock rate of CPU:
  - Speed of L2 cache only affects miss penalty of L1.
- **Inclusion Policy:**
  - Many designers keep L1 and L2 block sizes the same.
  - Otherwise on a L2 miss, several L1 blocks may have to be invalidated.
- **Multilevel Exclusion:**
  - L1 data never found in L2 ---Saves some L2 space
  - AMD Athlon follows exclusion policy .

Reducing Miss Penalty: Victim Cache

- How to combine fast hit time of direct mapped cache:
  - yet avoid conflict misses?
- **Add a fully associative buffer (victim cache) to keep data discarded from cache.**
- Jouppi [1990]:
  - **A 4-entry victim cache removed 20% to 95% of conflicts for a 4 KB direct mapped data cache.**
- Used in Alpha, HP machines.
- AMD uses 8-entry victim buffer.

Victim Cache [Jouppi'90]



Victim Cache Organization

- A small, fully associative structure
- Effective in direct-mapped caches
- Whenever a line is displaced from L1 cache, it is loaded into VC
- Processor checks both L1 and VC simultaneously
- **Swap** data between VC and L1 if L1 misses and VC hits
- When data has to be evicted from VC, it is written back to memory

Reducing Miss Penalty or Miss Rates via Parallelism

- Techniques:
  - Non-blocking caches
  - Prefetching

Non-blocking Caches

- Non-blocking cache:
  - Allow data cache to continue to serve other requests during a miss.
  - Meaningful only with out-of-order execution processor.
  - **Requires multi-bank memories.**
  - **Pentium Pro allows 4 outstanding memory misses.**

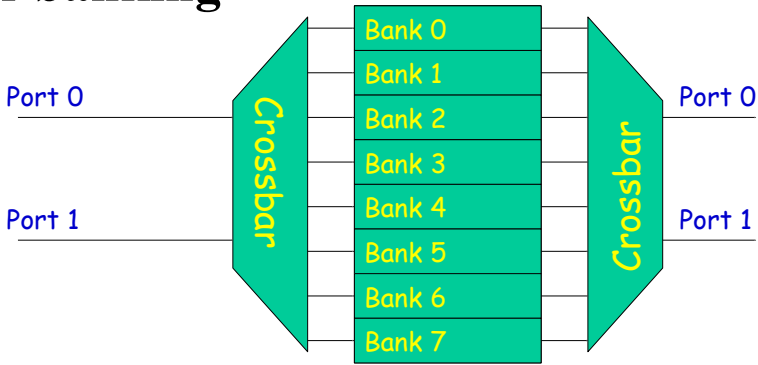
Non-blocking Caches

- Hit under miss reduces the effective miss penalty by working during miss.
- "Hit under multiple miss" may further lower the effective miss penalty:
  - By overlapping multiple misses.
  - Significantly increases the complexity of the cache controller as there can be multiple outstanding memory accesses.
  - Requires multiple memory banks.

Non-Blocking Caches

- Multiple memory controllers:
  - Allow memory banks to operate almost independently.
  - Each bank needs separate address lines and data lines.

Multi-banking



- Used in Intel Pentium (8 banks)
- Need routing network
- Must deal with bank conflicts

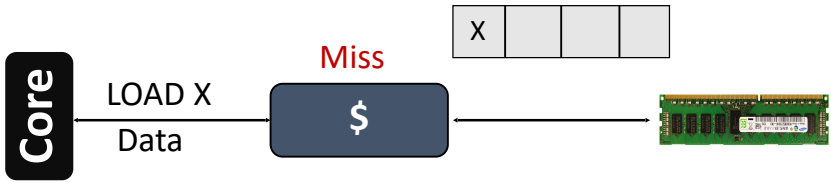
Core, cache, DRAM interaction

100s of cycles

I am an out-of-order core

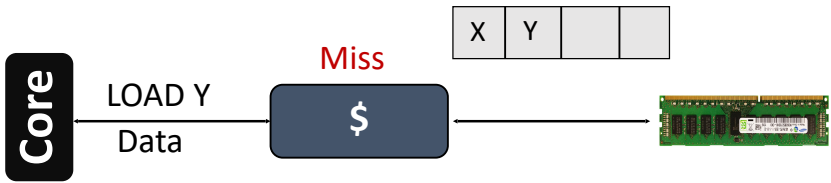
One cache miss and can't handle anymore misses

MSHRS (Miss-status holding registers)

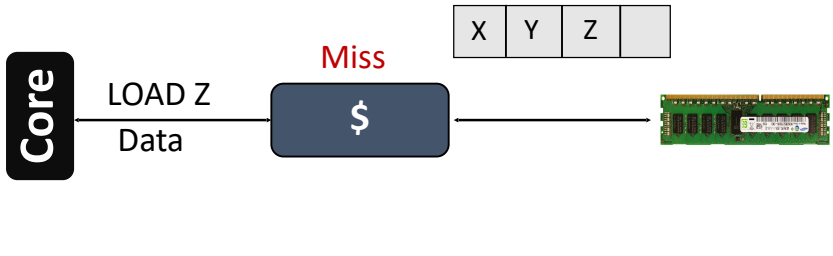


Non-blocking cache

MSHRS (Miss-status holding registers)

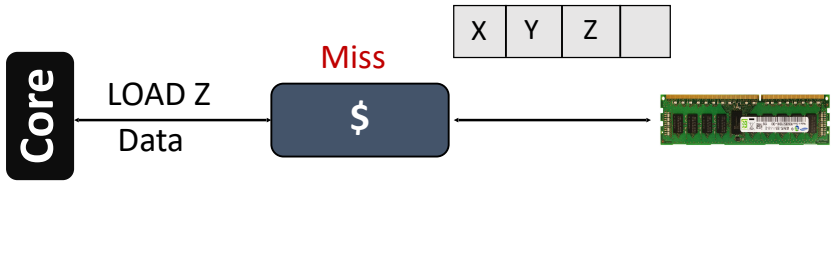


MSHRS (Miss-status holding registers)



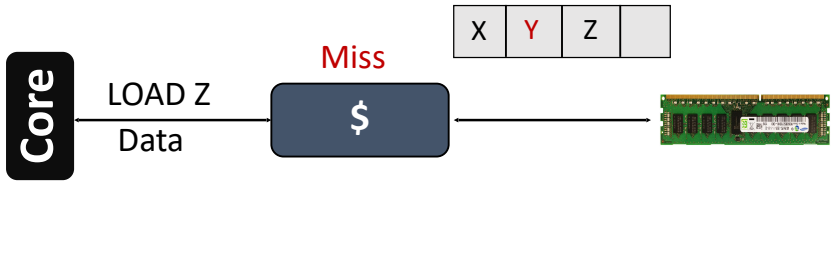
K-entry MSHR allows K outstanding misses: provides memory-level parallelism

MSHRS (Miss-status holding registers)



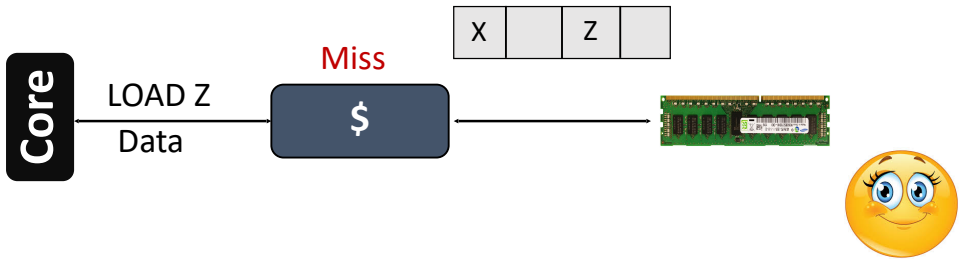
DRAM response time is not constant: can take from 60 cycles to 1000s of cycles (on a multi-core system).

MSHRS (Miss-status holding registers)



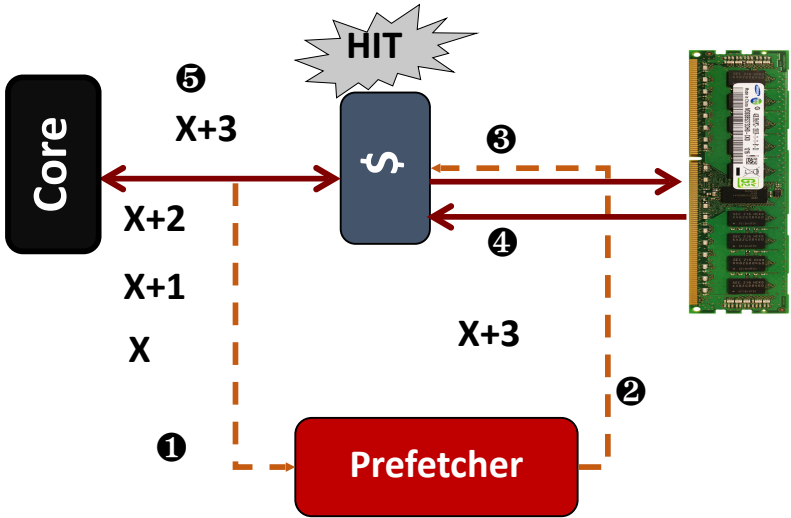
DRAM response time is not constant: can take from 60 cycles to 1000s of cycles (on a multi-core system).

MSHRS (Miss-status holding registers)



DRAM response time is not constant: can take from 60 cycles to 1000s of cycles (on a multi-core system).

Hardware Prefetching



10K Feet View

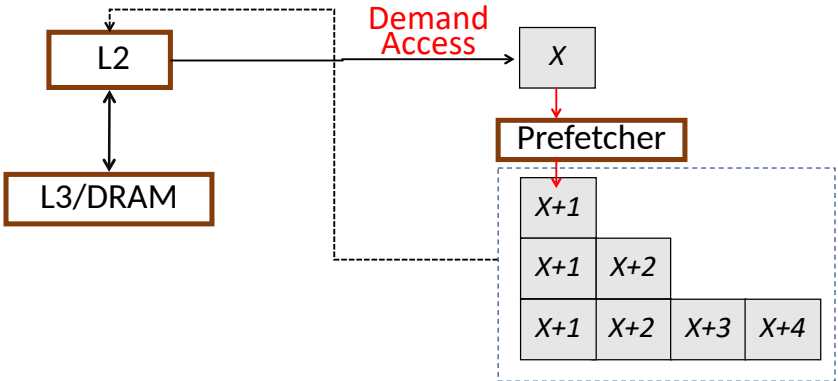
What?
Latency-hiding technique - Fetches data before the core demands.

Why?
Off-chip DRAM latency has grown up to 400 to 800 cycles.

How?
By observing/predicting the demand access (LOAD/STORE) patterns.

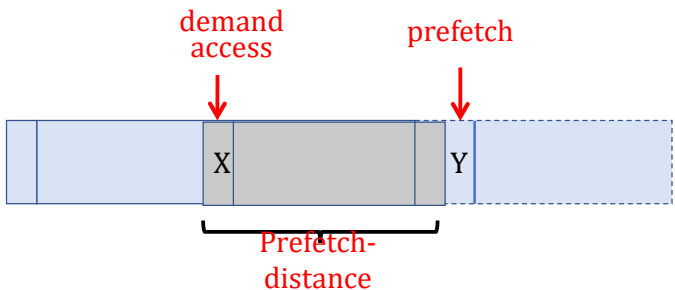
Prefetch Degree

Prefetch Degree: Number of prefetch requests to issue at a given time.



Prefetch Distance

Prefetch Distance: How far ahead of the demand access stream are the prefetch requests issued?



Next-line prefetcher

Next Line: Miss to cache block X , prefetch X+1. Degree=1, Distance=1

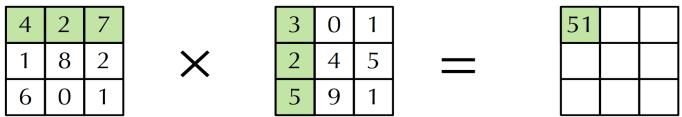
Works well for L1 Icache and L1 Dcache.

Compiler Level Optimization for Speedup

Matrix Multiplication: 101

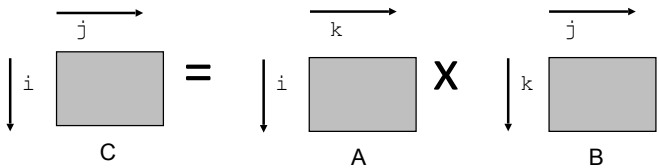
```
/* ijk */
for (i=0; i<n; i++)  {
  for (j=0; j<n; j++) {
    sum = 0.0;
    for (k=0; k<n; k++)
      sum += a[i][k] * b[k][j];
    c[i][j] = sum;
  }
}
```

$4\times3 + 2\times2 + 7\times5 = 51$



Miss Rate analysis

- Assume:
  - Block size = 32B (big enough for four doubles)
  - Matrix dimension (N) is very large
    - Approximate 1/N as 0.0
  - Cache is not even big enough to hold multiple rows
- Analysis Method:
  - Look at access pattern of inner loop



Effect of Cache Layout

C arrays allocated in row-major order

- each row in contiguous memory locations

Stepping through columns in one row:

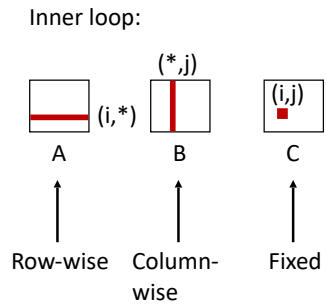
- **for (i = 0; i < N; i++)**
  sum += a[0][i];
- accesses successive elements
- if block size (B) > sizeof(a_{ij}) bytes, exploit spatial locality
  - miss rate = sizeof(a_{ij}) / B

Stepping through rows in one column:

- **for (i = 0; i < N; i++)**
  sum += a[i][0];
- accesses distant elements
- no spatial locality!
  - miss rate = 1 (i.e. 100%)

Effect of loop order (ijk)

```
/* ijk */
for (i=0; i<n; i++)  {
  for (j=0; j<n; j++) {
    sum = 0.0;
    for (k=0; k<n; k++)
      sum += a[i][k] * b[k]
[j];
    c[i][j] = sum;
  }
}
```

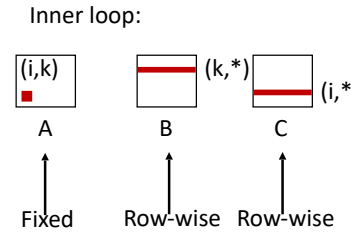


Miss rate for inner loop iterations:

A	B	C
0.25	1.0	0.0

Effect of loops (kij)

```
/* kij */
for (k=0; k<n; k++) {
  for (i=0; i<n; i++) {
    r = a[i][k];
    for (j=0; j<n; j++)
      c[i][j] += r * b[k][j];
  }
}
```

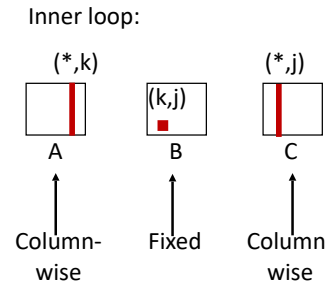


Miss rate for inner loop iterations:

A	B	C
0.0	0.25	0.25

Effect of loops (jki)

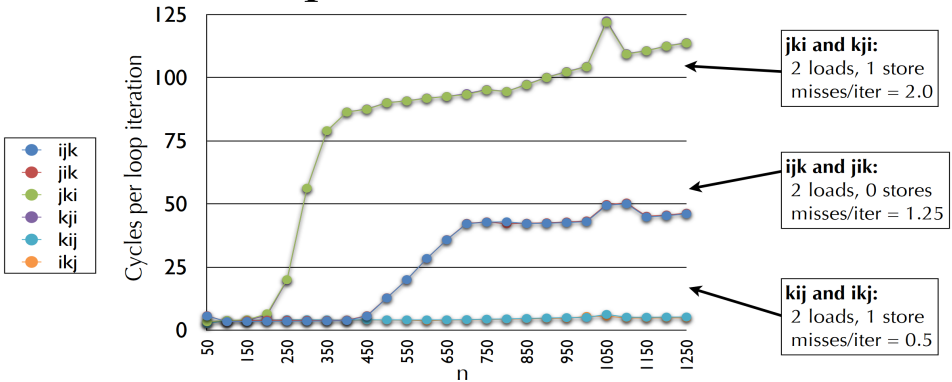
```
/* jki */
for (j=0; j<n; j++) {
  for (k=0; k<n; k++) {
    r = b[k][j];
    for (i=0; i<n; i++)
      c[i][j] += a[i][k] * r;
  }
}
```



Miss rate for inner loop iterations:

A	B	C
1.0	0.0	1.0

Effect of loops



- Miss rate better predictor or performance than number of mem. accesses!
- For large N, kij and ikj performance almost constant.
  Due to **hardware prefetching**, able to recognize stride-1 patterns.

Book

P & H, Chapter 4

Exercise
From H & P Sixth Edition Appendix B

Correction on AMAT Calculation

- Do not calculate the normalized AMAT in case of separate instruction and data caches unless explicitly specified. Not everybody does this normalisation

Exercise 1

Assume a fully associative write-back cache with many cache entries that starts empty. Following is a sequence of five memory operations (the address is in square brackets):

```
Write Mem[100];
Write Mem[100];
Read  Mem[200];
Write Mem[200];
Write Mem[100].
```

What are the number of hits and misses when using no-write allocate versus write allocate?

Exercise 1

For no-write allocate, the address 100 is not in the cache, and there is no allocation on write, so the first two writes will result in misses. Address 200 is also not in the cache, so the read is also a miss. The subsequent write to address 200 is a hit. The last write to 100 is still a miss. The result for no-write allocate is four misses and one hit.

For write allocate, the first accesses to 100 and 200 are misses, and the rest are hits because 100 and 200 are both found in the cache. Thus, the result for write allocate is two misses and three hits.

Exercise 2

Which has the lower miss rate: a 16 KiB instruction cache with a 16 KiB data cache or a 32 KiB unified cache? Use the miss rates in Figure B.6 to help calculate the correct answer, assuming 36% of the instructions are data transfer instructions. Assume a hit takes 1 clock cycle and the miss penalty is 200 clock cycles. A load or store hit takes 1 extra clock cycle on a unified cache if there is only one cache port to satisfy two simultaneous requests. Using the pipelining terminology the unified cache leads to a structural hazard. What is the average memory access time in each case? Assume write-through caches with a write buffer and ignore stalls due to the write buffer.

Size (KiB)	Instruction cache	Data cache	Unified cache
8	8.16	44.0	63.0
16	3.82	40.9	51.0
32	1.36	38.4	43.3
64	0.61	36.9	39.4
128	0.30	35.3	36.2
256	0.02	32.6	32.9

Figure B.6 Miss per 1000 instructions for instruction, data, and unified caches of different sizes. The percentage of instruction references is about 74%. The data are for two-way associative caches with 64-byte blocks for the same computer

Exercise 2

First let's convert misses per 1000 instructions into miss rates. Solving the preceding general formula, the miss rate is

$$\text{Miss rate} = \frac{\dfrac{\text{Misses}}{\text{1000 Instructions}} / 1000}{\dfrac{\text{Memory accesses}}{\text{Instruction}}}$$

Because every instruction access has exactly one memory access to fetch the instruction, the instruction miss rate is

$$\text{Miss rate}_{\text{16 KB instruction}} = \frac{3.82/1000}{1.00} = 0.004$$

Because 36% of the instructions are data transfers, the data miss rate is

$$\text{Miss rate}_{\text{16 KB data}} = \frac{40.9/1000}{0.36} = 0.114$$

The unified miss rate needs to account for instruction and data accesses:

$$\text{Miss rate}_{\text{32 KB unified}} = \frac{43.3/1000}{1.00 + 0.36} = 0.0318$$

Exercise 2

As stated herein, about 74% of the memory accesses are instruction references. Thus, the overall miss rate for the split caches is

This is Normalised

$$(74\% \times 0.004) + (26\% \times 0.114) = 0.0326$$

Thus, a 32 KiB unified cache has a slightly lower effective miss rate than two 16 KiB caches.

The average memory access time formula can be divided into instruction and data accesses:

$$\begin{aligned}&\text{Average memory access time}\\&= \%\,\text{instructions} \times (\text{Hit time} + \text{Instruction miss rate} \times \text{Miss penalty})\\&\quad + \%\,\text{data} \times (\text{Hit time} + \text{Data miss rate} \times \text{Miss penalty})\end{aligned}$$

Therefore, the time for each organization is

This is Normalised AMAT

$$\begin{aligned}&\text{Average memory access time}_{\text{split}}\\&= 74\% \times (1 + 0.004 \times 200) + 26\% \times (1 + 0.114 \times 200)\\&= (74\% \times 1.80) + (26\% \times 23.80) = 1.332 + 6.188 = 7.52\\&\text{Average memory access time}_{\text{unified}}\\&= 74\% \times (1 + 0.0318 \times 200) + 26\% \times (1 + 1 + 0.0318 \times 200)\\&= (74\% \times 7.36) + (26\% \times 8.36) = 5.446 + 2.174 = 7.62\end{aligned}$$

Hence, the split caches in this example—which offer two memory ports per clock cycle, thereby avoiding the structural hazard—have a better average memory access time than the single-ported unified cache despite having a worse effective miss rate.

Exercise 3

Let's use an in-order execution computer . Assume that the cache miss penalty is 200 clock cycles, and all instructions usually take 1.0 clock cycles (ignoring memory stalls). Assume that the average miss rate is 2%, there is an average of 1.5 memory references per instruction, and the average number of cache misses per 1000 instructions is 30. What is the impact on performance when behavior of the cache is included? Calculate the impact using both misses per instruction and miss rate.

Exercise 3

$$\text{CPU time} = \text{IC} \times \left(\text{CPI}_{\text{execution}} + \frac{\text{Memory stall clock cycles}}{\text{Instruction}} \right) \times \text{Clock cycle time}$$

The performance, including cache misses, is

$$\begin{aligned} \text{CPU time}_{\text{with cache}} &= \text{IC} \times [1.0 + (30/1000 \times 200)] \times \text{Clock cycle time} \\ &= \text{IC} \times 7.00 \times \text{Clock cycle time} \end{aligned}$$

Now calculating performance using miss rate:

$$\text{CPU time} = \text{IC} \times \left(\text{CPI}_{\text{execution}} + \text{Miss rate} \times \frac{\text{Memory accesses}}{\text{Instruction}} \times \text{Miss penalty} \right) \times \text{Clock cycle time}$$

$$\begin{aligned} \text{CPU time}_{\text{with cache}} &= \text{IC} \times [1.0 + (1.5 \times 2\% \times 200)] \times \text{Clock cycle time} \\ &= \text{IC} \times 7.00 \times \text{Clock cycle time} \end{aligned}$$

The clock cycle time and instruction count are the same, with or without a cache. Thus, CPU time increases sevenfold, with CPI from 1.00 for a "perfect cache" to 7.00 with a cache that can miss. Without any memory hierarchy at all the CPI would increase again to $1.0 + 200 \times 1.5$ or 301—a factor of more than 40 times longer than a system with a cache!

Exercise 4

What is the impact of 1-way vs 2-way cache organizations on the performance of a processor? Assume that the CPI with a perfect cache is 1.0, the clock cycle time is 0.35 ns, there are 1.4 memory references per instruction, the size of both caches is 128 KiB, and both have a block size of 64 bytes. One cache is direct mapped and the other is two-way set associative. for set associative caches we must add a multiplexor to select between the blocks in the set depending on the tag match. Because the speed of the processor can be tied directly to the speed of a cache hit, assume the processor clock cycle time must be stretched 1.35 times to accommodate the selection multiplexor of the set associative cache. the cache miss penalty is 65 ns for either cache organization. (In practice, it is normally rounded up or down to an integer number of clock cycles.) First, calculate the average memory access time and then processor performance. Assume the hit time is 1 clock cycle, the miss rate of a direct-mapped 128 KiB cache is 2.1%, and the miss rate for a two-way set associative cache of the same size is 1.9%.

Exercise 4

Average memory access time is

$$\text{Average memory access time} = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$$

Thus, the time for each organization is

$$\text{Average memory access time}_{\text{1-way}} = 0.35 + (.021 \times 65) = 1.72\,\text{ns}$$

$$\text{Average memory access time}_{\text{2-way}} = 0.35 \times 1.35 + (.019 \times 65) = 1.71\,\text{ns}$$

The average memory access time is better for the two-way set-associative cache.

The processor performance is

$$\begin{aligned}\text{CPU time} &= \text{IC} \times \left(\text{CPI}_{\text{execution}} + \frac{\text{Misses}}{\text{Instruction}} \times \text{Miss penalty}\right) \times \text{Clock cycle time} \\ &= \text{IC} \times \left[(\text{CPI}_{\text{execution}} \times \text{Clock cycle time}) \right. \\ &\quad \left. + \left(\text{Miss rate} \times \frac{\text{Memory accesses}}{\text{Instruction}} \times \text{Miss penalty} \times \text{Clock cycle time}\right)\right]\end{aligned}$$

Exercise 4

Substituting 65 ns for (Miss penalty $\times$ Clock cycle time), the performance of each cache organization is

$$\text{CPU time}_{\text{1-way}} = \text{IC} \times [1.0 \times 0.35 + (0.021 \times 1.4 \times 65)] = 2.26 \times \text{IC}$$

$$\text{CPU time}_{\text{2-way}} = \text{IC} \times [1.0 \times 0.35 \times 1.35 + (0.019 \times 1.4 \times 65)] = 2.20 \times \text{IC}$$

and relative performance is

$$\frac{\text{CPU time}_{\text{2-way}}}{\text{CPU time}_{\text{1-way}}} = \frac{2.26 \times \text{Instruction count}}{2.20 \times \text{Instruction count}} = 1.03$$

In contrast to the results of average memory access time comparison, the direct-mapped cache leads to slightly better average performance because the clock cycle is stretched for *all* instructions for the two-way set associative case, even if there are fewer misses. Because CPU time is our bottom-line evaluation and because direct mapped is simpler to build, the preferred cache is direct mapped in this example.

Exercise 5

Let's redo the preceding example, but this time we assume the processor with the longer clock cycle time supports out-of-order execution yet still has a direct-mapped cache. Assume 30% of the 65 ns miss penalty can be overlapped; that is, the average Miss penalty is now 45.5 ns.

Exercise 5

Average memory access time for the out-of-order (OOO) computer is

$$\text{Average memory access time}_{\text{1-way,OOO}} = 0.35 \times 1.35 + (0.021 \times 45.5) = 1.43\,\text{ns}$$

The performance of the OOO cache is

$$\text{CUP time}_{\text{1-way,OOO}} = \text{IC} \times \left| 1.0 \times 0.35 \times 1.35 + (0.021 \times 1.4 \times 45.5) \right| = 1.81 \times \text{IC}$$

Hence, despite a much slower clock cycle time and the higher miss rate of a direct-mapped cache, the out-of-order computer can be slightly faster if it can hide 30% of the miss penalty.

Exercise 6

Given the following data, what is the impact of second-level cache associativity on its miss penalty?

- Hit time$_{L2}$ for direct mapped $= 10$ clock cycles.
- Two-way set associativity increases hit time by 0.1 clock cycle to 10.1 clock cycles.
- Local miss rate$_{L2}$ for direct mapped $= 25\%$.
- Local miss rate$_{L2}$ for two-way set associative $= 20\%$.
- Miss penalty$_{L2} = 200$ clock cycles.

Exercise 6

For a direct-mapped second-level cache, the first-level cache miss penalty is

$$\text{Miss penalty}_{\text{1-way L2}} = 10 + 25\% \times 200 = 60.0 \text{ clock cycles}$$

Adding the cost of associativity increases the hit cost only 0.1 clock cycle, making the new first-level cache miss penalty:

$$\text{Miss penalty}_{\text{2-way L2}} = 10.1 + 20\% \times 200 = 50.1 \text{ clock cycles}$$

In reality, second-level caches are almost always synchronized with the first-level cache and processor. Accordingly, the second-level hit time must be an integral number of clock cycles. If we are lucky, we shave the second-level hit time to 10 cycles; if not, we round up to 11 cycles. Either choice is an improvement over the direct-mapped second-level cache:

$$\text{Miss penalty}_{\text{2-way L2}} = 10 + 20\% \times 200 = 50.0 \text{ clock cycles}$$
$$\text{Miss penalty}_{\text{2-way L2}} = 11 + 20\% \times 200 = 51.0 \text{ clock cycles}$$

Digital Logic Design + Computer Architecture

Sayandeep Saha

Assistant Professor
Department of Computer Science and Engineering
Indian Institute of Technology Bombay

DRAM

Slide Courtesy — Biswa

DRAM Organization

Channel

DIMM

Rank

Chip

Bank

Row

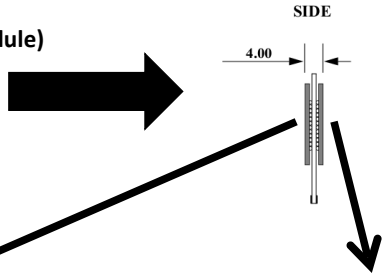
Column

Rank 1 with 8 chips

Rank 0 with 8 chips

Rank

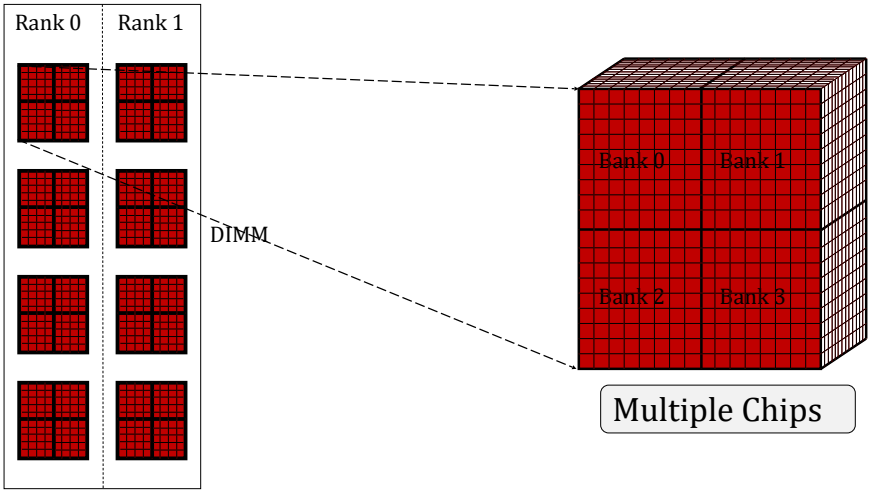
DIMM (Dual in-line memory module)



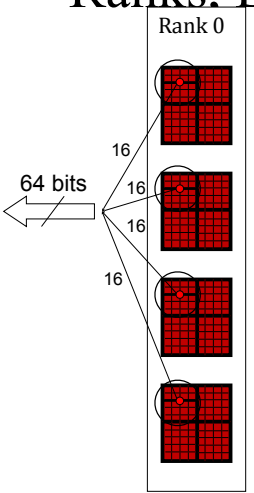
Rank 0: collection of 8 chips

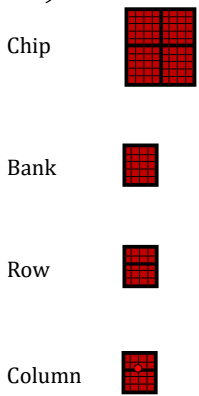
Rank 1

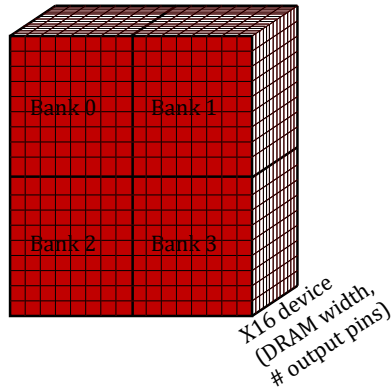
Ranks, Banks, Rows, Columns



Ranks, Banks, Rows, Columns







16-bit interface: 16 bits from each chip in one go

Ranks, Banks, Rows, Columns

Each rank has 64-bit wide data bus

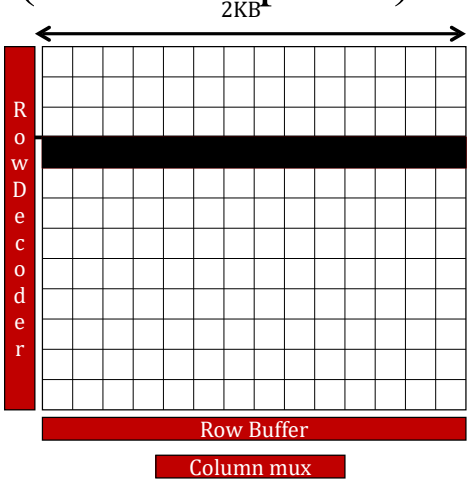
If a rank is of width x8 then # DRAM chips ??

What about x4, # DRAM chips ??

If a rank is of width x8 then # DRAM chips ?? 8

What about x4, # DRAM chips ?? 16

Row Decoder and Row buffer (Sense Amplifier)



- Each bank has a row buffer
- Stores the last used row

An Example

2Gb * 8 DRAM Chips (one side of the rank)

Total 16 chips + 2 chips for ECC (for both the ranks)

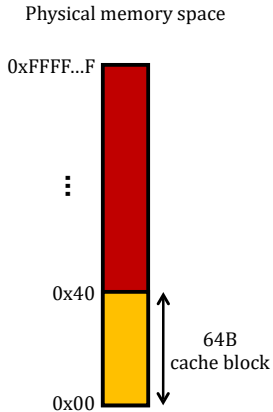
64 bit + 8 bit ECC interface (72 bit wide DIMM)

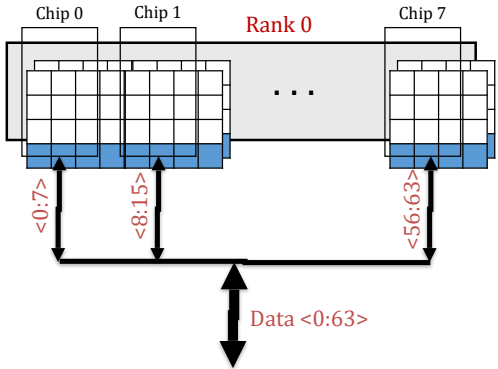
Transferring a 64B cache line will take 8 transfers of 8B each

8B will come from 8 chips (8 bits from one chip)

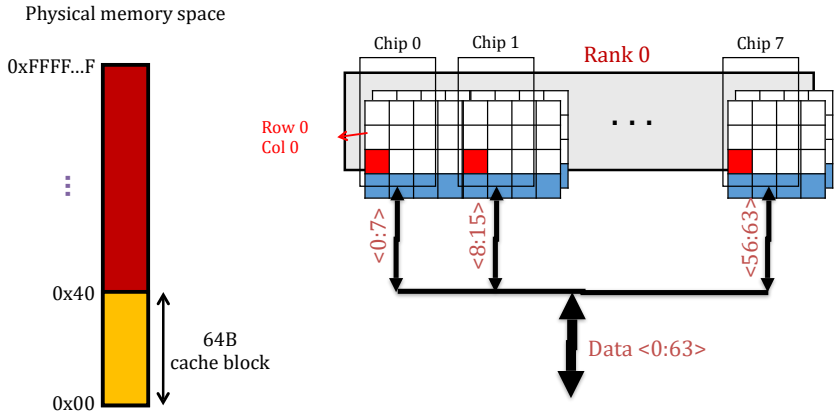
1 bit from each DRAM array assuming 8 DRAM arrays per bank

DRAM to LLC interaction

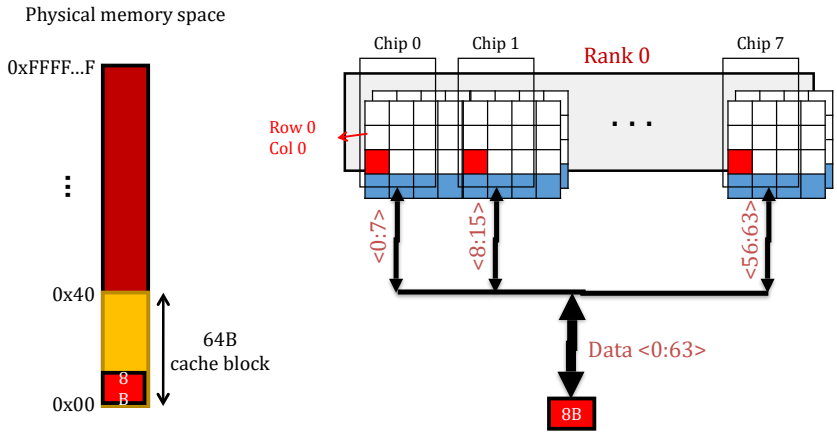




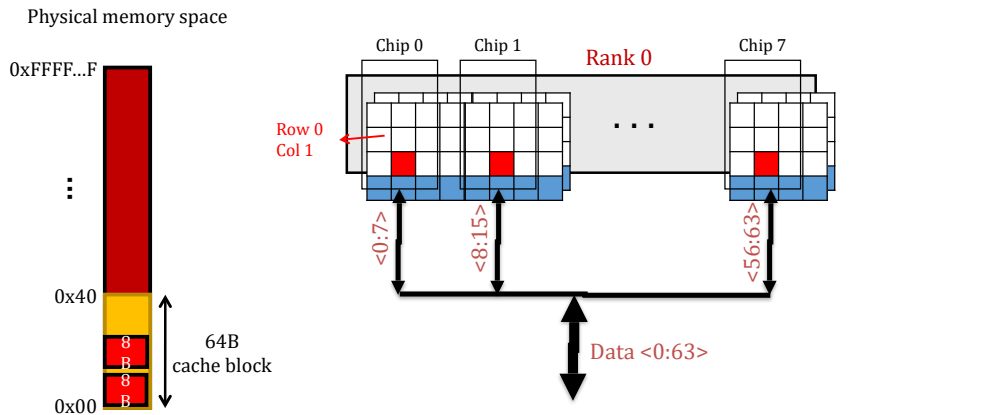
DRAM to LLC interaction



DRAM to LLC interaction



DRAM to LLC

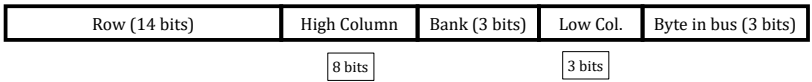


8 cycles (DRAM IO): 1 cycle transfers 8 bytes from a column

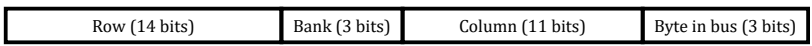
DRAM Address Mapping (One Channel)

2GB DRAM, 8 Banks, 16K rows, 2K Columns per bank

Cache Interleaving: Consecutive cache blocks in consecutive banks



Row Interleaving: Consecutive rows in consecutive banks



Row Access

A “DRAM row” is also called a “DRAM page”
“Sense amplifiers” also called “row buffer”

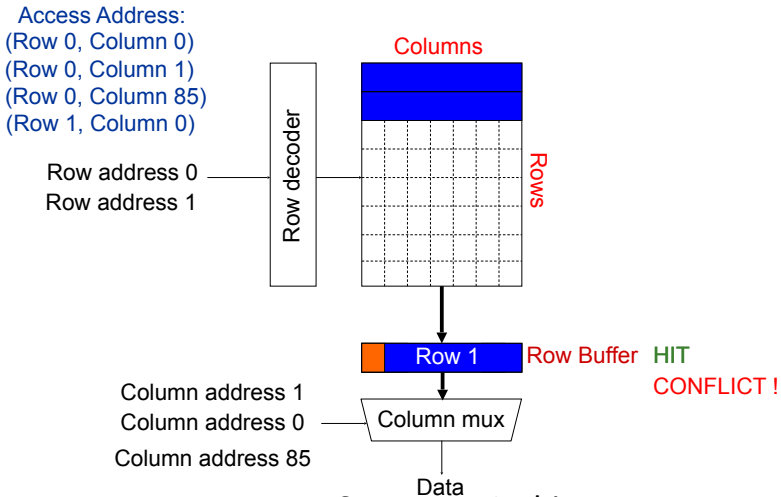
Each address is a <row,column> pair
Access to a “closed row”

- Activate command opens row (placed into row buffer)
- Read/write command reads/writes column in the row buffer
- Precharge command closes the row and prepares the bank for next access

Access to an “open row”

- No need for activate command

Row Buffer hit/miss/conflict



DRAM Timing Constraints

tRAS: Row address strobe latency

tRP: Precharge latency

tCAS: Column address strobe latency

tRC: Row cycle time: tRAS+tRP

Row (Page) Policies

Open Page (do not get confused with an OS page)

After an access: Keep the page in the row-buffer

Consecutive accesses to the same page : Row-buffer Hit

On an access to different page: Close the row and open the new one

Closed page:

After an access: Close the page if there are no accesses to the same row in the request queue.

Consecutive accesses to different page : Low latency

On an access to different page: No need to close the row

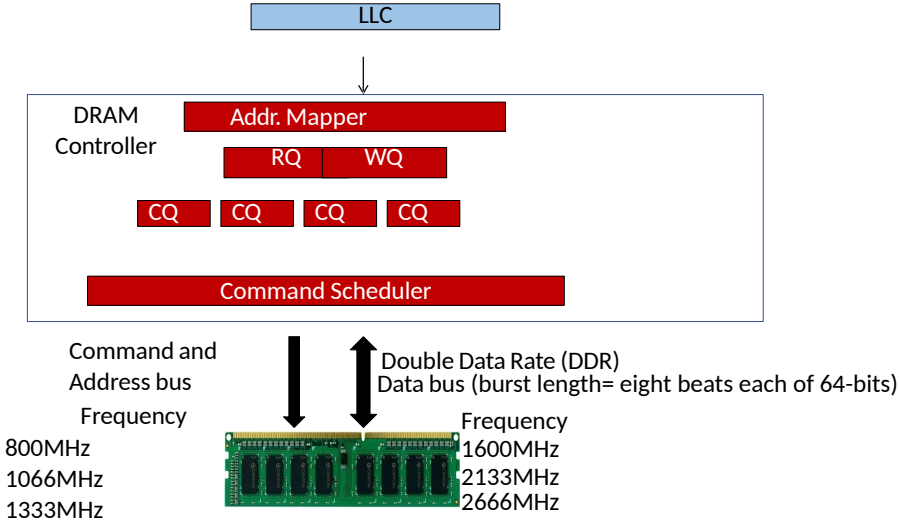
10K view on the latency

Page Empty: ACT + CAS

Page Hit: CAS

Page Miss: PRE+ACT+CAS

DRAM Controller



Reads and Writes(Writebacks)

Reads are critical to performance

Write Queue stores writes and the writes are serviced after # writes reach a threshold

Why?

The direction of the data bus changes from reads to writes. So ??

DRAM controller creates DRAM commands from based on the requests at read Q and write Q

DRAM Scheduling

Based on
Row-buffer locality,
Source of the request,
Loads/Stores
Load criticality

Satisfy all the timing constraints. Around 60

FR-FCFS

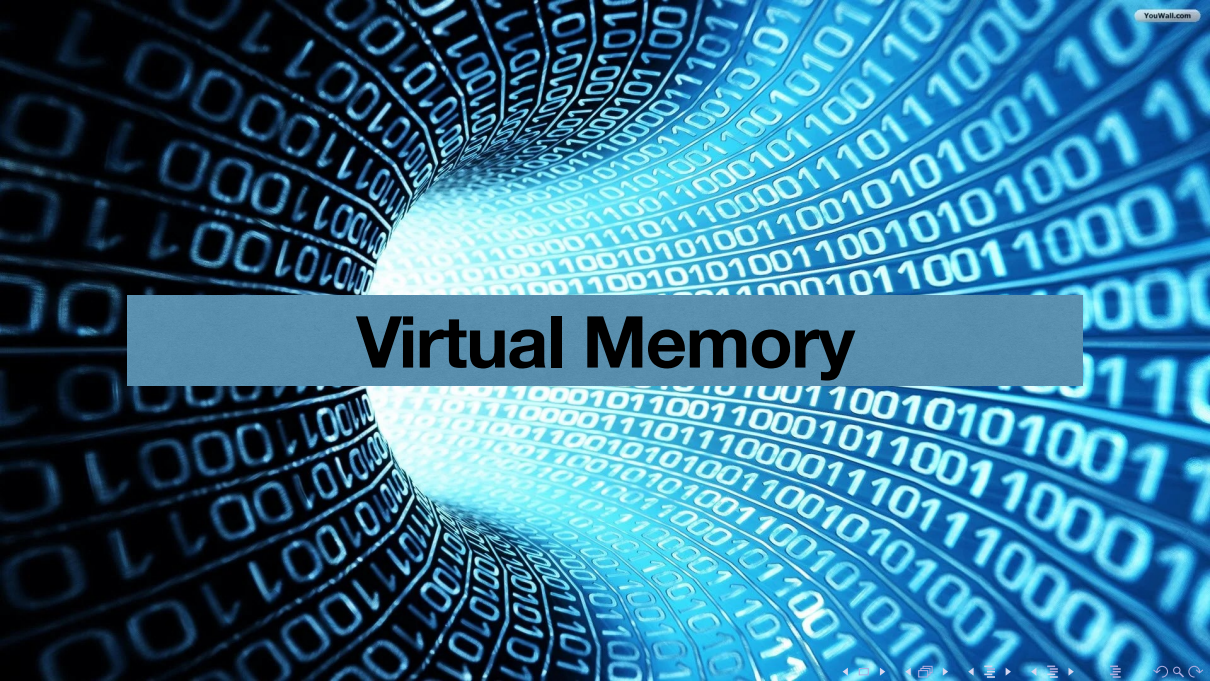
Prefers requests with Row hits (column-first) FR: First Ready

Names	Memory clock	I/O bus clock	Transfer rate	Theoretical bandwidth
DDR-200, PC-1600	100 MHz	100 MHz	200 MT/s	1.6 GB/s
DDR-400, PC-3200	200 MHz	200 MHz	400 MT/s	3.2 GB/s
DDR2-800, PC2-6400	200 MHz	400 MHz	800 MT/s	6.4 GB/s
DDR3-1600, PC3-12800	200 MHz	800 MHz	1600 MT/s	12.8 GB/s
DDR4-2400, PC4-19200	300 MHz	1200 MHz	2400 MT/s	19.2 GB/s
DDR4-3200, PC4-25600	400 MHz	1600 MHz	3200 MT/s	25.6 GB/s
DDR5-4800, PC5-38400	300 MHz	2400 MHz	4800 MT/s	38.4 GB/s
DDR5-6400, PC5-51200	400 MHz	3200 MHz	6400 MT/s	51.2 GB/s

Digital Logic Design + Computer Architecture

Sayandeep Saha

Assistant Professor
Department of Computer Science and Engineering
Indian Institute of Technology Bombay



Virtual Memory

Creating Illusion

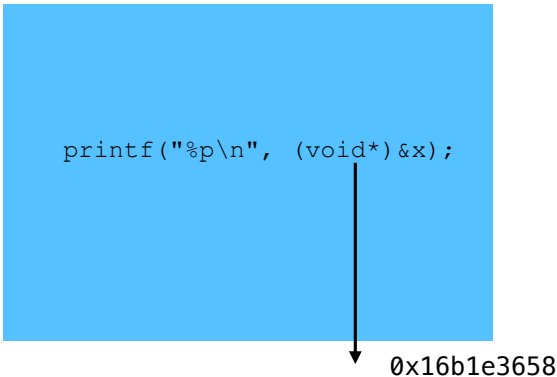
- Consider

```
printf("The address of x is: %p\n", (void*)&x);

printf("The address of x is: %llx\n”, (unsigned long long)&x);
```

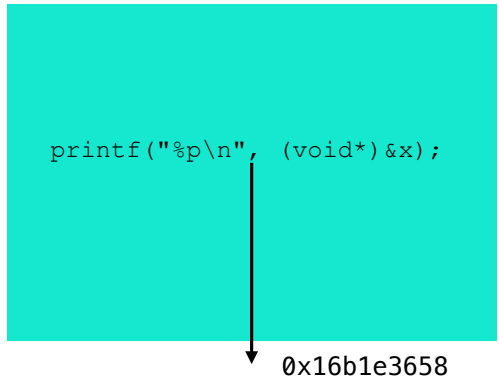
- These are not the actual addresses in DRAM !!! these are virtual addresses

Creating Illusion

Program 1 (or better Process 1)



Program 2 (or better Process 2)



These two maps in completely different locations in the DRAM

Creating Illusion

- Modern systems are multiprogramming-based
  - Run multiple programs on the same processor at the same time
  - Virtual memory simplifies programming
  - More importantly: it provides safe memory isolation between two processes.
    - Isolation is handled under the hood

Creating Illusion

View of the Programmer

- Large contiguous address space
  - Can allocate blocks of contiguous addresses, say using `malloc`
  - Do not need to think about how these addresses are organized physically.
  - Allows a program to (virtually) exceed the amount of physical memory — though not very significant these days.

Inside the System

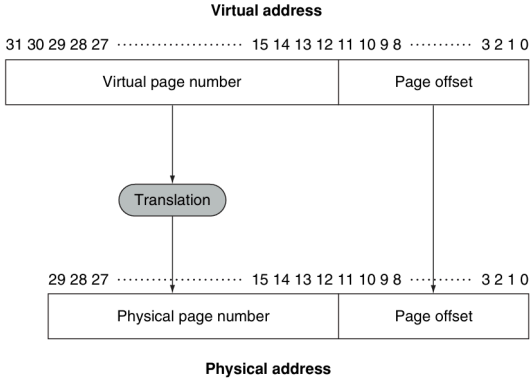
- User virtual address space created by mapping to set of **pages**
  - Addresses in main memory need not be contiguous
  - Two different process has isolated address space. They do not interfere with each other
- OS manages many processes simultaneously
  - Continually switching among processes (context switching)

How to Create Illusion: Paged Memory System

- Processor generates Virtual Address
- The address is translated to Physical Address
- Translation happens through a table called Page Table
- What is a “page”?
  - A fixed size contiguous set of virtual addresses — typically 4KiB (4*1024 bytes) to 16 KB. But can be large as 1 Gib (huge pages). — why?
  - Physical memory is also broken into page-sized chunks, called frames

How to Create Illusion: Paged Memory System

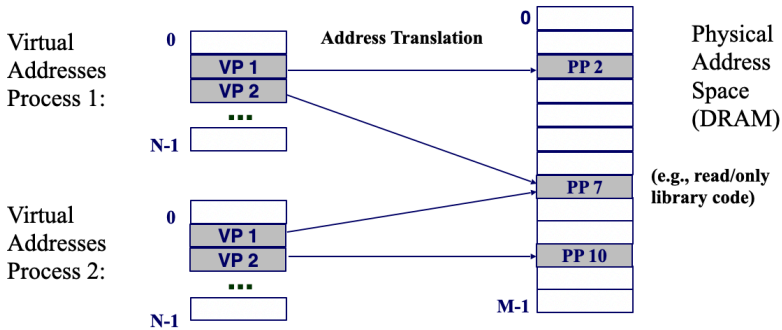
- A Page Table contains the address at the start of each page



- Physical page number is also called “frame number”

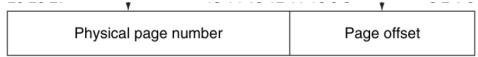
How to Create Illusion: Page Table

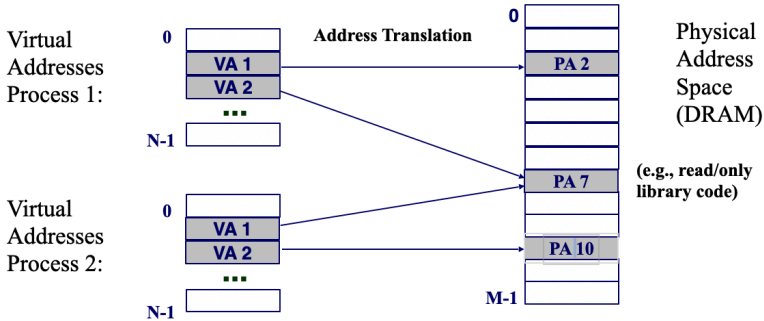
- Processor generates VA = | Page Number | Offset |



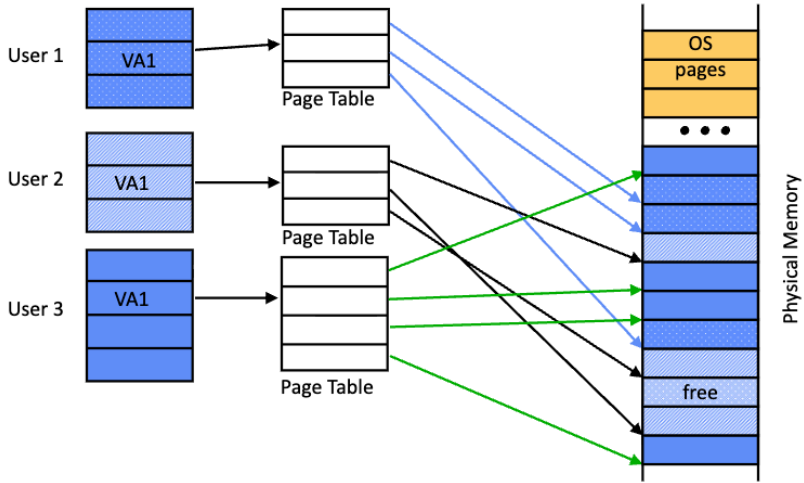
How to Create Illusion: Page Table

- Page table returns PA =





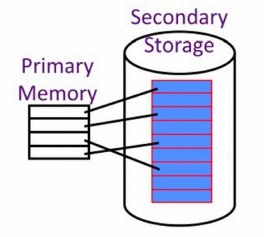
How to Create Illusion: Page Table



- Page table is user specific — each program has one.

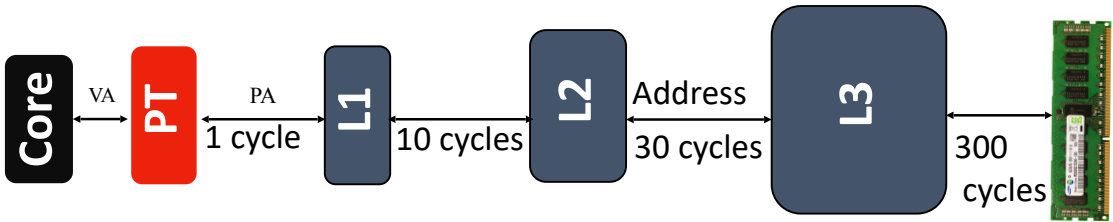
Demand Paging — When Your DRAM is a Cache

- Memory isolation through paging is understandable, but does all the pages in a program remains in the DRAM?



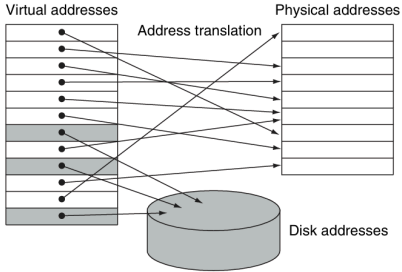
- Think about an LLM with 1 trillion parameters — size can be 300 GB..
  - But maybe you have only 128 GB RAM
- Demand Paging: Bring pages from disc to DRAM when you need it
- Disk has a space to store pages called swap space
- Page fault: A miss in the DRAM, bring page from disk
- Demand paging helps in running ultra large programs without bothering about your DRAM size.

Effect of Address Translation on Program Execution



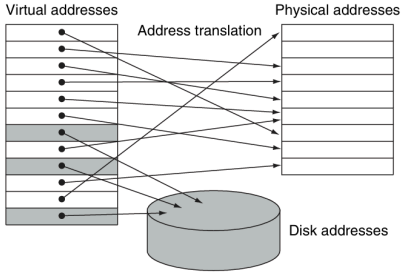
- This view is a bit simplistic — actually there are more thrills in the story
- But main question — how fast should be the PT access?
  - More fundamentally, what is the PT — hardware or software??

Let's See…



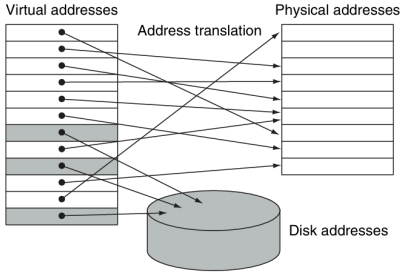
- Addresses can be in DRAM or in Disk — for the later, the page is loaded into the DRAM (demand paging)
- But where the is the PT located?

Let's See…



- Addresses can be in DRAM or in Disk — for the later, the page is loaded into the DRAM (demand paging)
- But where the is the PT located? — in memory

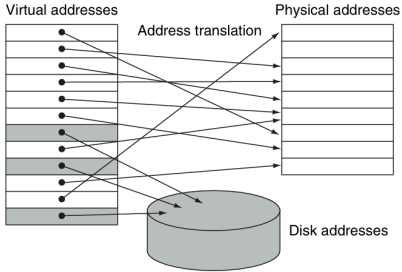
Let's See…



- Addresses can be in DRAM or in Disk — for the later, the page is loaded into the DRAM (demand paging)

- But where the is the PT located? — in memory
- Does the PT fits in your DRAM?

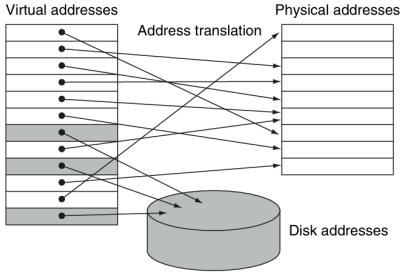
Let's See…



- Addresses can be in DRAM or in Disk — for the later, the page is loaded into the DRAM (demand paging)

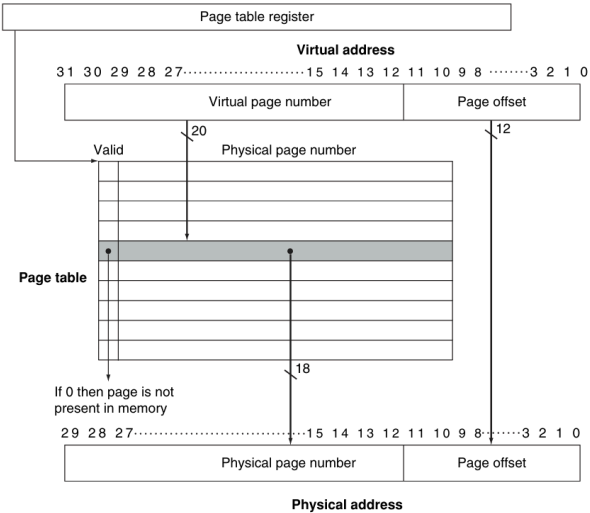
- But where the is the PT located? — in memory
- Does the PT fits in your DRAM? — depends

Let's See…



- Ok, PT is in memory — but how to search the PT to have address translation?
- Point!! PT is a part of the memory allocated to a program (process). Also, each process has a register called *page-table register* which stores the address of its page table

Let's See…



- What can be the size of a page table ??

Size of a Typical Page Table

- Let's assume a 32-bit virtual address space — 2^{32} bytes
- Page size is 4 KiB — 2^{12} bytes
- Let the physical address space is 1 GiB — 2^{32} bytes
- Number of pages in the page table — 2^{20} — 1 million!!
- Let's assume each entry is of 4 byte (32 bit) — VA is just an index
- Size of the page table — $2^{20} \times 4$ bytes $= 4$ MiB
- So each process needs 4 MiB for storing its page table

Size of a Typical Page Table

- So each process needs 4 MiB for storing its page table — not bad
- But you may have 100 processes
- Also, think about a 64-bit address space with 4KiB pages…
  - 2^{52} pages.
  - Typical size of each PT entry 8 bytes.
  - So size of page table — 2^{55} bytes — 32 petabytes..

The Page Table Dilemma

- How to store such large page tables efficiently?
- How to quickly search page tables?
- Intuition: Do you ever allocate 32 petabytes of memory?? — no!!

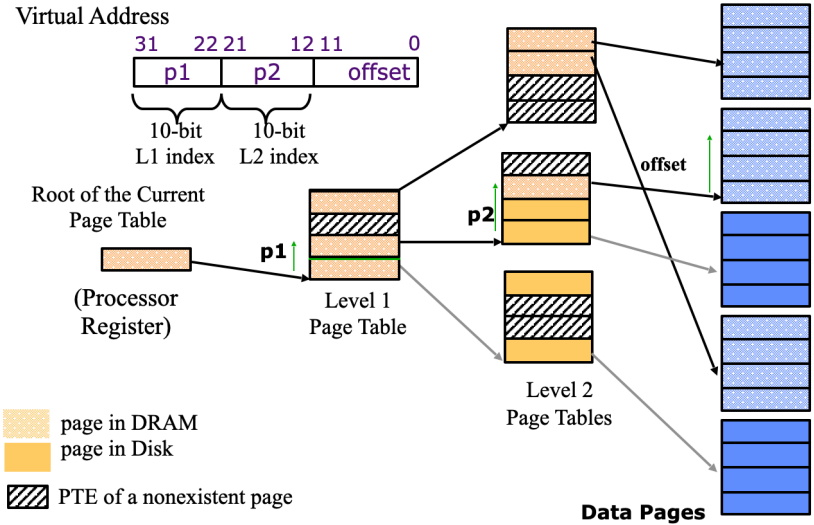
Hierarchical Page Table

- How to store such large page tables efficiently?
- How to quickly search page tables?
- Intuition: Do you ever allocate 32 petabytes of memory?? — no!!
- We should not store anything for a virtual address which maps to NONE
- Idea!! What if we only store a linked list of valid pages??

Hierarchical Page Table

- How to store such large page tables efficiently?
- How to quickly search page tables?
- Intuition: Do you ever allocate 32 petabytes of memory?? — no!!
- We should not store anything for a virtual address which maps to NONE
- Idea!! What if we only store a linked list of valid pages?? — bad idea; search complexity O(N).

Hierarchical Page Table

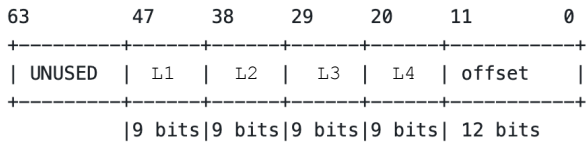


Hierarchical Page Table

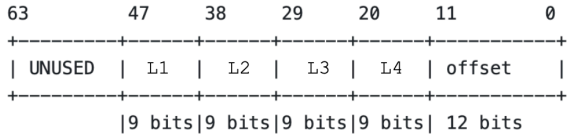
- There can be multiple levels of the page table.
- Level 1 page table contains the address of the Level 2 table and so on..
- Let’s quantify the saving in space…
- Let’s consider a 4 level page table….

Hierarchical Page Table

- Page size is 4KiB
- L1 contains (8 byte) entries of L2 table — how many entries in L1?
  - 4 KiB / 8 bytes per PTE = 512 entries — each level must fit in one page
  - 512 entries ==> 9 bits for indexing
- L2 again contains 512 entries == > 9 bits indexing and so on…
- So the addressing looks like this…

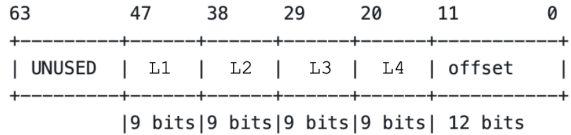


Hierarchical Page Table



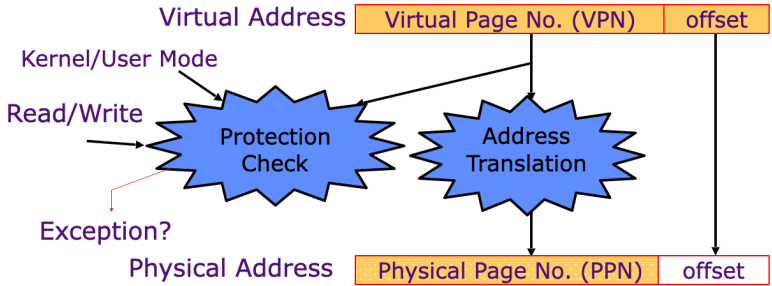
- Only 48 address bits are used (X86-64)
- There is 1 L1 table, 512 possible L2 tables, 512^3 possible L3 tables, and 512^4 possible L4 tables
- L4 actually holds the physical address…
- Note: Each level of table contains the physical address of the next level

Hierarchical Page Table Walk



- Use the address in the *page table register (cr3)* to find the L1 page table address
  - Actually this register contains the physical address of the L1 page table
- Use the L1 index from the virtual address to get the L2 page table address
- Use the L2 page table address and L2 index to get a L3 page table address
- Use the L3 page table address and L3 index to get a L4 page table address
- Use the L4 page table address and L4 index to get the destination physical page
- Use the destination physical page and the offset to get actual physical address within that destination physical page

Address Translation and Protection



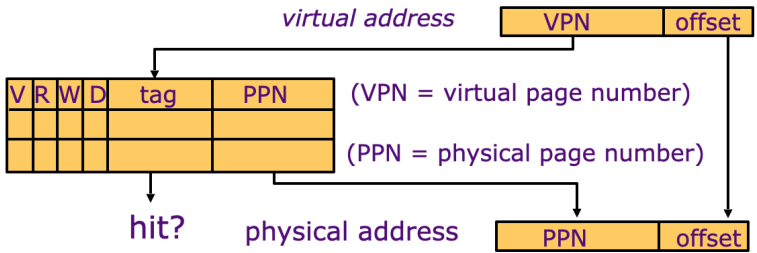
- Every address translation goes through a protection check

Translation Look Aside Buffer

- Address translation is extremely costly
  - DRAM access
  - Also disk access which is orders of magnitude slower than DRAM.
  - How frequently you need to do it?
    - Depends on the size of each page
  - How to make it faster?

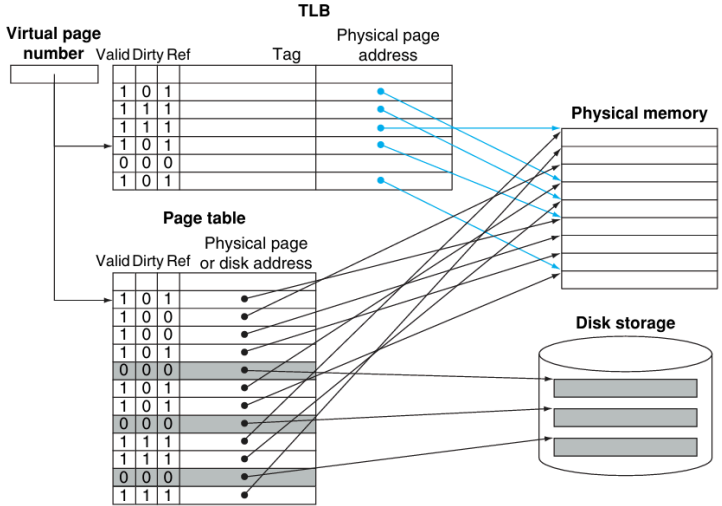
Translation Look Aside Buffer

TLB hit ⇒ *Single Cycle Translation*
TLB miss ⇒ *Page Table Walk to refill*



- Simple Solution: Use a Cache

Translation Look Aside Buffer — More Elaborate



Translation Look Aside Buffer — More Elaborate

- Typically 32-128 entries, usually fully associative
  - Each entry maps a large page, hence less spatial locality across pages
    - more likely that two entries conflict–
  - Sometimes larger TLBs (256-512 entries) are 4-8 way set-associative
  - Larger systems sometimes have multi-level (L1 and L2) TLBs
- Random or FIFO replacement policy
- No process information in TLB? — usually TLB is flushed in context switch..
- TLB Reach: Size of largest virtual address space that can be simultaneously mapped by TLB

How to Handle a TLB Miss

Software (MIPS, Alpha)

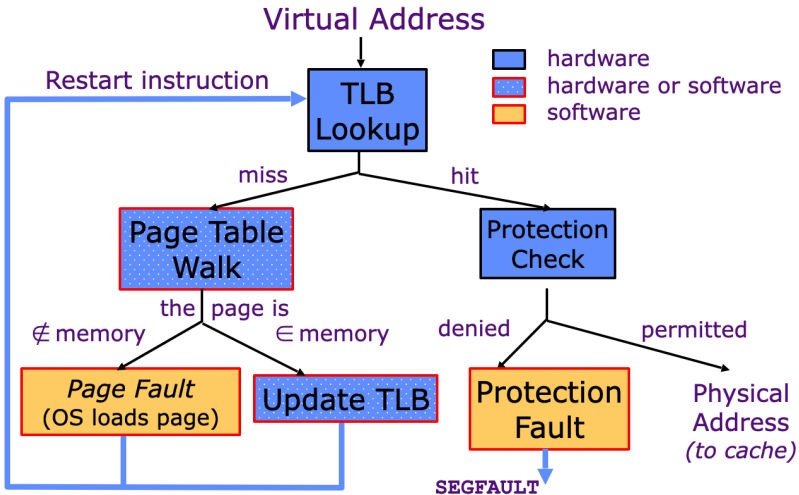
TLB miss causes an exception and the operating system walks the page tables and reloads TLB. *A privileged "untranslated" addressing mode used for walk*

Hardware (SPARC v8, x86, PowerPC)

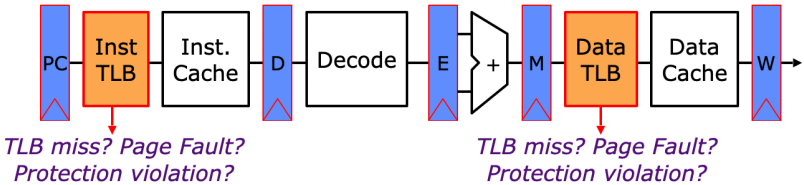
A memory management unit (MMU) walks the page tables and reloads the TLB

If a missing (data or PT) page is encountered during the TLB reloading, MMU gives up and signals a Page-Fault exception for the original instruction

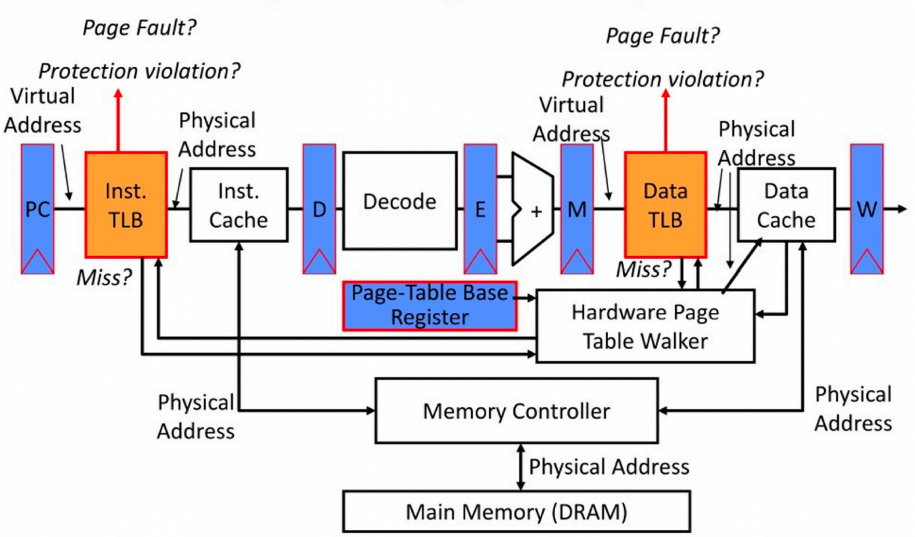
The Entire Picture



TLB, Cache and Pipeline

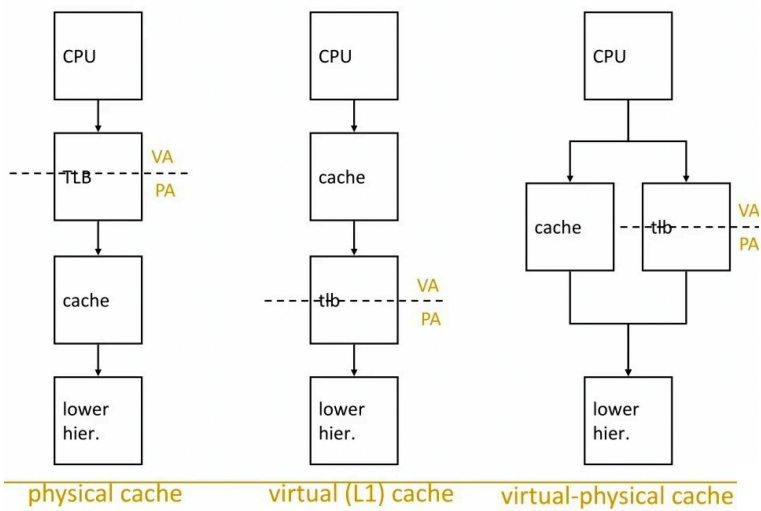


TLB, Cache and Pipeline



Virtual Caches

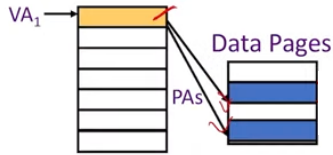
How Virtual Memory Interacts with Cache



Homonym and Synonym

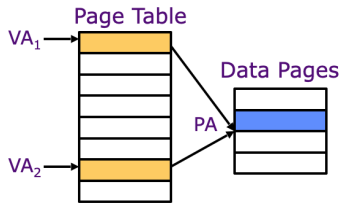
Homonym

- Same VA can map to two different PA
- VAs are from two different processes
- Tag may not uniquely identify the cache data
- Solution:
  - Add process ID
  - Flush cache on context switch
  - Anything else?

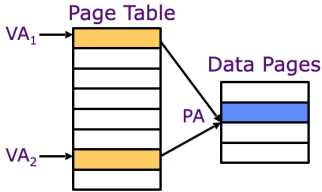


Synonym (or Alising)

- Different VAs can map to the same PA
- Shared library, shared data



Issues with Synonym



Two virtual pages share one physical page

Tag	Data
VA_1	1st Copy of Data at PA
VA_2	2nd Copy of Data at PA

Virtual cache can have two copies of same physical data. Writes to one copy not visible to reads of other!

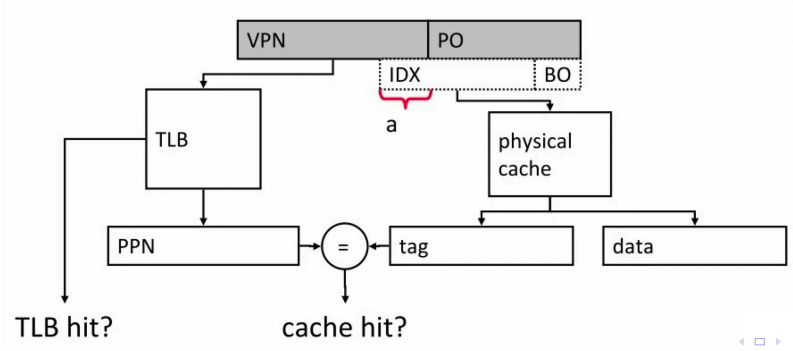
Synonym (or Alising)

- Different VAs can map to the same PA
- Solution:
  - Disallow alisas coexist in the cache
  - Direct mapped cache
  - Anything else?

Virtually Indexed Physically Tagged Cache

Idea

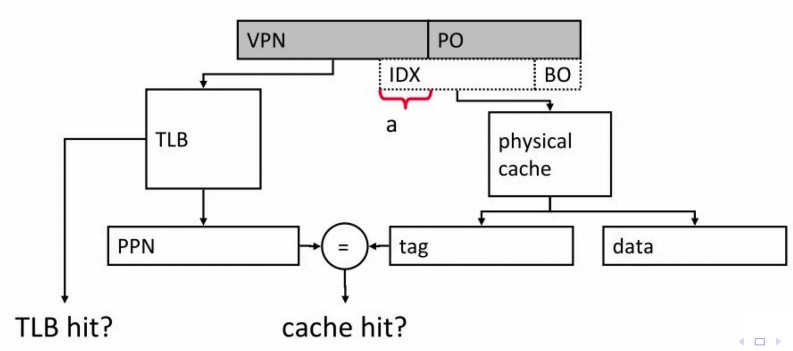
- Perform (L1) indexing based on virtual address
- Simultaneously, look into the TLB for the address translation and get the physical tag bits.
- Perform the tag check based on the physical tag.



Virtually Indexed Physically Tagged Cache: Catch

Idea

- If $|C| < |page| \times associativity$, then the index bits comes from the page offset — page offset is same in VA and PA
- What does it solve?

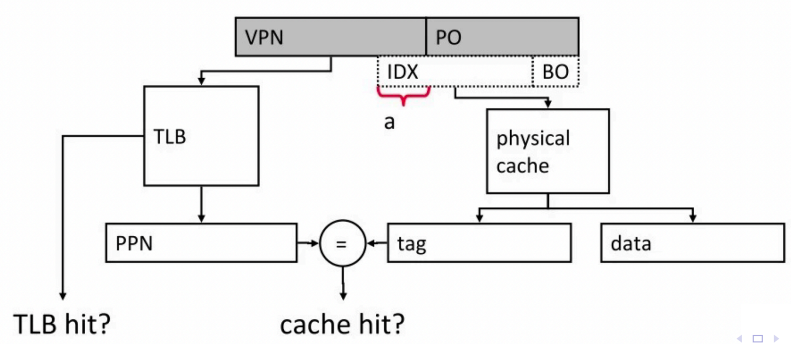


Virtually Indexed Physically Tagged Cache: Catch

Idea

- If $|C| < |page| \times associativity$, then the index bits comes from the page offset — page offset is same in VA and PA

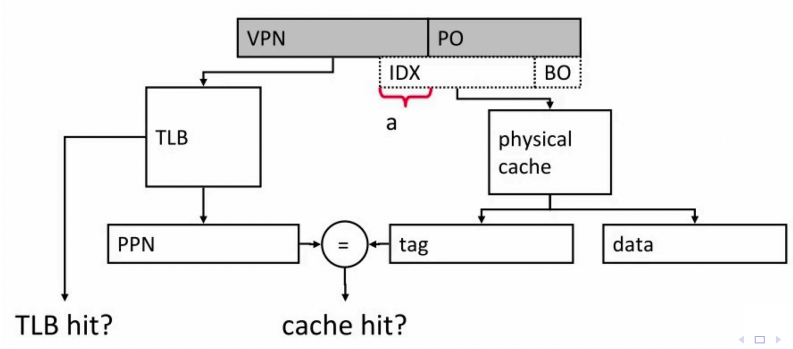
- What does it solve? — Homonym, even if the VAs are the same, physical tags for two different processes will never match.



Virtually Indexed Physically Tagged Cache: Catch

Idea

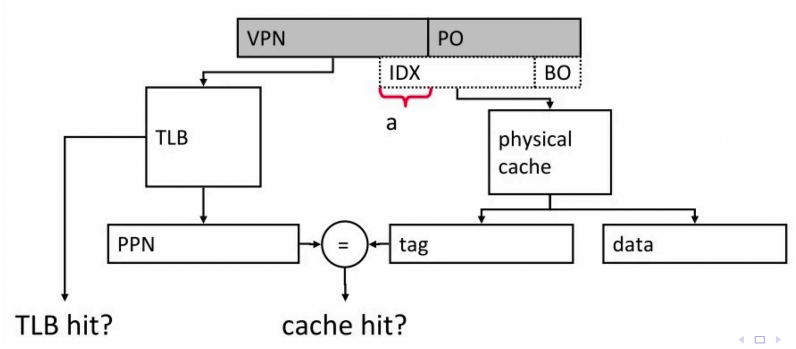
- If $|C| < |page| \times associativity$, then the index bits comes from the page offset — page offset is same in VA and PA
- What about synonym?



Virtually Indexed Physically Tagged Cache: Catch

Idea

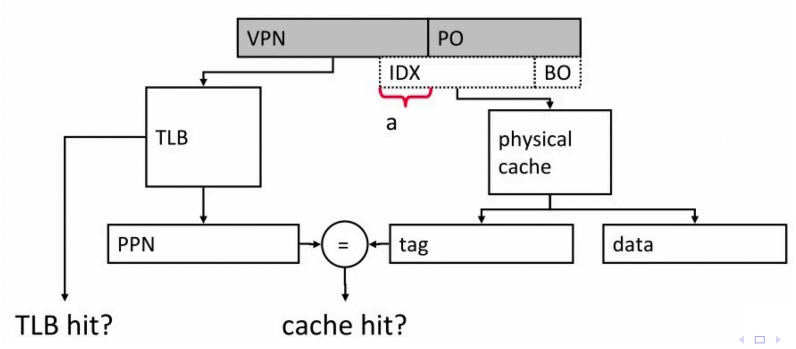
- If $|C| < |page| \times associativity$, then the index bits comes from the page offset — page offset is same in VA and PA
- What about synonym? — it is still there, but..



Virtually Indexed Physically Tagged Cache: Catch

Idea

- If $|C| < |page| \times associativity$, then the index bits comes from the page offset — page offset is same in VA and PA
- What about synonym? — it is still there, it will never happen if the indexing happens from page offset bits only



Overall…

VIVT
- Fastest — but homonym and synonym

PIPT
- Classical

VIPT
- No homonyms; synonyms can also be managed if small cache

PIVT?
- Extremely slow — cannot overlap TLB and cache access
- Homonym — as virtually tagged
- Synonym??

Book

P & H, Chapter 5